Informatica® Cloud Data Quality
July 2026

Data Quality Assets

Informatica Cloud Data Quality Data Quality Assets
July 2026

© Copyright Informatica LLC 2019, 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices and AI assistant acknowledgements are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-07-10

Table of Contents

Preface 11

Informatica Resources 11

- Informatica Documentation 11
- IDMC web site 11
- IDMC Communities 11
- IDMC Marketplace 11
- Informatica Knowledge Base 12
- IDMC Trust Center 12
- IDMC Pulse Dashboard 12
- Informatica Global Customer Support 12

Part I: Introduction 13

Chapter 1: Introduction to data quality assets 14

Part II: Cleanse assets 16

Chapter 2: Introduction to cleanse assets 17

Cleanse configuration 18

- Cleanse process flow 18
- Example: Customer data cleanup 19

Cleanse asset structure 20

- Configuration tab options 20
- Merge tab options 22

Working with steps 23

Configuring a cleanse asset 24

Cleanse instance configuration options 25

Adding a cleanse instance to the asset 26

Cleanse assets and mappings 26

Cleanse assets and dimensions 27

Chapter 3: Step configuration 28

Updating the character case of the input data 29

Removing instances of one or more values that you specify 29

Removing instances of values that appear in a dictionary 30

Removing character spaces from the data 31

Replacing instances of one or more values that you specify 31

Replacing instances of dictionary values with a value that you specify 32

Replacing instances of dictionary values with valid values 32

Dictionaries and cleanse assets 33

Rules and guidelines for search and replace operations. . . . 34

Chapter 4: Merge configuration. . . . 35

Merge configuration options. . . . 35

Merging records. . . . 36

Chapter 5: Test cleanse assets. . . . 37

Testing a cleanse instance. . . . 37

Rules and guidelines for test data. . . . 38

Testing the merge operation. . . . 38

Part III: Deduplicate assets. . . . 39

Chapter 6: Introduction to deduplicate assets. . . . 40

Duplicate analysis operations. . . . 41

Deduplicate asset structure. . . . 42

Deduplication tab options. . . . 42

Consolidation tab. . . . 46

Deduplication objectives. . . . 48

Identity reference data. . . . 52

Reviewing the Administrator properties for identity population data. . . . 53

Configuring the deduplication process. . . . 54

Configuring the consolidation process. . . . 54

Chapter 7: Test deduplicate assets. . . . 56

Testing a deduplicate asset. . . . 56

Understanding the test results. . . . 57

Rules and guidelines for test data. . . . 58

Chapter 8: Field types and identity objectives. . . . 59

Mandatory, required, and optional fields. . . . 59

Part IV: Labeler assets. . . . 69

Chapter 9: Introduction to labeler assets. . . . 70

When to use a labeler asset. . . . 71

Selecting the right labeling mode data for your data. . . . 72

Labeler assets and mappings 72

Labeler assets and dimensions. . . . 73

Labeler asset properties. . . . 74

Labeler asset structure 75

Configuring a labeler asset. . . . 75

Chapter 10: Token labeling operations. 77

Dictionary options for token labeling 77
Regular expression options 80
Configuring a dictionary step for token labeling. 81
Configuring a regular expression step. 82

Chapter 11: Character labeling operations 83

Dictionary options for character labeling. 84
Character set options for character labeling. 86
Configuring a dictionary step for character labeling. 87
Configuring a character set step. 88

Chapter 12: Test labeler assets. 90

Testing a labeler asset in token labeling mode. 90
Testing a labeler asset in character labeling mode. 91

Part V: Parse assets. 92

Chapter 13: Introduction to parse assets. 93

Parsing modes. 94
Unparsed data and overflow data. 94
Rules and guidelines for overflow and unparsed data fields. 95
Parse assets and mappings. 95
Parse assets and dimensions. 96
Parse asset properties 97
Configuring a parse asset. 98

Chapter 14: Custom parsing operations. 99

Parsing with dictionaries. 99
Parsing with regular expressions. 100
Parse asset structure. 101
Dictionary options on the Configuration tab. 101
Regular expression options on the Configuration tab. 103
Configuring a dictionary step. 104
Configuring a regular expression step. 105

Chapter 15: Pattern-based parsing operations. 106

Pattern-based parsing options on the Configuration tab. 107
Configuration summary for pattern-based parsing 108
Installing the asset bundle for pattern-based parsing. 109
Configuring the pre-built options for pattern-based parsing. 110
Testing and updating the pattern-based parsing configuration. 110

Pattern selection and row selection options. . . . 112
Rules and guidelines for user-defined pattern labels. . . . 114

Chapter 16: Test parse assets. . . . 116
Testing a parse asset in Custom mode. . . . 116
Rules and guidelines for test data. . . . 117

Part VI: Rule specification assets. . . . 118

Chapter 17: Introduction to rule specifications. . . . 119
Rule specification modes 119
Rule specification structure 120
Configuration tab options in basic mode 120
Configuration tab options in advanced mode 121
Rule sets 122
Inputs and outputs. . . . 122
Rule statements 123
Rule specifications and exception management. . . . 124
Rule specifications and mappings. . . . 124
Rule specifications and dimensions. . . . 125

Chapter 18: Rule specification configuration. . . . 126
Prerequisites to rule specification configuration. . . . 126
Verifying the business rule requirements. . . . 126
Verifying the business data properties 127
Determine the sequence of the steps in the rule. . . . 127
Configuring a rule specification in basic mode. . . . 127
Configuring a rule specification in advanced mode 128
Configuring a new rule specification in advanced mode. . . . 129
Converting a saved rule specification from basic mode. . . . 129
Creating an input in basic mode. . . . 130
Creating an input and output in advanced mode. . . . 131
Rules and guidelines for inputs. . . . 132
Rule specification data properties in basic mode. . . . 132
Rule specification data properties in advanced mode. . . . 133
Rule specification functions. . . . 133
Rule specifications and dictionaries. . . . 135

Chapter 19: Rule set configuration. . . . 137
Rules and guidelines about rule sets 137
Rule set configuration examples 138
Rule sets that use a single input. . . . 138
Rule sets with dependent conditions. . . . 138

Rule sets with independent conditions. . . . 139
Rule sets in parent and child relationships. . . . 139
Rule set properties . . . 140
Adding a rule set. . . . 140
Copying and moving rule sets . . . 141
Copying or moving a rule set within a rule specification. . . . 141
Copying or moving a rule set to another rule specification . . . 141

Chapter 20: Rule statement configuration . . . 143

Conditions. . . . 144
Rules and guidelines for mapplets in conditions and actions. . . . 145
Function configuration in conditions. . . . 145
Operators . . . 147
Actions. . . . 148
Function configuration in actions. . . . 150
Exception data properties. . . . 154
Function descriptions . . . 157
Rule statement menu options . . . 159
Configuring a rule statement. . . . 160
Configuring a rule statement that reads the result of another rule statement . . . 161
Configuring a rule statement with multiple conditions . . . 161
Adding a rule specification to a rule statement . . . 162
Cut and paste operations and rule statements . . . 163
Copying a rule statement within a rule set . . . 163
Copying a rule statement to another rule specification. . . . 163

Chapter 21: REST APIs for rule specifications. . . . 165

Enabling a rule specification as an API. . . . 165

Chapter 22: Rule specification configuration in advanced mode . . . 166

Rule specification logic in advanced mode. . . . 166
Rule logic configuration examples. . . . 167
Specifying a rule specification in an IF element. . . . 167
Specifying a rule specification in a THEN element. . . . 167
Specifying a mapplet in an IF element. . . . 168
Specifying a mapplet in a THEN element. . . . 169
Specifying a dictionary in the expression logic. . . . 169
Specifying exception data options in the expression logic. . . . 170
Rule specification functions . . . 171
Rule specification operators . . . 172

Chapter 23: Business rules and rule statements. . . . 173

Verifying the accuracy of business data . . . 174

Identifying errors in business data . . . 175
Standardizing project data values . . . 176
Improving the usability of business data . . . 177
Deriving information about business users . . . 178
Verifying that data values conform to business regulations . . . 179
Adding internal data to a data set . . . 180
Finding a value in a range of values . . . 181
Identifying data values that contain key terms or character strings . . . 181
Generating status data for a profile . . . 182
Using status data to identify exception records . . . 183
Resolving hidden data issues . . . 184
Finding null values . . . 185
Finding values that contain character spaces only . . . 185
Combining data values in advanced mode . . . 186
Standardizing character case in advanced mode . . . 187
Eliminating redundant character spaces in advanced mode . . . 188
Standardizing data values using a dictionary in advanced mode . . . 188
Writing the IF statement . . . 189
Writing the THEN statement . . . 189
Writing the ENDIF statement . . . 189
Adding optional parameters to the THEN statement function . . . 189
Rule logic example . . . 190
Standardizing custom data values in advanced mode . . . 190

Chapter 24: Validation and testing . . . 191

Rules and guidelines for test and validation operations . . . 191
Validating a rule specification . . . 191
Testing a rule specification in basic mode . . . 192
Testing a rule set . . . 193
Testing a rule specification in advanced mode . . . 193

Part VII: Verifier assets . . . 194

Chapter 25: Introduction to verifier assets . . . 195

Verifier operations . . . 195
Verifier structure . . . 196
Input model . . . 197
Inputs . . . 197
Outputs . . . 198
Understanding the address structure . . . 199
Address verification process flow . . . 200
Certified verification . . . 200
Address enrichments . . . 201

Verifier assets and mappings . . . 201
Verifier assets and dimensions. . . . 202
Reference data . . . 202

Chapter 26: Verifier configuration. . . . 203

Selecting an input model. . . . 203
Selecting the input fields. . . . 204
Selecting the output fields . . . 204
Selecting output fields to verify and correct addresses. . . . 204
Selecting output fields to update the address structure . . . 205
Selecting output fields to generate address suggestions . . . 206
Selecting output fields to measure address quality . . . 207
Selecting output fields to generate geocodes . . . 208
Selecting output fields to retrieve certification data . . . 209

Chapter 27: Verifier asset properties. . . . 211

Input properties. . . . 211
Process properties. . . . 213
Result properties . . . 215
Standardization properties. . . . 217
Certification report details. . . . 220

Chapter 28: Frequently asked questions. . . . 225

Why does the verifier asset show multiple instances of a single field?. . . . 225
How do I set the input and output address properties for an address?. . . . 225
How do I standardize street and directional descriptors? . . . 226
How do I activate suggestions for addresses that I cannot verify? . . . 226
How do I set the level of verification for an address? . . . 226
How do I standardize common terms in addresses that fail verification?. . . . 227
How do I set the output address data to upper case?. . . . 227
How do I set the destination country for the input addresses?. . . . 227
How do I specify the character set for the output address data?. . . . 227
Can I expand the numerical ranges in address suggestions?. . . . 228
How do I change the language of the output address data?. . . . 228
Can the Secure Agent download CASS data files? . . . 228

Chapter 29: Address verification properties. . . . 229

Verification properties in the CDQAV property set. . . . 230
Configuring the address verification properties. . . . 232

Chapter 30: Validation and testing. . . . 233

Validate a verifier . . . 233

Chapter 31: Input address fields. 234

Chapter 32: Output address fields. 237

Single address elements and preformatted data fields. 237
Status code fields. 237
Verification Status Codes. 238
Result Quality. 238
Address Type. 239
Enrichments: Geocoding fields. 240
Geocoding Accuracy Codes. 241
Enrichments: Certified fields. 241
Australia-AMAS. 242
Canada-SERP 243
France-SNA. 243
New Zealand-Send Right 244
United States-CASS. 245
Enrichments: Global fields. 250
Enrichments: Country Specific fields. 251
Enrichments for Australia addresses. 251
Enrichments for Austria addresses. 253
Enrichment for Belgium addresses. 254
Enrichments for Brazil addresses. 255
Enrichment for Czech Republic addresses. 255
Enrichments for France addresses. 255
Enrichments for Germany addresses. 256
Enrichments for Italy addresses 256
Enrichments for Japan addresses. 256
Enrichments for Poland addresses. 257
Enrichments for Russia addresses. 258
Enrichments for Serbia addresses. 258
Enrichments for South Africa addresses. 258
Enrichments for Spain addresses. 258
Enrichments for Switzerland addresses. 258
Enrichments for United Kingdom addresses. 259
Enrichments for United States addresses. 260
Enrichment and certification status fields. 261

Index. 262

Preface

Use *Data Quality Assets* to learn how to create and manage the assets that you'll add to Data Quality transformations when you configure a mapping in Data Integration.

Informatica Resources

Informatica provides you with a range of product resources through the Informatica Network and other online portals. Use the resources to get the most from your Informatica products and solutions and to learn from other Informatica users and subject matter experts.

Informatica Documentation

Use the Informatica Documentation Portal to explore an extensive library of documentation for current and recent product releases. To explore the Documentation Portal, visit https://docs.informatica.com.

If you have questions, comments, or ideas about the product documentation, contact the Informatica Documentation team at infa_documentation@informatica.com.

IDMC web site

You can access the Informatica® Intelligent Data Management Cloud™ (IDMC) web site at http://www.informatica.com/cloud.

IDMC Communities

Use the IDMC Community to discuss and resolve technical issues. You can also find technical tips, documentation updates, and answers to frequently asked questions.

Access the IDMC Community at:

https://network.informatica.com/community/informatica-network/products/cloud-integration

Developers can learn more and share tips at the Cloud Developer community:

https://network.informatica.com/community/informatica-network/products/cloud-integration/cloud-developers

IDMC Marketplace

Visit the Informatica Marketplace to try and buy Data Integration Connectors, templates, and mapplets:

https://marketplace.informatica.com/

Informatica Knowledge Base

Use the Informatica Knowledge Base to find product resources such as how-to articles, best practices, video tutorials, and answers to frequently asked questions.

To search the Knowledge Base, visit https://search.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

IDMC Trust Center

The IDMC Trust Center provides information about Informatica security, compliance, and privacy practices.

You can access the trust center at https://trust.informatica.com/.

IDMC Pulse Dashboard

The IDMC Pulse Dashboard provides system status and upgrade information for all PODs and services across IDMC.

Follow the services you use and subscribe to alerts to receive notifications about upgrades, maintenance, production incidents, and upcoming releases. The IDMC Pulse Dashboard displays the production status of all the IDMC products. All maintenance updates are posted to this page, and during an outage, it will have the most current information. To ensure you are notified of updates and outages, you can subscribe to receive updates for a single service or all IDMC services. Subscribing to all services is the best way to be certain you never miss an update.

For more information, see the Service Status and Communications guide.

Informatica Global Customer Support

You can contact a Global Support Center through the Informatica Network or by telephone.

To find online support resources on the Informatica Network, click **Contact Support** in the IDMC Help menu to go to the **Cloud Support** page. The **Cloud Support** page includes system status information and community discussions. Log in to Informatica Network and click **Need Help** to find additional resources and to contact Informatica Global Customer Support through email.

The telephone numbers for Informatica Global Customer Support are available from the Informatica web site at https://www.informatica.com/services-and-training/support-services/contact-us.html.

Part I: Introduction

This part contains the following chapter:

- Introduction to data quality assets, 14

CHAPTER 1

Introduction to data quality assets

An asset in a data quality project is a reusable object that defines part or all of a data quality operation. You can configure data quality assets in Data Quality and in Data Integration and add the assets to data quality transformations in a mapping.

The following table describes the data quality assets that you can create:

Asset Name	Description
Cleanse	A cleanse asset is a set of one or more data transformation steps that can standardize the form and content of your data.
Deduplicate	A deduplicate asset measures the level of similarity between records in a data set and optionally consolidates similar records into a single, preferred record.
Labeler	A labeler asset derives information about the content and structure of data. You can configure a labeler asset to analyze tokens, or delimited values, in an input field, or the individual characters in an input field.
Parse	A parse asset defines a set of operations that can identify discrete values in an input field and write the values to appropriate output fields. Use a parse asset to improve the structure of your data.
Rule specification	A rule specification asset represents the data requirements of a business rule in logical form.
Verifier	A verifier asset evaluates the accuracy and deliverability of address records. Use a verifier to determine the accuracy of input addresses, fix errors in addresses, and enhance addresses where possible with additional information.

You add a data quality asset to the corresponding data quality transformation when you configure a mapping in Data Integration. When you run a mapping with a data quality transformation, the transformation applies the logic that you define in the asset to the input fields that that the transformation specifies. The data quality asset doesn't identify or connect directly to source data.

You can also create dictionaries that other data quality assets can use. A dictionary is a table of reference data that a mapping can read to evaluate the accuracy or usefulness of source data.

Create a data quality asset in the following ways:

- Create the asset from the transformation options when you configure a mapping.
- Create and save the asset in Data Quality and add it to a transformation at a later date.

- Create and save the asset in Data Integration and add it to a transformation at a later date.

The asset configuration options are identical in all cases.

Part II: Cleanse assets

This part contains the following chapters:

- Introduction to cleanse assets, 17
- Step configuration, 28
- Merge configuration, 35
- Test cleanse assets, 37

CHAPTER 2

Introduction to cleanse assets

A cleanse asset is a set of one or more data transformation steps that can standardize the form and content of your data.

You might decide to standardize your data to achieve the following goals:

Improve data consistency in a data set

Your data might use a range of data values to represent the same data point. For example, a field of country names might contain a mix of full names, ISO three-character codes, and other non-standard country identifiers. You standardize the values to ensure uniformity in the data field.

Fix errors in data

Your data might include correct and incorrect spellings of given data values. You standardize the data to find and fix the errors.

Comply with regulatory standards

You might need to verify that your data meets the latest regulatory standards that the industry or government defines. For example, you might decide to search a data set for potentially fraudulent accounts. To start the initiative, you standardize the data to replace inaccurate values with a single term, such as REVIEW. You can proceed to review the records that contain the new term.

Prepare for downstream data quality initiatives

You might standardize your data as the first step in a data quality project. For example, you standardize common business terms and postal address terms before you run a mapping with a Verifier transformation on your address data.

Merge data fields to effectively manage your data set

Your data set might contain the data information in the discrete fields, which you might decide to combine and add into a single field. For example, you might need a single full name field in addition to the first name and surname fields in your data set. You can merge the two separate fields into a single field.

You create and test a cleanse asset in Data Quality or Data Integration, and then add the asset to a Cleanse transformation in a mapping in Data Integration. In Data Quality, you create a cleanse asset from the **New** dialog box. In Data Integration, you create a cleanse asset either from the **New Asset** dialog box or from a mapping.

When the mapping runs, the Cleanse transformation applies the operations that you define in the cleanse asset to the fields that you select in the input data. The outputs from the transformation contain the standardized field values and also any merged fields that you configured in the asset.

You can enable a mapplet that contains a cleanse asset transformation as an API. For more information about how to enable the mapplet, see the *Components* help in Data Integration.

Cleanse configuration

A cleanse asset is composed of instances and steps. An instance identifies the input fields on which you can perform cleanse and merge operations. A step is a discrete cleanse operation that you define for an input.

You define one or more cleanse steps for the input fields that an instance identifies. The input fields that you add to an instance and the steps that you define for the input fields depend on your business requirements and the content of your data. You can add one or more input fields to an instance.

Cleanse process flow

To cleanse your data, configure and run assets in Data Quality and in Data Integration.

The cleansing process includes the following steps:

1. Analyze the content of the source data, so that you can identify the fields that require cleanup. Work with a developer or data steward to understand the data sets. You will create inputs that represent the source data columns in the cleanse asset instances.

   During your analysis, perform the following steps:

   a. Verify your business requirements.

   b. Verify the content and structure of your data.

   c. Determine the sequence of the cleanse operations to apply to the data.
2. Configure a cleanse asset to translate your business requirements into one or more cleanse instances. The input fields within an instance are processed through the logic that you define in a particular instance.

   To configure the asset, perform the following steps:

   a. Add one or more instances for the input fields that need cleansing in the source data.

   b. Configure one or more steps for the input fields that each instance identifies.

   c. Optionally, configure the merge operation for the input fields.
3. In Data Integration, define a mapping that can run the cleansing operation:

   a. Add the Cleanse asset to a Cleanse transformation.

   b. Connect the cleanse asset input and output fields to the upstream and downstream objects in the mapping.
4. Run the mapping.

Note: The Cleanse transformation does not identify the instance on which each asset input originates. If you define multiple instances on the cleanse asset, make a record of the instances to which each input belongs. Use the record as a guide when you connect the asset inputs to the transformation input fields.

Example: Customer data cleanup

As a data analyst in an organization, you are trying to cleanse customer data. The customer data set includes multiple fields for customer contact data, including name, gender, and address fields. To perform cleanse operations on the different fields, you can configure a single cleanse asset.

The customers data record in your organization contains the following information:

Title	FirstName	MiddleName	Surname	Gender	Address	Country
Dr.	John	William	smith	Male	2101 massachusetts ave nw washington dc 20008-2811	usa
Mr.	Mr. Frances	folsom	cleveland	Man	18 broomfield ridge midleton co. cork p25 kn66	IRE
Miss	Miss. Abigail	powers	Fillmore	Female	shop 7 208 adelaide st brisbane city qld 4000	aus

You might want to perform the following operations on your record:

- Remove salutation data from the FirstName field.
- Change the character case to title case in the MiddleName and Surname fields.
- Replace `Man` with `Male` in the Gender field.
- Remove character spaces and change the character case to uppercase in the Address field and Country field.

Configure a cleanse asset with four instances, based on the types of cleanse operation that you want to perform on the data. Add one or more input fields to each instance and define the steps that you want to apply to the input fields in each case.

For example, to change the character case to title case in the MiddleName and Surname fields, create an instance in the asset to add two input fields. You can specify MiddleName and Surname as the input field names. Configure the step to perform the **Convert Case** operation on the fields that the instance identifies.

Likewise, you can configure an instance to change the character case of the address and country abbreviation data. Add a step to convert the case of the data to uppercase. Additionally, add a step to replace multiple character spaces with a single space. Provide suitable names for the inputs, such as Address and Country.

Note: The Country input does not contain any character spaces, and so the step logic to remove spaces does not alter the data in the associated input field.

Add the asset that you create to a Cleanse transformation in a mapping. When a mapping runs, the transformation applies the standardization operations that you define in the asset instances to the fields that you select.

Additionally, if you want to merge the cleansed data from the FirstName, MiddleName, and Surname fields into a single field, configure the asset to perform a merge operation on the input fields.

Create a merge record to add the fields that you want to merge. You can specify the merged field name as FullName.

Cleanse asset structure

A cleanse asset contains options on a **Definition** tab, a **Configuration** tab, and a **Merge** tab.

Use the Definition tab options to enter a name for the asset, optionally enter a description for the asset, and select the folder in which to store the asset. Use the Configuration tab options to configure the standardization operations that a mapping will perform. Optionally, use the Merge tab options to merge data from two or more fields into a single field.

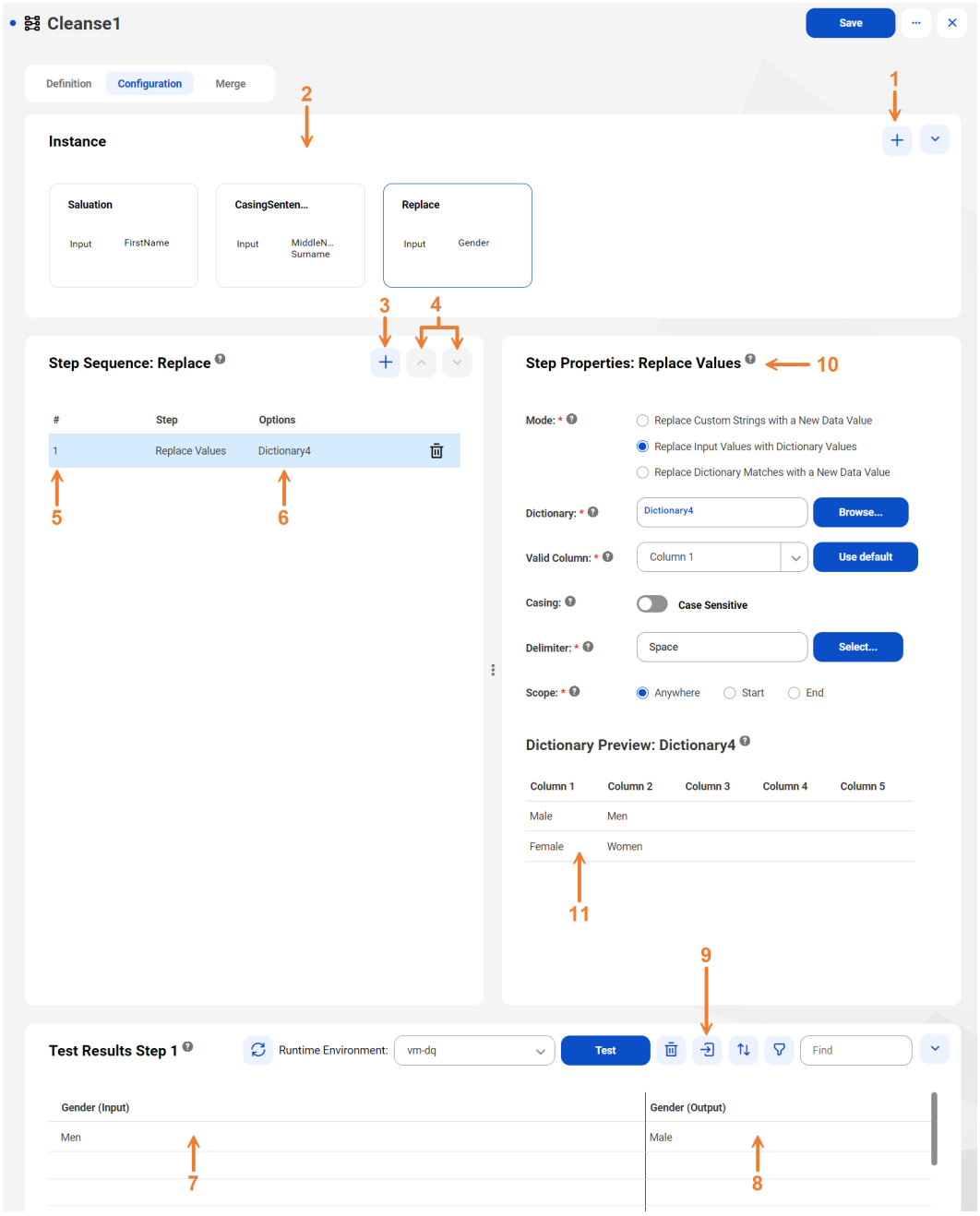
You configure the standardization operations as a sequence of steps that the mapping follows at run time.

Configuration tab options

Use the Configuration tab options to configure the type of cleansing operations that a mapping will perform on the input data. Each instance that you add identifies one or more input fields to which a mapping can

apply the cleanse operations. You add instances to configure different cleansing operations for different input fields within a single asset.

The following image shows the cleanse asset options in the **Configuration** tab:



The Configuration tab includes the following options:

1. Add Instance option.
   Adds a cleanse instance to the asset. You can add one or more cleanse instances.

   Define one or more input fields in each instance that you add.

2. Instance panel.
   Displays the cleanse instances that you add to the asset. You can edit or delete an instance.

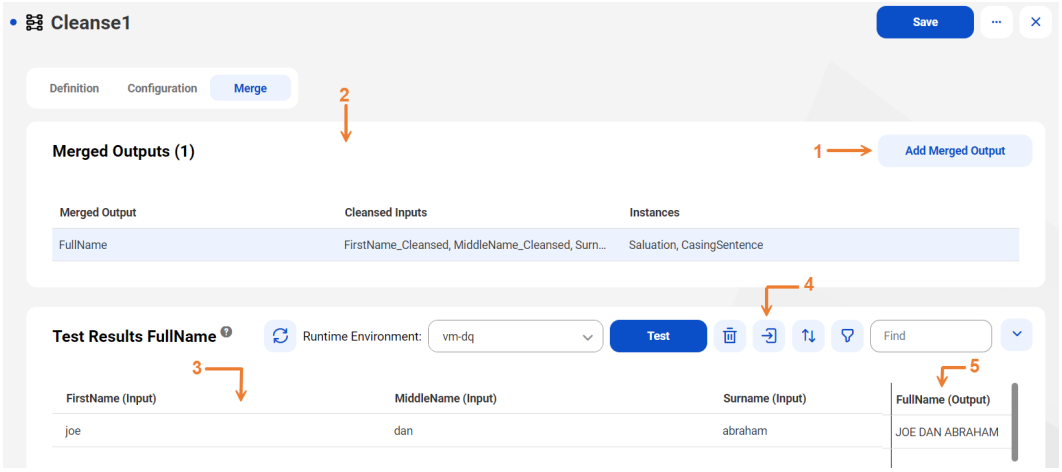
Expand or collapse the instance panel to show or hide the instances that you add. You might collapse the instance panel when you configure a step in an instance, so that the step configuration options are easier to view.

3. Add Step option.
   Adds a step to the asset. A step describes a standardization operation that a mapping can apply to an input data field.
4. Up and Down options.
   Moves a step that you select up or down within the step sequence. The mapping performs standardization operations in the order that you specify.
5. Step sequence.
   Defines the order in which the mapping applies each step to the input field.
6. Operation options.
   Lists the main element in the operation that you define within the step. For example, if you use a dictionary in a step, the list displays the dictionary name.
7. Test input fields.
   Contains the test data that you enter for the instance that you select.
8. Test output fields.
   Contains the result of the test for the instance that you select. The asset displays an output field for each input field that you add to an instance.
9. Import file option.
   Imports data to the test panel.
10. Step properties.
    Displays the configurable properties that you use to define the step.
11. Dictionary data.
    Shows the contents of any dictionary that you select in the step.

Merge tab options

Use the Merge tab options to merge multiple input fields into a single output field. You can merge any combination of the fields to suit your data requirements.

The following image shows the options in the Merge tab:



The Merge tab includes the following options:

1. Add Merged Output option.
   Opens the merge configuration options. Use the options to identify the fields to merge.
2. Merged outputs panel.
   Lists the merged fields that the asset will create at run time. Also lists the input fields in each case and the instances to which the inputs belong. You can edit or delete a merged field. When you hover over the merged field, the Edit and Delete options appear.
3. Test input fields.
   Contains input data that you enter to verify the structure of the merged field that you select.
4. Import file option.
   Imports data to the test panel.
5. Test output fields.
   Contains the result of the test.

Working with steps

When you configure a cleanse asset, you define one or more steps for the input fields that an instance identifies. A step describes a standardization operation that a mapping can apply to an input field on a Cleanse transformation. The mapping applies the steps in a sequence that you define in the cleanse asset.

You can configure multiple steps for the input fields in a cleanse asset. The mapping applies the steps to one or more input fields that you select in the Cleanse transformation.

When you add a step to a cleanse asset, you choose from the following options:

Convert Case

Updates the character case of the data in an input field. For example, you can convert a field of mixed-case characters to uppercase characters. You can select or clear an option to ignore uppercase values in the input data.

Remove Values

Searches an input field for one or more values that you specify, and removes the values.

You can enter one or more values that the mapping will remove. Or, you can use a dictionary to identify the values that the mapping will remove.

Remove Spaces

Removes redundant character spaces from an input field.

You can remove leading and trailing spaces, and you can replace multiple spaces with a single space.

Replace Values

Searches an input field for one or more values that you specify, and replaces the values. You can find and replace values in the following ways:

- Enter one or more values that the mapping can search for. Enter a single replacement value for every value that the mapping finds.
- Use a dictionary to identify multiple strings that the mapping can search for. Use the same dictionary to specify a replacement value for each value that the mapping finds.
- Use a dictionary to identify multiple strings that the mapping can search for. Enter a single replacement value for every value that the mapping finds.

Configuring a cleanse asset

Create and configure cleanse assets that you can then use in Cleanse transformations.

1. To create a cleanse asset, click **New** > **Cleanse**.
2. On the **Definition** tab, enter a name for the cleanse asset.
3. Optionally, enter a description.
4. Select the location in which to save the cleanse asset.
5. Optionally, select a data quality dimension to represent the type of data quality issue that you want the asset to examine.

   For more information about dimensions, see "Cleanse assets and dimensions" on page 27.
6. Save the cleanse asset.

   The **Asset References** panel appears. The **Asset References** panel shows additional assets that are referenced by data quality assets to perform transformation tasks. When you create a new asset, there are no referenced assets.
7. Optionally, add a tag to the cleanse asset. You can search for assets with a common tag on the Explore page.
8. Select the Configuration tab.

   The configuration workspace appears for the cleanse asset.

   When you create a cleanse asset, the Instance panel in the **Configuration** view displays a single default cleanse instance. By default, the instance contains a single input field. The input field name is the name of the asset. Optionally, you can add input fields and modify the default input field name and instance name.
9. Optionally, add one or more cleanse instances. For information about how to add a cleanse instance to the asset, see "Adding a cleanse instance to the asset" on page 26.
10. Add a step to each cleanse instance in the asset.

    Select one of the following step types:

    - Convert Case
    - Remove Values
    - Remove Spaces
    - Replace Values
11. Configure the properties on the step to create an operation that the mapping can apply to the data.
12. Optionally, add one or more additional steps to each instance.

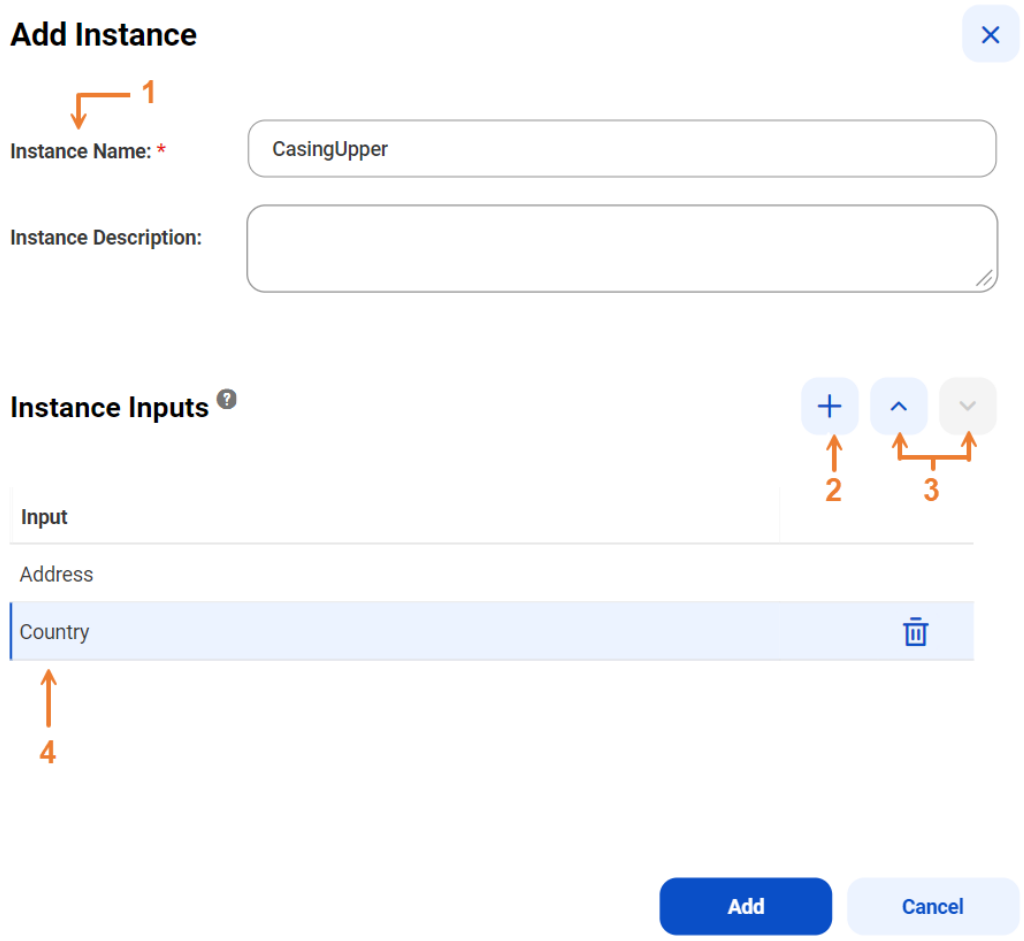
    Configure the properties on each step that you add.
13. Optionally, configure a merge operation for two or more input fields. For information about configuring the fields to merge, see "Merging records" on page 36.
14. Save the asset.

After you configure the cleanse asset, test the asset with sample data.

Cleanse instance configuration options

When you configure a cleanse instance, you add one or more input fields on which to perform cleanse operations.

The following image shows the options on the **Add Instance** dialog box:



The dialog box contains the following options:

1. Instance name.
   Specifies the name of the instance that you add.

   Enter an instance name that represents the type of input data fields that you add to the instance or the type of operation to perform on the input data.
2. Add Input option.
   Adds an input field to the instance. You can add one or more input fields.
3. Up and Down options.
   Moves an input field that you select up or down within the instance input.

4. Input fields.
   Displays the input fields that you add to the instance. You enter the input data field names.

   Choose names for each input that represent the data on the input fields that the Cleanse transformation will read.

   Note: The Maximum string length property applies to the asset inputs and outputs in test operations and at run time. You can set a maximum string length of 10,000 characters. The default value on the property is 255 characters.

Adding a cleanse instance to the asset

You can add one or more instances to a cleanse asset. By default, a cleanse asset contains a single cleanse instance. The Instance panel on the **Configuration** tab displays the cleanse instances.

1. Open a cleanse asset.
2. On the Configuration tab, click the **Add Instance** option present in the Instance panel.

   The **Add Instance** dialog box appears.
3. Enter a name for the instance.
4. Select the option to add the input fields to the instance.
5. Click **Add**.

   The asset adds the cleanse instance to the Instance panel.
6. Open the Instance panel to view the instance that you created.
7. Optionally, edit a cleanse instance to add additional input fields or to modify the input field name or instance name. You can also delete the instance that you added.

Cleanse assets and mappings

The cleanse assets that you create are available to the users who create mappings in Data Integration. You add a Cleanse transformation to a mapping and add a cleanse asset to the transformation. The Cleanse transformation applies the logic in the cleanse asset to one or more input fields that you specify.

A Cleanse transformation works in a similar way to a Mapplet transformation. You connect the transformation inputs and outputs to other objects in the mapping in the same manner as a mapplet connects to other mapping objects. When you run the mapping, the Cleanse transformation applies the cleanse asset logic to the input fields and generates an output field that you can connect to a downstream transformation.

After the mapping runs, you can evaluate the data output to determine if the data meets the standardization objectives.

Rules and guidelines for cleanse assets and mappings

Consider the following rules and guidelines when you work with mappings and cleanse assets:

- A cleanse asset is a reusable object. You can add a cleanse asset to transformations in multiple mappings. A Cleanse transformation is a nonreusable object. You select a Cleanse transformation when you configure a mapping.

- To view the list of cleanse assets that you created, navigate to the **Explore** page.
- If you update a cleanse asset that you previously added to a transformation, you must synchronize the transformation with the cleanse asset.

  To synchronize the transformation, open the transformation in the mapping and select the Cleanse properties. Data Integration displays a message that prompts you to synchronize the transformation with the cleanse asset.

Cleanse assets and dimensions

The data quality issues that you may find in your data can fall into a range of common categories. Data quality assets can identify the categories as dimensions. When you configure an asset, you can use the **Dimension** property to indicate the type of data quality issue that you want the asset to examine.

Find the Dimension property on the **Definition** tab of the asset.

For more information about data quality dimensions, see the *Introduction* help in Data Quality.

CHAPTER 3

Step configuration

Steps define the standardization operations that a mapping applies to input fields and the order in which the operations run. You add one or more steps to each cleanse instance in the asset. Each sequence of steps applies to the fields in the instance that you select.

You can configure steps to perform the following operations:

Update the character case of the data

For example, you might want to convert a set of postal address records to uppercase. The postal carrier in your country might declare a preference for address data in uppercase characters.

Select **Convert Case** as the step type, and select the uppercase option.

Remove instances of a single value from the data

For example, you might want to remove the country identifier from an address data field, so that the country name does not appear on envelopes that you mail to domestic addresses.

Select **Remove Values** as the step type, and select **Remove Custom Strings** as the operation. Next, enter the value to remove.

Remove instances of multiple values from the data

For example, you might want to remove state or province names from a field that also contains the state or province abbreviations.

Select **Remove Values** as the step type, and select **Remove Dictionary Values** as the operation. Next, select a dictionary that contains the state or province names.

Remove character spaces from the data

You can clean up a field that contains extraneous and redundant character spaces.

Select **Remove Spaces** as the step type, and select the options for leading spaces, trailing spaces, and multiple spaces.

Replace instances of a single value in the data

For example, you might want to update a vendor name in your address records.

Select **Replace Values** as the step type, and select **Replace Custom Strings with a New Data Value** as the operation. Next, enter the old vendor name and the new name.

Replace instances of multiple values with corresponding new values

For example, you might want to use abbreviated street descriptors in your address data, such as ST, RD, and AVE. The postal carrier in your country might declare a preference for abbreviated descriptors.

Select **Replace Values** as the step type, and select **Replace Input Values with Dictionary Values** as the operation. Next, select a dictionary that contains the full versions of the street descriptors and the abbreviated versions. Select the column of abbreviations as the valid column.

Replace multiple values in the data with a single value

For example, your address data might contain multiple values that are no longer valid. The values might identify customer coupons that have expired.

Select **Replace Values** as the step type, and select **Replace Dictionary Values with a New Data Value** as the operation. Next, select a dictionary that contains the coupon codes. Additionally, enter a value that the operation can return, such as EXPIRED.

Updating the character case of the input data

Configure a step to change the character case in a data field.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Convert Case** option.
3. In the step properties pane, select one of the following casing styles:
   - Upper Case
   - Lower Case
   - Toggle Case
   - Sentence Case
   - Title Case
4. If you select the Lower Case, Toggle Case, Sentence Case, or Title Case option, specify one or more delimiters for the input data. Optionally, select one or more dictionaries to apply to the input data.

   If you select a dictionary, select a valid column for the dictionary. In addition to converting the character case, the asset replaces any value in a non-valid dictionary column with the corresponding value in the valid column.
5. Optionally, select or clear the option to leave uppercase words unchanged.
6. Save the asset.

Removing instances of one or more values that you specify

Configure a step to remove instances of one or more values that you specify from a data field.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Remove Values** option.
3. In the step properties pane, select **Remove Custom Strings**.
4. Verify the delimiter that the mapping will use to recognize discrete values in the input field. You can specify one or more delimiters.

   The input delimiter determines how the mapping reads the input data when you select the Start or End option as the scope.
5. Set the scope for the search operation.

You can set the following options:

- Anywhere. Removes any instance of the value that occurs in the input field. Can remove multiple instances of the same value in a field.
- Start. Removes any instance of the value that is not preceded by another value in the input field.
- End. Removes any instance of the value that is not followed by another value in the input field.

6. Enter a string to search for and remove in the **Custom Strings** field.

   Note: The search operation is case-sensitive.

7. Add the string to the list of strings to search for.
8. Optionally, add one or more additional strings to the list.
9. Save the asset.

Removing instances of values that appear in a dictionary

Configure a step to remove instances of values that appear in a dictionary. You might use a dictionary when the list of values to remove is too long to enter as custom values. You might also use a dictionary when you will reuse the list of values in multiple cleanse steps.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Remove Values** option.
3. In the step properties pane, select **Remove Dictionary Values**.
4. Select the dictionary to use in the step.

   Click **Browse** to select the dictionary.

5. Select the valid column for the dictionary. The valid column contains the versions of the dictionary values that you designate as the preferred or correct values for the current operation.

   The search operation ignores the values in the valid column. However, if a value in the valid column also appears in another dictionary column, the search operation selects the value for removal.

6. Select or clear the option to perform a case-sensitive comparison between the dictionary data and the input data.
7. Verify the delimiter that the mapping will use to recognize discrete values in the input field. You can specify one or more delimiters.

   The input delimiter determines how the mapping reads the input data when you select the Start or End option as the scope.

8. Set the scope for the search operation.

   You can set the following options:

   - Anywhere. Removes any instance of the value that occurs in the input field. Can remove multiple instances of the same value in a field.
   - Start. Removes any instance of the value that is not preceded by another value in the input field.
   - End. Removes any instance of the value that is not followed by another value in the input field.

9. Save the asset.

Removing character spaces from the data

Configure a step to remove extraneous and redundant character spaces from a data field. The step does not remove instances of single spaces between values in the field.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Remove Spaces** option.
3. Select one or both of the following options:
   - Leading and trailing spaces. Removes one or more character spaces at the start or the end of the string.
   - Multiple spaces. Replaces instances of two or more spaces with a single space.
4. Save the asset.

Replacing instances of one or more values that you specify

Configure a step to replace instances of one or more values that you specify with a single new value.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Replace Values** option.
3. In the step properties pane, select **Replace Custom Strings with a New Data Value**.
4. Verify the delimiter that the mapping will use to recognize discrete values in the input field. You can specify one or more delimiters.

   The input delimiter determines how the mapping reads the input data when you select the Start or End option as the scope.
5. Set the scope for the search operation.

   You can set the following options:
   - Anywhere. Replaces any instance of the value that occurs in the input field. Can replace multiple instances of the same value in a field.
   - Start. Replaces any instance of the value that is not preceded by another value in the input field.
   - End. Replaces any instance of the value that is not followed by another value in the input field.
6. Enter a value to search for and replace in the **Custom Strings** field.

   Note: The search operation is case-sensitive.
7. Add the string to the list of strings to search for.
8. Optionally, add one or more additional strings to the list.
9. Enter the replacement value in the **New Data Value** field.
10. Save the asset.

Replacing instances of dictionary values with a value that you specify

Configure a step to replace instances of values that appear in a dictionary with a value that you enter in the step properties. You might use a dictionary when the list of values to replace is too long to enter as custom values. You might also use a dictionary when you will reuse the list of values in multiple cleanse steps.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Replace Values** option.
3. In the **Step Properties** pane, select **Replace Dictionary Matches with a New Data Value**.
4. Select the dictionary to use in the step.

   Click **Browse** to select the dictionary.
5. Select the valid column for the dictionary. The valid column contains the versions of the dictionary values that you designate as the preferred or correct values for the current operation.

   Note: The search operation ignores the values in the valid column. However, if a value in the valid column also appears in another dictionary column, the search operation selects the value for replacement with the new value.
6. Select or clear the option to perform a case-sensitive comparison between the dictionary data and the input data.
7. Verify the delimiter that the mapping will use to recognize discrete values in the input field. You can specify one or more delimiters.

   The input delimiter determines how the mapping reads the input data when you select the Start or End option as the scope.
8. Set the scope for the search operation.

   You can set the following options:

   - Anywhere. Replaces any instance of the value that occurs in the input field. Can replace multiple instances of the same value in a field.
   - Start. Replaces any instance of the value that is not preceded by another value in the input field.
   - End. Replaces any instance of the value that is not followed by another value in the input field.
9. Enter the replacement value in the **New Data Value** field.
10. Save the asset.

Replacing instances of dictionary values with valid values

Configure a step to search for instances of values that appear in a dictionary and to replace each value with a corresponding valid value from the dictionary. You can use a dictionary when the list of values to replace is too long to enter as custom values. You might also use a dictionary when you will reuse the list of values in multiple cleanse steps.

1. On the **Configuration** tab of the cleanse asset, click **Add Step**.
2. Select the **Replace Values** option.
3. In the step properties pane, select **Replace Input Values with Dictionary Values**.

4. Select the dictionary to use in the step.

   Click **Browse** to select the dictionary.

5. Select the valid column for the dictionary.

   Note: The valid column represents the preferred version of the values in the dictionary. The step searches for values that appear in other columns in the dictionary and replaces them with the corresponding values from the valid column.

6. Select or clear the option to perform a case-sensitive comparison between the dictionary data and the input data.

7. Verify the delimiter that the mapping will use to recognize discrete values in the input field. You can specify one or more delimiters.

   The input delimiter determines how the mapping reads the input data when you select the Start or End option as the scope.

8. Set the scope for the search operation.

   You can set the following options:

   - Anywhere. Replaces any instance of the value that occurs in the input field. Can replace multiple instances of the same value in a field.
   - Start. Replaces any instance of the value that is not preceded by another value in the input field.
   - End. Replaces any instance of the value that is not followed by another value in the input field.

9. Save the asset.

Dictionaries and cleanse assets

A dictionary is a reference data set that a cleanse step can use to evaluate data. Use dictionaries to verify that the data values on a data source or another object in a mapping are accurate and correctly formatted.

You add a dictionary to a cleanse asset, and you add the asset to the Cleanse transformation in a mapping.

At least one dictionary column must contain the set of standard or preferred values for your current data project. The other columns can contain alternative versions of the values. The column that contains the standard or preferred values is called the valid column.

In search and replace operations, a Cleanse transformation searches every column in the dictionary except the valid column that the asset specifies. If you want the search operation to include the valid column data, add a copy of the valid column to the dictionary. By default, the valid column is the first or left-most column in a dictionary. You can create a dictionary that contains two identical columns of data.

A dictionary might contain public terms, such as telephone area codes or address abbreviations. Or, a dictionary might contain values that are specific to an organization, such as employee codes or product codes. You can populate a dictionary with any combination of values that suits your project. The data in each column does not need to be formally correct.

For more information about dictionaries, see the *Data Quality Dictionaries* help.

Rules and guidelines for search and replace operations

When you configure a step to remove or replace values in an input data field, you can enter the values as properties on the step. You can also use a dictionary to specify the values.

The combination of dictionary values and custom values that you select and the order in which you select them can affect the search and replace operations at run time.

Consider the following rules and guidelines when you configure the properties on a step:

Searching in single-value and multi-value strings

- A search operation looks for complete values in an input field. The search operation ignores any value that occurs within a larger string. For example, a search operation will ignore the value DATA in the string METADATA.
- In a step to remove or replace values, the search operation applies the scope criterion that you select to the values in the input field. The search option does not reapply the scope as the operation progresses.

  For example, if you enter the values HARD and DISK as separate search terms in a step and you set the scope to Start, the search operation removes HARD from the string HARD DISK DRIVE. The value DISK remains at the start of the output string.

  If you set the scope to Anywhere in the same step, the search operation will remove both HARD and DISK from the input field.
- You can remove or replace more than one value in a string. For example, you can configure a step to remove the values VIRTUAL PRIVATE from VIRTUAL PRIVATE NETWORK.

Selecting the valid column in a dictionary

- When you configure a step to remove dictionary terms, the search operation ignores the terms in the valid column that you select.

  You can use a dictionary that contains a single column of data to remove values or to replace data values with a custom value. To do so, select any other column number on the Valid Column field as the valid column.
- You can use a dictionary to find values in the input data and to replace the values with values from the valid column in the dictionary. When you use a dictionary to find and replace values, select a dictionary that contains at least two columns.
- You can review the dictionary data in the Step properties pane and select the valid column that you need.

Differences in search criteria for dictionaries and custom strings

- When you configure a step to remove or replace a list of custom strings, the search operation searches for the strings in the order in which you enter them in the Step properties pane.

  For example, let's say that you first enter SITE and then enter WEB SITE as custom strings, and you set the scope to Anywhere. The search operation removes or replaces the value SITE first and cannot subsequently remove the string WEB SITE. The value WEB remains in the output data.

  If you enter WEB SITE before SITE in the same step, the search operation will remove both terms from the input field.
- When you configure a step to remove or replace a list of dictionary values with a new value, the search operation prioritizes longer strings in the dictionary.

  For example, let's say that you select a dictionary that contains the strings SITE and WEB SITE, and you set the scope to Anywhere. The search operation removes or replaces the value WEB SITE and also removes the value SITE.

CHAPTER 4

Merge configuration

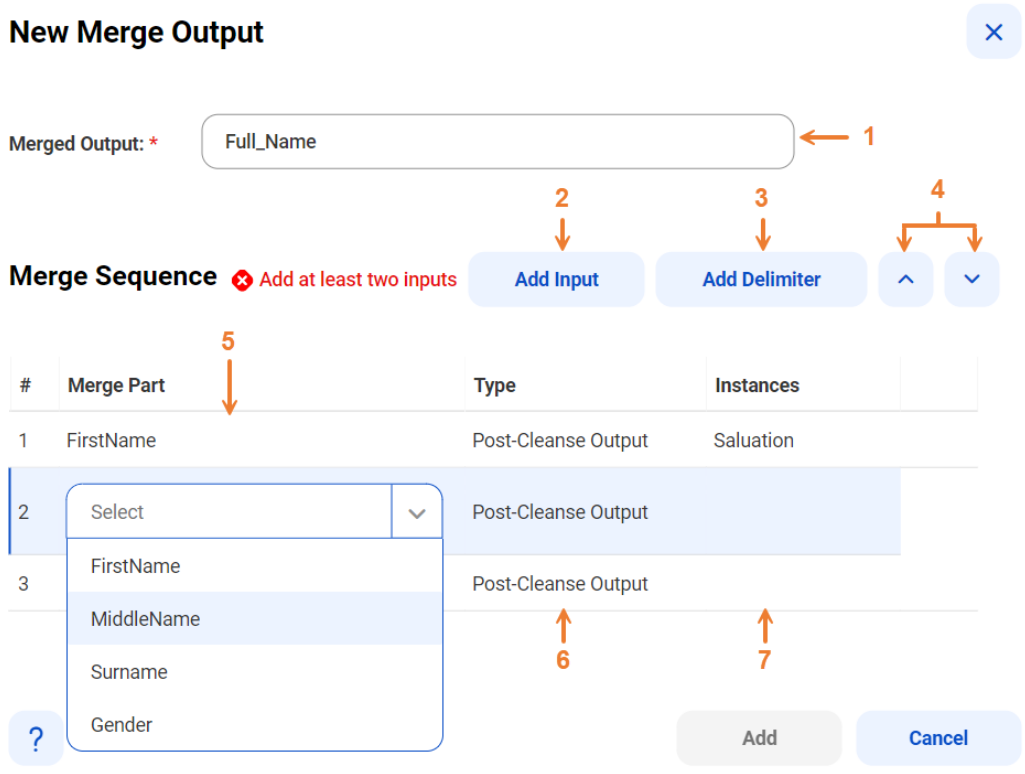
A cleanse asset contains a Merge tab where you can define merge operations for input fields. Use the Merge tab to create a field based on one or more fields across the instances or within an instance.

You configure the Merge tab options to merge the post-cleansed input fields. When you run a test on the Merge tab, the merged output column in the test panel displays the result of the merged record that you select.

Merge configuration options

To merge one or more input data fields into a single field, create a cleanse asset and configure the options in **Merge** tab.

The following image shows the options on the **New Merge Output** dialog box:



The dialog box contains the following options:

1. Merged output field name.
   Specifies the merged field name. You enter a name for the merged output field.
2. Add Input option.
   Adds an input field whose data you want to merge. You can add two or more input data fields.
3. Add Delimiter option.
   Specifies the delimiter to use between the input fields that you merge. The default delimiter is a character space.

   You can select a delimiter from a list of built-in delimiters or add a custom delimiter. Select the delimiters from the **Select Delimiter** dialog box.
4. Up and Down options.
   Moves an input field that you select up or down within the merge sequence.
5. Merged part names.
   Displays the lists of the fields that you want to merge.
6. Type.
   Identifies the type of the input field considered for merging. The operation merges the cleansed version of each input field.
7. Instances.
   Indicates the instance from which you added the input field.

Merging records

You can configure a cleanse asset to merge two or more fields into a single new field.

1. Select the **Merge** tab on the asset.
2. On the **Merge** tab, click **Add Merged Output**.

   The **New Merge Output** dialog box appears.
3. Enter a name for the new field.
4. To select the input fields to merge, click **Add Input**.
5. Optionally, to change the delimiter to apply between the merged fields, click **Add Delimiter**. The default delimiter is a character space.
6. Click **Add**.

   The **Merge** tab displays the new field.

CHAPTER 5

Test cleanse assets

Test a cleanse asset to verify that the asset logic generates the results that you expect.

Testing a cleanse instance

Test a cleanse asset instance to verify that the data flows through the instance in the ways that you expect. You test a single cleanse instance at a time.

1. Open the cleanse asset.
2. Select the **Configuration** tab, and select an instance.
3. Select a step in the step sequence that you configured in the instance.

   The test will run from the first step in the sequence through the step that you select. To test all steps in the instance, select the final step.

   Note: If any step that you select includes a dictionary, the Secure Agent downloads all of the dictionaries that the asset specifies when you run the test.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter data values in the test panel, or import the data to test. To import the data, click the **Import** option in the test panel.

   Note: Configure the asset as completely as you can before you enter or import the test data. The asset may discard your test data if you make further changes to the configuration.

   For more information see "Rules and guidelines for test data" on page 38.
6. Click **Test**.

   The Output column displays the test results. The results represent the application of the step logic in the sequence that the instance specifies.
7. Verify that the steps read the input values and write the output values that you expect.

   You can sort and filter the test data, and you use the Find feature to search for values in the test data.
8. Optionally, repeat the steps to test other instances in the asset.

Rules and guidelines for test data

You can import data to the test panel in the cleanse asset and save the test data in the asset configuration.

Consider the following rules and guidelines when you import or enter test data:

- The import option supports CSV and Microsoft Excel files.
- You can import up to 200 consecutive rows of data from a delimited file. You can specify the row at which the import starts.

  Note: Before you import, check the file for column headings. If the first row in the import file contains column headings, start the import at line 2 or lower.
- The Maximum string length property applies to the data in the test panel in addition to the strings that the asset reads and writes at run time. When you test the asset, import or enter a string on each row that does not exceed the value on the property.

  You can set a maximum string length of 10,000 characters. The default maximum length is 255 characters.
- If you import a CSV file that contains multiple columns or uses a text qualifier, verify that the file uses a delimiter or a text qualifier that the Secure Agent recognizes. By default, the *Comma* option is the delimiter for the column data. By default, the *No quotes* option is the text qualifier for the data. You can update the delimiter and text qualifier characters when you select the data to import. The Delimiter and Text Qualifier options are not required when you import a Microsoft Excel file.
- The Secure Agent saves the data that you import to the asset when you save the asset. If you change an option in the asset configuration, you may lose any unsaved test data.

Testing the merge operation

Test the merge field records that you configured in the **Merge** tab.

1. Select the **Merge** tab on the asset.
2. Select the merged field that you want to test.
3. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
4. Enter data values in the test panel, or import the data to test. To import the data, click the **Import** option in the test panel.
5. Click **Test**.

   The Output column displays the test results. The results represent the application of the step logic that you define in the **Configuration** tab for the input fields.
6. Verify the result of the test.

   You can sort and filter the test data, and you use the Find feature to search for values in the test data.

Part III: Deduplicate assets

This part contains the following chapters:

- Introduction to deduplicate assets, 40
- Test deduplicate assets, 56
- Field types and identity objectives, 59

CHAPTER 6

Introduction to deduplicate assets

A deduplicate asset measures the level of similarity between records in a data set and optionally consolidates similar records into a single, preferred record.

Deduplicate assets focus on *identity* information within the records that they analyze. In a data quality context, an identity is a set of data values in a record that identify a person, an organization, or another uniquely identifiable entity.

In the deduplication process, a Deduplicate transformation generates a set of numerical values that represent the degrees of similarity between the input records. When two or more records match each other with a high score, the transformation adds the records to a set of potential duplicates. You can review and update the score threshold that the transformation uses in the deduplicate asset.

In the consolidation process, the mapping further evaluates the sets of matching records that the duplicate analysis identified. The mapping builds a preferred or optimal version of the records in each set according to the criteria that you specify in the deduplicate asset.

You create and test a deduplicate asset in Data Quality or Data Integration, and then add the asset to a Deduplicate transformation in a mapping in Data Integration. In Data Quality, you create a deduplicate asset from the **New** dialog box. In Data Integration, you create a deduplicate asset either from the **New Asset** dialog box or from a mapping.

When the mapping runs, Data Integration performs the deduplication and consolidation operations that the asset defines.

Types of deduplication project

You might use a deduplicate asset in the following types of data project:

- Customer Relationship Management. For example, a store designs a mail campaign and must check the customer database for duplicate customer records.
- Regulatory compliance initiatives. For example, a business operates under government or industry regulations that insist all data systems are free of duplicate records.
- Financial risk management. For example, a bank may want to search for relationships between account holders.
- Any project that must identify or eliminate records that store duplicate identity information.

Duplicate analysis operations

You define duplicate analysis operations in a deduplicate asset and in the Deduplicate transformation that reads the asset.

At a high level, you complete the following steps:

Deduplicate asset steps

Select the type of identity information that the transformation will analyze.

Configure the search criteria that the transformation will apply to the input data.

Deduplicate transformation steps

Add the deduplicate asset to the transformation.

Select the fields that contain the relevant identity data.

Select a field on which the transformation can sort the input records at run time.

The deduplicate asset provides a list of identity types that you must choose from. Each identity is optimized for different types of information. When you configure the Deduplicate transformation, you map the identity fields that the asset specifies to the input fields on the transformation.

Additionally, you configure the Deduplicate transformation to sort the input records into groups based on the values in a field that you select. In duplicate analysis, a group is a set of records that contain identical values in a given field. At run time, the Deduplicate transformation analyzes records exclusively within each group and combines the results from each group into a single output data set. When you create groups on an appropriate field, you reduce the overall number of comparisons that the Deduplicate transformation must perform without any meaningful loss of accuracy in the duplicate analysis.

The **GroupKey** field in the Deduplicate transformation identifies the field on which the transformation sorts the records. For more information on groups in duplicate analysis, see the *Transformations* help in the Data Integration.

Note: To analyze all of your input records in a single group, create the group from a field that contains the same value for all records.

Rules and guidelines for duplicate analysis operations

When you configure the deduplicate analysis operations, consider the following rules and guidelines:

- The Deduplicate transformation compares field data from every input record with data from every other record in its group. The transformation begins with the first record in the group, or the record with the lowest sequence ID value.

  The transformation creates discrete sets of similar records called **clusters**. If a record matches any other record in the group, the transformations adds the records to the same cluster. If a record matches another record that is already part of a cluster, the transformation adds the record to the cluster. If a record does not match any other record in its group, the record becomes the sole member of its cluster.

  When the group analysis is complete, the Deduplicate transformation collates the records from each group into a single output data set.

- To determine the similarity between the identities in a set of records, the Deduplicate transformation creates an index from the values on the input fields that are relevant to the identity. The index contains raw values from the input fields and a range of alternative versions of the raw values.

  You select the identity type as the **Objective** in the deduplicate asset. You specify the input field that contains the principal data for the identity as the **Index Key** in the asset.

- The Deduplicate transformation calculates a numerical score for each pair of records in a group that represents the degree of similarity between the records. A pair of identical records will return a score of 1.00.

  The transformation uses a threshold value to identify records that match each other. The default value is 0.9, or 90 percent. If two records match each other with a score that exceeds the threshold, the transformation identifies the records as duplicates.

  You can optionally update additional asset properties that modify the speed, performance, and depth of the duplicate analysis.
- The final output from the Deduplicate transformation depends on the output properties that you specify in the deduplicate asset. By default, the asset instructs the transformation to retain every pair of matching records in the clusters that it creates. Alternatively, the asset can instruct the transformation to retain only the records that are the closest matches with each other. You can set the output type as a property on the asset.

Deduplicate asset structure

A Deduplicate asset contains options on a **Definition** tab, a **Deduplication** tab, and a **Consolidation** tab.

Use the **Definition** tab options to enter a name for the asset, optionally enter a description for the asset, and select the folder in which to store the asset. Use the **Deduplication** tab options to configure the type of identity analysis that a mapping will perform. Optionally, use the **Consolidation** options to configure the strategy that the mapping will apply to the discrete sets of duplicate records that arise from the identity analysis.

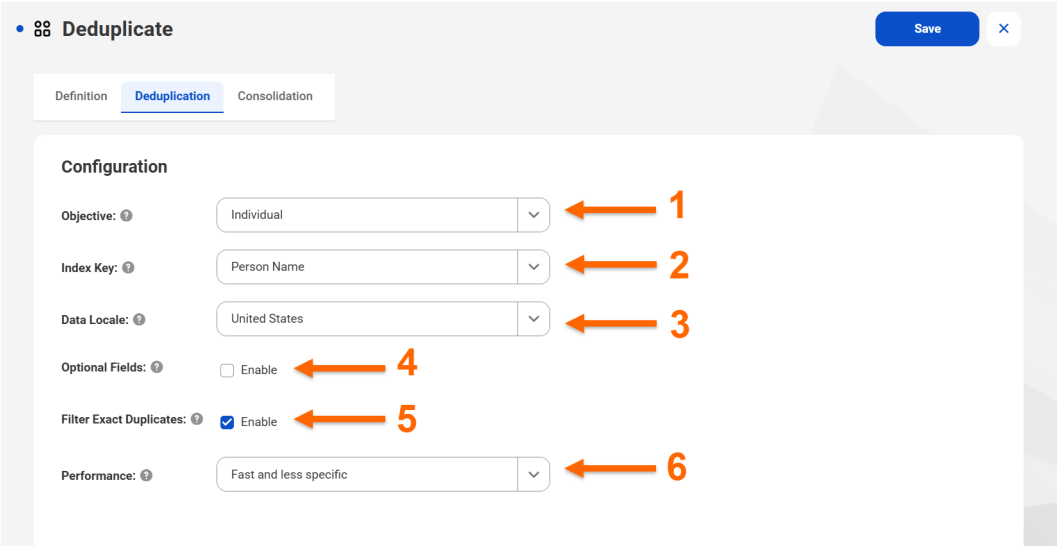
Deduplication tab options

Use the options on the **Deduplication** tab to configure the type of duplicate analysis that a mapping will perform. The **Deduplication** tab includes the **Configuration**, **Advanced Options** and the **Test Data** panels.

Configuration panel

Use the **Configuration** panel to configure the type of duplicate analysis that a mapping will perform.

The following image shows the **Configuration** panel on the **Deduplication** tab:



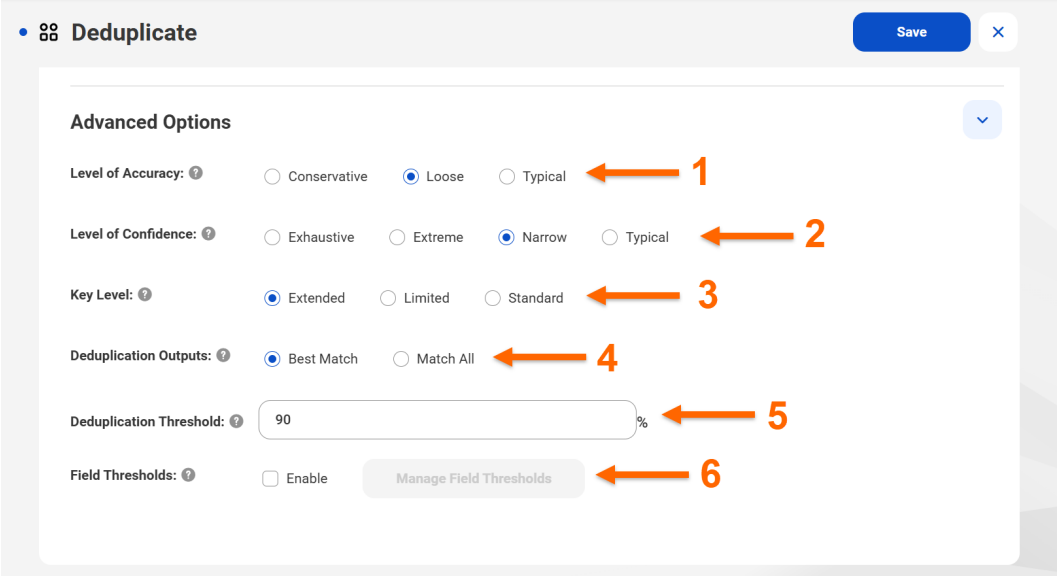
The **Configuration** panel includes the following options:

1. Objective. Identifies the type of information that the asset will prioritize when it compares the input records to each other. For more information about the objectives that you can configure, see "Deduplication objectives" on page 48.
2. Index Key. Identifies the type of information that the asset will use to create an index of the input records.
3. Data Locale. Identifies the county or region for which the Deduplicate transformation loads identity population reference data.
4. Optional Fields. Enables the deduplicate analysis to consider additional fields at run time. This field is disabled by default.
5. Filter Exact Duplicates. Determines whether the transformation passes identical records directly to the configuration stage without analysis. This field is disabled by default.
6. Performance. Specifies the speed and granularity of duplicate analysis. Fine-grained analysis takes longer to perform. To enable advanced options, select **Custom**.

Advanced Options panel

Use the **Advanced Options** panel to review or update the performance criteria that the Deduplicate transformation defines for duplicate analysis at run time.

The following image shows the **Advanced Options** panel on the **Deduplication** tab:



1. Level of Accuracy. Determines the degree of similarity that must exist between two identities before the deduplication process considers them to be good matches with each other.

   You can select one of the following accuracy levels:

   - **Conservative**. Requires a very high degree of similarity.
   - **Loose**. Allows for a wider degree of latitude when evaluating the similarity between identities.
   - **Typical**. Requires a reasonable degree of similarity.

2. Level of Confidence. Determines the extent of the differences between data values that the deduplication process will tolerate. The level of confidence represents the level of error tolerance that the deduplication process supports.

   You can select one of the following confidence levels:

   - **Exhaustive**. Configures the process to find a broader range of errors than the typical analysis. Mappings with exhaustive confidence may take longer to run.
   - **Extreme**. Configures the process to use every possibility to find a candidate match. Select the extreme level when you have a critical need to find every possible duplicate in the input data. Extreme confidence extends the time that the mapping takes to run.
   - **Narrow**. Configures the process to find a narrow range of errors. Narrow confidence prioritizes mapping speed and may miss some duplicate identities.
   - **Typical**. Configures the process to find common errors and variations in data values. The typical confidence level provides a practical balance between quality and mapping execution time.

3. Key Level. Determines the complexity of the index keys that the deduplication process creates.

   You can select one of the following key levels:

   - **Extended**. Improves match reliability by finding matches regardless of word order or concatenation. Extended keys increase disk space requirements and result in larger sets of matching candidates.
   - **Standard**. Addresses most variations in word order, missing words, and extra words. They also maximize the likelihood of finding candidate matches in cases of severe spelling errors in multi-word names.

- **Limited**. A subset of standard keys. Limited keys reduce the use of disk space but may also reduce the reliability of identity search operations.

4. Deduplication Outputs. Determines the composition of the clusters of matching records that the transformation creates at run time.

   You can select one of the following deduplication outputs:

   - **Best Match**. Creates a cluster that contains only the best match for each record in the cluster.
   - **Match All**. Creates a cluster that contains all records in the input data that match each record in the cluster.

5. Deduplication Threshold. Specifies the minimum score between two records that identify the records as plausible duplicates of each other. You can enter a threshold value from 0 through 100.

6. Field Thresholds. Enables custom deduplication thresholds for input fields.

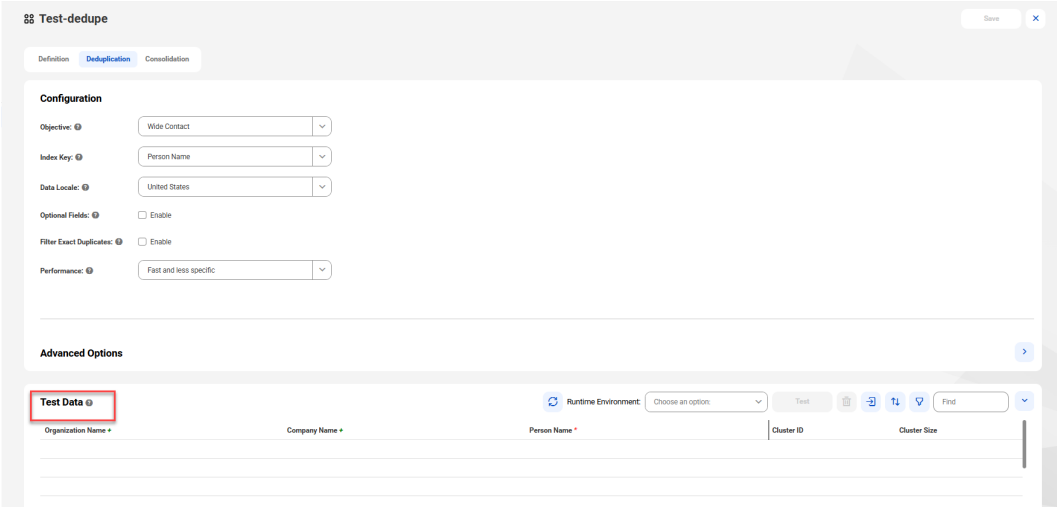
   You can use the field-level threshold options to change the emphasis on individual fields within a deduplication process and deliver results that are fine-tuned to your current deduplication project.

   To set the thresholds, click **Manage Field Thresholds**. In the **Manage Field Thresholdes** dialog box, you can configure a unique threshold value for each input field. You can enter a threshold value from 0 through 100.

Test Data panel

Use the **Test Data** panel to see the fields for which the asset expects data, based on the objective that you select.

The following image shows the **Test Data** panel:



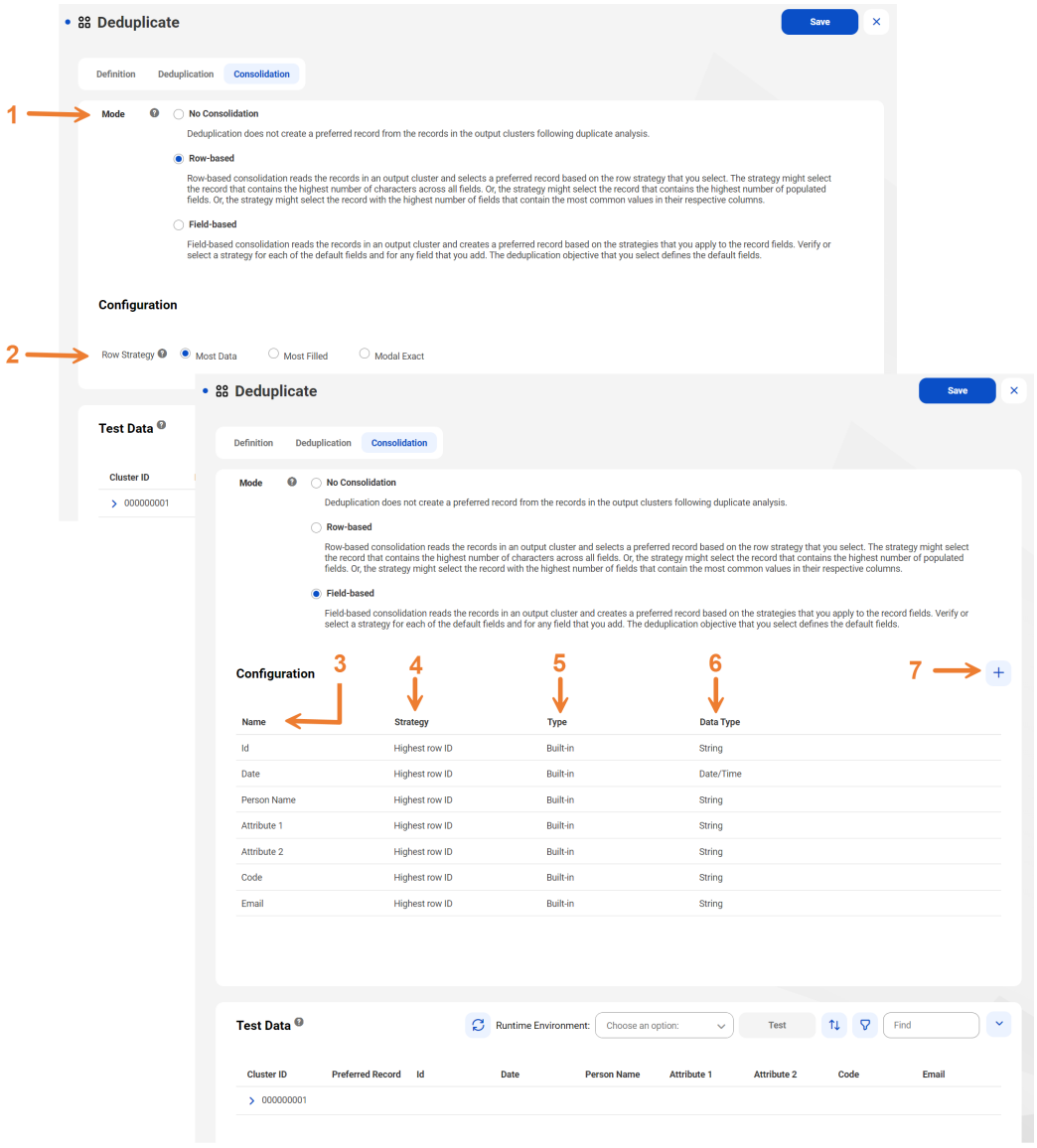
Each field appears as a column name. To test the similarity between data records, enter two or more rows of data. Populate at a minimum each mandatory field and one or more of any required fields that the asset specifies for the current objective.

The panel also includes a Runtime Environment option and options to search, sort, and filter the test data. Use the Runtime Environment option to specify a Secure Agent.

Consolidation tab

Use the options on the **Consolidation** tab to configure the type of consolidation that a mapping will perform.

The following image shows the **Consolidation** tab options:



The **Consolidation** tab includes the following options:

1. Consolidation mode.

   Identifies the type of consolidation that the Deduplicate transformation will perform when the mapping runs. The type that you select determines how the transformation selects the preferred record in each set of duplicate records.

   Choose the row-based option to select a preferred record based on the quantity of data in the identity fields. Choose the field-based to build a preferred record from the data values across one or more records. You can also choose not to consolidate the duplicate record sets.

2. Row strategy.

Determines how the transformation will select the preferred record when you choose the row-based consolidation mode.

Choose Most Data to specify the record with the greatest number of characters as the preferred record. Choose Most Filled to specify the row with the highest number of populated fields. Choose Modal Exact to select the record with the highest number of fields that contain the most common values in their respective columns.

3. Field name column.

   Lists the fields in the input records that the Deduplicate transformation will read. The field name column is visible when you select the field-based consolidation mode. You can specify a consolidation strategy for each field when you select field-based consolidation.

4. Strategy.

   Determines how the transformation selects the value in each field for the preferred record when you choose the field-based consolidation mode.

   You can select one of the following strategies:

   - Highest row ID. Use the value from the record with the highest row ID or sequence ID. Highest row ID is the default strategy.
   - Average. Use the average value across the records.
   - Longest. Use the longest value in the field across the records.
   - Maximum. Use the highest number in the field across the records. Or, choose the last value in alphabetical order.
   - Minimum. Use the lowest number in the field across the records. Or, choose the first value in alphabetical order.
   - Most frequent. Use the most frequently-occurring value in the field across the records, including blank, empty, or zero-length string fields.

     Note: The consolidation operation will not add a null value to the preferred record.
   - Most frequent non-blank. Use the most frequently-occurring value in the field across the records, excluding null, blank, empty, or zero-length string fields.
   - Shortest. Use the shortest value in the field across the records.

5. Type

   Indicates whether the asset created the field during the deduplication operation or whether you added the field to the asset in the Consolidation pane.

6. Data Type

   Identifies the data type of the field. The default data type on all fields is String. You can modify the data type in field-based consolidation to suit your data requirements.

   You can select one of the following data types for a field:

   - Date/Time
   - Float
   - Integer
   - String

   Note: If you modify the data type of a field, do not change the mode to No Consolidation or Row-based consolidation without first saving the asset. The asset discards any update that you make to a data type in Field-based consolidation mode if you change to another mode.

7. Add field button.

Adds one or more inputs to the consolidation operation when you select the field-based mode.

Add fields in the following cases:

- The identity analysis that you define on the **Deduplication** tab does not cover all of the fields that the transformation will analyze.
- You want to specify a non-default strategy for the additional fields.

The field-based strategies will apply to every input field that you map to the deduplicate asset in the transformation. If you do not specify a strategy for a field in field-based mode, the transformation applies the default strategy.

Deduplication objectives

The **Objective** option on the **Deduplication** tab defines the type of identity that the Deduplicate transformation analyzes when you run a mapping with the transformation. The objective also indicates the types of information that the transformation expects to read for the identity. When you configure the transformation, you map the input fields that contain the identity information to the most appropriate fields on the deduplicate asset.

Each objective supports one or more index key fields. You select an index key on the **Deduplication** tab in the asset. Some objectives define similar identity types, for example Family and Household. In each case, the deduplication process uses unique comparison logic.

The set of potential input fields on an objective contains at least one mandatory field and may also identify one or more required fields. To get the most accurate results from the deduplication operations, map each mandatory field on the asset to an appropriate input field on the transformation. Likewise, map at least one required field on the asset to an appropriate input field on the transformation. The mandatory and required input field names indicate the types of information that the input fields must contain.

Each objective also supports a set of input fields that can contain additional data about the identity. Optionally, map any additional asset field to a transformation field that contains the appropriate information. To optimize the analysis of the identity data, ensure that the Deduplicate transformation reads as many of the fields as possible.

For more information about mandatory and required fields on each objective, see Chapter 8, “Field types and identity objectives” on page 59.

The following table describes the objectives that you can select and identifies the fields that you can select as the index key field:

Identity objective	Description	Index Keys
Address	Identifies records that share an address.	Address Part 1 Code Date Exact Geocode Telephone Number
Address Code	Identifies records that share an address code.	Address Part 1

Identity objective	Description	Index Keys
Author ISBN	Identifies records that share information about an author who published work with an ISBN number.	Email Exact Person Name ISBN10 ISBN13
CC Issuer	Identifies records that share information about a credit card issuer.	Company Name Credit Card Exact Organization Name
CC Owner	Identifies records that share information about a credit card holder.	Address Part 1 Credit Card Date Email Exact Geocode Person Name Telephone Number
Contact	Identifies records that share a contact at a single organization and location.	Address Part 1 Code Company Name Date Email Exact Geocode Person Name Organization Name Telephone Number
Corp Entity	Identifies records that share corporate identification data.	Address Part 1 Code Company Name Date Exact Geocode Organization Name Telephone Number
Division	Identifies records that share an office location within an organization.	Address Part 1 Code Company Name Exact Geocode Organization Name Telephone Number

Identity objective	Description	Index Keys
Family	Identifies individuals that belong to the same family.	Address Part 1 Code Email Exact Geocode Person Name Telephone Number
Fields	Identifies records that share identity data across multiple fields that you select.	Address Part 1 Code Company Name Credit Card Date Email Exact Geocode ISBN10 ISBN13 Organization Name Person Name Product Description Product Name Telephone Number VIN [Generic field]
Generic	Identifies records that share identity information when different types of identity are represented in a single field. Select the objective for records that contain Australia, United Kingdom, or United States data. **Note:** The Generic field that you select might contain person names, organization names, or addresses. The objective looks for common values across these information types in the field.	[Generic field]
Geocode	Identifies records that share geocode data.	Geocode
Household	Identifies individuals that belong to the same household.	Address Part 1 Code Email Exact Geocode Person Name Telephone Number
Individual	Identifies duplicate individuals.	Date Code Email Exact Person Name

Identity objective	Description	Index Keys
Organization	Identifies records that share organization data.	Address Part 1 Code Company Name Date Exact Geocode Organization Name Telephone Number
Person Name	Identifies records that share information about a person.	Address Part 1 Code Date Email Exact Geocode Person Name Telephone Number
Product	Identifies records that share information about a product.	Code Company Name Exact Organization Name Product Description Product Name
Publisher ISBN	Identifies records that share information about a publishing company. The information includes ISBN data for published works.	Address Part 1 Company Name Exact Geocode ISBN10 ISBN13 Organization Name
Resident	Identifies duplicate individuals at the same address.	Address Part 1 Code Date Email Exact Geocode Person Name Telephone Number

Identity objective	Description	Index Keys
VIN Manufacturer	Identifies records that share information about a vehicle manufacturer.	Address Part 1 Company Name Exact Geocode Organization Name VIN
VIN Owner	Identifies records that share information about a vehicle owner.	Address Part 1 Code Company Name Date Email Exact Geocode Organization Name Person Name VIN
Wide Contact	Identifies records that share a contact at an organization.	Code Company Name Email Exact Organization Name Person Name
Wide Household	Identifies individuals that belong the same household.	Address Part 1 Code Email Exact Geocode Person Name Telephone Number

Identity reference data

The duplicate analysis process use reference data files called population files to evaluate the identity information in the input data. When you run a mapping with a Deduplicate transformation, the transformation compares the input data to the population file for the country in which the input data originates.

Data Quality downloads the population files to the Secure Agent host machine when you install the Secure Agent. You do not need to take any action to download the files.

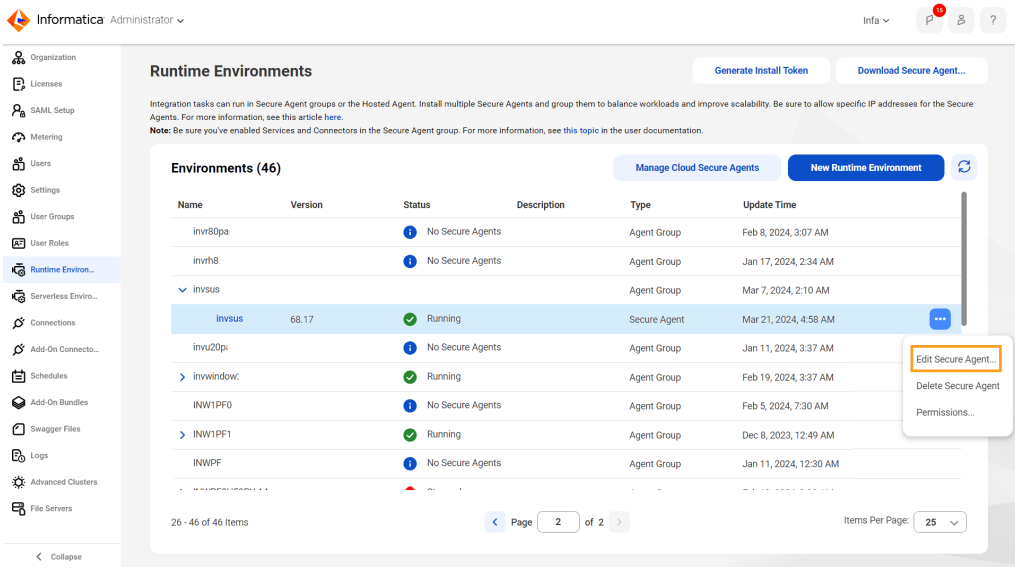
You can review the location of the population files in the Administrator.

Reviewing the Administrator properties for identity population data

You can review and configure the directory to which the Secure Agent downloads identity population files in the Administrator. You can also review and configure the directories that the deduplication process uses for index and cache data.

1. From the **My Services** page, select the **Administrator** service.
2. Choose the **Runtime Environments** option.
3. Select the **Secure Agent** that you will use to run mappings with the Deduplicate transformation.
4. Hover over the **Actions** icon for the Secure Agent, and select the **Edit Secure Agent** option.

   The following image shows the option:



   The **Secure Agent** page appears.
5. Under **System Configuration Details**, select **Data Integration Server** in the **Service** field and select **IDQ** in the **Type** field.

   The **System Configuration Details** pane returns a list of properties based on the type that you specified.
6. Review the following properties:
   - IdentityReferenceDataLocation. Identifies the location of the population files.
   - IdentityReferenceIndexLocation. Identifies the location of the index files that the deduplication process creates at run time.
   - IdentityCacheLocation. Identifies the location of the temporary files that the deduplication process creates at run time.

   By default, the properties identify directories in the Informatica installation.
7. Optionally, update the property values to suit your system and the mappings that you will run.

Configuring the deduplication process

To define duplicate analysis operations in the deduplicate asset, configure the options on the **Deduplication** tab.

1. Select the **Deduplication** tab on the asset.
2. Select an objective. The objective represents the type of identity that the Deduplicate transformation will look for during duplicate analysis.

   Tip: The objective also indicates the set of inputs that the transformation expects to read at run time. You can preview the fields in the Test Data panel.
3. Select an index key.

   The index key represents the field on which the transformation will build identity data index. The objective that you select determines the set of keys from which you can select the index key.
4. Select the data locale in which the data set originates.

   The duplicate analysis process reads identity reference data for the locale that you select.
5. Select or clear the option to define optional fields for the objective.

   Select the option if your source data contains one or more columns of relevant data that the objective does not specify. For example, your source data might contain a discrete field for corporate suffixes.
6. Select or clear the option to filter exact duplicates.

   When you select the option, the transformation passes records that are duplicates of each other directly to the consolidation stage or to the downstream objects in the mapping.

   You might select the option when the input data contains many identical rows.

   Note: The output from the analysis contains the same records whether you select or clear the option. The Deduplicate transformation might assign different scores to the output records when you select and clear the option.
7. Select a level of performance for the duplicate analysis.

   The performance level describes the relationship between the speed and the granularity of the analysis. Faster analysis is less granular and might miss some duplicate records.
8. Optionally, review or update the criteria that apply for a performance option.

   To review the criteria, select the option and expand the **Advanced Options**.

   To customize the criteria, select the **Custom** option as the performance level. For example, you might decide to update the threshold score value.
9. Save the asset.

After you configure the deduplication process, you can optionally configure a consolidation process for the duplicate records that deduplication identifies.
For more information about the deduplication options, see "Deduplication tab options" on page 42.

Configuring the consolidation process

To define a consolidation process, configure the options on the **Consolidation** tab.

1. Select the **Consolidation** tab on the asset.

2. Select a consolidation mode. The mode defines the method that the Deduplicate transformation uses to consolidate sets of duplicate records into single, preferred records.

   You can select one of the following modes:

   - No consolidation. The transformation does not create preferred records from the sets of matching records that the duplicate analysis defines.
   - Row-based consolidation. The transformation selects a preferred record from each set of matching records based on a strategy that you select.
   - Field-based consolidation. The transformation creates a preferred record based on a strategy that you specify for each field in the records. Field-based consolidation can combine values from more than one record to create a preferred record that is not present in the source data.

3. Configure the row-based or field-based strategies for the fields in the input records.
4. If you selected the field-based consolidation mode, optionally add one or more fields to the consolidation model.

   Note: The following field names are reserved on the deduplicate asset:

   - ClusterId
   - ClusterSize
   - DriverId
   - DriverScore
   - GroupKey
   - LinkId
   - LinkScore
   - RowId
   - SequenceId

   Do not create a field with a reserved name.

5. Save the transformation.

For more information about the consolidation options, see "Consolidation tab" on page 46.

CHAPTER 7

Test deduplicate assets

Test a deduplicate asset to verify that the asset logic generates the results that you expect.

To test an asset, you must have access to an active Secure Agent. Use the Runtime Environment option to specify the Secure Agent.

Testing a deduplicate asset

Test a deduplicate asset to verify that the data flows through the asset in the ways that you expect.

1. Open the deduplicate asset.
2. Select the **Deduplication** tab.
3. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
4. Enter data values in the test panel, or import the data to test. To import the data, click the **Import** option in the test panel.

   Consider the following guidelines before you add or import data to the test panel:

   - Configure the asset as completely as you can before you enter or import the test data. The asset may discard your test data if you make further changes to the configuration.
   - Verify that the test data structure matches the column structure in the test panel. Provide test data for each mandatory field in the test panel. If the test panel includes any required fields, provide test data for at least one required field.
   - If the objective accepts a date value, you can use the calendar and clock options to add a date and time to an input row. The calendar and clock options are synchronized. You can update the date or time after you add either value.

   For more information see “Rules and guidelines for test data” on page 58.
5. Optionally, save the asset to preserve the current test data and configuration.
6. Click **Test**.

   You can sort, search, and filter the test results in the following ways:

   - Click the Up and Down arrows to select a field on which to sort the test data. To reverse the sort order, select the field a second time. open a menu of the sorting categories. The categories reflect the configuration of the asset. The test results refresh to show the values for the category that you choose.

- Click the Filter icon to add a filter option to the test panel. Select a field on which to filter the data, and add a data filter for the field. The test results refresh to show any row that contains the filter value in the field that you specify.
- You can also enter a value in the Find field to search the test results.

The sort, search, and filter options work together. For example, if you apply a filter to the test data and you enter a value in the Find field, the asset displays any row that meets both the filter and search criteria.

7. Verify that the deduplication process analyzes the test data in the manner that you expect.
8. Optionally, select the **Consolidation** tab.

   Use the options on the tab to verify that the consolidation process generates preferred records in the manner that you expect.

   The test panel on the **Consolidation** tab contains the results of any test that you ran on the **Deduplication** tab.

   Note: If you add one or more fields to the Consolidation tab, bear in mind that several field names are reserved on the deduplicate asset. To read the list of reserved field names, see "Configuring the consolidation process" on page 54.
9. Click **Test**, and review the results of the test. You can test the consolidation options in row-based mode and field-based mode.

Understanding the test results

When you run a test on the **Deduplication** or **Consolidation** tab, the test results include a number of predefined fields. The input data and the test results on the Deduplication tab form the basis of the test that you can perform on the Consolidation tab.

The test results on the Deduplication tab include the following predefined fields:

Cluster ID

Contains the identifier of the cluster to which the input record belongs.

In the deduplication process, a cluster is a set of records whose data values match each other to a degree that exceeds the duplicate threshold. Records in the same set are likely to identify the same identity. A set may contain a single record, as every unique record is a perfect match with itself.

Cluster Size

Contains the number of records in the set to which the current record belongs. When a set contains a unique record, the cluster size is 1.

The test results on the Consolidation tab include the following predefined fields:

Cluster ID

Contains the identifier of the cluster to which the input record belongs. The Cluster ID fields on the Deduplication and Consolidation tabs contain identical information for a given test.

Preferred Record

Contains the values in the preferred record that the test creates for the current input.

The Deduplication and Consolidation tabs also display the mandatory and required fields that apply for the objective and index key that you select. In addition, the Consolidation tab displays all of the fields that the objective can use and any custom field that you add.

Rules and guidelines for test data

You can import data to the test panel in the deduplicate asset and save the test data in the asset configuration.

Consider the following rules and guidelines when you add data to the test panel:

- The import option supports CSV and Microsoft Excel files.
- You can import up to 200 consecutive rows of data from a delimited file. You can specify the row at which the import starts.

  Note: Before you import, check the file for column headings. If the first row in the import file contains column headings, start the import at line 2 or lower.
- You can import or enter an input string of up to 255 characters.
- If you import a CSV file that contains multiple columns or uses a text qualifier, verify that the file uses a delimiter or a text qualifier that the Secure Agent recognizes. By default, the *Comma* option is the delimiter for the column data. By default, the *No quotes* option is the text qualifier for the data. You can update the delimiter and text qualifier characters when you select the data to import. The Delimiter and Text Qualifier options are not required when you import a Microsoft Excel file.
- The Secure Agent saves the data that you import to the asset when you save the asset. If you change an option in the asset configuration, you may lose any unsaved test data.

The following rules and guidelines apply to the deduplicate asset:

- The test panel structure can change based on the objective and the index key that you select. The structure of the test data that you import must match the structure in the test panel.
  The test panel can include two types of input field:

  - Mandatory: Populate all mandatory fields with test data.

  - Required: Populate at least one required field with test data.

  To read a list of the mandatory and required fields on each objective, see Chapter 8, “Field types and identity objectives” on page 59.

CHAPTER 8

Field types and identity objectives

Each objective that you select on the Deduplication tab contributes data from one or more input fields to the process of identity analysis.

An objective can contribute data from one or more of the following field types:

Mandatory field

A field that contains essential information for the type of analysis that the objective specifies. Map every mandatory field to an input on the Deduplicate transformation.

Required fields

A field that contains high-priority information for the type of analysis that the objective specifies. Map at least one required field to an input on the Deduplicate transformation.

Optional fields

Any field that contains additional information about the identity and that might be useful during analysis. Optionally, map the fields to inputs on the Deduplicate transformation.

When you select an objective on the **Deduplication** tab, the test pane displays the mandatory and required fields on the objective. The test pane displays an asterisk (*) beside any mandatory field. The test pane displays a plus symbol (+) beside any required field.

An objective can specify any combination of one or more mandatory, required, or optional fields. The combination of fields can depend on the index key that you select for the objective. The test panel on the Deduplication tab displays the current mandatory and required fields for the objective and index key that you select. The index key is always a mandatory field for an objective.

Tip: If you browse the objectives, the index keys for the objectives can change. Before you save and close a deduplicate asset, verify that the Deduplication tab displays the index key that you intend for the objective that you select.

For more information about the fields that you can choose in each objective, see "Mandatory, required, and optional fields" on page 59.

Mandatory, required, and optional fields

Each objective on the **Deduplication** tab specifies a set of fields that the deduplication process can analyze at run time. To get the best results from the deduplication operation, provide input data for every mandatory

field that the objective specifies and provide data for at least one required field. You can also provide data for the other fields that the objective specifies.

Note: The mandatory and required fields can vary according to the index key that you select for the objective. The current index key on any objective is always a current mandatory field. For example, the deduplicate asset specifies *Organization Name* as a single index key option for the *CC Issuer* objective, which means that *Organization Name* is mandatory in all cases for *CC Issuer*.

The following tables list the default sets of mandatory, required, and optional fields on each objective:

Address

Field	Field Type
Address Part1	Mandatory
Code	Optional
Date	Optional
Exact	Optional
Geocode	Optional
Telephone Number	Optional

Address Code

Field	Field Type
Address Part1	Mandatory

Author ISBN

Field	Field Type
Email	Optional
Exact	Optional
Person Name	Mandatory
ISBN10	Required, unless you specify ISBN13
ISBN13	Required, unless you specify ISBN10

CC Issuer

Note: As the Organization Name is the single index key option on the CC Issuer objective, Organization Name becomes a mandatory key in addition to Credit Card.

Field	Field Type
Company Name	Required, unless you specify Organization Name
Credit Card	Mandatory
Exact	Optional
Organization Name	Required, unless you specify Company Name

CC Owner

Field	Field Type
Address Part1	Optional
CreditCard	Mandatory
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Mandatory
Telephone Number	Optional

Contact

Field	Field type
Address Part1	Mandatory
Code	Optional
Company Name	Required, unless you specify Organization Name
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional

Field	Field type
Person Name	Mandatory
Organization Name	Required, unless you specify Company Name
Telephone Number	Optional

Corp Entity

Note: As Organization Name is the single required field on the Corp Entity objective, Organization Name becomes a mandatory field.

Field	Field type
Address Part1	Optional
Code	Optional
Company Name	Required, unless you specify Organization Name
Date	Optional
Exact	Optional
Geocode	Optional
Organization Name	Required, unless you specify Company Name
Telephone Number	Optional

Division

Field	Field type
Address Part1	Mandatory
Code	Optional
Company Name	Required, unless you specify Organization Name
Exact	Optional
Geocode	Optional
Organization Name	Required, unless you specify Company Name
Telephone Number	Optional

Family

Field	Field type
Address Part1	Mandatory
Code	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Mandatory
Telephone Number	Mandatory

Fields

Note: In the core field list for the Fields objective, all fields are optional. The deduplicate asset identifies the current index key as a manadatory field.

Field	Field type
Address Part1	Optional
Code	Optional
Company Name	Optional
Credit Card	Optional
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional
ISBN10	Optional
ISBN13	Optional
Organization Name	Optional
Person Name	Optional
Product Description	Optional
Product Name	Optional
Telephone Number	Optional

Field	Field type
VIN	Optional
Generic Field	Optional

Generic

Field	Field Type
Generic Field	Mandatory

Geocode

Field	Field Type
Geocode	Mandatory

Household

Field	Field type
Address Part1	Mandatory
Code	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Mandatory
Telephone Number	Optional

Individual

Field	Field type
Date	Required, unless you specify ID
Code	Optional
Email	Optional
Exact	Optional
Person Name	Mandatory

Organization

Field	Field type
Address Part1	Optional
Code	Optional
Company Name	Required, unless you specify Organization Name
Date	Optional
Exact	Optional
Geocode	Optional
Organization Name	Required, unless you specify Company Name
Telephone Number	Optional

Person Name

Field	Field type
Address Part1	Optional
Code	Optional
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Mandatory
Telephone Number	Optional

Product

Field	Field type
Code	Optional
Company Name	Optional
Exact	Optional
Organization Name	Optional

Field	Field type
Product Description	Required
Product Name	Required

Publisher ISBN

Field	Field Type
Address Part1	Optional
Company Name	Required, unless you specify Organization Name
Exact	Optional
Geocode	Optional
ISBN10	Required, unless you specify ISBN13
ISBN13	Required, unless you specify ISBN10
Organization Name	Required, unless you specify Company Name

Resident

Field	Field type
Address Part1	Mandatory
Code	Optional
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Mandatory
Telephone Number	Optional

VIN Manufacturer

Field	Field Type
Address Part1	Optional
Company Name	Required, unless you specify Organization Name

Field	Field Type
Exact	Optional
Geocode	Optional
Organization Name	Required, unless you specify Company Name
VIN	Mandatory

VIN Owner

Field	Field Type
Address Part1	Mandatory
Code	Optional
Company Name	Required, unless you specify Organization Name
Date	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Organization Name	Required, unless you specify Company Name
Person Name	Optional
VIN	Mandatory

Wide Contact

Field	Field type
Code	Optional
Company Name	Required, unless you specify Organization Name
Email	Optional
Exact	Optional
Organization Name	Required, unless you specify Company Name
Person Name	Mandatory

Wide Household

Field	Field type
Address Part1	Mandatory
Code	Optional
Email	Optional
Exact	Optional
Geocode	Optional
Person Name	Optional
Telephone Number	Mandatory

Part IV: Labeler assets

This part contains the following chapters:

- Introduction to labeler assets, 70
- Token labeling operations, 77
- Character labeling operations , 83
- Test labeler assets, 90

CHAPTER 9

Introduction to labeler assets

A labeler asset derives information about the content and structure of data. You can configure a labeler asset to analyze tokens, or delimited values, in an input field, or the individual characters in an input field.

A token labeling operation identifies the types of information in the input field. At run time, the asset writes a label for each type that it identifies to a corresponding output field. The label is a string of characters that indicates a type of information, such as a person name, or a date, or a post code.

A character labeling operation analyzes the character structure of the data in the input field, including punctuation and spaces. At run time, the asset writes a label for each character in the input data field that matches the labeling criteria. In character labeling, a label is a single character. The output data contains a label for each matching input character.

You create and test a labeler asset in Data Quality or Data Integration, and then add the asset to a Labeler transformation in a mapping in Data Integration. In Data Quality, you create a labeler asset from the **New** dialog box. In Data Integration, you create a labeler asset either from the **New Asset** dialog box or from a mapping.

When the mapping runs, the Labeler transformation enables you to understand the types of information in your data fields and to identify fields that do not contain the types of information that you expect.

You can enable a mapplet that contains a labeler asset transformation as an API. For more information about how to enable the mapplet, see the *Components* help in Data Integration.

You can configure a labeling operation to label values in the following ways:

Use a dictionary to label values

The labeling operation compares the values in the input string to the values in a dictionary that the labeler asset specifies. When the operation finds an input value that matches a dictionary value, it writes a label that you specify for the value to the output.

You can use dictionaries in token labeling and character labeling. In token labeling, you can also configure a labeling operation to assign labels to values that do not match any dictionary value.

Use a regular expression to label values

The labeling operation applies a regular expression to the input string and finds values that match the expression logic. When the operation finds an input value that matches the expression logic, it writes a label that you specify for the value to the output.

Use a regular expression to find values that match a character format or structure. You can use a predefined regular expression, or you can enter your own regular expression.

You can use regular expressions in token labeling.

Use a character set to label values

The labeling operation examines the characters in an input string and returns labels for the characters that match the character set.

You can use a predefined character set, or you can add a custom character set.

You can use character sets in character labeling. In character labeling, the label is a single character.

Each operation that you define in a labeler asset is called a step. In token labeling, you can combine steps that use dictionaries and steps that use regular expressions in a single asset. In character labeling, you can combine steps that use dictionaries and steps that use character sets in a single asset.

When to use a labeler asset

The labeler asset assigns a descriptive label to values in an input string.

The following examples describe some of the types of analysis you can perform with a Labeler asset.

Verify business information with dictionaries

A data set might contain a field of values that correspond to a finite set of known values, such as a set of stock-keeping unit (SKU) numbers in your organization. You can use a dictionary of the values to verify that the field contains the data that you expect.

Create a token labeling step that reads a dictionary, and add a dictionary that contains the SKU values to the step. Next, specify a label name for the step. For example, you might specify SKU as the label.

Add the asset that you create to a Labeler transformation in a mapping. When a mapping runs, the transformation compares the input field values to the values in the dictionary that the steps specifies. The mapping writes the text label for each SKU value that it finds to an output field.

You can also configure the step to label any value that does not match a dictionary value. In this case, the mapping writes a text label to the output field when an input value does not match a value in the dictionary. You might specify a different label in this case, such as INCORRECT. To find the incorrect values, select the *Exclusive* option in the step.

Identify data by character format

A customer data set might include a column for contact data. You expect the column to contain email addresses, but users might enter other values, such as phone number, country name, or postal code values into the column. You can use a regular expression to verify the fields that contain email addresses.

For example, you can configure the asset to label the following string as an email address:

```
info@informatica.com
```

Create a token labeling step for regular expressions, and add an expression that represents the email data format. You can enter a regular expression that describes the format that you want to find. Or, select a regular expression from the list of built-in expressions in the asset. You can specify EMAIL as the label name that the step applies to values that match the expression format.

At run time, the Labeler transformation applies the regular expression logic to the values in the input field. When the transformation finds a value with a format that matches the expression logic, it writes the label that you provided to an output field. The output fields will contain the label EMAIL for well-formatted email addresses and will contain any non-email addresses in their original form.

Review the structure of your input data

An organization might store the telephone number of employees in the following patterns: `(212)555-1212`, `2125551212`, and `+212-555-1212`. You can use a character set to verify the telephone number structures.

Create a step in character labeling mode for each telephone number structure that you support. Add a custom character set or select a built-in character set in the asset. Configure the asset to label any input character that matches the content of a character set. You might specify the following label names for your telephone data: P for punctuation characters, D for digits, and S for spaces.

When the transformation finds a character that matches a member of the character set that you define, it writes the label that you provided for the characters in the output. For example, the labeling operation reads the telephone number `(212)555-1212` and returns the label PDDDPDDDPDDD

Selecting the right labeling mode data for your data

Token labeling and character labeling can perform equally well in identifying correct and incorrect data values.

The labeling mode that you choose can depend on the types of error that you expect to find in your data. Character labeling can be more useful when a user adds valid data to the wrong field. Token labeling can be more useful when the accuracy of the field data is paramount and you want to find inaccurate data.

Consider the following cases:

- Your organization maintains an address data set in which users can enter valid data values to the wrong fields. For example, the users may enter street name information to a field for city names.

  You configure a character labeling operation that applies a dictionary of street terminology to the city name field. At run time, the operation returns a label for street terms such as STREET, ROAD, and AVENUE.

  Because character labeling returns the labeled and unlabeled characters in the same field, you can determine whether the values are incorrect or simply entered in the wrong field.
- Your organization maintains a list of batch codes for ingredients in a product recipe.

  You configure a token labeling operation to apply a dictionary of the code values to the code data field. At run time, the operation returns a label for each correct value in the field.

  You might alternatively configure the token labeling operation to return a label for each incorrect value in the field.

Labeler assets and mappings

The labeler assets that you create are available to the users who create mappings in Data Integration. You add a Labeler transformation to a mapping and add a labeler asset to the transformation. The Labeler transformation applies the logic in the asset to the data source that the mapping identifies.

A Labeler transformation works in a similar way to a Mapplet transformation. You connect the transformation inputs and outputs to other objects in the mapping in the same manner as a mapplet connects to other mapping objects. When you run the mapping, the Labeler transformation applies the labeler asset logic to an input field and generates output data for downstream objects.

The asset logic comprises the labeling criteria that you define in one or more the steps in the asset. The transformation applies the asset logic to a single input field.

Token labeling output fields

When the mapping runs, the Labeler transformation creates the following output fields for token labeling:

LabeledOutput

A copy of the input field data in which any value that matches the logic in an asset step is replaced with the label that the step specifies.

TokenizedData

A copy of the input field data. If the asset includes a dictionary step, you can optionally configure the step to replace any value that matches a dictionary value with the corresponding valid value from the dictionary.

You can test the run-time performance of a labeler asset. The asset writes test results to the *Labeled Output* field and the *Tokenized Data* field.

Character labeling output fields

When the mapping runs, the Labeler transformation creates the following output fields for character labeling:

LabeledOutput

A copy of the input field data in which any character that matches the logic in an asset step is replaced with the label that the step specifies.

You can optionally configure a step to ignore any input term that you specify.

If the asset includes a dictionary step, you can optionally configure the step to override any later step in the sequence.

Input

Contains the input data.

You can test the run-time performance of a labeler asset. The asset writes test results to the *Labeled Output* field.

Labeler assets and dimensions

The data quality issues that you may find in your data can fall into a range of common categories. Data quality assets can identify the categories as dimensions.

When you configure an asset, you can use the **Dimension** property to indicate the type of data quality issue that you want the asset to examine.

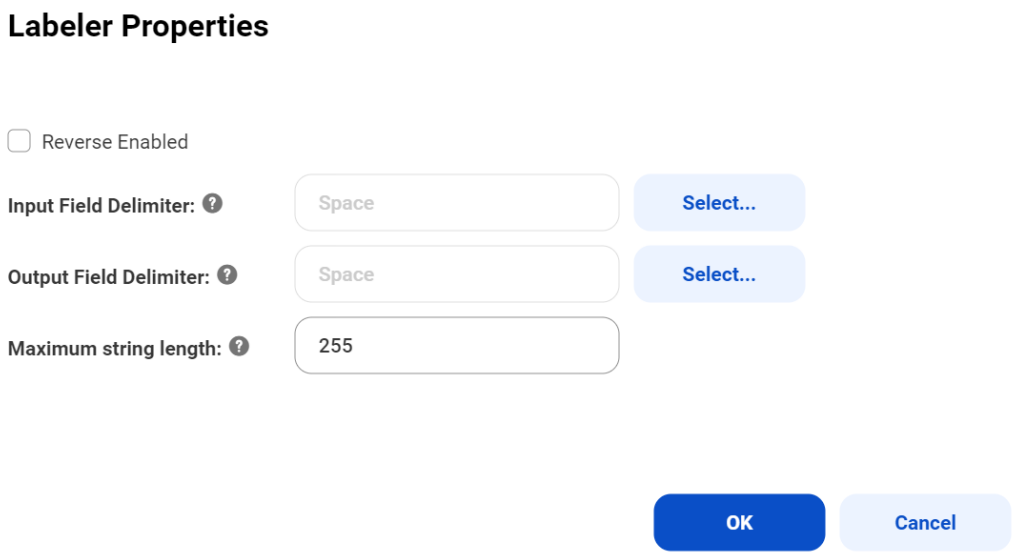
Find the Dimension property on the **Definition** tab of the asset.

For more information about data quality dimensions, see the *Introduction* help in Data Quality.

Labeler asset properties

Configure properties on a labeler asset that determine how a mapping that includes the asset will read, write, and process data.

The following image shows the properties:



You can set the following properties:

Reverse Enabled

When selected, indicates that the labeling operation reads an input string from right to left. By default, a labeling operation reads the data in an input field from left to right.

Input Field Delimiter

Specifies the delimiter that the labeling operation recognizes between discrete values in an input field. Use the **Select** option to select the delimiter or to add a custom delimiter. You can select one or more delimiters. The default input delimiter is a character space.

Output Field Delimiter

Specifies the delimiter that the labeling operation applies in an output field. Use the **Select** option to select the delimiter or to add a custom delimiter. The default output delimiter is a character space.

Maximum string length

Specifies the maximum number of characters that a labeling operation can read or write in an input or output field. You can set a maximum string length of 10,000 characters. The default maximum length is 255 characters.

The Maximum string length property also applies to the data that you import or enter in the test panel. Import or enter a string on each row that does not exceed the value on the property.

Labeler asset structure

A labeler asset contains options on a **Definition** tab and a **Configuration** tab.

Use the **Definition** tab options to enter a name for the asset, optionally to enter a description for the asset, and to select the folder in which to store the asset.

Use the **Configuration** tab options to configure one or more labeling operations that a mapping will perform. You configure the labeling operations as a sequence of steps that the mapping follows at run time.

Configuring a labeler asset

Create and configure labeler assets that you can then use in Labeler transformations.

1. To create a labeler asset, click **New** > **Labeler**.
2. On the **Definition** tab, enter a name for the labeler asset.
3. Optionally, enter a description.
4. Select the location in which to save the labeler asset.
5. Save the labeler asset.

   The **Asset References** panel appears. The **Asset References** panel shows additional assets that are referenced by data quality assets to perform transformation tasks. When you create a new asset, there are no referenced assets.
6. Optionally, select a data quality dimension to represent the type of data quality issue that you want the asset to examine.

   For more information about dimensions, see "Labeler assets and dimensions" on page 73.
7. Optionally, add a tag to the labeler asset. You can search for assets with a common tag on the **Explore** page.
8. Select the **Configuration** tab.

   The configuration workspace appears for the labeler asset.
9. Select the labeler mode to apply to your data.

   Select one of the following modes:

   - Token. Displays the token labeling options.
   - Character. Displays the character labeling options.
10. Add a step to the asset.

    Select one of the following step types, depending on the labeling mode that you selected:

    - Dictionary. By default, labels any value in an input field that matches a value in the dictionary. Token labeling returns a label for each complete input value that matches a dictionary value. Character labeling returns a single-character label for each character in an input value that matches a dictionary value.
    - Regular expression. Labels any value in an input field that conforms to the regular expression logic.
    - Character set. Labels any character in an input field that matches a member of the character set.
11. Configure the properties on the step to create an operation that the mapping can apply to the data.
12. Optionally, add one or more additional steps.

Configure the properties on each step that you add.

13. Save the asset.

The following pages provide additional information on configuring the asset:

- To read more about dictionaries and token labeling, see “Configuring a dictionary step for token labeling” on page 81.
- To read more about dictionaries and character labeling, see “Configuring a dictionary step for character labeling” on page 87.
- To read more about regular expressions and token labeling, see “Configuring a regular expression step” on page 82.
- To read more about character sets and character labeling, see “Configuring a character set step” on page 88.

CHAPTER 10

Token labeling operations

A token labeling operation analyzes the data values in an input field to discover the types of information that the values contain. The operation returns a label for each input value that matches an expected type of information. You can use a regular expression or a dictionary to analyzes the data values.

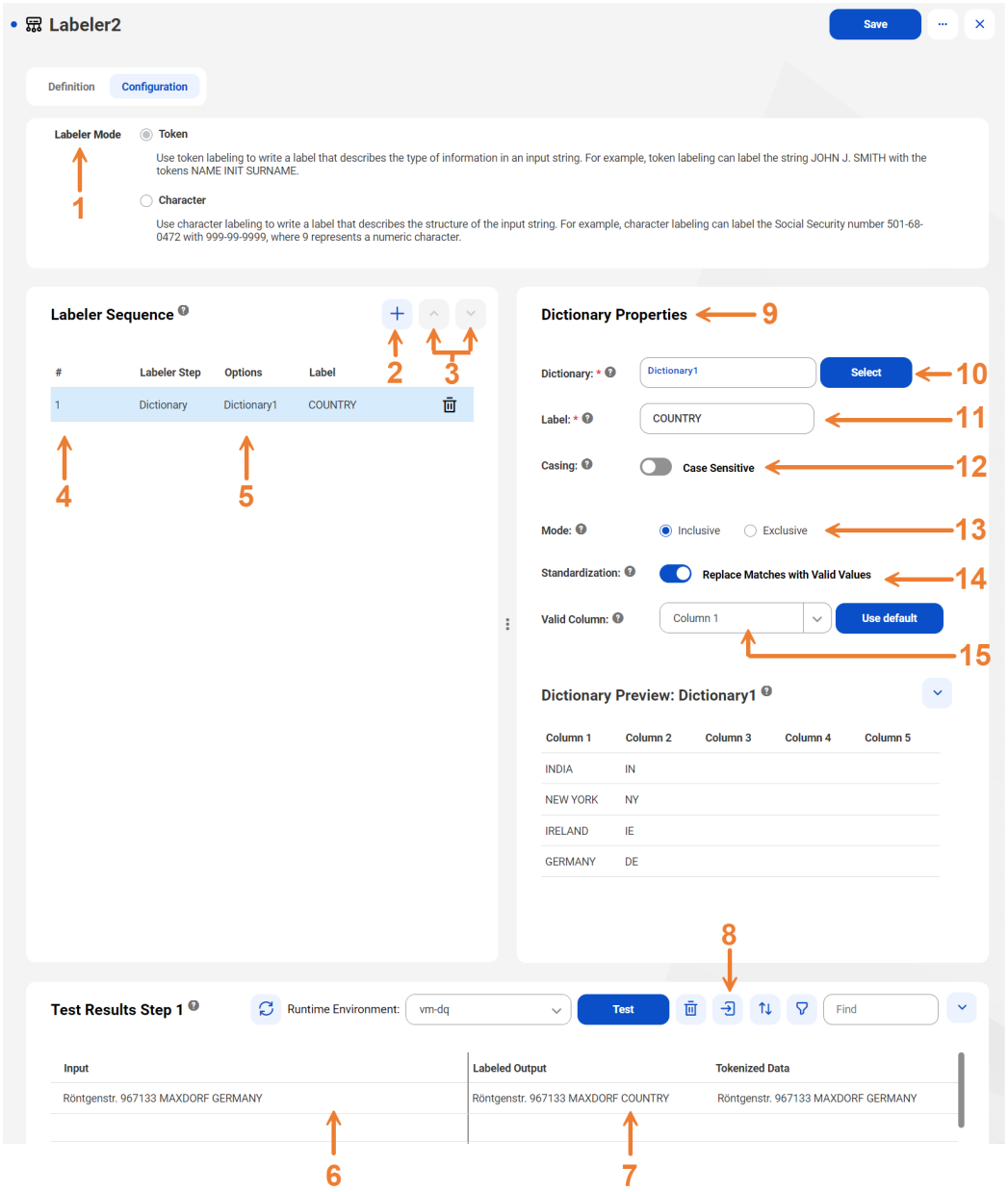
To define a token labeling operation, configure a labeler asset in **Token** mode.

Dictionary options for token labeling

When you add a step to a labeler asset, you are prompted to define a step with a regular expression or with a dictionary.

You add a dictionary to a labeler asset, and then add the asset to the Labeler transformation in a mapping.

The following image shows the options that you configure when you define a step with a dictionary:



The dictionary options include the following properties:

1. Labeler mode.
   Indicates the type of labeling operations to perform on the input data.
2. Add Step option.
   Adds a step to the asset. A step describes a labeling operation that a mapping can apply to an input data field.
3. Up and Down options.
   Moves a step that you select up or down within the step sequence.
4. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs labeling operations in the order that you specify.

5. Options name.
   Identifies the dictionary that you add to the step.
6. Test input field.
   Contains the input data that the Secure Agent uses to test the steps in the labeler sequence.
7. Test output fields.
   Contain the result of the test.

   The test output fields contain the following data:

   - A copy of the input field data in which any dictionary value is replaced by the label that you specify.
   - A copy of the input field data in which the discrete input values use the output field delimiter that you specify. The field can also return the valid dictionary version of any input value that matches a dictionary value.
8. Import file option.
   Imports data to the test panel.
9. Step type.
   Identifies the type of step to which the properties apply.
10. Dictionary name.
    Identifies the dictionary that the step applies to the input data.
11. Label name.
    Specifies the label that the step applies to the values in an input string. You enter the label.
12. Casing option.
    Specifies whether the step considers the character case of the input values that it compares to the dictionary values. For example, the character case may be relevant if you plan to label person names from an input string, as the person names may begin with an uppercase letter.

    Clear the option if the character case is not relevant to the data that the step will examine.
13. Mode option.
    Determines the labeling method. Choose Inclusive or Exclusive mode to label values that match or do not match the dictionary values. The default mode is Inclusive.

    In Inclusive mode, the labeling operation assigns the label to any value that matches a value in the dictionary.

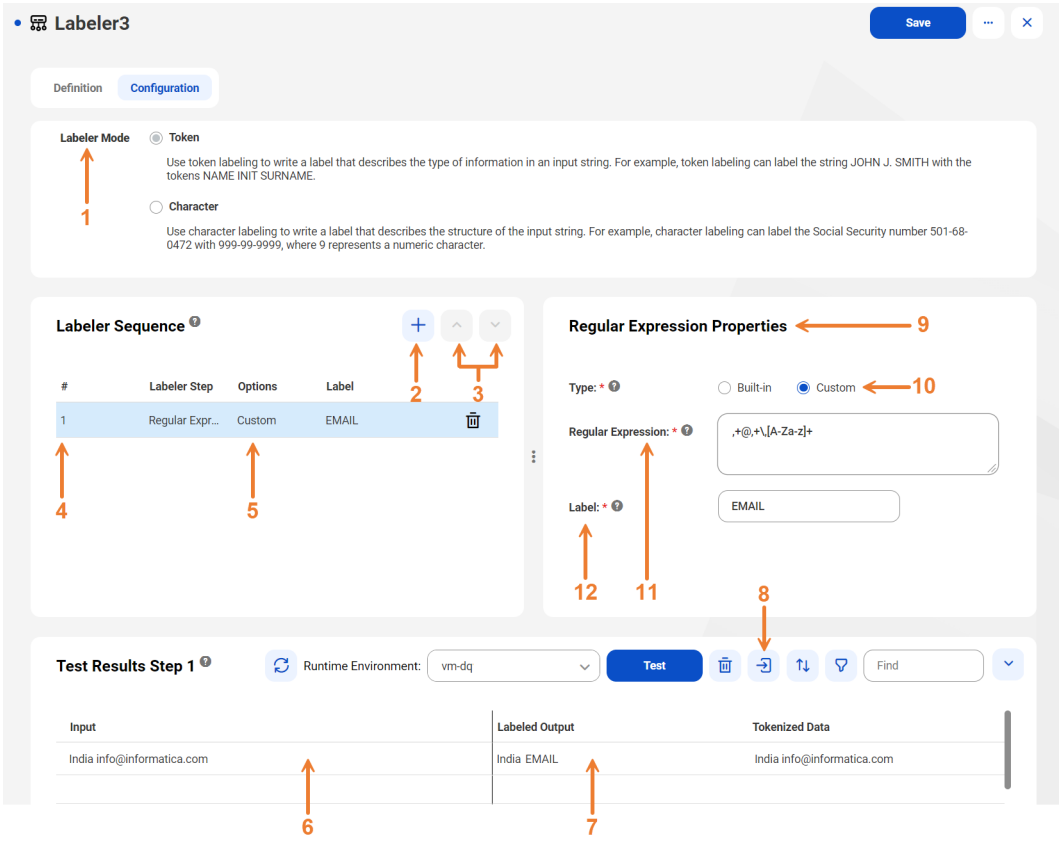
    In Exclusive mode, the operation assigns a label to any value that does not match a value in the dictionary.
14. Standardization option.
    Specifies whether the step will replace any input value that matches a dictionary value with the corresponding value from the valid column in the dictionary.

    The step compares the input values to every value in the dictionary. If the input value matches a valid value, the step does not update the value. If you clear the option, the step does not standardize any input value that matches a dictionary value.
15. Valid dictionary column name.
    Identifies the valid column in the dictionary that you select. The valid column contains the preferred versions of the values in the dictionary. The valid dictionary column is active when you select the standardization option.

Regular expression options

When you add a step to a labeler asset, you are prompted to define a step with a regular expression or with a dictionary.

The following image shows the options that you configure when you define a step with a regular expression:



The regular expression options include the following properties:

1. Labeler mode.
   Indicates the type of labeling operations to perform on the input data.
2. Add Step option.
   Adds a step to the asset. A step describes a labeling operation that a mapping can apply to an input data field.
3. Up and Down options.
   Moves a step that you select up or down within the step sequence.
4. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs labeling operations in the order that you specify.
5. Options name.
   Identifies the type of regular expression in the step.
6. Test input field.
   Contains the input data that the Secure Agent uses to test the steps in the labeler sequence.
7. Test output fields.

Contain the result of the test.

The test output fields contain the following data:

- A copy of the input field data in which any value that conforms to the regular expression logic is replaced by the label that you specify.
- A copy of the input field data.

8. Import file option.
   Imports data to the test panel.
9. Step type.
   Identifies the type of step to which the properties apply.
10. Regular expression type.
    Specifies whether the step uses a built-in regular expression or a regular expression that you define.

    A built-in regular expression is one that you select from a list that the asset provides. A custom regular expression is one that you define.
11. Regular Expression.
    In Custom mode, the field contains the regular expression that you specify. You enter the regular expression logic in the field.

    In Built-in mode, the Regular Expression Name field displays the name of the regular expression that you select. Additional fields display the regular expression logic and a description of the regular expression.
12. Label name.
    Specifies the label that the step applies to the values in an input string.

    You provide the label for a custom regular expression. A built-in regular expression has a default label. You can accept the default label or specify a label you prefer.

Configuring a dictionary step for token labeling

You can configure a step to compare the values in an input string to a dictionary. The step writes a label for each matching value to the output. The step can also return the valid column entry for any input value that matches a dictionary value.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.
2. Select **Dictionary** as the step type.
3. In the **Dictionary Properties** pane, select a dictionary.

   Use a dictionary that contains a range of data values that you expect the step to find at run time.

   The step compares each value in an input string to every value in the dictionary that you select.
4. In the **Label** field, enter a label name that represents the data that you expect to find in an input string.
5. Select or clear the **Casing** option.

   If you select the option, the step searches for values in the input string that match both the spelling and the character case of the values in the dictionary. By default, the step ignores the character case of the input values.
6. Select one of the following modes:
   - Inclusive. The step applies a label to each input value that matches a dictionary value.

- Exclusive. The step applies a label to each input value that does not match a dictionary value.

7. Select or clear the **Standardization** option.

   If you select the option, the step returns the valid value from the dictionary for any input value that matches a dictionary value.

8. If you select the Standardization option, use the **Valid Column** options to identify the dictionary column that contain the valid values.

   The valid column contains the standard or preferred version of a value in cases where the dictionary contains more than one value on a given row.

   You can specify any dictionary column as the valid column within a step. To use the column that the dictionary designer set as the valid column, click **Use default**.

   You can see a preview of the Reference 360 dictionary columns, but can't edit the columns. To edit the dictionary data, go to Reference 360.

9. Save the asset.

Configuring a regular expression step

You can configure a step to apply a regular expression to the values in an input string. The operation compares the values in an input string to the values in a regular expression. The step writes the label that you specify for the value in the output.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.

2. Select **Regular Expression** as the step type.
3. Select the Built-in or Custom option.
   - To select a regular expression from a list of built-in expressions, select Built-in.
   - To enter a regular expression that the step will use, select Custom.

   Select or enter a regular expression that describes the character composition of the values that you want the step to find at run time.

4. If you select Custom mode, enter a label that describes the data that the regular expression finds in the input string. If you select Built-in mode, the regular expression adds a default label.

   You can accept the default label in Built-in mode or enter a label that you prefer. To change the label, select the User-Defined option.

5. Save the asset.

CHAPTER 11

Character labeling operations

A character labeling operation determines the character structure of an input string and writes a label as an output that describes the character structure. To define a character labeling operation, configure a labeler asset in **Character** mode.

When you configure a character labeling step, you enter a single character as the label. The operation returns the label for each input character that matches the conditions that the step defines.

You can configure a step to compare input characters to the values in a dictionary or to the characters in a character set.

Character labeling with a dictionary

When you configure character labeling with a dictionary, the labeling operation returns a label for each character when an input value matches a dictionary value. The labeling operation returns other characters unchanged in the output.

For example, you might compare the values in a column of surname data with the values in a dictionary of person names. You add N as the label in the character labeling step. The input data and the dictionary both include the name SMITH. The labeling operation returns the string NNNNN as output for the name SMITH. However, the dictionary does not contain the name SMYTH. If the labeling operation finds the name SMYTH in an input string, it writes SMYTH as output.

The labeling operation can recognize a dictionary value within a larger value. For example, the operation can return GOLDNNNNN as the label for the input name GOLDSMITH.

Character labeling with a character set

When you configure character labeling with a character set, the labeling operation returns a label for each character in the input value that matches a character in the character set. The labeling operation returns other characters unchanged in the output.

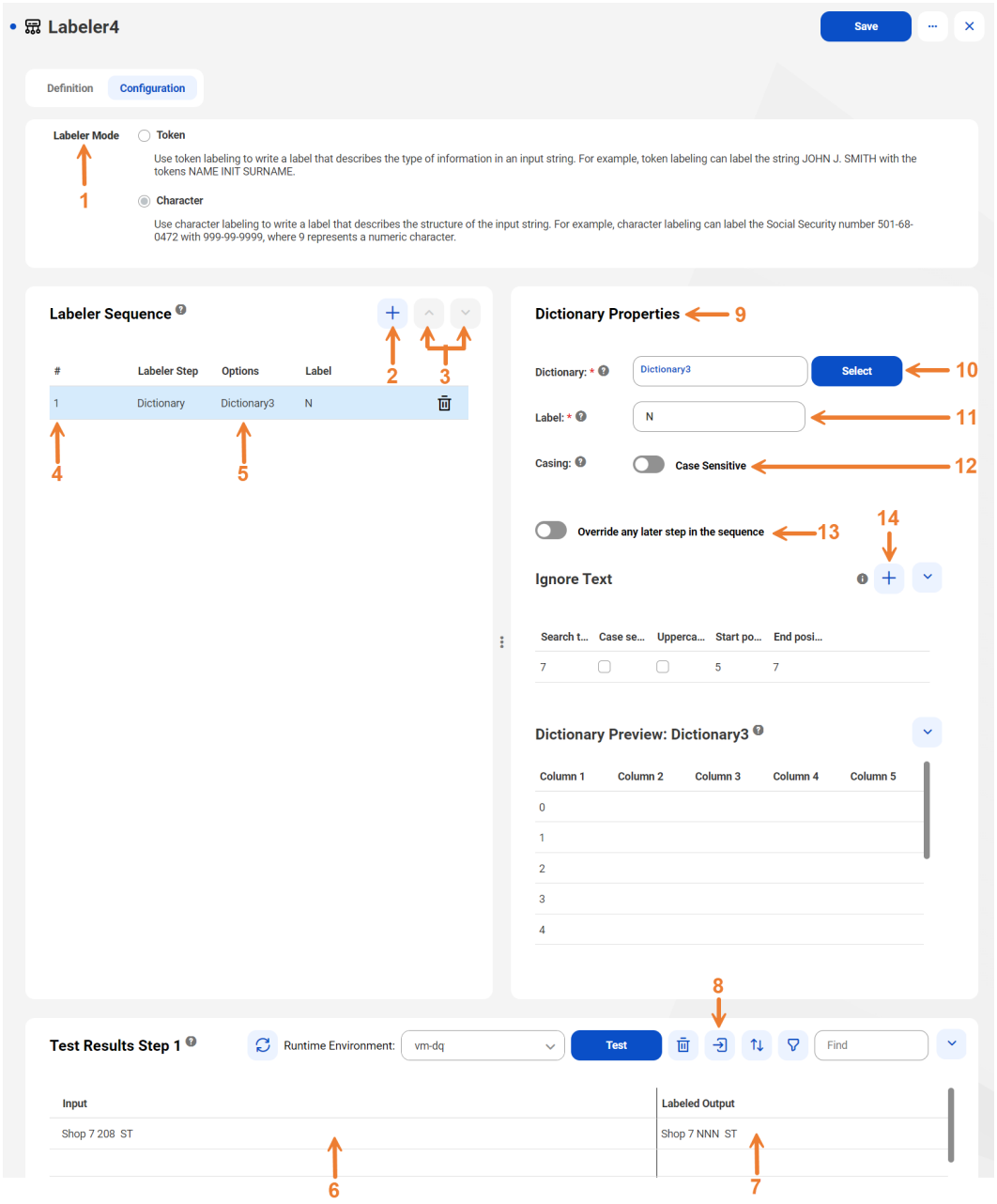
For example, you might compare the values in a column of Irish postcodes with a character set. Irish post codes have the format A99SAA99, where A is an alphabetic character, 9 is a digit, and S is an optional character space. You define a character set that includes the valid characters and excludes characters such as I,O, and Z that the postcodes do not use.

The labeling operation returns the labels A99SAA99 for the input D08 E1W3, as D08 E1W3 represents a valid postcode format. However, Z08 IO34 does not conform to the format. If the labeling operation finds Z08 IO34 in an input string, it writes Z99SIO99 as output.

Dictionary options for character labeling

When you add a step to a labeler asset, you are prompted to define a step with a character set or with a dictionary.

The following image shows the options that you configure when you define a step with a dictionary:



The dictionary options include the following properties:

1. Labeler mode.
   Indicates the type of labeling operations to perform on the input data.
2. Add Step option.

Adds a step to the asset. A step describes a labeling operation that a mapping can apply to an input data field.

3. Up and Down options.
   Moves a step that you select up or down within the step sequence.
4. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs labeling operations in the order that you specify.
5. Options name.
   Identifies the dictionary that you add to the step.
6. Test input field.
   Contains the input data that the Secure Agent uses to test the steps in the labeler sequence.
7. Test output fields.
   Contain the result of the test. The test output field contains a copy of the input field data in which any dictionary value is replaced by the label that you specify.
8. Import file option.
   Imports data to the test panel.
9. Step type.
   Identifies the type of step to which the properties apply.
10. Dictionary name.
    Identifies the dictionary that the step applies to the input data.
11. Label name.
    Specifies the label that the step applies to the characters in an input string. You enter a single character as the label.

    The labeling operation returns the label for each character in a matching dictionary value.
12. Casing option.
    Specifies whether the step considers the character case of the input values that it compares to the dictionary values. For example, the character case may be relevant if you plan to label person names from an input string, as person names begin with an uppercase letter.

    Clear the option if the character case is not relevant to the type of data that you plan to label from the inputs.
13. Override any later step in the sequence option.
    Determines whether the current step output overrides the output from later steps in the sequence.
14. Ignore Text option.
    Adds a character to ignore during a labeling operation. You can add one or more characters. The labeling operation does not assign the label to any character that matches a character in the dictionary.

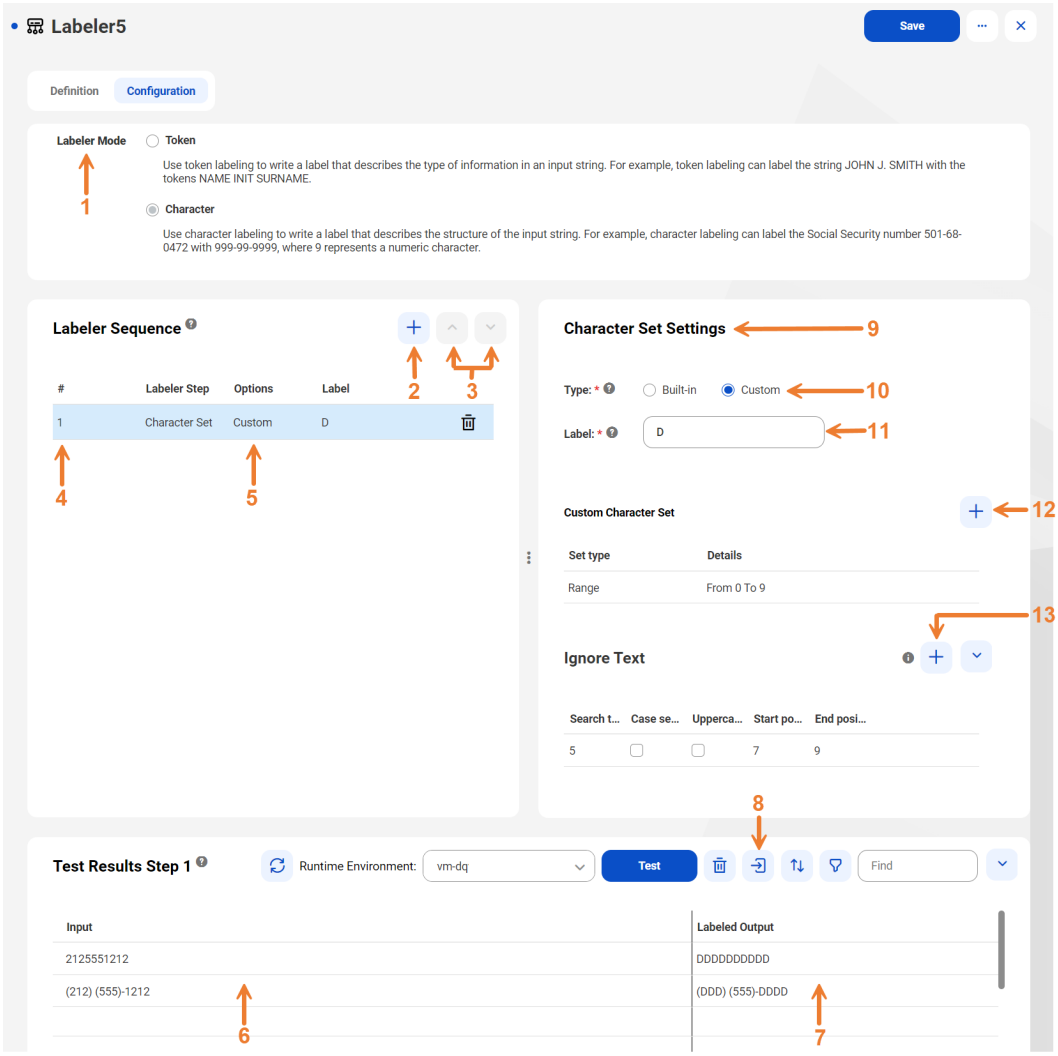
    The **Ignore Test** option includes the following properties:

    - Search term. Specifies the characters to ignore when you perform a labeling operation.
    - Case sensitive. Determines whether the input characters must match the case of the search characters.
    - Uppercase. Converts the characters in the input string that match the search term to uppercase.
    - Start position. Specifies the character position in the input string at which the asset starts to analyze the characters.
    - End position. Specifies the character position in the input string at which the asset ends the analysis.

Character set options for character labeling

When you add a step to a labeler asset, you are prompted to define a step with a character set or with a dictionary.

The following image shows the options that you configure when you define a step with a character set:



The character set options include the following properties:

1. Labeler mode.
   Indicates the type of labeling operations to perform on the input data.
2. Add Step option.
   Adds a step to the asset. A step describes a labeling operation that a mapping can apply to an input data field.
3. Up and Down options.
   Moves a step that you select up or down within the step sequence.
4. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs labeling operations in the order that you specify.

5. Options name.
   Identifies the type of regular expression in the step.
6. Test input field.
   Contains the string that the Secure Agent uses to test the steps in the labeler sequence.
7. Test output fields.
   Contain the result of the test. The test output field contains a copy of the input field data in which any character that matches the content of a character set is replaced by the label that you specify.
8. Import file option.
   Imports data to the test panel.
9. Step type.
   Identifies the type of step to which the properties apply.
10. Character set type.
    Specifies whether the step uses a built-in or custom character set.

    A built-in character set is one that you select from a list that the asset provides. A custom character set is one that you define.
11. Label name.
    Specifies the label that the step applies to the characters in an input string. You provide a single-character label for a custom character set.
12. Add Character Set option.
    Adds a custom character set. You can specify individual characters or a range of characters. Select the characters from the **Custom Character Set** dialog box.
13. Ignore Text option.
    Adds one or more strings to ignore during a labeling operation. The labeling operation does not assign the label to any input character that matches a character in the strings.

    The **Ignore Test** option includes the following properties:

    - Search term. Specifies the characters to ignore when you perform a labeling operation.
    - Case sensitive. Determines whether the input characters must match the case of the search characters.
    - Uppercase. Converts the characters in the input string that match the search term to uppercase.
    - Start position. Specifies the character position in the input string at which the asset starts to analyze the characters.
    - End position. Specifies the character position in the input string at which the asset ends the analysis.

Configuring a dictionary step for character labeling

You can configure a step to compare the values in an input field to the values in a dictionary. The step writes a label that you specify to the output field for each character in a matching value.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.
2. Select **Dictionary** as the step type.
3. In the **Dictionary Properties** pane, select a dictionary.

   Use a dictionary that contains values that you expect the step to find at run time. The step compares each value in an input field to every value in the dictionary that you select.

4. In the **Label** field, enter a label name that represents the characters that you expect to find in the input field.

   Enter a single character as a label in character labeling. The step returns the label for each character in an input value that matches a dictionary value.

   The string may contain additional characters that do not match a dictionary value. The labeling operation returns the characters unchanged in the output.

5. Select or clear the **Casing** option.

   If you select the option, the step searches for characters in the input string that match both the spelling and the character case of the characters in the dictionary. By default, the step ignores the character case of the characters in the input string.

6. Select or clear the **Override any later step in the sequence** option.

   If you select the option, the output from the current step overrides the output from any later step in the sequence.

7. Optionally, use the in the **Ignore Text** options to specify one or more values that the step will skip during a labeling operation. Configure the **Ignore Text** properties for each character that you add.

8. Save the asset.

Configuring a character set step

You can configure a step to compare the characters in an input string to a character set. The step writes the label that you specify to the output field for each character that matches the character set.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.

2. Select **Character Set** as the step type.

3. Select the Built-in or Custom option.

   - To select a character set from a list of built-in character sets, select Built-in.
   - To enter a character set that the step will use, select Custom.

   Select or define a character set that identifies the characters that you want to find at run time.

   Note: Complete step 4 or step 5, based on your selection of Built-in or Custom option.

4. If you select the Built-in option, the character set adds the label name.

5. If you select the Custom option, configure the following options:

   1. In the **Label** field, enter a single character as a label. The step applies the label to any input character that matches a character in the character set.

   2. Select the option to add a character set.

   3. In the **Custom Character Set** dialog box, select the **Range** or **Individual character(s)** option.

   4. Select the characters that you need from the character selection grid.

      Note: When you define a range, ensure that you follow the character order in the character selection grid. The character that you select in the *From* field must precede the character that you select in the *To* field. For example, do not select 'Z' in the *From* field and 'A' in the *To* field. The labeler reads the characters in the grid from top to bottom and from left to right.

6. Optionally, use the in the **Ignore Text** options to specify one or more values that the step will skip during a labeling operation. Configure the **Ignore Text** properties for each character that you add.

7. Save the asset.

CHAPTER 12

Test labeler assets

Test a labeler asset to verify that the asset logic generates the results that you expect.

Testing a labeler asset in token labeling mode

Test a labeler asset to verify that the data flows through the asset in the ways that you expect. You can test all of the steps in the asset or a subset of the steps.

1. Open the labeler asset.
2. Select the **Configuration** tab.
3. Select a step in the step sequence.

   The test runs on the steps in the asset and ends with the step that you select. The test runs in the order that the step sequence specifies.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter one or more data values in the input column, or import the data to test. To import the data, click the **Import** option in the test panel.
6. Click **Test**.

   The test results contain the following columns:

   Labeled Output

   A copy of the input string in which values that match the label criteria are replaced with the label that the steps define.

   Tokenized Data

   A copy of the input string. If you selected the Standardization option in a dictionary step, the test replaces any input value that matches the label criteria with the corresponding value from the valid column in the dictionary.
7. Verify that the steps read the input values and write the output values that you expect.

   You can sort and filter the results of the test.

Testing a labeler asset in character labeling mode

Test a labeler asset to verify that the data flows through the asset in the ways that you expect. You can test all of the steps in the asset or a subset of the steps.

1. Open the labeler asset.
2. Select the **Configuration** tab.
3. Select a step in the step sequence.

   The test runs on the steps in the asset and ends with the step that you select. The test runs in the order that the step sequence specifies.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter one or more data values in the input column, or import the data to test. To import the data, click the **Import** option in the test panel.
6. Click **Test**.

   The test results contain the following column:

   Labeled Output

   A copy of the input string in which any characters that match the label criteria are replaced with the label that the steps define.

   If you selected the **Ignore Text** option in any step, the test ignores any term that you specified.

   If you selected the **Override any later step in the sequence** option in a dictionary step, the step output takes precedence over the output from any later step in the sequence.
7. Verify that the steps read the input data and write the output characters that you expect.

   You can sort and filter the results of the test.

Part V: Parse assets

This part contains the following chapters:

- Introduction to parse assets, 93
- Custom parsing operations, 99
- Pattern-based parsing operations, 106
- Test parse assets, 116

CHAPTER 13

Introduction to parse assets

A parse asset defines a set of operations that can identify discrete values in an input field and write the values to appropriate output fields. Use a parse asset to improve the structure of your data.

You create and test a parse asset in Data Quality or Data Integration, and then add the asset to a Parse transformation in a mapping in Data Integration. In Data Quality, you create a parse asset from the **New** dialog box. In Data Integration, you create a parse asset either from the **New Asset** dialog box or from a mapping.

When the mapping runs, the Parse transformation reads a single input field and writes the values that it identifies to new fields based on the criteria that the parse asset specifies.

You can enable a mapplet that contains a parse asset transformation as an API. For more information about how to enable the mapplet, see the *Components* help in Data Integration.

You can configure a parse operation to identify values in the following ways:

Use a dictionary to identify values

The operation compares the values in the input field to the values in a dictionary that the parse asset specifies. When the operation finds an input value that matches the dictionary, it writes the value to a new output column.

Use a dictionary to find values based on their content.

Use a regular expression to identify values

The operation applies a regular expression to the input field and finds values that match the expression logic. When the operation finds an input value that matches the expression logic, it writes the value to a new output column.

Use a regular expression to find values based on their structure.

Use a pre-built pattern to identify values

The operation uses pre-built parsing logic to analyze the values in the input field and assign each value to a pre-defined output field. The parse asset includes built-in parsing logic for person name data. You can add patterns to the asset to enhance the parsing logic.

A parsing operation that uses a dictionary or a regular expression is called a step. You can combine steps that use dictionaries and steps that use regular expressions in a single asset. Each step parses data values to one or more output fields that you specify. The Parse transformation creates the outputs that you specify at run time and writes data to the outputs that it creates.

A parse asset can contain a single pattern-parsing operation. The Parse transformation creates the outputs that the pattern specifies and writes data to the outputs that it creates.

Parsing modes

When you configure a parse asset, you must select the mode in which the parsing operations will run.

You can select one of the following modes:

Custom mode

Select custom mode to define a parsing operation that uses a dictionary or a regular expression to analyze the input data. When you work in custom mode, you configure all elements of the parsing operation.

Pre-built mode

Select pre-built mode to define a parsing operation that applies built-in pattern logic to the input data. When you work in pre-built mode, you do not need to configure the parsing operation. You can optionally add one or more patterns to the asset logic in order to enhance the parsing performance.

Unparsed data and overflow data

When a parsing step cannot identify a value in an input field, the step writes the value to a field for unparsed data. When a parsing step successfully identifies a value but cannot assign the value to the designated output field, the step writes the value to a field for overflow data. A step writes a value to an overflow field when it has already parsed another value successfully to the designated output field.

When a step assigns a value to an unparsed data field, the value remains available for parsing from its original input field by subsequent steps in the asset. When a step writes a value to an overflow field, the value is no longer available to subsequent steps in the asset. In this way, a value is not finally written to the unparsed data field until all of the steps in the asset are complete.

The number of overflow fields that the asset generates at run time depends on the types of step that you configure and the asset properties that you define. For example, you might select the **Detailed Overflow** property to enable the creation of a dedicated overflow field for each step. The **Detailed Overflow** property is available in custom mode.

Example: using the overflow data field

You might configure a parsing step to split a field of multiple values into discrete fields based on the types of information that the values represent. You can use the fields for parsed data and overflow data to capture values that contain the same type of information.

For example, the following product description contains color and product type information:

EMERALD GREEN WOODSTAIN

The product description uses two values define the color. You can define a dictionary step to parse the color names and the product type to discrete fields. The step reads a dictionary of color names and specifies a single overflow field.

At run time, the step identifies EMERALD and GREEN as colors that match values in the dictionary. The step writes EMERALD to the field for parsed data and writes GREEN to the overflow field. The step additionally writes WOODSTAIN to the unparsed data field. As a result, the step writes the terms EMERALD, GREEN, and WOODSTAIN to discrete fields.

You can optionally merge the fields that contain colors in a downstream transformation.

Example: using an unparsed data field to create new fields

You might configure a parsing step to write a uniform or consistent set of data values to an unparsed data field.

For example, the following contact field includes a name and a prefix:

PROFESSOR INDIRA SINGH

You define a step that uses a dictionary of prefix values to parse the prefix PROFESSOR to an output field. The step parses all other information to an unparsed data field.

Because the unparsed data field contains a set of values that uniformly represent a person name, you can use the field as person name data in downstream operations.

Rules and guidelines for overflow and unparsed data fields

The parse asset configuration determines how the associated Parse transformation creates overflow and unparsed data fields at runtime.

Consider the following rules and guidelines about the overflow and unparsed fields:

- When you configure an asset in Custom mode, the **Detailed Overflow** option determines how the Parse transformation creates overflow fields. If you clear the option, the transformation creates a single overflow field for all overflow data from the asset. If you select the option, the transformation creates an overflow field for each step in the asset.

  Find the Detailed Overflow option in the **Parse Properties** dialog box when you configure the asset in custom mode. The option is cleared by default.
- When you configure an asset in Pre-built mode, the locale that you select determines whether the Parse transformation creates an overflow field. The transformation creates an overflow field when you set the locale to Brazil or Portugal. The transformation does not create an overflow field in other locales.
- A Parse transformation creates a single field for all unparsed data when you configure the asset in Custom mode.
- The locale that the asset specifies in Pre-built mode determines whether the Parse transformation creates an unparsed data field. The transformation does not create an unparsed data field when you set the locale to Brazil, Canada, Portugal, or the United States.

Parse assets and mappings

The parse assets that you create are available to the users who create mappings in Data Integration. You add a Parse transformation to a mapping and add a parse asset to the transformation. The Parse transformation applies the logic in the parse asset to an input data field that you specify.

A Parse transformation works in a similar way to a Mapplet transformation. You connect the transformation inputs and outputs to other objects in the mapping in the same manner as a mapplet connects to other mapping objects. When you run the mapping, the Parse transformation applies the parse asset logic to the input field and generates output fields that you can connect to a downstream transformation.

After the mapping runs, you can evaluate the data output to determine if the data meets the parsing objectives.

Rules and guidelines for parse assets and mappings

Consider the following rules and guidelines when you work with mappings and parse assets:

- A Parse transformation contains a single parse asset. You configure the transformation to perform parsing operations on a single field of input data. You select the field on the **Field Mappings** tab in the transformation.
- When you run a mapping that includes a Parse transformation, the mapping creates the output fields that the parse asset specifies.

  When the parse asset specifies a regular expression or a dictionary, the Parse transformation creates an output field for the results of each step. The transformation also creates a field for unparsed data and one or more fields for overflow data.

  When the parse asset specifies pattern-based parsing, the Parse transformation creates multiple output fields for name data, including discrete fields for first names, family names, and full names. The transformation can also create fields for overflow data and for unparsed data.

  For more information about the creation of overflow and unparsed data fields, see "Unparsed data and overflow data" on page 94.
- A parse asset is a reusable object. You can add a single parse asset to transformations in multiple mappings.

  A Parse transformation is a nonreusable object. You select a Parse transformation when you configure a mapping.
- To view the list of parse assets that you created, navigate to the **Explore** page.
- If you update a parse asset that you previously added to a transformation, you must synchronize the transformation with the parse asset.

  To synchronize the transformation, open the transformation in the mapping and select the Parse properties. Data Integration displays a message that prompts you to synchronize the transformation with the parse asset.

Parse assets and dimensions

The data quality issues that you may find in your data can fall into a range of common categories. Data quality assets can identify the categories as dimensions. When you configure an asset, you can use the **Dimension** property to indicate the type of data quality issue that you want the asset to examine.

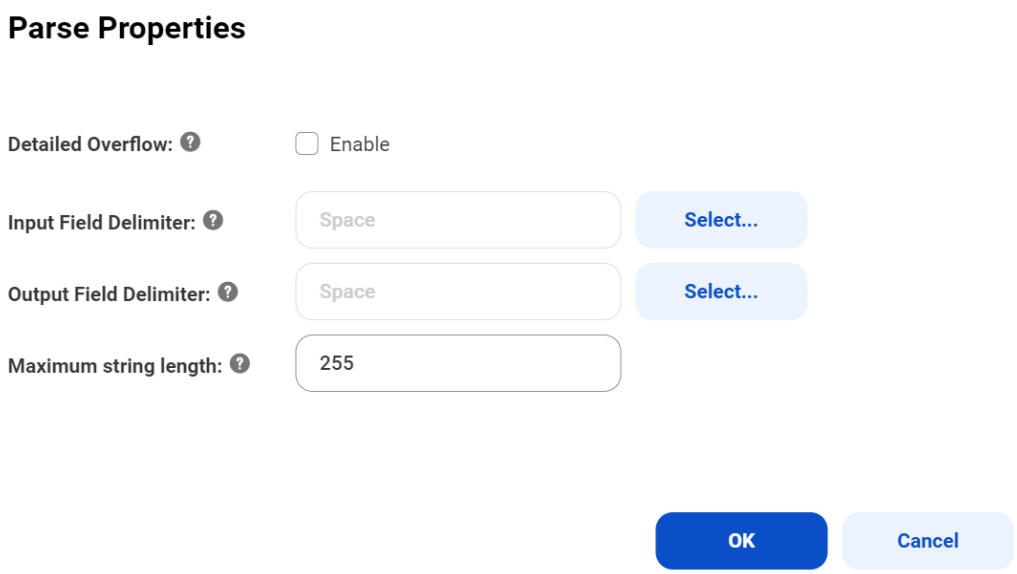
Find the Dimension property on the **Definition** tab of the asset.

For more information about data quality dimensions, see the *Introduction* help in Data Quality.

Parse asset properties

You can configure properties on a parse asset that determine how a mapping that includes the asset will read, write, and process data.

The following image shows the properties:



You can set the following properties:

Detailed Overflow

Determines whether the parsing operation creates an Overflow field for each step in the asset at run time. Select or clear the option in Custom mode.

When you select the option, the transformation writes overflow data to a separate field for each step in the asset. When you clear the option, the transformation writes all overflow data that the asset identifies to a single field. The option is cleared by default.

The Detailed Overflow option does not apply in Pre-built mode, as the asset specifies a single parsing operation in Pre-built mode.

Input Field Delimiter

Specifies the delimiter that the parsing operation recognizes when you parse data with a dictionary or regular expression. Set the option in Custom mode. Use the **Select** option to select the delimiter or to add a custom delimiter. You can select one or more delimiters. The default input delimiter is a character space.

Output Field Delimiter

Specifies the field delimiter that the parsing operation applies in any output field when you parse data with a dictionary or regular expression. Set the option in Custom mode. Use the **Select** option to select the delimiter or to add a custom delimiter. The default output delimiter is a character space.

Maximum string length

Specifies the maximum number of characters that a parsing operation can read or write in an input or output field. You can set a maximum string length of 10,000 characters. The default maximum length is 255 characters.

Configuring a parse asset

Create and configure parse assets that you can then use in Parse transformations.

1. To create a parse asset, click **New** > **Parse**.
2. On the **Definition** tab, enter a name for the parse asset.
3. Optionally, enter a description.
4. Select the location in which to save the parse asset.
5. Optionally, select a data quality dimension to represent the type of data quality issue that you want the asset to examine.

   For more information about dimensions, see "Parse assets and dimensions" on page 96.
6. Save the parse asset.

   The **Asset References** panel appears. The **Asset References** panel shows additional assets that are referenced by data quality assets to perform transformation tasks. When you create a new asset, there are no referenced assets.
7. Optionally, add a tag to the parse asset. You can search for assets with a common tag on the Explore page.
8. Select the Configuration tab.

   The configuration workspace appears for the parse asset.
9. Select the parsing mode to apply to your data.

   Select one of the following modes:

   - Custom. Select the custom mode to define one or more parsing steps with a dictionary or a regular expression.
   - Pre-built. Select the pre-built mode to use the built-in parsing logic in the asset.

   Note: Complete step 9 or step 10, based on your selection of Custom or Pre-built mode.
10. In Custom parsing mode, configure one or more steps in the asset.

    Select one or more of the following step types:

    - Regular expression. Use a regular expression to parse one or more data values in an input string to discrete outputs. You can enter a regular expression or select one from a list of built-in expressions.
    - Dictionary. Use a dictionary to parse one or more data values in an input string to one or more discrete outputs.
11. In Pre-built parsing mode, select the locale in which your input data originates and select the format in which the data is stored.

    Optionally, test the data and use the test results to enhance the pattern logic.

    You can repeat the testing process to fine-tune the pattern logic.
12. Save the asset.

For more detailed information on configuring an asset to use regular expressions, see "Configuring a regular expression step" on page 105.

For more detailed information on configuring an asset to use dictionaries, see "Configuring a dictionary step" on page 104.

For more detailed information on configuring an asset to perform pattern-based parsing, see "Configuration summary for pattern-based parsing " on page 108.

CHAPTER 14

Custom parsing operations

Select custom mode to define a parsing operation that uses one or more dictionary steps or regular expression steps.

You can create multiple steps in a single parse asset. At run time, the Parse transformation runs the steps that you specify.

Note: The Parse transformation runs any dictionary step in an asset before it runs any regular expression step. First, the transformation runs the dictionary steps in the order in which they appear in the step sequence. Next, the transformation runs the regular expression steps in the order in which they appear in the step sequence.

Parsing with dictionaries

You can use a dictionary to find known values in an input field. Populate the dictionary with the values that you want to find.

At run time, the Parse transformation compares the values in the input field to the values in a dictionary that you select in the parse asset. The transformation writes any values that match the dictionary values to an output field.

Example: product data

A product data set might combine data values about multiple product attributes in the same field. You can configure a parse asset to find the different values and to specify discrete fields for each type of value.

For example, you can configure the asset to split the following paint description into separate inventory elements:

```
500ML Red Matt Exterior
```

Create a step for each element that you want to find. In each step, add a dictionary that contains reference data for an element, such as size, color, style, or finish. When a mapping runs with the Parse transformation, the transformation will compare the input field values to the dictionaries that the steps specify.

The mapping will write the values to the following columns:

Size	Color	Style	Finish
500ML	Red	Matt	Exterior

Note: The input field data may contain data values in more than one format. For example, the quantity 500ML may appear as 500 millilitres, 500ml, 0.5L, or simply 500. To capture the data in all formats, use a dictionary

that contains a column of values in each format. You can select the format that you prefer when you configure the step. Select the column that uses the preferred format as the Valid column.

Parsing with regular expressions

You can use a regular expression to find values that match a given character structure in an input field. Create a regular expression that matches the structure of the values that you want to find. Or, select a regular expression from the list of built-in expressions in the asset.

Use a regular expression in place of a dictionary when you cannot predict the content of every value or when the range of values that you will search for is too great to add to a dictionary.

At run time, the Parse transformation applies the regular expression logic to the values in the input field. When the transformation finds a value with a structure that matches the expression logic, the transformation writes the value to the output field that the step specifies.

Example: United States telephone numbers and Social Security numbers

A customer data set might include a column for telephone numbers. Over a period of time, many users incorrectly enter Social Security numbers into the column. You can configure a parse asset to find values that match both formats.

The following table displays the types of errors that can appear in the column:

Value	Format
212-555-1234	Telephone number
910-22-5555	Social Security number
(518)555-8466	Telephone number
(718) 555-2907	Telephone number
2125550987	Telephone number
922-823-5746	Social Security number
974-43-0202	Social Security number
212-555-3287	Telephone number

Create a step for each data format, and add a regular expression to each step.

For example, the parse asset contains the following built-in regular expression for United States telephone numbers:

```
1?[-. ]?\(?[0-9]{3}\)?[-. ]?[0-9]{3}[-. ]?[0-9]{4}?(?:EXT|ext|Ext|X|x|#|\.| |,)*[0-9]{3,5}|1?[-. ]?\(?[0-9]{3}\)?[-. ]?[0-9]{3}[-. ]?[0-9]{4}
```

The asset contains the following built-in regular expression for United States Social Security numbers:

```
(.*)([0-9]{3}[- ]?[0-9]{2}[- ]?[0-9]{4})(.*)
```

Add a single output to each step for telephone numbers and Social Security numbers respectively.

Parse asset structure

A parse asset contains options on a **Definition** tab and a **Configuration** tab.

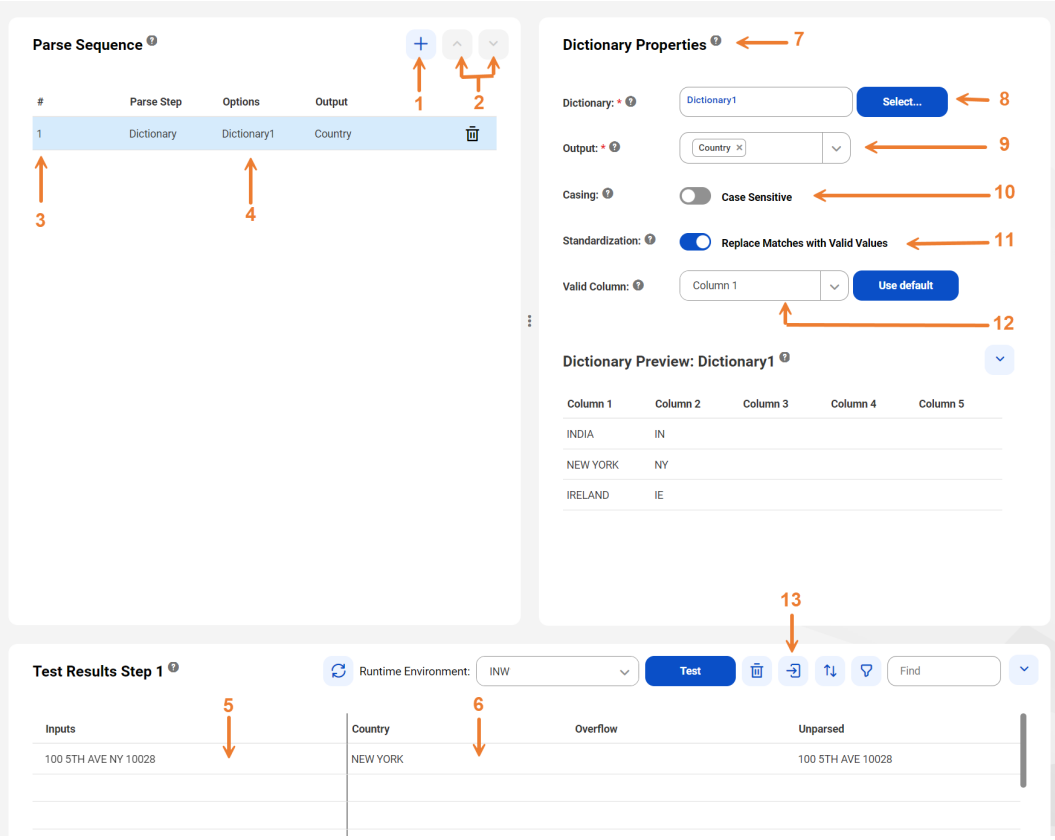
Use the Definition tab options to enter a name for the asset, optionally enter a description for the asset, and select the folder in which to store the asset.

Use the Configuration tab options to configure one or more parsing operations that a mapping will perform. You configure the parsing operations as a sequence of steps that the mapping follows at run time.

Dictionary options on the Configuration tab

When you add a step to a parse asset, you are prompted to define a step with a regular expression or with a dictionary.

The following image shows the options that you configure when you define a step with a dictionary:



The dictionary options includes the following properties:

1. Add Step option.
   Adds a step to the asset. A step describes a parsing operation that a mapping can apply to an input data field.
2. Up and Down options.
   Moves a step that you select up or down within the step sequence.
3. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs dictionary parsing operations in the order that you specify.

4. Option description.
   Identifies the dictionary that you add to the step.
5. Test input field.
   Contains the string that the Secure Agent uses to test the steps in the parse sequence.
6. Test output fields.
   Contain the result of the test. The asset displays an output field for each step and also displays output fields for overflow dat and unparsed data.
7. Step type.
   Identifies the type of step to which the properties apply. The step type can be dictionary or regular expression.
8. Dictionary name.
   Identifies the dictionary that the step applies to the input data.
9. Output name.
   An output to which the step can write each input data values that matches a dictionary value. You enter the output name. The mapping creates the output on the Parse transformation at run time.
10. Casing option.
    Specifies whether the step considers the character case of the input values that it compares to the dictionary values. For example, the character case may be relevant if you plan to parse person names from an input string, as person names begin with an uppercase letter.

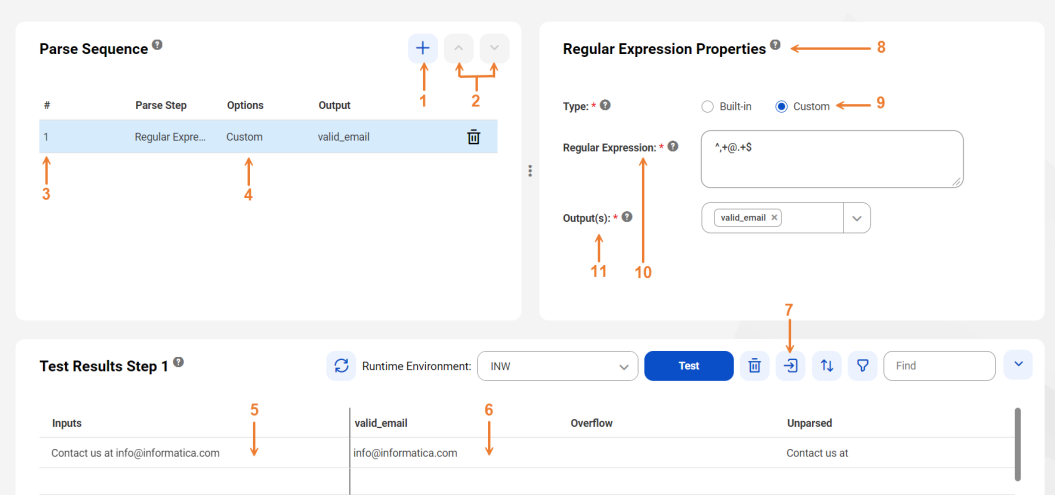
    Clear the option if the character case is not relevant to the type of data that you plan to parse from the inputs.
11. Standardization option.
    Specifies whether the step will replace any input value that matches a dictionary value with the corresponding value from the valid column in the dictionary.

    The step compares the input values to every value in the dictionary. If you clear the option, the step does not standardize any input value that matches a dictionary value.
12. Valid dictionary column name.
    Identifies the valid column in the dictionary that you select. The valid column contains the preferred versions of the values in the dictionary. The valid dictionary column is active when you select the standardization option.
13. Import file option.
    Imports the first column of data from a comma-separated file to the test panel.

Regular expression options on the Configuration tab

When you add a step to a parse asset, you are prompted to define a step with a regular expression or with a dictionary.

The following image shows the options that you configure when you define a step with a regular expression:



The regular expression options includes the following properties:

1. Add Step option.
   Adds a step to the asset. A step describes a parsing operation that a mapping can apply to an input data field.
2. Up and Down options.
   Moves a step that you select up or down within the step sequence.
3. Step sequence.
   Defines the order in which a mapping applies each step to the input field at run time. The mapping performs regular expression parsing operations in the order that you specify.
4. Option description.
   Identifies the type of regular expression in the step. A custom regular expression is a regular expression that you define.
5. Test input field.
   Contains the string that the Secure Agent uses to test the steps in the parse sequence.
6. Test output fields.
   Contain the result of the test. The asset displays an output field for each step and also displays output fields for overflow dat and unparsed data.
7. Import file option.
   Imports the first column of data from a comma-separated file to the test panel.
8. Step type.
   Identifies the type of step to which the properties apply. The step type can be dictionary or regular expression.
9. Regular expression type.

   Specifies whether the step uses a built-in regular expression or a regular expression that you define.
10. Regular expression field.

In Custom mode, the field contains the regular expression that you specify. You enter the regular expression logic in the field.

In Built-in mode, the field displays the name of the regular expression that you select. Additional fields display the regular expression logic and the number of output fields that you must create.

11. Output name(s).
One or more outputs to which the step can write each input data value that matches the regular expression logic. You enter the output names.

    The mapping creates the outputs on the Parse transformation at run time. If you expect an input field to contain more than one value that matches the regular expression, create an output for each value. For example, you might add two outputs if you will use the step to find data values that occur in pairs, such as map coordinates.

Configuring a dictionary step

You can configure a step to compare the values in an input field to the values in a dictionary. The step writes matching values to one or more output fields.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.
2. Select **Dictionary** as the step type.
3. In the **Dictionary Properties** pane, select a dictionary.

   Use a dictionary that contains a range of data values that you expect the step to find at run time.

   You add a dictionary to a parse asset, and then add the asset to the Parse transformation in a mapping.

   The step compares each value in an input field to every value in the dictionary that you select, regardless of the number of columns in the dictionary.
4. In the **Outputs** field, enter the name of the field to which the step will write any input value that matches a dictionary value. The step creates the field at run time.

   Press the Enter key after you enter the field name.
5. Select or clear the **Casing** option.

   If you select the option, the step searches for values in the input fields that match both the spelling and the character case of the values in the dictionary. By default, the step ignores the character case of the values in the input fields.
6. Select or clear the **Standardization** option.

   If you select the option, the step returns the valid value from the dictionary for any input value that matches a dictionary value.
7. If you select the Standardization option, use the **Valid Column** options to identify the dictionary column that contain the valid values.

   The valid column contains the standard or preferred version of a value in cases where the dictionary contains more than one value on a given row.

   You can specify any dictionary column as the valid column within a step. To use the column that the dictionary designer set as the valid column, click **Use Default**.
8. Save the asset.

Configuring a regular expression step

You can configure a step to apply a regular expression to the values in an input field. The step writes any value that matches the expression logic to one or more output fields.

1. On the **Configuration** tab, select the option to add a step to the asset.

   The **Add Step** dialog box opens onscreen.

2. Select **Regular Expression** as the step type.
3. Select the Built-in or Custom option.
   - To select a regular expression from a list of built-in expressions, select Built-in.
   - To enter a regular expression that the step will use, select Custom.

   Select or enter a regular expression that describes the character composition of the values that you want the step to find at run time.
4. In the **Outputs** field, enter the name of the field to which the step will write any input value that matches the regular expression logic. The step creates the field at run time.

   Press the Enter key after you enter each field name.

   You can create more than one output. Create additional outputs if the step might find more than one value that matches the expression logic within a single input field.

   A built-in regular expression might require more than one output. When you select the Built-in option, the Number of Outputs field displays the number of outputs to create.
5. Save the asset.

CHAPTER 15

Pattern-based parsing operations

A pattern-based parsing operation reads an input string that contains multiple values and parses values to output fields based on the information that they contain. To define a pattern-based parsing operation, configure a parse asset in **Pre-built** mode.

Pattern-based parsing compares the values in the input string to a set of patterns that are built into the asset. The parsing operation selects values from the input string based on their similarity to the values in the patterns. The parsing operation additionally considers the location of each value relative to the other values that it parses from the inputs string.

You can enhance the pattern parsing logic in the asset that you configure. To enhance the logic, add one or more patterns to the asset. Use the test panel options in Pre-built mode to identify the patterns that you might add.

Parse assets support pattern-based parsing for person name data. Use pattern-based parsing to identify different types of person name data in an input string and to create an output data structure that can assign different name values to appropriate fields.

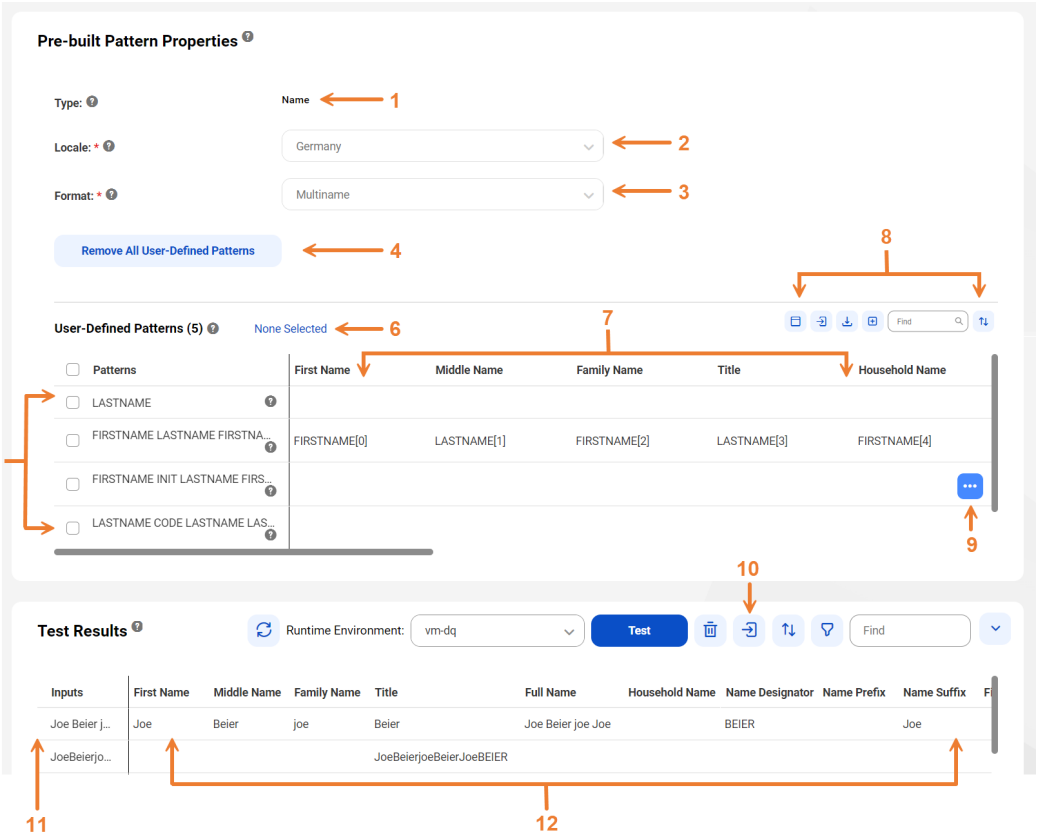
Note: Pattern-based parsing uses dictionaries to identify person name values and to select the appropriate output field for the values. Informatica provides the dictionaries in an asset bundle. To enable pattern-based parsing operations, install the bundle.

For more information on bundle installation, see “Installing the asset bundle for pattern-based parsing” on page 109.

Pattern-based parsing options on the Configuration tab

When you select Pre-built mode in the Configuration tab, Data Quality prompts you to specify the criteria for person name parsing.

The following image shows the options that you configure when you define the parsing operation:



The pattern-based parsing options include the following properties:

1. Type
   Identifies the type of information that the parsing operation analyzes. Pattern-based parsing analyzes person name information.
2. Locale
   Indicates the country of origin of the data that the parsing operation analyzes.
3. Format option
   Specifies the format of the input data that the parsing operation will read. The format indicates the sequence in which the parsing operation expects to read the input data values.

   The parsing operation applies the built-in data patterns based on the format that you select.
4. Add or Remove User-Defined Patterns option
   Adds patterns that you define to the built-in patterns that the current operation uses. Or, removes all user-defined patterns from the current operation.

   The asset imports user-defined patterns from a CSV or Microsoft Excel file that you specify.
5. User-Defined Patterns

Lists any pattern that you add to the current parsing operation.

6. Pattern selection menu
   Indicates the number of patterns that you select. You can use the menu to delete the rows or to erase the field mappings that you defined for the rows.
7. Pattern input fields
   Displays the fields to which you can map the values in a user-defined pattern. Select the appropriate field for each value in any pattern that you add. The **Configuration** tab displays the fields based on the locale and format you select.
8. Pattern data options
   Provides options to switch to full-screen mode, to import a pattern, to download a pattern as a Microsoft Excel file, to add a row to a pattern, to search for a pattern, and to sort the rows in your pattern by the output column names.
9. Row selection menu
   Opens a set of options that you can use delete the row or to erase the field mappings that you defined for the row.

   For more information about the pattern selection and row selection options, see "Pattern selection and row selection options" on page 112.
10. Import test data option
    Imports data that the Secure Agent can test.

    The asset imports the test data from a CSV or Microsoft Excel file that you specify.
11. Test input fields
    Displays the test data that you import.
12. Test output fields
    Displays the result of any test that you run.

Configuration summary for pattern-based parsing

To perform pattern-based parsing, create a parse asset and select the **Pre-built** parsing mode.

Optionally, test the asset configuration. Use the test results to update the pattern logic that the asset uses.

Prerequisites

Before you perform pattern-based parsing, verify that the *CDQ_Name_Parsing_Reference_Data_Bundle* is present in the **Add-On Bundles** folder in Explorer.

Testing strategies for pattern-based parsing

You can test the asset before you use it in a mapping and after you use it in a mapping. For example, you might test the asset and update the pattern data before your first mapping run. You might then use the results of the first mapping run to update the pattern data so that the mapping parses the source data more comprehensively.

When you test the asset before you run a mapping, you add one or more patterns from the test results to the pattern-based logic in the asset. When you test the asset with the results of a mapping, you can add both the patterns and the names that the patterns identify to the pattern-based logic in the asset.

Process flow

The following steps summarize the configuration process:

1. If necessary, install the *CDQ_Name_Parsing_Reference_Data_Bundle* asset bundle.
2. Select **Pre-built** parsing mode.
3. Select the locale and the format of the data that the operation will read.

   Note: After you verify the locale and the data format, the asset is ready to use in a Parse transformation. You can use the steps that follow to enhance the pattern-based logic that the asset applies to your data.

   You can test and enhance the pattern-based logic before you add the asset to the Parse transformation. Or, you can run a mapping that contains the transformation and use the mapping results to update the pattern-based logic.
4. Test a sample of your data in the parse asset. Enter the values that you want to test, or import a file that contains the values.

   You can import a file and perform the test on your data in the following ways:

   - Import a file that contains name data, and run the test. You might perform this step to enhance the pattern logic before you add the asset to a transformation in a mapping.
   - Import a file that includes both names and patterns, and run the test. You might perform this step to enhance the pattern logic with the results of the mapping that you ran.
5. Review the results of the test.

   Find any name that the test failed to parse. Copy the pattern for each name to a CSV or Microsoft Excel file. Optionally, copy the name along with the pattern.
6. Import the file that contains the pattern data and optionally the name data to the asset. Use the **Add User-Defined Patterns** option to import the file data.
7. Map the values in each pattern to appropriate fields in the user-defined pattern grid. Then, save the asset.

   Note: When you map a pattern to the appropriate fields, you train the asset to recognize names with a structure that matches the pattern.
8. Run the test again and review the results.

   You may decide to import additional patterns to the asset in order to further improve the pattern parsing logic.

When you are satisfied with the performance of the parsing operation on your sample data, save the asset. A user can then add the asset to a Parse transformation in a mapping and run the mapping on your data.

Installing the asset bundle for pattern-based parsing

Before you perform pattern-based parsing, verify that the dictionaries that the parsing operation reads are installed for your organization. Informatica provides the dictionaries in a bundle named *CDQ_Name_Parsing_Reference_Data_Bundle*. Find the bundle in the **Add-On Bundles** folder on the Explore page.

If the bundle is not present, install the bundle. You install the bundle in the Administrator.

Note: *CDQ_Name_Parsing_Reference_Data_Bundle* contains 84 dictionaries. If you find the bundle on the Explore page and the bundle contents are incomplete, uninstall the bundle from the **Add-On Bundles** page in Administrator. Then, install the bundle.

To install a bundle, perform the following steps:

1. In Administrator, select **Add-On Bundles**.

2. Click **Available Bundles**.

   The Available Bundles tab lists the public and private bundles that are available for installation or copying.
3. If the bundle that you want to install is an unlisted bundle, enter the bundle access code in the **Find** field.
4. Click the bundle name to open the Bundle Details page.
5. Verify that the **Allow** field is set to **Reference** or to **Reference and Copy**.

   You cannot install a bundle that is configured for copying only.
6. Click **Install**.

Administrator displays a notification to indicate the status of the installation.

You can find the installed bundle name on the **Installed Bundles** tab of the **Add-On Bundles** page in Administrator. On the **Explore** page, the bundle is added to the **Add-On Bundles** project.

Configuring the pre-built options for pattern-based parsing

If your data matches the default settings in **Pre-built** mode on the parse asset, you can save the asset for use in a Parse transformation with minimal configuration.

1. On the **Configuration** tab, select the **Pre-built** parsing mode.
2. In the **Pre-built Pattern Properties** pane, configure the following options:
   - Locale. Specifies the country to which the input data belongs. The configuration specifies the United States by default.

     Note: Each locale has a unique set of output fields.
   - Format. Specifies the format of the data that the operation will read. The default format is *Last Name, First Name, Middle Name*.
3. Save the asset.

After you complete the configuration steps, test the asset configuration.

Testing and updating the pattern-based parsing configuration

Test a parse asset to verify that data flows through the asset in the ways that you expect. You can then update the asset with the pattern for any name that the test failed to parse. The asset stores the patterns that you add and includes them in the pattern logic that the Parse transformation uses at run time.

You can update the asset with pattern data, and you can update the asset both with patterns and with the names in your input data that are associated with the patterns.

You might update an asset exclusively with pattern data when you test the asset before you add it to a Parse transformation. You might update an asset with both pattern data and name data after you run a mapping with the Parse transformation and review the output from the transformation.

Iterative testing

Testing the asset and updating the pattern-based logic for a source data set can be an iterative process. You might test the asset first with a sample of your source data and later test the asset with the name data and associated patterns that you read from the transformation output data.

Testing the parsing configuration with name data

The following steps describe the process to test the parse asset with name data that you import. When you test the data, you might find names that the asset does not parse. You can then add the patterns for the unparsed names to the asset logic.

1. Open the parse asset that you created for pattern-based parsing.
2. Select the **Configuration** tab.
3. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
4. Import a sample of the source data that the Parse transformation will run on.

   Use the Import option in the **Test Results** pane to import the data from a CSV or Microsoft Excel file. The sample data must populate the first column in the file.

   The input data appears in the **Inputs** column.
5. Click **Test**.

   The output columns display the test results.
6. Review the results of the test:
   - Find any input name that the test did not parse. Look for input names in the **Unparsed Data** field.
   - If you find a name that did not parse, make a copy of the pattern that the test assigned to the name.

     The test writes the pattern for each name to the **Labeled Name** field. Copy the pattern from the Labeled Name field to a CSV or Microsoft Excel file.
7. Import the file that contains the pattern to the asset.

   Use the **Add User-Defined Patterns** option to import the data. The patterns must populate the first column in the file.

   You can import up to 5,000 user-defined patterns to a parse asset from a file that you specify.
8. Map the values in each pattern to an appropriate field in the pattern grid. For each value, select the field that best matches the type of information that the value represents.

   Use the number in each pattern value as a guide when you map the values. The numbers match the order in which the data values appear in the input row. The numbered values in each pattern begin at (0).
9. After you map the pattern values to the appropriate fields, run the test again and review the results.

   You may decide to import additional patterns to further update the pattern parsing logic.
10. When you are satisfied with the performance of the parsing operation on your sample data, save the asset.

Testing the parsing configuration with pattern and name data

Test the parse asset with pattern data and associated name data.

To import the names and associated patterns, copy the data to a CSV or Microsoft Excel file. The pattern values must populate the first column in the file, and the name values must populate the second column.

1. Open the parse asset that you created for pattern-based parsing.
2. Select the **Configuration** tab.
3. Select a runtime environment in which to test the configuration.
4. Use the **Add User-Defined Patterns** option to import the unparsed name and pattern data that the Parse transformation wrote as output.

   You can import up to 5,000 user-defined patterns to a parse asset from a file that you specify.

When you select the file that contains the data, the **Import Patterns** dialog box opens. Select the **Import Input Data** option in the dialog box.

The pattern data appears in the **User-Defined Patterns** pane and the name data appears in the Inputs column of the **Test Results** pane.

5. Map the values in each pattern to an appropriate field in the pattern grid. For each value, select the field that best matches the type of information that the value represents.

   Use the number in each pattern value as a guide when you map the values. The numbers match the order in which the data values appear in the input row. The numbered values in each pattern begin at (0).

6. Click **Test**.

   The output columns display the test results.

7. Review the results of the test:

   - Find any input name that the test did not parse. Look for input names in the **Unparsed Data** field.
   - If you find a name that did not parse, make a copy of the pattern that the test assigned to the name. The test writes the pattern for each name to the **Labeled Name** field.

     You can copy the patterns from the Labeled Name field to the file that you previously imported, or you can copy the patterns to a new file. Bear in mind that when you import data to the asset, you erase the prior test and pattern data on the Configuration tab.

8. Import the file that contains the latest pattern and name data.
9. Map the pattern values to the appropriate fields in the pattern grid.
10. Run the test again and review the results.

    You may decide to import additional patterns to further update the pattern parsing logic.

11. When you are satisfied with the performance of the parsing operation on your sample data, save the asset.

For more information about importing files, see "Rules and guidelines for test data" on page 117.

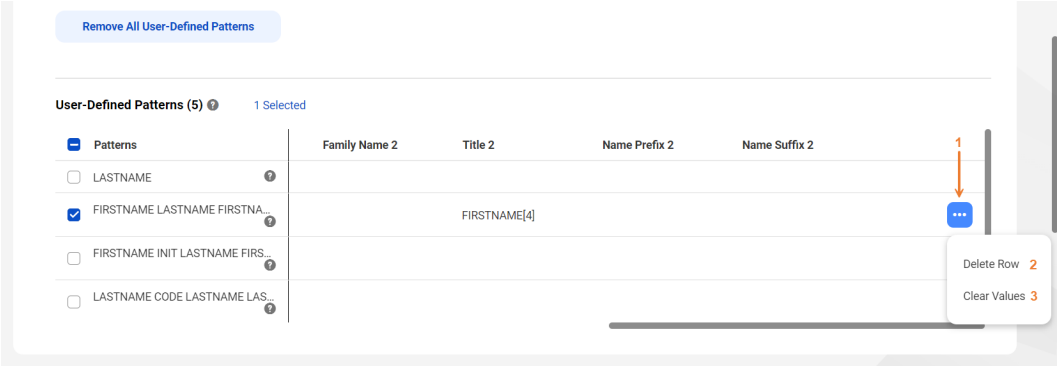
For more information about the meaning of the pattern label values, see "Rules and guidelines for user-defined pattern labels" on page 114.

Pattern selection and row selection options

When you import data to the **User-Defined Patterns** list, you can use the pattern and row selection options to undo any pattern match that you define and to delete unwanted rows of data from the list.

Use the row selection options to clear the pattern matches from a single row or delete a single row. Use the pattern selection options to clear the pattern matches from multiple rows or delete multiple rows in a single action.

The following image shows the row selection options:



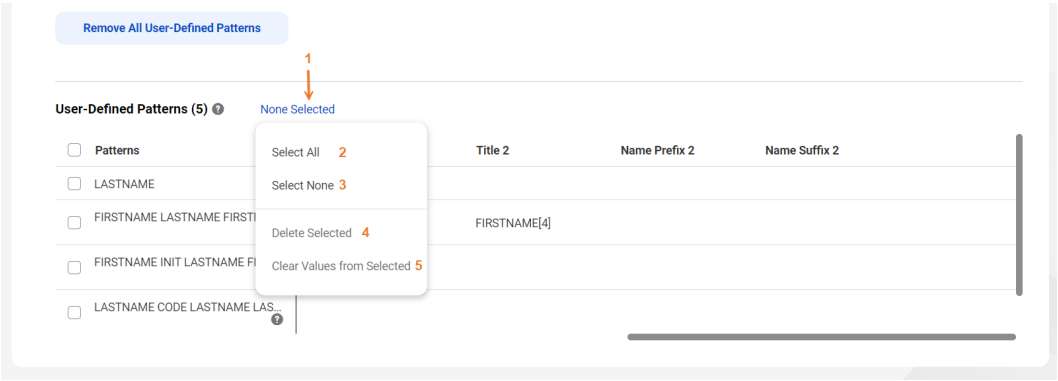
The row selection options include the following properties:

1. Row properties option
   Opens the menu of row selection options.
2. Delete Row
   Deletes the current row of data.

   Clear Values

   Removes all pattern matches that you selected in the current row. The option does not delete the pattern that you imported.

The following image shows the pattern selection options:



The pattern selection options include the following properties:

1. Pattern properties option
   Opens the menu of pattern selection options. The option also displays the number of rows that you selected.
2. Select All
   Selects all rows under User-Defined Patterns.
3. Select None
   Delesects all rows under User-Defined Patterns.

   Note: You can also use the check boxes beside each row to select or deselect one or more rows.
4. Delete Selected
   Deletes the rows of data that you selected.
5. Clear Values from Selected
   Removes all pattern matches from the rows that you selected. The option does not delete the patterns that you imported.

Rules and guidelines for user-defined pattern labels

When you create a user-defined pattern, you must use labels that the parse asset recognizes.

Consider the following rules and guidelines when you add labels to a user-defined pattern:

- Use label values that appear in the **Labeled Name** field in the test panel.
- Your pattern can contain a label that represents more than one token in a name. For example, you might define a label with two values to represent a hyphenated last name. Use the **Custom Token** label option to define the label.
- The following table describes the labels that the parse asset recognizes:

Label	Description
FIRSTNAME	First name of a person.
LASTNAME	Family name.
MIDDLENAME	Middle name of a person.
LN_ID	Affix to family name. For example, Mac, Van, al.
INIT	Initial, or single alphabetic character.
TITLE	Occupation name.
SUFFIX	Person name suffix, such as a professional qualification.
INITSET	Two or more initials separated by spaces or periods.
PREFIX	Person name prefix. For example, DR, MISS.
DESIGNATOR	Designated recipient identifier. For example: CARE OF.
HOUSEHOLD	Household identifier. For example: Current Resident.
WORD	Alphabetic string.
WORDSYMBOL	Combination of one or more alphabetic characters and punctuation symbols.
CONNECTOR	Conjunction. For example: and, or.
CODE	Alphanumeric string.
NUMBER	Numeric string.
SYMBOL	Punctuation symbol.
UNDEFINED	Undefined string.
CONNECT	Conjunction. For example: and, or.
dq_dash	Dash symbol in a Spanish name.

Label	Description
dq_comma	Comma symbol in a Spanish name.
dq_quote	Quote symbol in a Spanish name.
dummy	Value that is not associated with a name. e.g. Generico.
found	Portuguese name prefix.
COMMA	Comma symbol.
#DASH	Dash symbol.
#FNDOLN	Family name that begins with [O'].
#!FND	Family name affix [O'] without an apostrophe.
amb	Ambiguous Spanish name. May be first or family name.
apell	Spanish family name.
nom	Spanish first name.
inicial	Initial letter in a Spanish name.
Custom Token	A set of multiple tokens that the parsing operation can apply to an output field.

CHAPTER 16

Test parse assets

Test a parse asset to verify that the asset logic generates the results that you expect.

Note: The steps to test a parse asset in pattern-based parsing mode are integral to the process flow for pattern-based parsing. To read more about the steps test the asset in pattern-based parsing mode, see "Testing and updating the pattern-based parsing configuration" on page 110.

Testing a parse asset in Custom mode

Test a parse asset to verify that the data flows through the asset in the ways that you expect.

1. Open the parse asset.
2. Select the **Configuration** tab.
3. Select a step in the step sequence.

   You will test the logic in the sequence up to the step that you select. To test the logic in the complete asset, select the final step in the sequence.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter one or more data values in the input column, or import the data to test. To import the data, click the **Import** option in the test panel.

   For more information on importing data, see "Rules and guidelines for test data" on page 117.
6. Click **Test**.

   The test runs the steps in the asset in the following sequence:

   - The test runs all dictionary steps in the order in which they appear in the step sequence.
   - The test runs all regular expression steps in the order in which they appear in the step sequence.

   The output columns display the test results.
7. Verify that the steps read the input values and write the output values that you expect.

Rules and guidelines for test data

You can import data to the test panel in the parse asset and save the test data in the asset configuration.

Consider the following rules and guidelines when you add data to the test panel in a parse asset:

- The import option supports CSV and Microsoft Excel files.
- You can import up to 200 consecutive rows of data from a delimited file. You can specify the row at which the import starts.

  Note: Before you import, check the file for column headings. If the first row in the import file contains column headings, start the import at line 2 or lower.
- The **Import** option on the test panel and the **User-Defined Patterns** option can import up to 10 MB of data from a file that you specify.
- The Maximum string length property applies to the data that you import or enter in the test panel. Import or enter a string on each row that does not exceed the value on the property.
- If you import a CSV file that contains multiple columns or uses a text qualifier, verify that the file uses a delimiter or a text qualifier that the Secure Agent recognizes. By default, the *Comma* option is the delimiter for the column data. By default, the *No quotes* option is the text qualifier for the data. You can update the delimiter and text qualifier characters when you select the data to import. The Delimiter and Text Qualifier options are not required when you import a Microsoft Excel file.
- The Secure Agent saves the data that you import to the asset when you save the asset. If you change an option in the asset configuration, you may lose any unsaved test data.

Part VI: Rule specification assets

This part contains the following chapters:

- Introduction to rule specifications, 119
- Rule specification configuration, 126
- Rule set configuration, 137
- Rule statement configuration , 143
- REST APIs for rule specifications, 165
- Rule specification configuration in advanced mode , 166
- Business rules and rule statements, 173
- Validation and testing, 191

CHAPTER 17

Introduction to rule specifications

A rule specification asset represents the data requirements of a business rule in logical form.

A rule specification breaks down the logic in a business rule into the following tasks:

- Define the types of data that a business data set contains.
- Define a set of conditions that the business data must satisfy.
- Define the actions to take when the data satisfies the conditions of the business rule.
- Define the actions to take when the data fails to satisfy the conditions of the business rule.

You create and test a rule specification in Data Quality or Data Integration, and then add the rule specification to a Rule Specification transformation in a mapping in Data Integration. In Data Quality, you create a rule specification from the **New** dialog box. In Data Integration, you create a rule specification either from the **New Asset** dialog box or from a mapping.

When the mapping runs, the Rule Specification transformation applies the conditions and actions that you define to the values in each row in the input data. The outputs from the transformation indicate whether the row data meets the requirements that you set in the business rule.

Rule specification modes

You can configure a rule specification in basic mode or advanced mode. By default, a rule specification displays the configuration options in basic mode.

Configure a rule specification in basic mode to define business rule logic using the set of graphical elements that the asset provides. Use basic mode if you are not familiar with text-based conditional rule logic. You might also use basic mode if the business rule that the rule specification will represent is easily transferrable into a graphical format.

Configure a rule specification in advanced mode to represent the business rule in expression logic in a text editor. You can use simple expressions, you can use IF-THEN-ELSE syntax to build the rule.

To switch from basic mode to advanced mode, click the **Convert to Advanced** option on the asset toolbar.

Note: You cannot switch to basic mode from advanced mode.

Rule specification structure

A rule specification asset contains options on a **Definition** tab and a **Configuration** tab.

Use the **Definition** tab options to enter a name for the asset, optionally enter a description for the asset, and to select the folder in which to store the asset. Use the **Configuration** tab options to configure the rule specification logic.

Configuration tab options in basic mode

In basic mode, the **Configuration** tab contains the **Design** panel and the **Properties** panel.

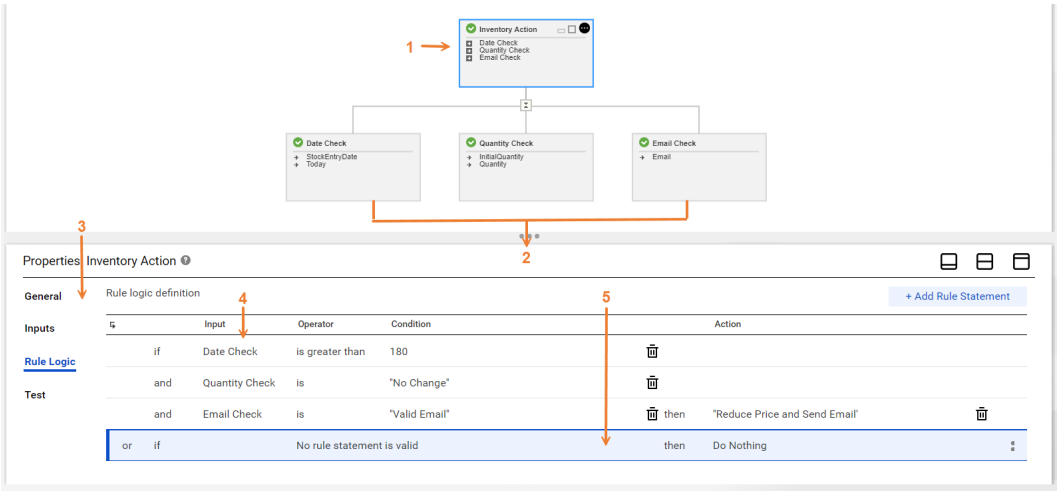
You can expand or collapse the **Design** panel to show or hide the rule sets that you add for configuration. You might collapse the **Design** panel when you configure a rule set with a rule statement, so that the rule set configuration options are easier to view.

When you create a rule specification, you configure a series of shapes in the **Configuration** tab. The shapes are called rule sets. Each rule set describes an aspect of the business rule.

A rule set contains one or more rule statements that define the business rule requirements at a low level. Each rule statement reads a column of input data and verifies that the input data meets the conditions that you specify.

The top-level rule set is the primary rule set. The primary rule set summarizes the business rule. The output from the primary rule set specifies whether each row of input data meets the overall requirement of the business rule.

The following image shows a rule specification in the **Configuration** tab in basic mode:



The rule specification contains the following elements:

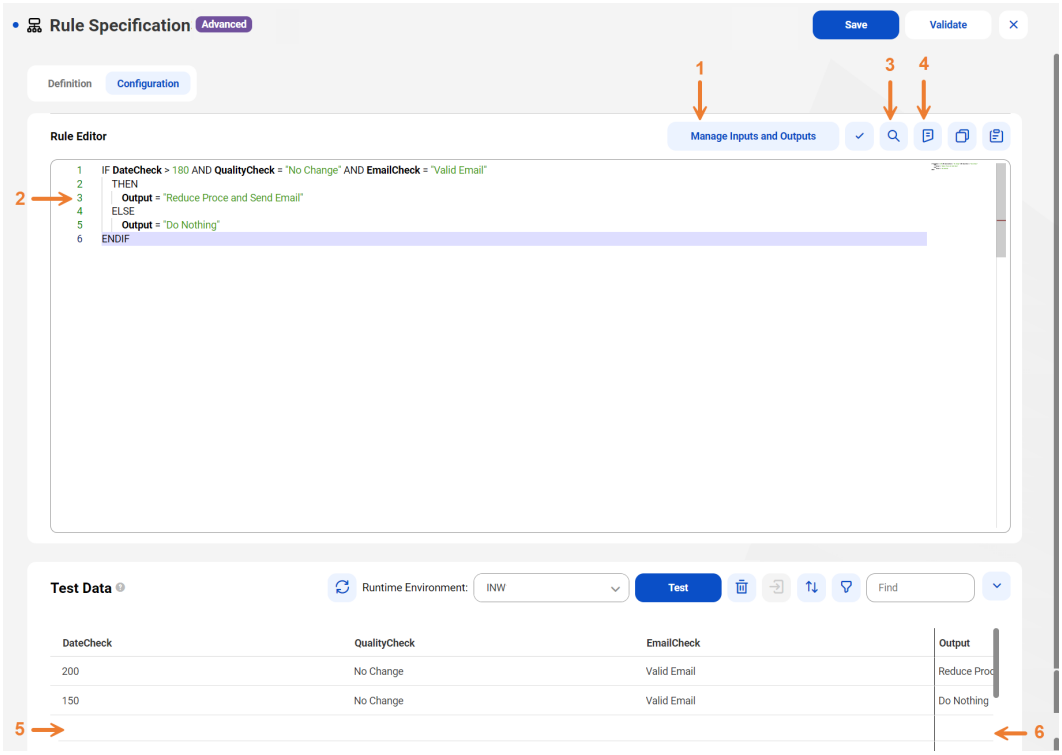
1. Primary rule set.
2. Child rule sets of the primary rule set.
3. Property options on the rule set that you select.
4. Rule statements in the rule set that you select.
5. Rule statement that the system defines.

Configuration tab options in advanced mode

In advanced mode, use the **Configuration** tab to write rule logic in simple expressions or in IF-THEN-ELSE syntax.

The **Configuration** tab in advanced mode contains an editor that allows you to directly write your business rule in expression logic.

The following image shows a rule specification in the **Configuration** tab in advanced mode:



The Configuration tab includes the following options:

1. Manage Inputs and Outputs.
   Opens the input and output configuration options. Use the options to add one or more inputs and outputs.
2. Rule Editor.
   Provides a canvas in which you can write the rule logic.
3. Search and replace.
   Searches and replaces a value that you specify.
4. Select all.
   Selects the entire script in the editor panel.
5. Test input field.
   Contains the input data that the Secure Agent uses to test the expression logic that you enter.
6. Test output field.
   Contains the result of the test.

Rule sets

Rule sets define the logical flow of data through the rule specification. You organize the rule logic in one or more rule sets when you configure a rule specification in basic mode. Data flows upward through the rule specification from the lowest rule set to the primary rule set. The number of rule sets in the rule specification depends on the logical requirements of the business rule. A valid rule specification might have a single rule set.

You can add a rule set below any rule set in the rule specification. The rule sets have a parent-to-child relationship. When you add a rule set, the output of the child rule set becomes an input to the parent rule set.

Note: You must select the output from any child rule set as an input in a rule statement in the parent rule set.

Rule sets contain the rule statements that analyze and update the input data. You can configure a rule set with a single rule statement, or you can add multiple rule statements to the rule set. Within a rule set, data flows from the first rule statement to the final rule statement.

You can copy or move a rule set to another location in a rule specification, and you can copy or move a rule set to another rule specification.

Inputs and outputs

An input describes a field of data that a rule statement can analyze in basic mode or that expression logic can analyze in advanced mode. An output represents the result of an operation within the rule specification. Inputs and outputs are specific to the rule specification in which you create them.

Define an input to represent a column in a data set that the rule specification will analyze at run time. Define an output to represent a column of data that the rule specification will generate at run time. In basic mode, a rule set generates an output. You define the outputs in advanced mode. The outputs that you define in advanced mode correspond to the results of a rule set in basic mode.

When you define an input or output, you can set the following properties:

- The input or output name.
- The data type of the data that the input or output represents. You can create an input with a string, date/time, float, or integer data type. You can additionally create an input of double data type in advanced mode.
- The maximum number of characters that a value in the input or output field can contain.

You can specify an integer data type for numbers in the range -2147483648 through 2147483647. To read numbers that are outside the integer range, use the float or double data type.

You can set the scale, or the number of digits that can follow a decimal point, in basic mode. In a float value, the default scale is 4. In a date/time value, the scale is preset to 9. You cannot change the scale in a date/time value.

The properties in basic mode also include a Usage value. The Usage value indicates the number of times that the input appears in a rule statement in the rule specification.

To view the inputs that a rule set uses in basic mode, select the rule set on the Configuration tab.

Rules and guidelines for inputs

Consider the following rules and guidelines for rule specification inputs:

- Create an input in basic mode to add the input to a rule statement.
- When you create a child rule set, the output from the rule set becomes an input to the parent rule set. You must use the input in a rule statement in the parent rule set.

- An input in a rule specification asset does not store information about business data, such as the name of a source column, table, or file.
- When you add a rule specification asset to a Rule Specification transformation in a mapping, you can connect a rule specification input to any column of data that is compatible with the input properties.

Input data types and Amazon S3 connections

You might use a Rule Specification transformation in a mapping that connects to a file source over an Amazon S3 V2 connection. For example, you might run such a mapping in Data Integration Elastic.

Consider the following factors before you configure the rule specification asset that the Rule Specification transformation will use:

- A mapping that uses an Amazon S3 V2 connector cannot process a date/time input on a rule specification.
- A mapping that reads a source file over an Amazon S3 V2 connector can read a string data type without additional configuration, as the string data type corresponds to the Amazon S3 STRING data type.

Rule statements

In basic mode, a rule statement is a set of operators, conditions, and actions that analyze a column of data and generate an output based on the result of the analysis. You add a rule statement to a rule set.

A condition is a data operation that determines a single fact about a data value. You can add multiple conditions to a rule statement. An action is a data operation that generates a potential output from the rule set. An action generates data when the input that you add to the rule statement satisfies the conditions that you define.

A rule specification reads the rule statements in a rule set from top to bottom. For a given row of input data, the rule set accepts the output from the first rule statement that generates output data.

Each rule set contains a system-defined rule statement that specifies the action to take if no other rule statement generates output data. The system-defined rule statement is the final rule statement in the rule set. You can edit the action in the system-defined rule statement. By default, the rule statement specifies that the rule set does not generate any output data if the other rule statements do not generate output data.

In advanced mode, you can write the equivalent rule in expression logic.

Status values in rule statements

The actions that a rule statement can perform include the generation of status values. A status value is a predefined value that a rule statement can generate as the output from an action. You can configure an action to generate a status value as an alternative to user-defined value.

A rule statement can return *Valid* or *Invalid* as status values. You cannot modify the status values.

You can use status values to achieve the following objectives:

Provide data to profiles

Scorecards in Data Profiling can recognize status values. When a rule specification returns a status value as an output, the scorecard can report the number of status values as a data quality category. To enable the scorecard to read the rule specification output, add the rule specification as a rule to the profile that generates the scorecard.

Provide information to downstream users about exception records

You can configure a rule specification to identify a record as an exception. An exception is a record that contains unresolved data quality issues.

To identify records as exceptions in basic mode, configure a rule statement to return the status value *Invalid*. Define the exception properties on a rule statement in the primary rule set.

To identify records as exceptions in advanced mode, configure exception indicators for one or more status values that you define in the rule logic that you write. In advanced mode, you can define custom status values that more closely describe the data quality issue in each exception record. You can associate each status value with a corresponding data quality issue. You can configure a different set of exception properties for each status value.

Rule specifications and exception management

An exception is a record that contains unresolved data quality issues. You can configure a rule specification to find exception records as part of an exception management process. In an exception management process, you add a rule specification that you configured for exception management to a profiling task. You then create an exception task from the profiling task.

A rule specification can add data to each exception record that describes the type of data quality issue that the record represents. The exception data properties include an output field for status values that can summarize the data quality issue that the rule logic identifies in a record. The properties also include an output for values that indicate the priority of each issue and an output to contain a description of each issue. The output values allow a downstream user to find exception records and to understand the reasons why a record is an exception.

For more information about the exception data properties, see "Exception data properties" on page 154.

Rule specifications and mappings

The rule specifications that you create are available to the users who create mappings in Data Integration. You add a Rule Specification transformation to a mapping and then add a rule specification asset to the transformation. The Rule Specification transformation applies the logic in the asset to the input fields that you specify.

After the mapping runs, you can evaluate the data output to determine if the data conforms to the business rule that the rule specification describes.

Rules and guidelines for rule specifications and mappings

Consider the following rules and guidelines when you work with mappings and rule specifications:

- A rule specification is a reusable object. You can add a rule specification to transformations in multiple mappings. A Rule Specification transformation is a nonreusable object. You select a Rule Specification transformation when you configure a mapping.
- To view the list of rule specifications that you created, navigate to the **Explore** page.
- If you rename a rule specification that you previously added to a transformation, you must synchronize the transformation with the rule specification.

  To synchronize the transformation, open the transformation in the mapping and select the rule settings. Data Integration displays a message that prompts you to synchronize the transformation with the rule specification.

Rule specifications and dimensions

The data quality issues that you may find in your data can fall into a range of common categories. Data quality assets can identify the categories as dimensions. When you configure an asset, you can use the **Dimension** property to indicate the type of data quality issue that you want the asset to examine.

Find the Dimension property on the **Definition** tab of the asset.

For more information about data quality dimensions, see the *Introduction* help in Data Quality.

Dimensions and scorecards

A rule specification that specifies a dimension is a key element in a scorecard. If you will use a rule specification in a scorecard, set a dimension on the scorecard.

A scorecard is a graphical representation of the values across the columns in a profile. You can use scorecards to monitor changes in profile data over time. You create scorecards in Data Profiling.

CHAPTER 18

Rule specification configuration

When you configure a rule specification, you define a series of inputs, rule sets, and rule statements. Each element in the rule specification represents a data operation that interacts with the business data to validate a business rule.

Before you configure the rule specification, verify that you understand the properties of the business data and the requirements of the business rule.

Prerequisites to rule specification configuration

When you configure a rule specification, you translate the requirements of a business rule into one or more rule statements. The rule statements represent the logic that determines whether a data set conforms to the business rule.

Before you configure the rule specification, perform the following steps:

1. Verify the business rule requirements.
2. Verify the business data properties.
3. Determine the sequence of the steps in the rule.

Note: When you verify the business rule requirements, you may determine that the business rule addresses a key dimension of data quality in your data. You can optionally select the data quality dimension that your business rule represents when you configure the rule specification. For more information about data quality dimensions, see "Rule specifications and dimensions" on page 125.

Verifying the business rule requirements

Before you configure a rule specification, discuss the business rule with the data owners in the organization. Verify that the business rule is valid and ready to apply to the business data.

1. Identify the business rule that the rule specification represents.
2. Identify the business data set that the business rule validates.
3. List the business rule requirements that apply to the data inputs.

   The business rule requirements indicate the types of rule logic that you define in the rule specification.
4. Identify the information types that the business rule applies to.

   The information types indicate the data types of the inputs that you create in the rule specification.

Verifying the business data properties

Before you create a rule specification, identify the business data that the rule specification can apply to. Work with a developer or data steward to identify the data source.

1. Identify one or more data sets that you can select in a mapping that reads the rule specification. For example, identify the database and the table that contain the mapping source data.
2. Verify the data types of the data columns that the rule specification will analyze. You specify the data types when you create the inputs in the rule specification.

   Note: You might not create an input for every column in the data set.

Determine the sequence of the steps in the rule

At a high level, a business rule defines a single objective that the business data must satisfy. To represent the objective in the rule specification, you'll define at least one rule statement in basic mode or you'll write at least one expression in advanced mode. If the business rule defines more than one data outcome, you might define more complex rule logic.

In basic mode, you might add rule sets and rule statements below the primary rule set to validate the data that the primary rule set analyzes. In advanced mode, you might nest multiple IF-THEN-ELSE conditions within the primary condition that you define.

1. Create the inputs that the rule statements analyze. The inputs are a key prerequisite for the rule statements. The inputs represent the columns in the business data set that the business rule applies to.
2. Identify the business rule conditions that the business data must satisfy. Plan a rule statement or an IF-THEN-ELSE condition for each condition.
3. Determine the sequence in which the rules must analyze the data.

   Start from the lowest dependencies in the business rule, and plan a rule set or IF-THEN-ELSE condition for each dependency.

Configuring a rule specification in basic mode

In basic mode, you configure inputs, rule sets, and rule specifications to represent the data that the business rule will read and the steps that the business rule must perform. Basic mode is the default mode in the asset.

Before you create a rule specification, read the "Prerequisites to rule specification configuration" on page 126.

1. Click **New** > **Rule Specification**.

   The **Rule Specification** page opens.
2. On the Definition tab, enter a name for the rule specification.
3. Optionally, enter a description.

   Tip: Enter a summary of the underlying business rule as the description.
4. Select the location in which to save the rule specification.
5. Optionally, select a data quality dimension to represent the type of data quality issue that you want the asset to examine.

   For more information about dimensions, see "Rule specifications and dimensions" on page 125.
6. Save the rule specification.

The **Asset References** panel appears. The **Asset References** panel shows additional assets that are referenced by data quality assets to perform transformation tasks. When you create a new asset, there are no referenced assets.

7. Optionally, add a tag to the rule specification. You can search for assets with a common tag on the Explore page.
8. Select the **Configuration** tab.

   The configuration workspace appears for the rule specification. The **Configuration** tab contains the **Design** panel and the **Properties** panel.
9. Expand the **Design** panel, and configure a rule statement in the primary rule set.

   The primary rule set defines the data output from the rule specification.

   The primary rule set contains a preconfigured rule statement and reads a preconfigured input by default. Based on your business rule requirements, you can modify the default components.
10. Optionally, update the primary rule set name.
    - Select the primary rule set, and select **General**.
    - Update the rule set name.
11. Add any rule set that the rule specification requires.

    You add a rule set below another rule set.
12. Add one or more inputs to each rule set.

    Use the **Input Management** dialog box to create the inputs.

    You can modify the properties on the default input.
13. Add any rule statement that the rule specification requires.

    You can add a rule statement to the primary rule set or to another rule set. For more information on rule statements, see Chapter 20, “Rule statement configuration ” on page 143.
14. Save the rule specification.

After you configure the rule specification, test the rule specification with sample data.

Configuring a rule specification in advanced mode

In advanced mode, you define the inputs and outputs for the business rule, and you write expression logic that represents the steps that the business rule must perform.

To configure a rule specification in advanced mode, you convert the asset from basic mode. The steps you follow depend on whether you save the rule specification in basic mode before you convert to advanced mode.

If you saved the asset in basic mode, your asset configuration is converted to advanced mode. A new asset is created with the advanced configuration and maintains the original rule specification in basic mode.

If you did not save the asset in basic mode, the asset is converted to advanced mode in its default configuration.

Before you create a rule specification, read the “Prerequisites to rule specification configuration” on page 126.

Configuring a new rule specification in advanced mode

In this task, you'll create a rule specification from scratch and save it in advanced mode.

1. Create a rule specification asset.
2. On the asset toolbar, click **Convert to Advanced**.
3. On the **Definition** tab, enter a name for the rule specification.
4. Optionally, enter a description.

   Tip: Enter a summary of the underlying business rule as the description.
5. Select the location in which to save the rule specification.
6. Optionally, select a data quality dimension to represent the type of data quality issue that you want the asset to examine.

   For more information about dimensions, see "Rule specifications and dimensions" on page 125.
7. Save the rule specification.

   The **Asset References** panel appears. The **Asset References** panel shows additional assets that are referenced by data quality assets to perform transformation tasks. When you create a new asset, there are no referenced assets.
8. Select the **Configuration** tab.

   The configuration workspace appears for the rule specification.
9. Click **Manage Inputs and Outputs**.

   The **Input and Output Management** dialog box appears.

   The asset reads a preconfigured input and output by default. You can modify the properties on the default input and output and add any other input or output that the rule specification requires.
10. In the **Rule Editor** panel, write the rule logic. Build the rule logic in an IF... THEN... ENDIF structure.

    The asset adds a preconfigured rule to the editor when you begin to type. You can modify or delete the default rule.
11. Save the rule specification.

After you configure the rule specification, test the rule specification with sample data.

Converting a saved rule specification from basic mode

In this task, you'll convert a rule specification that you saved in basic mode.

1. Open a rule specification that you configured in basic mode.
2. On the asset toolbar, click **Convert to Advanced**.

   A copy of the rule specification is created in advanced mode.
3. Save the rule specification in advanced mode with a new name.

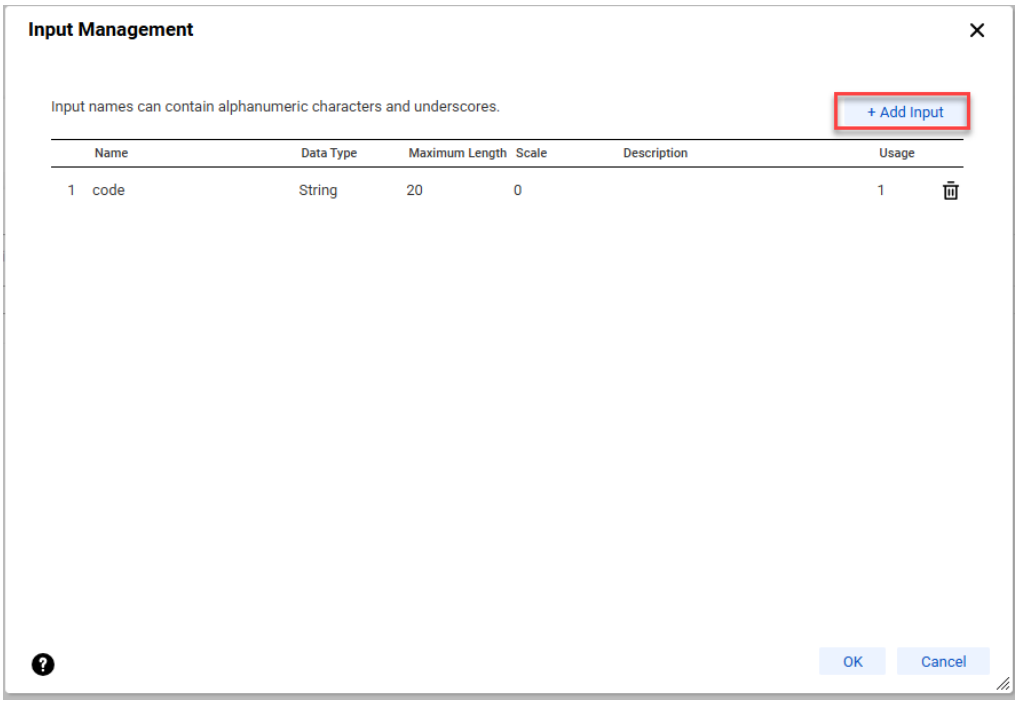
   Review the description, location, and other properties on the **Definition** tab.
4. Verify or update the configuration of the asset to ensure that the inputs, outputs, and expression logic meet your requirements.
5. Save the rule specification.

Creating an input in basic mode

When you create an input, you can add the input to any rule set in the rule specification.

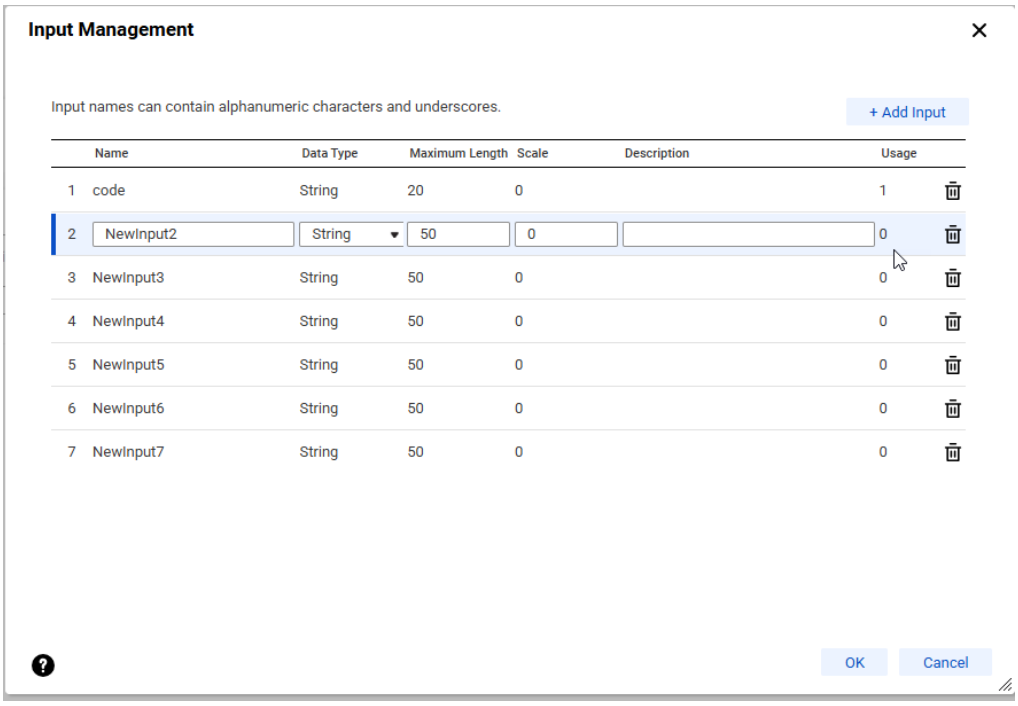
1. On the **Explore** page, open the rule specification.
2. In the toolbar, select **Manage Inputs**.

   The **Input Management** dialog box opens. The dialog box lists the current inputs in the rule specification.
3. Click **Add Input**.



You can see up to 100 inputs in the **Input Management** dialog box. Any additional inputs will not appear in the dialog box.

4. Set the following properties on the input:



- Name. The name of the input. You can enter alphanumeric characters, underscore characters, and spaces.
- Data Type. The data type of the column in the business data set that the input represents. You can select a date/time, float, integer, or string data type.
- Maximum Length. The maximum character length of the data values in the input data column.
- Scale. The number of digits that can follow a decimal point in a float value.
- Description. A text description of the input. The description is optional.
- Usage. A column that indicates the number of times that each input appears in a rule statement in the rule specification.

5. To save the input and close the dialog box, click **OK**.

Creating an input and output in advanced mode

When you create an input and output, you can add the input and output to the rule logic that you write to an editor in the rule specification.

1. On the **Explore** page, open the rule specification.
2. In the toolbar, select **Manage Inputs and Outputs**.

   The **Input and Output Management** dialog box opens. The dialog box lists the current inputs and outputs in the rule specification.
3. Select the **Inputs** or **Outputs** tab.
   - To create inputs to use in the rule logic, select the **Inputs** tab.

- To create outputs to use in the rule logic, select the **Outputs** tab.

4. Click **Add Field**.
5. Set the following properties on the field:
   - The input or output name. You can enter alphanumeric characters and underscore characters.
   - The data type of the column in the business data set that the input represents. You can select a date/time, float, integer, double or string data type.
   - The maximum character length of the data values in the input data column.
   - The number of digits that can follow a decimal point in a float value.
6. Click **OK**.

Rules and guidelines for inputs

Consider the following rules and guidelines when you create an input:

- Where possible, use input names that match to the names of the input fields that the rule specification will read in a mapping. When you add the rule specification to a Rule Specification transformation in a mapping, you can use an Automatch option to connect the rule specification inputs to transformation input fields with similar names. You can also manually connect any rule specification input to any input field on the transformation.
- When you perform a computation that involves a string data type and an integer data type in advanced mode, the asset can cast the digits in the string as an integer. If the string contains non-numeric characters in the computation with an integer, the asset assigns a value of zero to the non-numeric characters in the string. For example, if you specify a string input value of ABC and an integer input value of 10 in the expression `Output = String_Value + Integer_Value`, the asset generates an output value of 10.

  If you specify a string input value of 10 and an integer input value of 10 in the expression `Output = String_Value + Integer_Value`, the asset generates an output value of 20.

Rule specification data properties in basic mode

The rule specification properties define the data characteristics for the outputs on each rule set in the rule specification, including the primary rule set.

To view or edit the properties, select **Rule Properties** from the Actions menu on the toolbar.

You can define the following properties:

- Maximum string length. Determines the maximum length for text data. Default is 100.
- Maximum number length. Determines the maximum length for numeric data. Default is 10.
- Number of decimal places. Determines the maximum number of decimal places to include in the output data. Default is 4.

Rule specification data properties in advanced mode

The rule specification properties in advanced mode define the data characteristics for the outputs on each rule set in the rule specification.

To view or edit the properties, select **Rule Properties** from the Actions menu on the toolbar.

You can define the following properties:

- Maximum string length. Determines the maximum length for text data. Default is 2048.
- Maximum number length. Determines the maximum length for numeric data. Default is 28.
- Number of decimal places. Determines the maximum number of decimal places to include in the output data. Default is 8.

To use your own property values instead of the default settings, select the **Override Default Precision/Scale** check box.

Rule specification functions

The rule specification asset includes a range of functions that you can use in rule logic in basic and advanced modes.

You can use the following functions in basic and advanced modes:

- ABS
- ADD_TO_DATE
- ASCII
- CEIL
- CHOOSE
- CHR
- CHRCODE
- CONCAT
- CONTAINS
- CONVERT_BASE
- COS
- COSH
- CRC32
- CUME
- CURDATE
- DATE_COMPARE
- DATE_DIFF
- DATECONVERT
- DECODE
- EXP
- FLOOR

- FV
- GET_DATE_PART
- GREATEST
- IFF
- IN
- INDEXOF
- INITCAP
- INSTR
- IS_DATE
- IS_NUMBER
- IS_SPACES
- ISNULL
- LAST_DAY
- LEAST
- LEFTSTR
- LENGTH
- LN
- LOG
- LOWER
- LPAD
- LTRIM
- MAKE_DATE_TIME
- MD5
- METAPHONE
- MOD
- MONTHCOMPARE
- MOVINGAVG
- MOVINGSUM
- NPER
- PMT
- POWER
- PV
- RAND
- RATE
- REG_EXTRACT
- REG_MATCH
- REG_REPLACE
- REPLACECHR
- REPLACESTR

- REVERSE
- RIGHTSTR
- ROUND
- RPAD
- RTRIM
- SET_DATE_PART
- SIGN
- SIN
- SINH
- SOUNDEX
- SQRT
- SUBSTR
- SYSTIMESTAMP
- TAN
- TANH
- TIMECOMPARE
- TO_CHAR
- TO_DATE
- TO_DECIMAL
- TO_FLOAT
- TO_INTEGER
- TRUNC
- UPPER
- XOR

You can use the following functions in advanced mode only:

- MIN
- MAX

Rule specifications and dictionaries

A dictionary is a data set that contains one or more columns of reference data.

You can specify a dictionary in a rule statement condition or action in basic mode or in expression logic in advanced mode. For example, add a dictionary to a rule statement condition or an IF statement to verify that the input data contains the data values that you expect. Add a dictionary to rule statement action or a THEN statement to return a value that corresponds to the input data value.

You add a dictionary to a rule specification asset, and then add the asset to the Rule Specification transformation in a mapping.

A dictionary might contain public terms, such as telephone area codes or address abbreviations. Or, a dictionary might contain values that are specific to an organization, such as employee codes or product codes.

At least one dictionary column must contain a set of standard or required values as defined by a business rule. The other columns can contain alternative versions of the values. The column that contains the correct or required values is called the valid column. By default, the first or left-most column in a dictionary is the valid column. You can populate a dictionary with any combination of values that suits your business rule logic. The data in each column does not need to be formally correct.

When you add a dictionary to a condition or IF statement, the rule specification compares the input data to the values in each column in the dictionary and passes the results to the action in the rule statement.

When you add a dictionary to an action or THEN statement, the rule specification compares the input data to the values in each column in the dictionary and returns a value according to the operation that you define. For example, the action might return the value in the valid column on the same row in the dictionary.

You can change the valid column that an action uses in the **Define Actions** dialog box in basic mode.

Note: In a search and replace operation, a rule specification searches every column in the dictionary except the valid column that the asset specifies. If you want the search operation to include the valid column data, add a copy of the valid column to the dictionary. By default, the valid column is the first or left-most column in a dictionary. You can create a dictionary that contains two identical columns of data.

You can use any dictionary in a condition regardless of the number of columns that the dictionary contains. Some of the dictionary options in the **Define Actions** dialog box require a dictionary with two or more columns.

Dictionary example

The following table contains rows from a dictionary of city names in India:

Name1	Name2
Mumbai	Bombay
Chennai	Madras
Bengaluru	Bangalore
Puducherry	Pondicherry
Kadapa	Cuddapah
Kochi	Cochin
Kolkata	Calcutta

The Name1 column contains the current names of cities in India. The Name2 column contains the previous version of each city name. You might use the dictionary in an action to replace the older city names with the current city names.

You might also create a dictionary with a single column of data that contains the older city names. You can use the dictionary in a condition to determine if an input data set contains the older names.

CHAPTER 19

Rule set configuration

You configure one or more rule sets on a rule specification in basic mode. The number of rule sets that you configure depends on the facts that you need to verify about the business data.

A fact is a piece of information that you derive from a data value when you measure it or compare it to other data values. For example, a date value might indicate a birthday, or it might indicate the date that you sent an invoice to a customer. You might use a date to determine if a person is an adult or if a customer account is overdue. You can use the facts that you determine about the person or about the customer to make business decisions. A rule specification can determine a single fact, or it can determine multiple facts that contribute to an overall result for a row of business data.

When one fact about the business data depends on multiple other facts, create rule sets in a parent and child relationship. Use the child rule sets to generate inputs that the parent rule set can verify. If a fact relates to a single item of data, you can configure rule statements to analyze the data in a single rule set. If a fact depends on multiple independent facts about the business data, create child rule sets in parallel to determine each fact.

Rules and guidelines about rule sets

When you configure rule sets, you define the flow of data through the rule specification. The position of the rule sets and the order of the rule statements in the rule sets determine the data flow.

Consider the following rules and guidelines when you configure rule sets:

- Data flows upwards through a rule specification from the lowest rule set to the primary rule set. When the rule specification includes parallel rule sets, the rule specification runs the parallel rule sets concurrently.
- A rule set generates a single output for a row of input data. The output from the first rule statement to generate an action becomes the rule set output for the row.
- The output from the primary rule set is the primary fact that the rule specification generates. The primary rule set output represents the result of the business rule analysis of the data row.
- The output from a child rule set is an input to a parent rule set. Every rule set that you add to a rule specification is a child of the primary rule set or a child of another rule set.

  You must add the output from a child rule set to a rule statement in the parent rule set.
- The input properties on a rule set display the inputs that you select in the rule set and the inputs from any child rule set. To view all of the inputs in the rule specification, open the **Input Management** dialog box.
- The rule statements in a rule set must generate outputs of the same data type.

Rule set configuration examples

The number of rule sets that you configure in a rule specification depends on the facts that you need to determine about the input data.

You can configure rule sets in the following ways:

- Configure rule sets that read a single input.
- Configure rule sets that use rule statement conditions in a dependent relationship.
- Configure rule sets that use rule statements in an independent relationship.
- Configure a parent rule set and child rule sets.

Rule sets that use a single input

Configure a rule set to read a single data input when you want to analyze a range of data values in an input column. Define rule statements that apply the same condition logic to the input, and define a different action for the outcome of each condition.

Single input example

An airline sells airplane tickets online. The airline sells tickets at different prices for travelers in different age groups. The airline defines a business rule that requires the customer database to identify the age group of each traveler. You configure an input that represents the date-of-birth data in a customer data set. You create a series of rule statements that compare the data values in the input data to the current date.

You configure a rule set with the following rule statements:

```
    IF AGE >= 65 THEN SENIOR
OR IF AGE >= 18 THEN ADULT
OR IF AGE < 18 THEN STUDENT
```

The rule set returns data from the first rule statement that can generate an action. Therefore, the order of the rule statements is relevant. If the rule set reads the second rule statement first, the rule set cannot identify any traveler in the senior age category. When the rule set reads the second rule statement first, the rule set identifies all traveler who are 18 years or older as adults.

Rule sets with dependent conditions

Configure a rule set with rule statement conditions in a dependent relationship when the business rule requires multiple conditions to be true. Define a rule statement that applies a set of related conditions to the inputs, and define a single action for the conditions.

Use an AND operator in the rule statement to join the conditions.

Dependent conditions example

A city bus company wants to hire bus drivers. The bus company requires all drivers to pass an eyesight test and to hold a license to drive a bus. The company defines a business rule that requires the employee database to verify the eyesight test status and the license status for each driver. You create inputs that represent the data for the eyesight test and the license. You configure a rule statement with a condition for each input. You configure the default rule set to return a string when an input to the earlier rule statement is not valid.

You configure a rule set with the following rule statements:

```
        IF EYESIGHT TEST IS YES
        AND DRIVERS_LICENSE IS YES THEN OK
    OR IF NO RULE STATEMENT IS VALID THEN NOT_OK
```

Rule sets with independent conditions

Configure a rule set with rule statements in an independent relationship when the business rule requires one of multiple conditions to be true.

Independent conditions example

A bank wants to offer credit cards to customers. The bank decides that any customer with a credit balance greater than $5,000 or a salary greater than $50,000 qualifies for a credit card. The bank defines a business rule that requires the customer database to identify the eligible customers. You configure inputs that represent the customer salary and the customer balance. You configure a rule statement to test each input.

You configure a rule set with the following rule statements:

```
       IF CUSTOMER SALARY >= 50000 THEN OFFER_CARD
    OR IF CUSTOMER BALANCE >= 5000 THEN OFFER_CARD
```

The rule set returns data from the first rule statement that can generate an action. However, the order of the rule statements is not relevant in the example. If the data on any input is valid, the input row is valid.

Rule sets in parent and child relationships

When a business rule describes a data requirement that depends on other data requirements, configure rule sets in a parent-and-child relationship.

Parent and child example

A hardware store sells wood preserver in multiple colors and styles. The store must verify that the product database tracks the inventory of wood preserver in the warehouse. The store defines a business rule that requires the product database to maintain accurate data for the colors, styles, and units of wood preserver. You configure child rule sets with rule statements that verify the data for each type of wood preserver. You configure the parent rule set to verify the output from the child rule sets.

You configure the following child rule sets:

Rule set name: Color

```
        IF COLOR = (PINE, OAK, CHESTNUT) THEN GOOD_COLOR
```

Rule set name: Size

```
        IF SIZE = (250, 500, 1000) THEN GOOD_SIZE
```

Rule set name: Style

```
        IF STYLE = (GLOSS, MATT) THEN GOOD_STYLE
```

You configure the following parent rule set:

Rule set name: Wood Preserver

```
        IF COLOR = GOOD_COLOR
        AND SIZE = GOOD_SIZE
        AND STYLE = GOOD_STYLE THEN WOOD PRESERVER IS GOOD_PRODUCT
```

To configure a condition to compare an input to a range of values, select the *Input: List of Values* option in a function condition. You can configure similar parent rule sets and child rule sets for other products in the inventory.

Note: You might also create dictionaries that contain lists of the acceptable colors and styles. Configure the conditions to compare the input values to the dictionaries.

Rule set properties

To view the properties on a rule set, select a rule set in the Configuration tab. Use the properties to describe the rule set, to add inputs and rule statements, and to test the rule set.

The rule set properties contain the following views:

General

Displays the rule set name and any description that you add to the rule set. You can update the name and the description.

Enter a name that summarizes the type of data operations that the rule statements perform in the rule set. The rule set name must be unique in the rule specification.

Inputs

Lists the inputs that the current rule set uses. If a rule set is a child to a parent rule set, the parent rule set reads the child rule set name as an input name.

To view all of the inputs in the rule specification or to add an input to the rule specification, open the **Input Management** dialog box.

Rule Logic

Displays the rule statements in the rule set. You can add and update a rule statement.

Test

Displays the rule set inputs in a table that you can use to test the rule set logic. The table includes the inputs from any child rule set below the current rule set.

You can enter sample data values and verify that the rule set generates the results that you expect. To test the complete rule specification, select the primary rule set.

Adding a rule set

By default, a rule specification contains a primary rule set. Add a rule set under the primary rule set or under another rule set in the rule specification.

1. On the **Explore** page, open the rule specification.
2. On the **Configuration** tab, select a rule set as a parent for the rule set that you create.
3. Click the **Add rule set** button at the base of the rule set.

   Or, right-click the rule set and select **Add rule set**.

   A rule set is added below the parent rule set.

Copying and moving rule sets

Copy a rule set to reuse the rule statements in the rule set. Move a rule set when you determine that the rule set belongs in another location.

You might copy a rule set to another rule specification to reuse the rule statements that the rule set contains. You might copy a rule set within a rule specification to create rule statements that are similar to the rule statements in the rule set.

You might move a rule set when you determine that the rule set is a dependency for a different business rule requirement.

You copy or move a rule set to a position below another rule set. The rule set that you copy or move becomes a child rule set of the destination rule set. When you copy or move a rule set to another rule specification, you also copy the inputs that the rule set uses. You cannot move a primary rule set.

The copy and cut options are disabled on a primary rule set when you configure exception management on the rule set.

Copying or moving a rule set within a rule specification

To create multiple similar rule sets in a rule specification, copy the rule set. Move a rule set when you determine that the rule set belongs in another location.

After you copy or move a rule set, update the rule statements to reflect the business requirements that the rule set represents.

1. Open the rule specification.
2. Select a rule set, and right-click it to open the menu.
   - To copy the rule set, select **Copy**.
   - To move the rule set, select **Cut**.
3. Select a parent rule set for the rule set that you copy or move. Right-click the parent rule set to open the menu.
4. Click **Paste**.
5. The rule set that you copy or move appears below the rule set that you select.

   The string _COPY is added to the rule set name.
6. Save the rule specification.

Copying or moving a rule set to another rule specification

If you create multiple rule specifications with similar logic, you can copy the rule sets from one rule specification to another to save time. Move a rule set when you determine that the rule set belongs in another location.

After you copy or move a rule set, you might update it to reflect the business requirements of the current rule specification. Or, you might apply the rule set to different inputs.

1. Open the rule specification.
2. Select a rule set, and right-click it to open the menu.
   - To copy the rule set, select **Copy**.
   - To move the rule set, select **Cut**.
3. Click **Explore**.

4. Select a destination rule specification for the rule set.

   The rule specification opens.
5. Select a parent rule set for the rule set that you copy or move. Right-click the parent rule set to open the menu.
6. Click **Paste**.
7. The rule set that you copy or move appears below the rule set that you select.
8. Save both rule specifications.

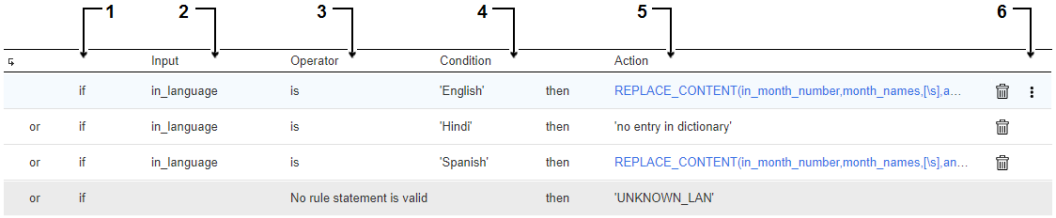
CHAPTER 20

Rule statement configuration

Rule statements define the data analysis and data transformation operations that a rule specification applies to input data in basic mode. Rule statements contain inputs, operators, conditions, and actions.

A condition defines the type of analysis that the rule statement performs on the input data. An operator is a mathematical function that determines how the rule statement processes the result of the condition analysis. An action defines the output that the rule statement generates if the input data satisfies the condition.

The following image shows a series of rule statements on the Configuration tab:



The rule statements have the following elements:

1. Condition type.
   Indicates the condition or conditions in which the input data can be valid. When you create a rule statement, you create an IF condition. When you add a condition to a rule statement, you create an AND relationship between the conditions in the statement.
2. Input.
   Identifies the data to analyze or update. You can select an input from the current rule set, or you can select the output from a child rule set.
3. Operator.
   Identifies the type of validation operation that the condition applies to the input data.
4. Condition.
   Describes a fact that the rule statement verifies about the input data.
5. Action.
   Describes the output that the rule statement generates when the input data satisfies the condition. If you configure multiple conditions in the rule statement, the input data must satisfy all conditions to generate an action.

   If a function in a condition or an action reads a dictionary, the condition or action description includes the dictionary name. Click the dictionary name to open the dictionary.
6. Menu options.
   Displays a list of options for the rule statement.

An input, operator, and condition correspond to an IF statement in the logic that you define in advanced mode. An action corresponds to a THEN statement in the logic that you define in advanced mode. The final rule statement in a rule set describes the action to take if no previous rule statement can generate an action. The final rule statement corresponds to an ELSE statement in the logic that you define in advanced mode.

Conditions

A condition is a data operation that a rule statement specifies for one or more input data values. Every rule statement contains at least one condition.

When a condition validates an input data value, the rule specification performs the action in the rule statement. The action represents a potential output from the rule set. If a condition cannot validate an input data value, the rule specification does not perform the action.

Configure a condition to perform one of the following types of analysis:

Compare the input data values to a single value

The rule statement compares the values in the input column to the value that you enter.

Compare the input data values to the values on another input

The rule statement compares the values in the input column to the values on the same row in another input column that you specify.

Compare the input data values to the current date and time

The rule statement compares the values in the input column to the current date and time.

Search for null values

The rule statement searches the input column for null or empty strings.

Compare the input data values to a range of values

The rule statement compares the values in the input column to a range of values that you enter.

Compare the input data values to the values in a dictionary

The rule statement compares the values in the input column to the values in a dictionary. The rule statement returns a value from the dictionary or a value that you enter.

Compare the input data values to the output from another rule specification

The rule statement compares the values in the input column to the output from a rule specification that you select. The rule specification runs within the rule statement logic.

Specify at least one input for the rule specification. You can select any input available to the current rule specification, including the current condition input, or you can enter a value as a rule specification input. You also select the field that the condition reads as the rule specification output.

Compare the input data values to the output from a mapplet

The rule statement compares the values in the input column to the output from a mapplet that you select. The mapplet runs within the rule statement logic.

Specify at least one input for the mapplet. You can select any input available to the current rule specification, including the current condition input, or you can enter a value as a mapplet input. You also select the field that the condition reads as the mapplet output.

Note: A rule specification can read mapplets that contain passive transformations.

Apply a function expression to the input data values

The rule statement applies a function expression that you select to the values in one or more input columns.

Apply a custom expression to the input data values

The rule statement applies a custom expression that you define to the values in one or more input columns.

Rules and guidelines for mapplets in conditions and actions

You can optionally add a mapplet to a function in a rule statement condition or action. When you add a mapplet, you add the logic of any asset or transformation in the mapplet to the condition or action.

The mapplet can contain any passive data quality transformation and can additionally contain a passive Java transformation and Expression transformation.

Consider the following rules and guidelines when you add a mapplet to a condition or action:

Passive transformations

You can use the following transformations in a mapplet that you add to a condition or action:

- Parse transformation
- Cleanse transformation
- Labeler transformation
- Rule specification transformation
- Verifier transformation
- Expression transformation
- Java transformation in passive mode

Note: Do not define an in-out parameter on a mapplet that you add to a condition or action. Before you add a mapplet with a Java transformation to a condition or action, you must first compile the transformation. For information about how to use a Java transformation that consumes external Java packages in a mapplet, see the Knowledge Base article KB 000227229.

Nested mapplets

You can configure a condition or action with up to four nested mapplets and rule specification assets in an alternating manner.

Consider the following example:

- You select the Mapplet function in a condition.
- You configure the condition to read a mapplet that contains a Rule Specification transformation named `RuleSpec1`.
- The Rule Specification transformation reads a rule specification asset with a condition that also specifies a Mapplet function.
- The mapplet that you specify in this condition contains a Rule Specification transformation named `RuleSpec2`, which reads an additional rule specification asset.

In this way, you can create four levels of mapplets and rule specifications beneath a condition or action.

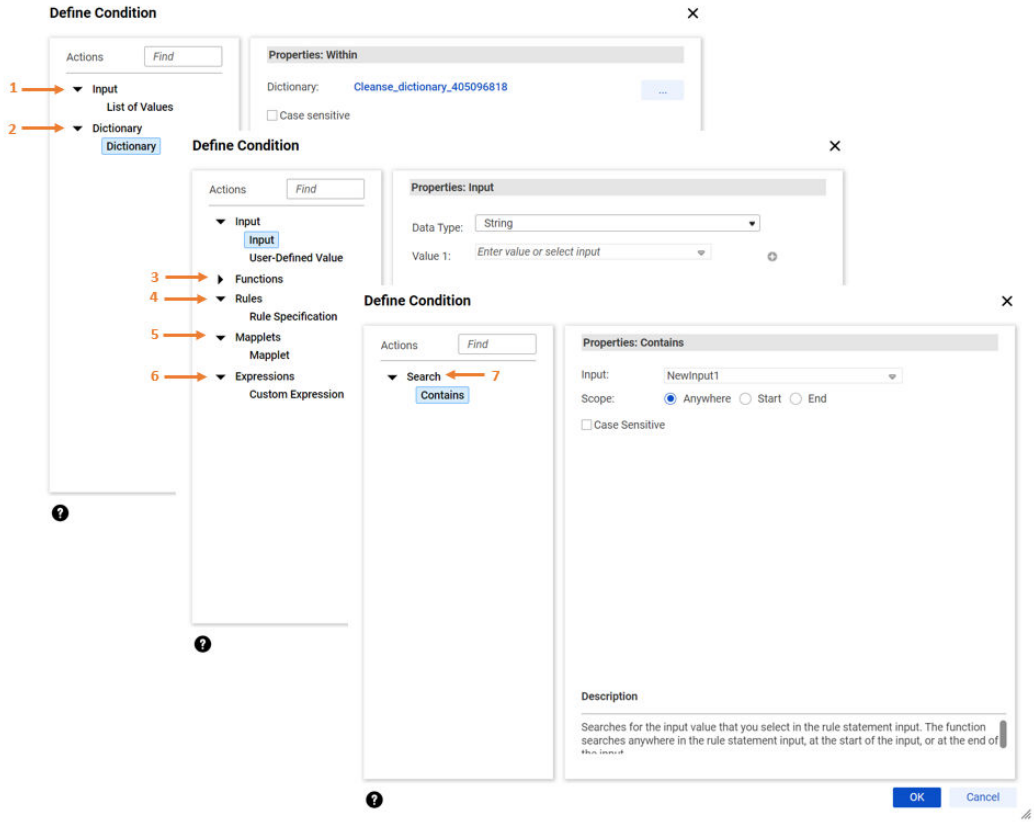
Function configuration in conditions

You can configure the data operation in a condition as a function expression. You can also select a function expression from a list of expressions in the rule specification. Use the function options in the condition to configure or select the expression.

You can use the function options as an alternative to the inline condition options that the rule statement displays in the workspace. You can configure a function to read multiple inputs. You can also use the function options to add a dictionary or another rule specification to the condition.

Use the **Define Condition** dialog box to configure or select a function expression. To open the dialog box, select *function* as the condition type in the rule statement. The function expressions that you can use depend on the operator that you select.

The following image shows the options on the **Define Condition** dialog box for different operators:



The dialog box contains the following options:

1. Input
   Use the Input options to compare an input data value to other data values in the following ways:
   - Compare the rule statement input with an input from another column on the same row.
   - Compare the rule statement input with a value that you enter.
   - Compare the rule statement input with a range of values that you enter.

   Select the Inputs option with the following operators: *is, is not, is greater than, is greater than or equal to, is less than, is less than or equal to, is within,* and *is not within*.
2. Dictionary
   Use the **Dictionary** options to compare an input data value with the values in a dictionary.

   Select the Dictionary option with the following operators: *is within* and *is not within*.
3. Functions
   Use the Functions options to apply an expression to the input data. Select an expression from the list in the dialog box.

   Select the Functions option with the following operators: *is, is not, is greater than, is greater than or equal to, is less than* and *is less than or equal to*.
4. Rules
   Use the Rules options to compare an input data value to the output from another rule specification. The rule specification that you select runs as part of the rule statement at run time.

   Select the Rules option with the following operators: *is, is not, is greater than, is greater than or equal to, is less than* and *is less than or equal to*.

5. Mapplets
   Use the Mapplets options to compare an input data value to the output from mapplet. The mapplet runs as part of the rule statement at run time.

   Select the Mapplets option with the following operators: *is, is not, is greater than, is greater than or equal to, is less than* and *is less than or equal to*.

   For more information on mapplet configuration, see "Mapplet configuration options" on page 152.
6. Expressions
   Use the Expressions options to define and apply a custom expression to the input data. You can enter an expression as text, or you can select from a range of built-in expressions.

   Select the Expressions option with the following operators: *is, is not, is greater than, is greater than or equal to, is less than* and *is less than or equal to*.
7. Search
   Use the Search options to find an input data value in another input column. The rule statement searches all of the values in the column for the current rule statement input value.

   Select the Search option with the *contains* operator.

Operators

An operator is a mathematical function that determines the outcome of a condition.

You can select one of the following operators:

is

Verifies that the input data matches the data that the condition specifies.

is not

Verifies that the input data does not match the data that the condition specifies.

is within

Verifies that the input data matches a value in a dictionary or in a list of values that you enter.

is not within

Verifies that the input data does not match a value in a dictionary or in a list of values that you enter.

contains

Verifies that the input data contains a sequence of characters that you specify.

is less than

Verifies that the input data value is less than the value that the condition specifies. Select the operator with numeric input data.

is less than or equal to

Verifies that the input data value is less than or equal to the value that the condition specifies. Select the operator with numeric input data.

is greater than

Verifies that the input data value is greater than the value that the condition specifies. Select the operator with numeric input data.

is greater than or equal to

Verifies that the input data value is greater than or equal to the value that the condition specifies. Select the operator with numeric input data.

Actions

An action specifies the output from a rule statement. An action generates an output value when the conditions in the rule statement validate an input value.

Configure an action to generate an output in one of the following ways:

Return a value that you enter

Configure an action to return a data value that you specify.

For example, you might configure an action to return the word VALID when the input data matches a dictionary value.

Return a value from an input column

Configure an action to return a data value from an input in the rule specification. The action reads the data value on the corresponding row of the input that you select.

For example, a business might define a rule that requires all product records to use Quick Response (QR) codes in place of bar codes. You configure an action to replace an input column of bar code values with a corresponding column of QR code values.

Return a value from a dictionary

Configure an action to return a data value from a dictionary. The action compares the input data to the values in the dictionary that you specify. If the action finds the input data in the dictionary, the action returns the dictionary value that corresponds to the input data.

For example, you might configure an action to compare first name data to a dictionary that identifies the gender of the first name. The action might read the word "JOHN" from the input data and return the word "MALE" from the dictionary.

Return a value that you enter when an input data value matches a dictionary value

Configure an action to return a data value from a dictionary. The action compares the input data to the values in the dictionary that you specify. If the action finds the input data in the dictionary, the action returns the string that you entered. if the action does not find the input data in the dictionary, the action returns the input value.

For example, you might configure an action to compare employee code data to a dictionary that contains the valid employee codes in an organization. You define a function that returns the word "VALID" when an employee code matches a value in the dictionary. If the function does not find an employee code in the dictionary, the action returns the employee code from the input column.

Return the current date and time

Configure an action to return the current date and time. The rule statement returns the date and time values in a date/time data type.

Remove data values from the input data

Configure an action to remove a data value that you specify.

For example, the input data might include a product name that the business no longer uses. Configure the action to return a version of the input data that omits the product name.

Remove dictionary values from the data

Configure an action to remove dictionary values from an input.

The action compares the input data to the values in a dictionary that you specify. If the action finds an input data value in the dictionary, the action returns a version of the input without the data value.

For example, you might configure an action to compare an input string to a dictionary of salutation terms. The action might read the input string "MR JOHN SMITH" and return the string "JOHN SMITH" as the output.

Remove character spaces from the input data

Configure an action to delete character spaces from the input data. Remove character spaces when the input data includes redundant character spaces.

For example, you might configure an action to remove character spaces from the following telephone number:

```
212 555 1234
```

Use a function expression to determine the output value

Configure an action to apply a function expression to the input data.

For example, you might configure the action to return the character length of the input data.

Use a custom expression to determine the output value

Configure an action to apply an expression that you define to the input data.

For example, you might configure the action to concatenate two input data values.

Merge values from multiple inputs

Configure an action to merge data from two or more inputs.

For example, you might configure an action to merge a column of first name data and a column of second name data. The action returns a single field that contains the first name and second name.

Change the character case of the input data

Configure an action to convert the character case of an input that you select.

For example, a rule statement might read a data set that stores data values in sentence case. Configure an action to return the data in uppercase.

Return an output form a rule specification

Configure an action to run a rule specification on one or more inputs that you specify. Select at least one input on the rule specification, or enter a value as a rule specification input. You can select any input available to the current rule specification as an input. You also select the field that the action reads as the output of the rule specification.

Return an output from a mapplet

Configure an action to run a mapplet on one or more inputs that you select. Select at least one input on the mapplet, or enter a value as a mapplet input. You can select any input available to the current rule specification as an input. You also select the field that the action reads as the mapplet output.

You can configure a condition or an action with up to four nested mapplets and rule specification assets in an alternating manner. For more information, see “Rules and guidelines for mapplets in conditions and actions” on page 145.

Note: A rule specification can read mapplets that contain passive transformations.

Generate a status value

Configure an action to generate a predefined status value when the input data satisfies the condition on the rule statement.

You can configure an action to return the status values *Valid* and *Invalid*. If you select *Invalid* as the status value, you can add data to the record to explain the status and to identify the priority of the issue that the value represents.

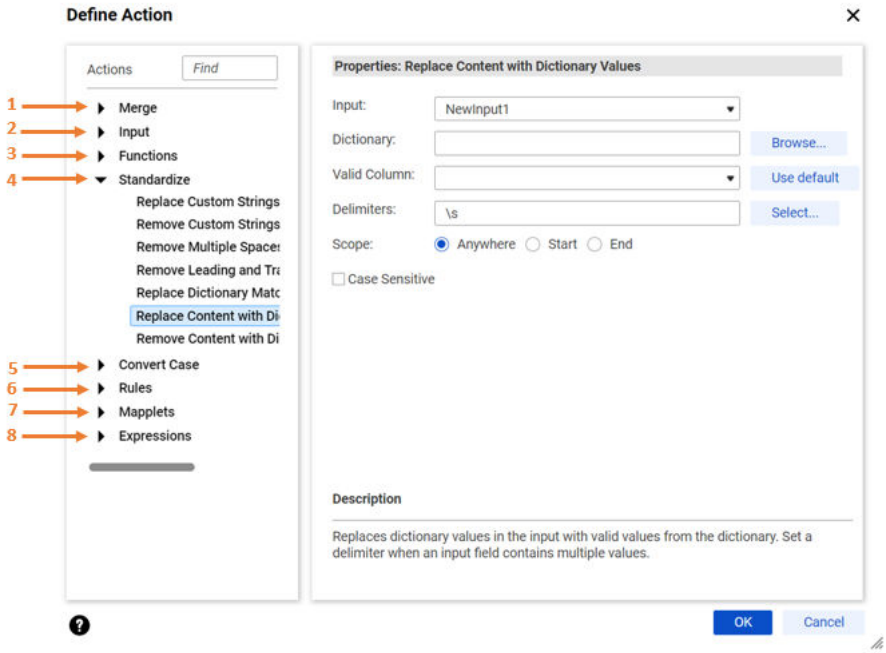
Function configuration in actions

You can configure the data operation in an action as a function expression. You can also select a function expression from a list of expressions in the rule specification. Use the function options in the action to configure or select the expression.

You can use the function options as an alternative to the inline action options that the rule statement displays in the workspace. You can configure a function to read multiple inputs. You can also use the function options to add a dictionary or another rule specification to the condition.

To configure an expression, select the function option from the action menu. When you select the field, you open the **Define Action** dialog box. The dialog box displays a common list of function expressions for all operators.

The following image shows the **Define Action** dialog box:



The dialog box contains the following options:

1. Merge
   Use the Merge options to return data values from multiple inputs as a single output.
2. Input
   Use the Input options to return a data value. You can specify a constant value, or you can select an input. If you select an input, the action returns the value from the input column on the same row as the condition input.

3. Functions
   Use the Functions options to apply a function expression to the input that you select and to return the output of the expression.
4. Standardize
   Use the Standardize options to remove an input value or to update a copy of the input data value and return the copy. The section below describes the standardization options.
5. Convert Case
   Use the Convert Case options to change the character case of the input data values.
6. Rules
   Use the Rules options to return the output of a rule specification that you add to the function.
7. Mapplets
   Use the Mapplets options to return the output of a mapplet that you add to the function.
8. Expressions
   Use the Expressions options to define and apply a custom expression to the input that you select and to return the output of the expression. You can enter an expression as text, or you can select from a range of built-in expressions.

Standardization options

The Standardization options contain several properties that you can configure to remove or replace data.

You can configure the properties to perform the following actions:

Remove Values

Searches an input field for one or more values that you specify, and removes the values.

You can enter one or more values that the asset will remove. Or, you can use a dictionary to identify the values that the asset will remove.

Remove Spaces

Removes redundant character spaces from an input field.

You can remove leading and trailing spaces, and you can replace multiple spaces with a single space.

Replace Values

Searches an input field for one or more values that you specify, and replaces the values. You can find and replace values in the following ways:

- Enter one or more values that the asset can search for. Enter a single replacement value for every value that the asset finds.
- Use a dictionary to identify multiple strings that the asset can search for. Use the same dictionary to specify a replacement value for each value that the asset finds.
- Use a dictionary to identify multiple strings that the asset can search for. Enter a single replacement value for every value that the asset finds.

When you select a dictionary in an option, you must specify the column in the dictionary that contains the valid values. You must also confirm one or more delimiters for the data values that the action reads.

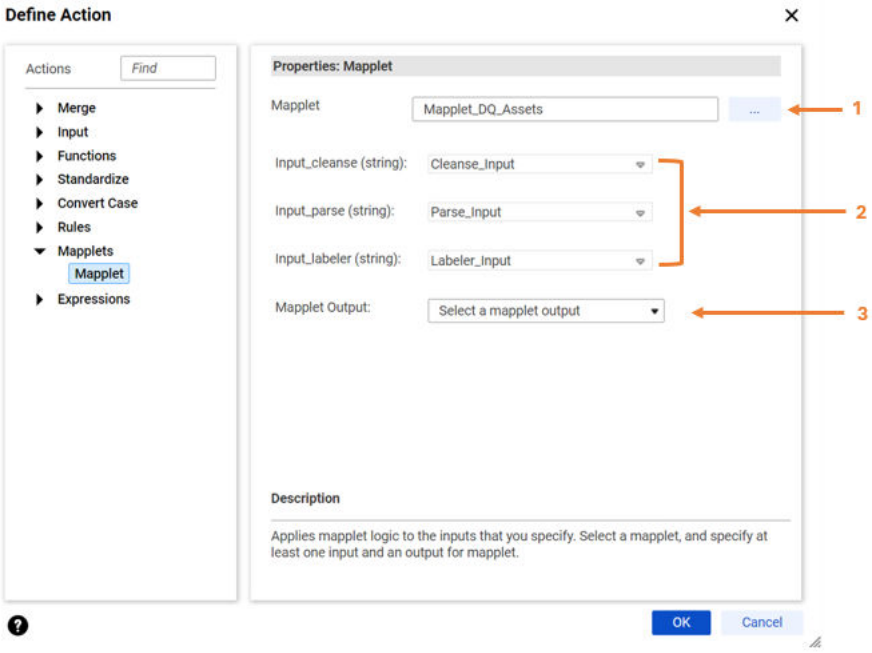
Mapplet configuration options

When you add a mapplet to a condition or action, you must configure at least one input on the mapplet. A mapplet may specify one or multiple inputs, depending on its design. The rule statement additionally reads a single output from the mapping.

Before you configure the inputs, review the design of the mapplet. Verify that your input selections enable the mapplet to generate the data outcomes that the condition or action requires.

For example, you may decide to configure a single input on a mapplet that supports multiple inputs. The data on the field that you specify will flow through the mapplet at run time, provided that the mapplet logic does not require any additional data input. If the input that you select has a dependency on other inputs on the mapplet, configure those inputs also.

The following image shows the options for a sample mapplet in the **Define Action** dialog box:



The dialog box includes the following options:

1. Mapplet name and selection options.

   The options open the **Select Mapplet** dialog box, which displays the mapplets on the system.

2. Mapplet inputs.

   The number of inputs depends on the mapping design. You can assign an input from the rule specification to a mapplet input, or you can enter a value as the input.

3. Mapplet output.

   The rule statement reads a single output from the mapplet as a condition to evaluate or as the data outcome of an action.

Custom expression configuration options

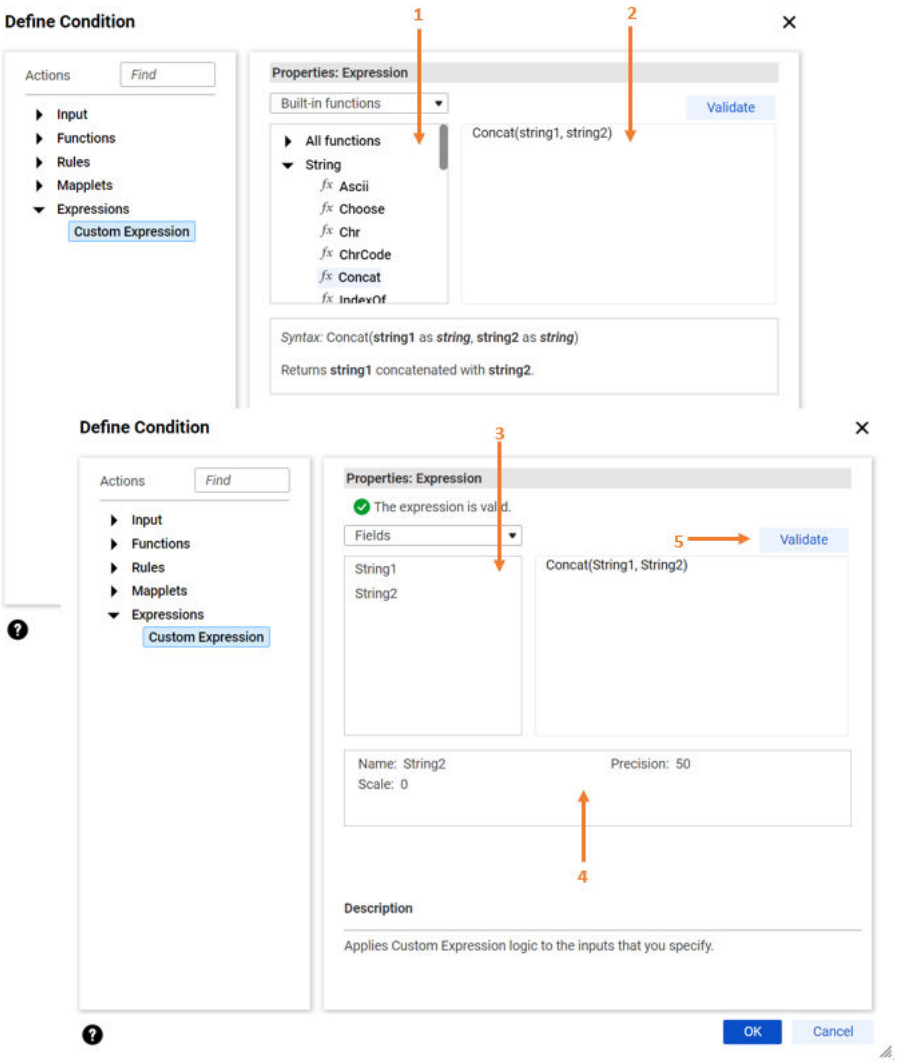
A custom expression is a logical element in a condition or an action. When you configure a custom expression, you enter or select the expression logic and you select the fields that the expression will apply to.

When you add a custom expression to a condition, the custom expression applies the expression logic that you specify to the input data that you select and returns the result to the condition. The condition uses the rule statement operator to determine the next step. When you add a custom expression to an action, the action applies the expression logic to the input data that you select and returns the result as the rule statement output.

To add a custom expression to a condition, use the Custom expression option in the **Define Condition** dialog box. To add a custom expression to an action, use the Custom expression options in the **Define Action** dialog box.

Before you add the custom expression to a condition or action, you can validate the expression.

The following image shows the options for a sample custom expression in the **Define Condition** dialog box:



The dialog box includes the following options:

1. Built-in functions option.

   Displays a library of custom expressions that you can apply to the input data.

2. Expression field.
   Displays the expression logic.

3. Fields option.
   Displays the input fields that you can select.

   Click **Add** next to the field that you want to use.

4. Properties field.
   Displays the details of the input field or the built-in function that you select.

5. Validate option.
   Validates the custom expression.

Note: You select the Built-in functions option and the Fields option from the same menu.

Exception data properties

You can configure a rule specification to add exception data indicators to a record. Use the exception data properties to add outputs for the indicators to the rule specification.

When you configure the exception data properties, you enable an output field for status values that can summarize the data quality issue that the rule logic identifies in a record. You also enable an output for values that indicate the priority of each issue and an output to contain a description of each issue. The output values allow a downstream user to find exception records and to understand the reasons why a record is an exception.

A rule specification asset can specify a single set of output fields for exception indicators. In basic mode, configure the exception indicators as outputs from a rule statement in the primary rule set. If you add a rule specification within a rule statement in basic mode, take care that the child rule specification does not create outputs for exception indicators.

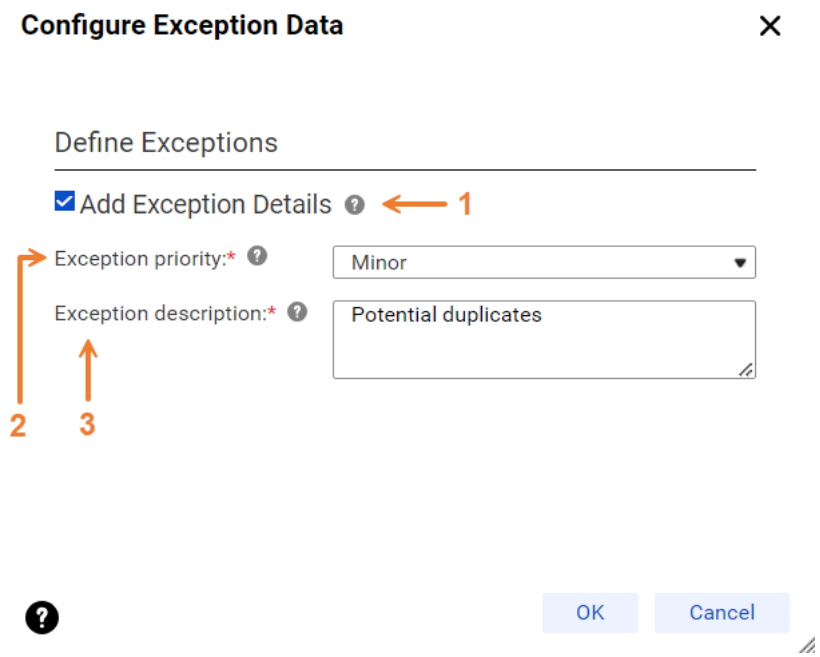
Note: Configure the options and add the rule specification to a profiling task before you create the exception task that reads the profiling task. If a profiling task reads a rule specification in which the exception data options are not configured, configure the exception options and refresh the rule specification in the profiling task before you create the exception task.

Exception data options in basic mode

To add exception indicators to a record, activate the exception indicators in a rule statement that specifies the status value *Invalid* as an action.

Use the **Configure Exception Data** dialog box options to define the indicators. To open the dialog box, click the **+ Exception** option beside the action properties.

The following image shows the options on the **Configure Exception Data** dialog box:



The dialog box contains the following options:

1. Add Exception Details.
   Indicates whether the rule statement adds exception indicators to records. The option is cleared by default.

   The rule statement can add indicators to a record if it assigns *Invalid* as a status value to the record. You specify the indicators in the **Exception priority** and **Exception description** fields.
2. Exception priority.
   Indicates the priority of the data quality issue that defines the record as an exception. The rule statement adds a field to the output records for the priority indicators.

   You can select one of the following options:

   - Minor
   - Major
   - Critical
3. Exception description.
   Describes the data quality issue that defines the record as an exception. The rule statement adds a field to the output records for the description.

Configure all of the dialog box options if you will use the rule specification in a profiling task from which you create an exception task.

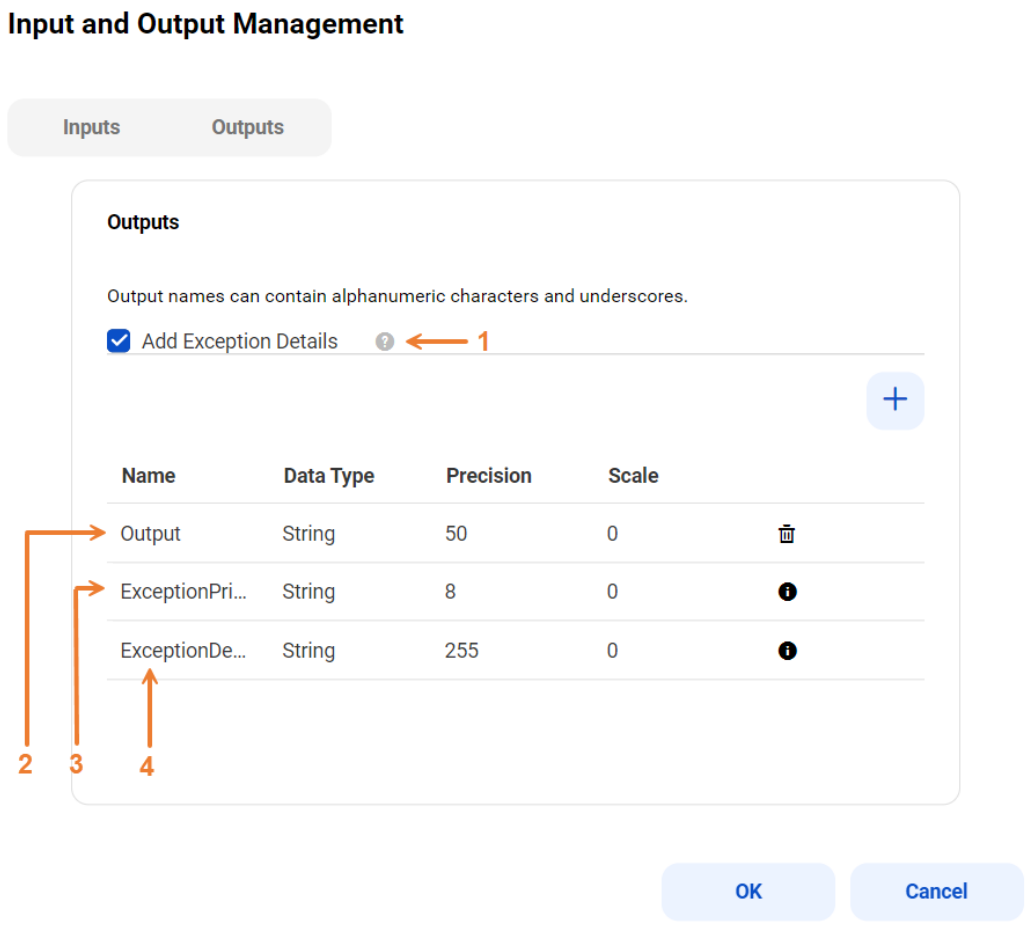
Note: The dialog box does not identify an output field for status values. You activate the output for status values when you configure the Invalid option in the rule statement action.

Exception data options in advanced mode

To add exception indicators to a record, activate exception indicators in the rule specification. You can include the exception indicators in the rule logic that you write.

Use the **Input and Output Management** dialog box options to activate the exception indicators. To open the dialog box, click **Manage Inputs and Outputs**.

The following image shows the options on the **Outputs** tab in the **Input and Output Management** dialog box:



The dialog box contains the following options:

1. Add Exception Details.
   Indicates whether the expression logic can add exception indicators to records. The option is cleared by default.
2. Output field for status values.
   Identifies an output field that the rule specification can use for status values. You can rename the status field in advanced mode.
3. Exception priority field.
   The name of the output field that indicates the priority of the data quality issue in the output field. The statement adds the ExceptionPriority field to the output data.

   You can specify one of the following options when you configure the exception indicator:

   - Minor
   - Major
   - Critical
4. Exception description field.
   The name of the output field that describes the data quality issue in the output field. The statement adds the Exception Description field to the output data.

Function descriptions

A function expression is a logical element in a condition or an action. A function expression reads the input data that you specify, performs a calculation on the input data, and returns a result.

When you add a function expression to a condition, the function expression performs the calculation and returns the result to the condition. The condition uses the rule statement operator to determine the next step. When you add a function expression to an action, the action performs the calculation and returns the result as the rule statement output.

To add a function expression to a condition, use the Function options in the **Define Condition** dialog box. To add a function expression to an action, use the Function options in the **Define Action** dialog box.

You can add the following function expressions to a condition or an action:

Add to Date

Reads a date value and adds a numeric value to an element in the date. Enter the value to add to the element, or select an input that contains the value to add. Select the function expression in a condition or an action.

Choose

Reads input data that contains multiple values and selects the value at the position that you specify. Select the function expression in a condition or an action.

For example, the following string contains values at four positions:

```
Q1, Q2, Q3, Q4
```

You can enter a position, or you can select an input to specify a position.

Concatenate

Reads two inputs that you select and concatenates the values from the inputs. Select the function expression in a condition or an action.

Convert to Date

Converts a value in a string data type to a date/time data type. You identify the input data format when you configure the function expression. Select the function expression in a condition or an action.

Convert to Float

Converts a value in a string data type to a float data type. Select the function expression in a condition or an action.

Convert to Integer

Converts a string or numeric value to an integer data type. Select the function expression in a condition or an action.

You can set the function expression to truncate an input value at the decimal point or to round an input value to the nearest integer.

The function expression treats an alphanumeric value that begins with an alphabetical character as a null value and returns 0 as the output. The function expression treats an alphanumeric string that begins with a digit as a numeric value and truncates the value at the first alphabetical character.

Date Difference

Reads date values from two inputs that you select and calculates the numeric difference between the dates. You select the date element that the function expression uses to measure the difference. Enter the date element, or select an input that contains the date element. Select the function expression in a condition or an action.

Date Part

Reads a date from an input and identifies the numeric value of a date element that you specify. Enter the date element, or select an input that contains the date element. Select the function expression in a condition or an action.

Date Time

Uses multiple values to specify a date and time. Enter a value for each date element, or select an input to add a value to each element. Select the function expression in a condition or an action.

Greatest

Reads two or more values and identifies the highest value. Enter the values, or select rule set inputs to add each value. You can specify numeric values, alphabetic values, or date values. Select the function expression in a condition or an action.

When you specify numeric values, the function expression identifies the highest number.

When you specify alphabetic values, the function expression identifies the value that is last in alphabetic order.

When you specify date values, the function expression identifies the latest date.

Is Date

Returns a value that indicates whether the input string is valid date. Select the function expression in a condition or an action.

Is Number

Returns a value that indicates whether the input string is valid number. Select the function expression in a condition or an action.

Is Spaces

Returns a value that indicates whether the input string contains character spaces only. Select the function expression in a condition or an action.

Last Day

Reads a date value from an input and identifies the last day in the month that the date specifies. The function expression identifies the last day as a numeric value. Select the function expression in a condition or an action.

Least

Reads two or more values and identifies the lowest value. Enter the values, or select rule set inputs to add each value. You can specify numeric values, alphabetic values, or date values. Select the function expression in a condition or an action.

When you specify numeric values, the function expression identifies the lowest number.

When you specify alphabetic values, the function expression identifies the value that is first in alphabetic order.

When you specify date values, the function expression identifies the earliest date.

Length

Reads an input value and calculates the number of characters in the value. Select the function expression in a condition or an action.

Lower

Reads an input string and returns the lowercase version of the characters in the string. Select the function expression in a condition.

Mod

Returns the remainder of a division operation. Select the function expression in a condition or an action.

Null

Reads an input value and determines if the value is null. Select the function expression in a condition or an action.

Replace Character

Reads an input value and replaces a sequence of characters in the value with a sequence that you specify. Select the function expression in a condition.

Replace String

Reads an input value and replaces the value with a value that you specify. Select the function expression in a condition.

Reverse

Reads an input value and reverses the order of the characters in the value. Select the function expression in a condition or an action.

Substring

Returns a substring from an input value of a length that you specify, beginning at a position that you specify. Select a string or numeric data type as an input. Select the function expression in a condition or an action.

To Char

Converts a numeric value or a date value to a string value. Select the function expression in a condition or an action.

Truncate

Reads an input value and truncates the value to the number of places that you specify. The function expression reads a numeric value. Enter the value, or select an input to add the value. Select the function expression in a condition or an action.

Upper

Reads an input string and returns the uppercase version of the characters in the string. Select the function expression in a condition.

Rule statement menu options

You can open a set of menu options for each rule statement. Use the menu options to organize the rule statements in the rule set.

The menu has the following options:

Add Condition

Add a condition to the current rule statement. When you configure multiple conditions in a rule statement, you create an AND relationship between the conditions.

Add Rule Statement

Add an empty rule statement to the rule set.

Add Rule Statement Using Result

Add a rule statement to the rule set below the current rule statement. The rule statement that you add uses the output from the current rule statement as an input.

Copy Rule Statement

Copy the current rule statement. You can paste the rule statement to the current rule set or to another rule set.

Cut Rule Statement

Remove the current rule statement from the rule set. You can paste the rule statement to another rule set.

Move Down

Reorder the rule statements so that the current rule statement changes places with the rule statement below it.

Move to Bottom

Move the current rule statement to the bottom of the rule set.

Move to Top

Move the current rule statement to the top of the rule set.

Move Up

Reorder the rule statements so that the current rule statement changes places with the rule statement above it.

Paste Rule Statement

Paste a rule statement that you copied or moved in an earlier action.

Configuring a rule statement

Before you configure the rule statement, verify that the rule specification contains the inputs that the rule statement requires.

1. Open the rule specification.
2. Select the rule set to contain the rule statement.
3. Click **Rule Logic**.
4. Click **Add Rule Statement**.
5. Add an input to the rule statement. The condition reads the input that you add.
6. Select an operator. The operator specifies the type of comparison operation that a condition performs.
7. Configure the condition.

   Identify one or more data values that the condition can compare to the rule statement input. Or, configure a data operation to apply to the input.
8. Configure the action to perform when the condition validates the input data.

   Identify the input data or enter a data value that the action can return as the rule statement output. Or, configure a data operation that defines the output. Use the function options to configure the operation.
9. Save the rule specification.

Configuring a rule statement that reads the result of another rule statement

You can create a chain of rule statements in which each statement reads the result of the statement above it. Use the rule statements to derive information from the relationship between multiple inputs.

1. Open the rule specification.
2. Select the rule set.
3. Click **Rule Logic**.
4. Select the rule statement that generates the result that another rule statement must read.

   Or, configure a rule statement.
5. From the rule statement menu, select **Add Rule Statement Using Result**.

   An empty rule statement is added under the rule statement that you select.
6. Configure and save the rule statement.

Note: Data passes through a chain of rule statements when all of the logical operations in the statements are valid for the inputs that you select. If a rule statement in the chain does not generate a result for a data row, the chain performs no further analysis on the row.

Configuring a rule statement with multiple conditions

You can configure multiple conditions in a single rule statement. When you use multiple conditions, you create an AND relationship between the conditions.

You configure a single action in the rule statement.

1. Open the rule specification, and click **Edit**.
2. Select the rule set to contain the rule statement.
3. Click **Rule Logic**.
4. Click **Add Rule Statement**.
5. Add an input to the rule statement. The condition reads the input that you add.
6. Select an operator. The operator specifies the type of comparison operation that the condition performs.
7. Configure the condition.

   Identify one or more data values that the condition can compare to the rule statement input. Or, configure a data operation to apply to the input.
8. From the rule statement menu, select **Add Condition**.

   A condition is added to the workspace.
9. Configure the condition.

   You can add multiple conditions.
10. Configure the action for the rule statement.
11. Save the rule specification.

Adding a rule specification to a rule statement

You can reference a rule specification in a condition or an action in a rule statement. Add a rule specification to a rule statement when the rule specification contains function logic that you want to reuse in the rule statement. Use the rule specification so that you do not have to define the logic in the current rule specification.

Configuring a condition to run a rule specification

1. Create or select a rule statement in a rule set.
2. Select one of the following operators:

   `is, is not, is greater than, is greater than or equal to, is less than` or `is less than or equal to`.

   The condition name changes to function.
3. Open the **Define Condition** dialog box.
4. Select **Rules** > **Rule Specification**.
5. In the **Select Rule Specification** dialog box, browse to and select a rule specification to add to the condition.
6. The **Define Condition** dialog box shows the inputs on the rule specification that you select.

   Select an input in the current rule set to map to each input in the selected rule specification. The data types of the inputs must be compatible in each case.
7. The **Define Condition** dialog box also shows an output field for the rule specification that you select. The output field identifies the rule set that generates an output for the condition.

   To provide an output from the rule specification that the condition can use, select a rule set name.
8. Click **OK**.

Configuring an action to run a rule specification

1. Create or select a rule statement in a rule set.
2. Select the function option on the action.
3. Open the **Define Action** dialog box.
4. Select **Rules** > **Rule Specification**.
5. In the **Select Rule Specification** dialog box, browse to and select a rule specification to add to the action.
6. The **Define Action** dialog box shows the inputs on the rule specification that you select.

   Select an input in the current rule set to map to each input in the selected rule specification. The data types of the inputs must be compatible in each case.
7. The **Define Action** dialog box also shows an output field for the rule specification that you select. The output field identifies the rule set that generates an output for the condition.

   To provide an output from the rule specification that the action can use, select a rule set name.
8. Click **OK**.

Cut and paste operations and rule statements

Copy a rule statement to reuse the rule statement logic in the same rule set or in another rule set. Move a rule statement that belongs in another rule set. You can copy or move a rule statement to a rule set in the same rule specification or in another rule specification.

You might copy a rule statement to create a rule statement based on the logic of the rule statement that you copy. You might move a rule statement when you determine that the rule statement applies to a different business rule requirement.

When you copy or move a rule statement to another rule specification, you also copy the inputs that the rule statement uses. The rule statement that you copy must be valid within the destination rule set. For example, the rule statement and the inputs on the rule set must be compatible.

Copying a rule statement within a rule set

To create multiple similar rule statements in a rule specification, copy the rule statement. You can copy and paste a rule statement within the same rule set or to another rule set. After you copy the rule statement, update it to reflect the business rule logic.

Move a rule statement when you determine that the rule statement belongs in another rule set.

1. Open the rule specification.
2. Select the rule set that contains the rule statement to copy or move.
3. Select **Rule Logic**.
4. Select the rule statement. If the rule statement contains more than one condition, select the first line of the rule statement.
5. Open the actions menu.
   - To copy the rule statement, select **Copy Rule Statement**.
   - To move the rule set, select **Cut Rule Statement**.

   Note: Find the actions menu option beside the Action options.
6. Select a destination rule set for the rule statement.

   You can create a copy of a rule statement in the current rule set or in another rule set.
7. Select a rule statement in the rule set.
8. Open the rule statement context menu, and select **Paste Rule Statement**.

   The rule statement that you copied or moved appears in the rule set.
9. Verify the position of the rule statement in the rule set.

   If necessary, move the rule statement up or down in the list of list of rule statements.
10. Save the rule specification.

Copying a rule statement to another rule specification

When you create multiple rule specifications with similar logic, you can copy the rule statements from one rule specification to another to save time. After you copy the rule statement, you might update it to reflect the business requirements of the current rule specification.

Move a rule specification when you determine that the rule set belongs in another location.

1. Open the rule specification.
2. Select the rule set that contains the rule statement to copy or move.

3. Select **Rule Logic**.
4. Select the rule statement. If the rule statement contains more than one condition, select the first line of the rule statement.
5. Open the actions menu.
   - To copy the rule statement, select **Copy Rule Statement**.
   - To move the rule set, select **Cut Rule Statement**.

   Note: Find the actions menu option beside the Action options.
6. Click **Explore**.
7. Select a destination rule specification for the rule set.

   The rule specification opens in the Configuration tab.
8. Select a rule set, and select **Rule Logic**.
9. Select a rule statement in the rule set.
10. Open the actions menu on the rule statement, and select **Paste Rule Statement**.

    The rule statement that you copied or moved appears in the rule set.
11. Verify the position of the rule statement in the rule set.

    If necessary, move the rule statement up or down in the list of list of rule statements.
12. Save the rule specification.

    If you moved a rule statement from another rule specification, save both rule specifications.

CHAPTER 21

REST APIs for rule specifications

You can expose a rule specification asset as a public REST API. Design a custom endpoint for the API, publish the endpoint, and invoke the endpoint to integrate the rule specification capabilities with external applications.

For example, if your organization generates a large volume of data in real time, you may need to perform frequent data quality checks and run business rules on the data. To run real-time rules on your data, invoke the rule specification API endpoints in an external application. You can also use the API to periodically run rules on incremental data.

As a first step, you need to enable a rule specification asset as an API on the user interface. After you enable the asset, you can design your endpoints, manage and control the use of your API in API Center.

Enabling a rule specification as an API

Enable a rule specification as an API on the **Explore** page. You can then import the API-enabled rule in order to design an API endpoint in API Center.

1. Click the **Actions** button on rule specification and select **Enable for APIs**.

   The **Enable for APIs** dialog box appears.

2. Enter a name for the API and click **Enable**. The name can contain upto 100 characters.

   You've sent a request for API enablement of the rule specification asset. You'll be notified about the status of the enablement request.

To design an endpoint in API Center, import the rule specification asset that you enabled. After you design an API endpoint, publish the endpoint. You can view your published endpoint in the **Published APIs** tab in the **API Console** page of API Center. After you publish the API endpoint, you can integrate the rule specification capabilities with external applications.

For more information about designing API endpoints with the API-enabled rule specification assets, see REST API assets.

For more information about enabling a mapplet that contains a rule specification asset transformation as an API, see the *Components* help in Data Integration.

CHAPTER 22

Rule specification configuration in advanced mode

A rule specification asset in advanced mode provides a rule editor in which you can write rule logic in simple expressions or in IF-THEN-ELSE syntax.

You can nest an IF-THEN-ELSE statement within another statement. You can specify other rule specification assets, dictionaries, mapplets, and complex expressions in the logic. You receive suggestions as you type in the editor, such as input or output field names, functions, and mapplets.

You can define the same business rule logic in basic mode and advanced mode.

Rule specification logic in advanced mode

In advanced mode, you write your business rule in expression logic. Within an IF-THEN-ELSE conditional statement, you can use ELSEIF conditions and nest additional IF-THEN-ELSE statements.

An IF-THEN-ELSE statement uses the following format:

```
// Primary condition
IF  <Boolean expression>
THEN <Rule Block>
// Optional - Multiple ELSEIF conditions
ELSEIF  <Boolean expression>
THEN  <Rule Block>
// Optional ELSE condition
ELSE  <Rule Block>
ENDIF
```

Before you write the logic, verify that you understand the business data and the requirements of the business rule.

You receive suggestions as you type in the editor. Enter a character space after you type each element in the editor.

Rule logic configuration examples

The elements that you include in your rule logic depend on the facts that you need to determine about the input data.

You might can write the rule logic to include the following elements:

- Other rule specifications
- Mapplets
- Dictionaries
- Exception data options

Specifying a rule specification in an IF element

You can configure an IF element in your rule specification to read a rule specification that you identify:

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
4. Type the name of a rule specification that you want to add.

   The editor can suggest rule specification names from the set of assets in your organization folders or projects based on the text that you type.
5. Map each input in the current rule specification to an input in the rule specification that you select above. Append the input names from the rule specification that you selected as parameters to the rule specification name.

   Note: The data types of the inputs that you map must be compatible.
6. Enter an operator value. For example, enter the `=` value to verify whether the input value matches the value that you specify.
7. Enter the value to compare to the input value. For example, enter `VALID`.
8. Type the `THEN` element.
9. To specify the output, type the output name.
10. Enter an operator value. For example, enter the `=` value to write a value for the data that satisfies the rule logic.
11. Enter the value that the output returns. For example, enter `Input not null`.
12. Type the `ENDIF` element.

The following example shows rule logic that contains a rule specification:

```
IF rule:Auto_Project1.TC593_InnerRule_AdvancedRulespec(Input) = 'VALID'
THEN
Output = 'Input not null'
ENDIF
```

The italicized text represents the rule specification name.

Specifying a rule specification in a THEN element

You can configure a THEN element in your rule specification to read a rule specification that you identify:

1. Open a rule specification in advanced mode.

2. Verify that the rule specification contains the inputs and outputs that you require.
3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
4. To specify the input, type the input name.
5. Enter an operator value. For example, enter the `!=` value to write a value for the data that does not satisfy the rule logic.
6. Specify a condition for the operator. For example, enter `NULL`.
7. Type the `THEN` element.
8. To specify the output, type the output name.
9. Enter an operator value. For example, enter the `=` value to write a value for the data that satisfies the rule logic.
10. Type the name of a rule specification that you want to add.

    The editor can suggest rule specification names from the set of assets in your organization folders or projects based on the text that you type.
11. Map each output in the current rule specification to an output in the rule specification that you select above. Append the output names from the rule specification that you selected as parameters to the rule specification name.

    Note: The data types of the outputs that you map must be compatible.
12. Type the `ENDIF` element.

The following example shows rule logic that contains a rule specification:

```
IF Input != NULL
    THEN
    Output = rule:Default.rule 1 for TC53(Input1)
ENDIF
```

The italicized text represents the rule specification name.

Specifying a mapplet in an IF element

You can configure an IF element in your rule specification to read a mapplet that you identify:

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
4. Type the name of a mapplet that you want to add.

    The editor can suggest rule specification names from the set of assets in your organization folders or projects based on the text that you type.
5. Map each input in the current rule specification to an input in the mapplet that you select above. Append the input names from the rule specification that you selected as parameters to the mapplet name.

    Note: The data types of the inputs that you map must be compatible.
6. Enter an operator value. For example, enter the `=` value to verify whether the input value matches the value that you specify.
7. Enter the value to compare to the input value. For example, enter `DIGIT`.
8. Type the `THEN` element.
9. To specify the output, type the output name.
10. Enter an operator value. For example, enter the `=` value to write a value to the data that satisfies the rule logic.

11. Enter the value that the output returns. For example, enter `Input given is DIGIT`.
12. Type the `ENDIF` element.

The following example shows rule logic that contains a mapplet:

```
IF mapplet:Validation_RuleSpec_1.Mapplet_RuleSpecVal(Input)='DIGIT'
THEN
Output= 'Input given is  DIGIT'
ENDIF
```

The italicized text represents the mapplet that you use in the rule specification.

Specifying a mapplet in a THEN element

You can configure a THEN element in your rule specification to read a mapplet that you identify:

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
4. To specify the input, type the input name.
5. Enter an operator value. For example, enter the `!=` value to write a value for the data that does not satisfy the rule logic.
6. Specify a condition for the operator. For example, enter `NULL`.
7. Type the `THEN` element.
8. To specify the output, type the output name.
9. Enter an operator value. For example, enter the `=` value to write a value to the data that satisfies the rule logic.
10. Type the name of a mapplet that you want to add.

    The editor can suggest rule specification names from the set of assets in your organization folders or projects based on the text that you type.
11. Map each output in the current rule specification to an output in the mapplet that you select above. Append the output names from the rule specification that you selected as parameters to the rule mapplet name.

    Note: The data types of the outputs that you map must be compatible.
12. Type the `ENDIF` element.

The following example shows rule logic that contains a mapplet:

```
IF Input != NULL
    THEN
    Output = mapplet:ER_11227.MappletRS(Input1)
ENDIF
```

The italicized text represents the mapplet that you use in the rule specification.

Specifying a dictionary in the expression logic

You can configure an IF or THEN element in your rule specification to read a dictionary that you identify.

The following task describes the steps to add a dictionary to an IF element.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.

3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement. To specify the input, type the input name.
4. Enter an operator value. For example, enter the `in` value to identify input values that match the dictionary values.
5. To apply a dictionary to the input data, type the name of the dictionary that you created.

   The editor can suggest dictionary names from the set of assets in your organization projects or folders based on the text that you type.
6. Type the `THEN` element.
7. To specify the output, type the output name.
8. Enter an operator value. For example, enter the `=` value to write a value for the data that satisfies the rule logic.
9. Enter the value that the output returns. For example, enter `Input present`.
10. Type the `ELSE` element.
11. To specify the output, type the output name.
12. Enter an operator value. For example, enter the `=` value to write a value for the data that does not satisfy the rule logic.
13. Enter the value that the output returns. For example, enter `Input not present`.
14. Type the `ENDIF` element.

The following example shows rule logic that contains a dictionary:

```
IF Input in dictionary:Validation_RuleSpec_1.Dictionary_RuleSpec
THEN
Output = 'Input present'
ELSE
Output = 'Input not present'
ENDIF
```

The italicized text represents the dictionary that you use in the rule specification.

Specifying exception data options in the expression logic

You can configure exception properties in the expression logic to identify exception records.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
3. Select the **Add Exception Details** option on the **Outputs** tab in the **Input and Output Management** dialog box.

   The option activates the exception priority, exception description, and status indicators that you will use in the rule logic.
4. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
5. To specify the input, type the input name.
6. Enter the operator value.
7. Specify a condition for the operator.
8. Type the `THEN` element.
9. Define a status indicator:
   - Define an `Output`.

- Enter the = operator.
- Enter a status valuethat reflects the data quality issue that the rule specification finds in the record.

10. Add the `ExceptionPriority` indicator.

    Enter the = operator value to write a priority value.

11. Enter the priority value for the data quality issue that defines the record as an exception.

    You can enter one of the following priority values: Minor, Major, Critical.

12. Add the `ExceptionDescription` indicator.

    Enter the = operator value to write a descriptive text.

13. Enter a description of the data quality issue that defines the record as an exception.
14. To identify records that contain another data quality issue, add an `ELSEIF` element. Write the element logic in the same manner as the IF-THEN sequence in the earlier steps.
15. Type the `ENDIF` element.

The following example shows exception rule logic that identifies individuals who may be old enough to obtain a driver's license:

```
IF Input < 17
THEN
Output = 'Child'
ELSEIF Input = 17
THEN
Output = 'Verify age'
ExceptionPriority = 'Major'
ExceptionDescription = 'Driver may qualify this year'
ELSEIF Input > 17
THEN
Output = 'Adult'
ELSE
Output = 'Not known'
ENDIF
```

The italicized text represents the exception data options.

Rule specification functions

Advanced mode supports a range of built-in functions that you can use in the rule logic. The built-in functions match the functions that you can use in basic mode in the asset.

You receive suggestions as you type function names in the editor. When you select a function in the logic, the asset displays the function signature.

For more information about the functions that you can use in the expression logic, see "Rule specification functions" on page 133.

Rule specification operators

Use the rule specification operators when you write business rule logic.

You can use the following operators:

Operator	Description
=	Verifies that the input value matches the value that the condition specifies.
!=	Verifies that the input value does not match the value that the condition specifies.
in	Verifies that the input value matches a value in a dictionary or in a list of values that you enter.
not in	Verifies that the input value does not match a value in a dictionary or in a list of values that you enter.
contains	Verifies that the input value contains a sequence of characters that the condition specifies.
<	Verifies that the input value is less than the value that the condition specifies.
<=	Verifies that the input value is less than or equal to the value that the condition specifies.
>	Verifies that the input value is greater than the value that you specify.
>=	Verifies that the input value is greater than or equal to the value that you specify.

CHAPTER 23

Business rules and rule statements

You can define all or part of a business rule in a rule statement. Define a rule statement for each fact that a business rule must discover. The type of rule statement that you define depends on the business rule requirements.

Use rule statements to verify and update the following business facts:

Verify the accuracy of business data

Define a rule statement that compares input data to a dictionary.

Use the rule statement to verify that the business data is accurate. For example, define a rule statement to verify that a product description table uses the current product codes.

Find obsolete business data

Define a rule statement to find values that are no longer valid or relevant to the business.

Use the rule statement to return a user-defined value that identifies any row that contains an obsolete term.

Standardize project data values

Define a rule statement that searches for values that you specify and replaces them with other values.

Use the rule statement to verify that the project data follows a standard format or reflects the current project terminology. For example, define a rule statement to verify that network machinery names reflect a change from the Development stage to the Production stage in a project. The rule statement can replace instances of Development with Production in the data.

Improve the usability of business data

Define a rule statement that removes redundant characters and redundant strings.

Use the rule statement to ensure that users and software applications can read the business data correctly. For example, define a rule statement to delete character spaces at the start and end of a data field.

Use data values to discover information about business users

Define a rule statement that correlates information from different inputs. Use different conditions to analyze the inputs, and connect the conditions with AND logic. For example, define conditions to link different facts about customers and to derive information that can improve customer service.

Use data values to verify a business policy

Define a rule statement that verifies that the business follows a stated policy.

Use the rule statement to define a function expression that represents the policy.

Verify that you applied a business rule to a data set

Define a rule statement that adds the current date and time to a data set.

Use the rule statement to add a time stamp to the data set that indicates when the mapping ran.

Use a list of values to find data in a column

Define a rule statement that looks for the values that you enter in a column of data.

Use the rule statement to define a list of data values that a dictionary does not contain.

Identify data values that contain key words or character strings

Define a rule statement that finds a data value or a character string in fields that contain multiple values or longer character strings.

Use the rule statement to find values that contain similar data.

Generate status values

Define a rule statement that generates a predefined status value when the data in an input field satisfies a given condition. For example, define the rule statement to return the value of *Valid* when an input field is not null.

If you define a status value of *Invalid* in the rule statement, you can add data to describe the status of the record.

Combine data values in advanced mode

Write rule logic that combines two or more data values in your data set. For example, write logic to concatenate the data values that you require into a single string. Use AND operator to connect two or more inputs.

Standardize character case in advanced mode

Write rule logic to convert the character case of the input data.

Eliminate redundant character spaces in advanced mode

Write rule logic that removes redundant character spaces.

Standardize data values using a dictionary in advanced mode

Write rule logic that searches for instances of values that appear in a dictionary and replaces each value with a corresponding valid value from the dictionary.

Standardize custom data values in advanced mode

Write rule logic that searches for instances of one or more values that you specify and replaces the values with a single new value.

Verifying the accuracy of business data

To verify the accuracy of a column of business data, configure a rule statement that compares the column to a dictionary. For example, you might configure a rule statement to verify that a data column contains the current product codes for the organization.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.

4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To identify input values that match the dictionary values, select the following operator:

     ```
     is within
     ```
6. Select the type of condition to apply to the input.
   - To use a dictionary in a condition, select the following condition type:

     ```
     function
     ```
7. Configure the condition to apply a dictionary to the input data.
   - Open the **Define Condition** dialog box.
   - Select **Dictionary**.
   - Browse to the dictionary that contains the standard versions of the business data. For example, select a dictionary that contains a set of product codes.
   - Select or clear the option to perform a case-sensitive search of the dictionary.
   - Click **OK**.
8. Select the type of action to apply to the data that satisfies the condition logic.
   - To return a string value, select the following action type:

     ```
     string value
     ```
9. Enter the value that the action returns. For example, enter VALID.
10. Save the rule specification.

Identifying errors in business data

To find values that are no longer valid in a data set, configure a rule statement to read a dictionary that contains known data errors. For example, a national organization updates the codes that identify branch offices. The organization changes the code structure from eight digits to ten digits. You might configure a rule statement to read a dictionary that contains the obsolete eight-digit codes.

If the rule statement finds an obsolete branch code in the input data, it writes a value that you specify as output.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the branch code data.
5. Select an operator to validate the results of the condition analysis.
   - To specify that the input data must match the data in the condition, select the following operator:

     ```
     is
     ```
6. Select the type of condition to apply to the input.

- To compare the input data to the data from another input, select the following condition type:

  ```
  input
  ```

7. Select the input that you selected in step 4.

   The rule statement compares the input to itself. Therefore, the rule statement passes all of the input data values from the condition to the action.
8. Configure the action to search the input data for obsolete branch codes.
   - Open the **Define Action** dialog box.
   - Select **Standardize**, and select **Replace Dictionary Matches with Custom Strings**.
   - Select the input that you specified in the condition.
   - Select the dictionary that contains the obsolete values. You must also select or verify the valid value column in the dictionary.
   - Enter the data value that the action returns when an input data value matches a dictionary value. For example, enter OBSOLETE_CODE.
9. Click **OK**.
10. Optionally, update the default rule statement to return the input data value if the action does not find a dictionary match with the value. Select *input* as the action type, and then select the input column that you selected in step 4.

    When you update the default rule statement in this way, the action can write an output for each row of input data.
11. Save the rule specification.

Standardizing project data values

To standardize data values, configure a rule statement that replaces an input value with a value that you require. For example, you might configure a rule statement to replace terms that identify a Development environment with terms that identify a Production environment.

To standardize multiple values, you can create additional rule statements. Alternatively, use a dictionary that identifies the values.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To specify that the input data must match the data that you enter, select the following operator:

     ```
     is
     ```
6. Select the type of condition to apply to the input.
   - To compare the input data to a string value, select the following condition type:

     ```
     string value
     ```
7. Enter the value to compare to the input data. For example, enter DEV.

8. Select the type of action to apply to the data that satisfies the condition logic.
   - To return a string value, select the following action type:
     ```
     string value
     ```
9. Enter the value that the action returns. For example, enter PROD.
10. Save the rule specification.

Improving the usability of business data

To improve the usability of data values, you can delete character spaces at the start and at the end of the values. The redundant spaces might introduce errors when you sort or compare the values.

Configure a rule statement that deletes the redundant spaces. To delete longer character strings, create a rule statement that uses the "Replace Character" function.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To specify that the condition does not apply to null data, select the following operator:
     ```
     is not
     ```
6. Select the type of condition to apply to the input.
   - To verify that the input values are not null, select the following condition type:
     ```
     null value
     ```
7. Select the type of action to apply to the data that satisfies the condition logic.
   - To standardize the data, select the following action type:
     ```
     function
     ```
8. Configure the action to remove the redundant spaces from the input data.
   - Open the **Define Action** dialog box.
   - Select **Standardize**, and select **Remove Leading and Trailing Spaces**.
   - Select the input that you specified in the condition.
   - Click **OK**.
9. Save the rule specification.

Deriving information about business users

To discover information about business users, configure a rule statement that measures a fact about the users. When you combine multiple conditions to measure related facts, you discover additional information.

For example, a financial institution decides to offer a credit card to all customers with high salaries and credit account balances. You might configure a rule statement to identify customers who hold credit balances. You might also configure a rule statement to identify customers who earn a high salary.

Use the AND feature to combine the rule statements.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.

     Note: The condition analyzes salary data.
5. Select an operator to validate the results of the condition analysis.
   - To identify input values that indicate a minimum salary of $100,000, select the following operator:

     `is greater than or equal to`
6. Select the type of condition to apply to the input.
   - To compare the input data to an integer, select the following condition type:

     `integer value`
7. Enter the value to compare to the input data. For example, enter 100000.
8. Select **Add Condition** in the rule statement menu.

   The rule specification adds a condition under the current condition and creates an AND relationship between the conditions.
9. Select an input for the condition that you added.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.

     Note: The condition analyzes the customer account balances.
10. Select an operator to validate the results of the condition analysis.
    - To identify input values that indicate a minimum balance of $1,000, select the following operator:

      `is greater than or equal to`
11. Select the type of condition to apply to the input.
    - To compare the input data to an integer, select the following condition type:

      `integer value`
12. Enter the value to compare to the input data. For example, enter 1000.
13. Select the type of action to apply to the data rows that meet all of the conditions in the rule statement.
    - To return a string value, select the following action type:

      `string value`
14. Enter the value that the action returns. For example, enter HIGH.
15. Save the rule specification.

Verifying that data values conform to business regulations

To verify that the business follows a stated policy, define a rule statement that applies an expression to key data columns.

For example, you might configure a rule statement that validates mortgage loan applications for a mortgage broker. The mortgage broker refuses any loan application that requires the applicant to repay more than twenty-five percent of their salary each month. The rule statement tests the monthly repayment amount against an expression that calculates the salary percentage.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.

     Note: The input represents the monthly repayment amount on the mortgage loan.
5. Select an operator to validate the results of the condition analysis.
   - To identify input values that are greater than twenty-five percent of another value, select the following operator:

     ```
     is greater than
     ```
6. Select the type of condition to apply to the input.
   - To define an expression to test the input data, select the following condition type:

     ```
     function
     ```
7. Configure the condition to apply an expression to the input data. The expression uses an input that represents the monthly salary of the loan applicant.
   - Open the **Define Condition** dialog box, and select **Input**.
   - Select the float or the integer data type.
   - In the Value 1 field, select the input column that represents the monthly salary data.
   - Add a value field under the current value.
   - Enter the number 4 in the Value 2 field.
   - Select the division sign [/] to indicate that the expression divides the first value by the second value.
   - Click **OK**.
8. Select the type of action to apply to the data that satisfies the condition logic.
   - To return a string value, select the following action type:

     ```
     string value
     ```
9. Enter the value that the action returns. For example, enter DENY_LOAN.
10. Save the rule specification.

Adding internal data to a data set

To demonstrate that you applied a business rule to a data set, you might add a date and time stamp to each record. The date and time stamp indicate the time at which the mapping ran to validate the business rule on the data set.

For example, you might define a business rule that states that primary key columns in a data set cannot contain null values. You define a parallel business rule that sets a monthly schedule to review the primary key column data. You configure a rule statement that analyzes the primary key columns and that adds a date and time stamp to each record to indicate the time of the analysis.

Add a condition for each primary key column. Use the AND feature to combine the conditions.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the primary key data.
5. Select an operator to validate the results of the condition analysis.
   - To specify that the condition does not apply to null data, select the following operator:

     `is not`
6. Select the type of condition to apply to the input.
   - To verify that the input values are not null, select the following condition type:

     `null value`
7. Optionally, select **Add Condition** in the rule statement menu.

   The rule specification adds a condition under the current condition and creates an AND relationship between the conditions.
8. Repeat steps four through six to configure a condition for any additional primary key column in the data set.
9. Select the type of action to apply to the data rows that meet all of the conditions in the rule statement.
   - To return the date and time at which the mapping runs, select the following action type:

     `current time stamp`
10. Optionally, update the default rule statement to return a value if any condition in the earlier rule statement finds a null value. Select `string value` as the action type, and then enter a value such as NULL.

    When you update the default rule statement in this way, the action can write an output for each row of input data.
11. Save the rule specification.

Finding a value in a range of values

To determine if any value in a list of values appears in a column of business data, add the values on the list to a condition. The rule statement compares the values in the input that you select to the values in the list.

Add a list of values to a condition in the following scenario:

- The business rule requirement that the condition represents applies to multiple values.
- The organization does not maintain a dictionary of the values.

For example, you might work for a manufacturing company that must recall a small number of products because of a product defect. You know the serial numbers of the products that you must recall. You configure a rule statement with an input that represents the column of serial number data. You add the list of serial numbers to recall to the rule statement condition.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To identify input values that match the dictionary values, select the following operator:

     ```
     is within
     ```
6. Select the type of condition to apply to the input.
   - To enter a list of values, select the following condition type:

     ```
     function
     ```
7. Configure the condition to apply a list of values to the input data.
   - Open the **Define Condition** dialog box.
   - Select **Input**, and select **List of Values**.
   - Enter a list of values. By default, the dialog box displays a single data field. Add a data field for each value on the list.
   - Click **OK**.
8. Select the type of action to apply to the data that satisfies the condition logic.
   - To return a string value, select the following action type:

     ```
     string value
     ```
9. Enter the value that the action returns. For example, enter RECALL.
10. Save the rule specification.

Identifying data values that contain key terms or character strings

To find a word or a character string in a column of input data, configure a condition to use the "contains" operator. You might use the operator to find a single data value when each field in the input contains multiple

values. Or, you might use the operator to find input data values that contain a character string within a longer string.

For example, you might work for an organization that stores inventory items in a warehouse. You want to measure the number of items in inventory for a range of products. The product names are West Wood, West Star, and West Land. You design a condition to find product names that contain the string "West."

The rule statement that you configure might represent a single element in a business rule. For example, the business rule might describe a policy to reduce the sale price of products that spend a long time in inventory. The current rule statement generates an output that a rule statement in a parent rule set can read as an input.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.

   When you use the `contains` operator, the rule statement prompts you to use the `function` condition type.
6. In the **Define Condition** dialog box, select Contains.
7. Enter the value to compare to the input data in the Input field. For example, enter "West."

   The function searches the rule statement input for the string value that you enter.

   By default, the function looks for the value anywhere in the input string. You can also configure the function to look for the value at the start or at the end of the input string.

   Note: You can also configure the Contains function to compare the condition input to a value on another input.
8. Select the type of action to apply to the data that satisfies the condition logic. For example, you might configure an action that returns the inventory numbers for each product.
   - To return the data values from another column on the same row, select the following action type:

     `input`
9. Select the input that represents the inventory levels for each product in the warehouse.
10. Save the rule specification.

Generating status data for a profile

To generate status values, configure an action with the *status value* option.

For example, you might work for a bank that monitors the account status of its account holders on a daily basis. You define a rule statement that returns the predefined status value *Invalid* for accounts that are overdrawn and returns the *Valid* status value for other accounts. You can add the rule specification to a profile and configure the profile to generate metrics for the status values.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.

4. Select an input for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To evaluate whether the account balances are below zero, select the following operator:

   `is greater than or equal to`
6. Enter the value to compare to the data in the Input field. In this case, enter the digit 0.

   The condition evaluates whether the input field contains positive or negative values.
7. Select the type of action to apply to the data that satisfies the condition logic.
   - To return a status value that represents the result of the condition, select the following action type:

     `status value`
   - Select the status value to return when the input data satisfies the condition. In this case, select *Valid*.
8. Define an action to perform when no rule statement is valid.
   - To return a status value that represents the result of the condition, select the following action type:

     `status value`
   - Select the status value to return when the input data does not satisfy the earlier condition. In this case, select *Invalid*.
9. Save the rule specification.

Using status data to identify exception records

To generate status values that identify exceptions, configure an action with a status value of *Invalid* and use the exception properties to indicate why the records meet your exception criteria. Users who analyze the data in downstream processes can use the data that you add to find the exception records and to understand why they qualify as exceptions.

For example, you might decide to examine the output from a mapping that performed duplicate analysis, on the basis that the output may contain additional duplicate records. Let's say that the deduplicate asset in the mapping applied a match threshold value of 90% to the records.

You decide to identify the records that retuned a match threshold value of 89% in the deduplicate output. You define a rule statement that reads the Out_LinkScore field from the output and updates any records with a score between 89% and 90% as Invalid. Additionally, you define exception properties to add `Major` as the priority value and `Potential Duplicates` as the descriptive text.

1. Select the primary rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input of type float for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.

     Note: The input represents the Out_LinkScore value.
5. Select an operator to validate the results of the condition analysis.

- To identify input records with a score of 89% and above, select the following operator:

  ```
  is greater than or equal to
  ```

6. Select the type of condition to apply to the input.
   - To compare the input data to a float, select the following condition type:

     ```
     float value
     ```
7. Enter the value to compare to the input data. In this case, enter 0.89.
8. Select **Add Condition** in the rule statement menu.

   The rule specification adds a condition under the current condition and creates an AND relationship between the conditions.
9. Select the input that you added in step 4.
10. Select an operator to validate the results of the condition analysis.
    - To identify input records with a score of below 90%, select the following operator:

      ```
      is less than
      ```
11. Select the type of condition to apply to the input.
    - To compare the input data to a float, select the following condition type:

      ```
      float value
      ```
12. Enter the value to compare to the input data. In this case, enter 0.9.
13. Select the type of action to apply to the data that satisfies the condition logic.
    - To return a status value that represents the result of the condition, select the following action type:

      ```
      status value
      ```
    - Select the status value to return when the input data satisfies the condition. In this case, select *Invalid*.
14. Configure the exception properties to add data that describe the status of the record.
    - Click the **+ Exception** option to open the **Configure Exception Data** dialog box.
    - Select the **Add Exception Details** option.
    - Add a priority value. For example, Minor.
    - Enter a text description.
    - Click **OK**.
15. Save the rule specification.

Resolving hidden data issues

Your data may contain issues that are not immediately evident. For example, you may define a rule specification to find null values in a data column. However, the rule specification does not find values that contain character spaces exclusively. Such values can appear to be null when you view them onscreen.

To find the values, you define a rule specification with a series of rule statements. One rule statement searches for null values. Two additional rule statements search for fields that contain character spaces exclusively. To find the fields that contain character spaces only, you configure one of the rule statements to read the output from the condition of the rule statement that precedes it.

Finding null values

In this task, you define a rule statement that searches for null or empty strings. To find fields that contain null or empty strings, set the condition type to `null value` and use the `is` operator to determine if the input value is null.

1. Select a rule set in a rule specification.
2. In the rule set properties, click **Rule Logic**.
3. Click **Add Rule Statement**.
4. Select an input of type string for the rule statement.
   - If the rule specification does not contain an input that you can use, create an input. Configure the input properties to represent the type of column that contains the business data.
5. Select an operator to validate the results of the condition analysis.
   - To specify that the input data must match the data in the condition, select the following operator:

     `is`
6. Select the type of condition to apply to the input.
   - To verify that the input values are null, select the following condition type:

     `null value`
7. Select the type of action to apply to the data that satisfies the condition logic.
   - To return an integer value, select the following action type:

     `integer value`
8. Enter the value that the action returns. For example, enter 2.

   Note: Enter a value other than 1 or 0. The values 1 and 0 represent the true and false outcomes from the function that identifies character spaces in the next task.

Finding values that contain character spaces only

In this task, you define rule statements that search for fields that exclusively contain character spaces. You configure two rule statements, where one rule statement reads the result of another rule statement. To find fields that contain character spaces only, you create a rule statement that uses the `Is Spaces` function.

1. Click **Add Rule Statement**.
2. Select the input that you added in the earlier task.
3. Select an operator to validate the results of the condition analysis.
   - To specify that the condition does not apply to null data, select the following operator:

     `is not`
4. Select the type of condition to apply to the input.
   - To identify input values that are not null, select the following condition type:

     `null value`
5. Select the type of action to apply to the data that satisfies the condition logic.
   - To return an integer value, select the following action type:

     `integer value`
6. Enter the value that the action returns. For example, enter 3.

   Note: Enter a value other than 1 or 0 and other than the value that you specified in the earlier task.
7. Select **Add Rule Statement Using Result** from the actions menu on the current rule statement.

The rule specification adds an empty rule statement under the current rule statement. The rule statement reads the result of the rule statement above it.

8. In the rule statement that you added in step 7, select an operator to validate the results of the condition analysis.
   - To specify that the condition reads the output from the earlier rule statement, select the following operator:

     `is`
9. Select the type of condition to apply to the input.
   - To verify the output from the earlier rule statement, select the following condition type:

     `integer value`
10. Enter the value that you added in step 6 in the current task.

    You configure the condition to search for values that indicate that the input string is not null. For example, enter 3 if the earlier rule statement returns the value 3 when an input string is not null.
11. Select the type of action to apply to the data that satisfies the condition logic.
    - To use a function expression in an action, select the following action type:

      `function`
12. Configure the action to search the input data for character spaces only.
    - Open the **Define Condition** dialog box.
    - Select **Functions**, and select **Is Spaces**.
    - Select the string input that you specified in the earlier rule statements.
    - Click **OK**.
13. Save the rule specification.

The rule specification searches first for null strings and then for strings that contain character spaces only. The `Is Spaces` function returns 1 for any string that exclusively contains character spaces and 0 for any other string that it reads.

Combining data values in advanced mode

To combine two or more data values in your data set, use the || operator. The operator combines the data values that you require into a single string.

For example, your organization might combine values to generate unique identifiers or keys. You might concatenate relevant attributes in your data set, such as a customer ID and an order ID to create a composite key that ensures unique identification of a data value.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
   - If the rule specification does not contain an input and an output that you can use, create an input and output. Configure the input and output properties to represent the type of column that contains the business data.
3. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
4. To specify the input, type the input name.
5. Enter the input value. For example, enter `12345`.

6. Enter an operator value. For example, enter the `AND` value to connect the inputs.
7. To specify another input, type the input name.
8. Enter the input value. For example, enter `1111`.
9. Type the `THEN` element.
10. To specify the output, type the output name.
11. Enter an operator value. For example, enter the = value to write a value for the data that satisfies the rule logic.
12. Enter the input names and use the `||` operator between the inputs to combine the values in the inputs.
13. Type the `ENDIF` element.

The following example shows the use of concatenate operator:

```
IF (Input = "12345" AND Input1 = "1111")
    THEN
        Output = (Input || Input1)
ENDIF
```

Standardizing character case in advanced mode

To convert your input data to a consistent case format, write rule logic that changes the character case of the data values. For example, your organization might want to store data values in a specific case format to ensure consistency in the data values.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
   - If the rule specification does not contain an input and an output that you can use, create an input and output. Configure the input and output properties to represent the type of column that contains the business data.
3. In the rule editor, type the output name.
4. Enter an operator value. For example, enter the = value to write a value for the data that satisfies the rule logic.
5. To change the character case to sentence case, select the `convertToSentenceCase` function in the operation.

   You can select the function from the suggestion list.
6. Type the name of an input that represents the column of data that the rule specification will search at run time.

The following example shows rule logic that contains a case convert operation:

```
Output = CONVERTCASE("convertToSentenceCase",Input)
```

Eliminating redundant character spaces in advanced mode

To eliminate redundant spaces in a data set, write rule logic to replace multiple character spaces with a single space.

For example, you organization might decide to identify duplicate records in a data set. When you perform deduplication operation on your data set, the redundant spaces in the data values might lead to incorrect results. If you do not eliminate the redundant character spaces, the deduplication operation might not correctly identify the records that are identical as duplicates, which leads to duplicate entries in the data set.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.
   - If the rule specification does not contain an input and an output that you can use, create an input and output. Configure the input and output properties to represent the type of column that contains the business data.
3. In the rule editor, type the output name.
4. Enter an operator value. For example, enter the = value to write a value for the data that satisfies the rule logic.
5. To remove redundant characters, select the `removeMultipleSpaces` function in the operation.

   You can select the function from the suggestion list.
6. Type the name of an input that represents the column of data that the rule specification will search at run time.

The following example shows rule logic that contains a standardize operation:

```
Output = STANDARDIZE("removeMultipleSpaces",Input);
```

Standardizing data values using a dictionary in advanced mode

To standardize data values, you can write rule logic that searches for instances of values that appear in a dictionary and replaces each value with a corresponding valid value from the dictionary.

For example, a bank might decide to standardize address terms in customer address data so that the data is consistent and uniform. The bank wants to use abbreviated street descriptors that the postal carrier might prefer, such as St for Street, Rd for Road, and Ave for Avenue.

Your data quality assets include a dictionary of address terms. You write a rule specification that standardizes any value in the input data that matches a value in the dictionary.

At a high level, you'll complete the following steps:

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.

   If the rule specification does not contain an input and an output that you can use, create an input and output. Configure the input and output properties to represent the type of column that contains the business data.
3. Write an IF statement to define the condition in which the rule will standardize data.

4. Write a THEN statement to define the action to take when the data satisfies the IF condition.

   Include a `replaceWithDictionary` function in the THEN statement. You can optionally add parameters to the function.

5. Add an ENDIF element.

Writing the IF statement

1. In the rule editor, type the `IF` element of the IF-THEN-ELSE statement.
2. Type an input name.
3. Add an `in` value to associate the input name with the dictionary name.
4. Type the name of the dictionary that contains the address terms.

   The rule editor can suggest available dictionaries based on the text that you type.

Writing the THEN statement

1. In the rule editor, type the `THEN` element.
2. Type an output name.
3. Add an = operator value to associate the output with the function that you'll add.
4. Define a function that returns the dictionary values that match input values.

   To define the function, use the STANDARDIZE command with a `replaceWithDictionary` function, add the input name, and specify the dictionary.

   Use the dictionary that you selected in the IF statement. The rule editor can suggest available dictionaries and function names based on the text that you type.

Writing the ENDIF statement

- To complete the rule logic, type `ENDIF`.

Adding optional parameters to the THEN statement function

- Optionally, add one or more of the following parameters to the `replaceWithDictionary` function:
  - Specify the dictionary column that contains the preferred versions of the address terms. If you do not identify a valid column, the rule specification uses the first column in the dictionary.
  - Enter the delimiter that the rule specification will use to recognize discrete values in the input field. Specify the delimiters within square brackets. If you do not specify a delimiter, the rule specification uses a space as the delimiter.
  - Enter the scope for the search operation. You can set the following options:
    - `Anywhere`. Replaces any instance of the value that occurs in the input field.
    - `Start`. Replaces any instance of the value that is not preceded by another value in the input field.
    - `End`. Replaces any instance of the value that is not followed by another value in the input field.
  - Enter `NULL` to indicate that you are not replacing input values with a custom value.
  - Enter `TRUE` to indicate that the function will perform a case-sensitive comparison between the dictionary data and the input data.

Rule logic example

The following example shows rule logic that represents the standardize operation:

```
IF Input in dictionary:MRS.Merge_Cleanse_Strategy.AddressDictionary
THEN Output =
STANDARDIZE("replaceWithDictionary",Input,dictionary:MRS.Merge_Cleanse_Strategy.AddressDi
ctionary, "Column 1",[" "],"Anywhere","NULL", TRUE)
ENDIF
```

Standardizing custom data values in advanced mode

To standardize data values, you can write rule logic that searches for instances of one or more values that you specify and replaces the values with a single new value.

For example, your organization might decide to standardize a list of department names to a common format. The organization decides to replace the terms `Information Technology` and `Information Tech` with the term `IT` in a column of department name data.

1. Open a rule specification in advanced mode.
2. Verify that the rule specification contains the inputs and outputs that you require.

   If the rule specification does not contain an input and an output that you can use, create an input and output. Configure the input and output properties to represent the type of column that contains the business data.
3. In the rule editor, type an output name.
4. Add an = operator value to associate the output with the function that you'll add.
5. Define a function that replaces the string values that you specify with a new value.

   To define the function, use the STANDARDIZE command with a `replaceCustomString` function, add the input name, and enter the string values that the rule specification will search for and replace.

   Specify the list of custom string values within square brackets.

   The rule editor can suggest available function names based on the text that you type.
6. Enter the delimiter that the rule specification will use to recognize discrete values in the input field.

   Specify the delimiters within square brackets. If you do not specify a delimiter, the rule specification uses a character space as the delimiter.
7. Enter the scope for the search operation.

   You can set the following options:

   - `Anywhere`. Replaces any instance of the value that occurs in the input field.
   - `Start`. Replaces any instance of the value that is not preceded by another value in the input field.
   - `End`. Replaces any instance of the value that is not followed by another value in the input field.
8. Enter the replacement value.

The following example shows rule logic that represents the standardize operation:

```
Output = STANDARDIZE("replaceCustomString",Input,["Information Technology","Information
Tech"],[","," "],"Anywhere","IT")
```

CHAPTER 24

Validation and testing

Validate a rule specification to verify that the rule specification is valid for use.

Test the rule specification to verify that the output satisfies the business rule requirements. To test a rule specification or a rule set, enter sample values in one or more data columns on the Test tab.

You can cut, copy, and paste test data within the test area and between rule sets in the rule specification. If you open more than one rule specification in concurrent browser tabs, you can cut, copy, and paste test data between the rule specifications.

You can test a complete rule specification, and you can test each rule set in the rule specification. If a rule specification is not valid, test the rule sets to find and fix the validation error.

Rules and guidelines for test and validation operations

Before you add a rule specification to a Rule Specification transformation, validate the rule specification.

Consider the following rules and guidelines when you validate a rule specification:

- A rule specification can fail validation for the following reasons:
  - A condition or an action in a rule statement specifies a data type that differs from the input data type.
  - The rule statements in a rule set generate output data values of different data types.
  - A parent rule set does not read an output from a child rule set.
  - A rule statement is incomplete.
- When you move or copy a rule set to another rule specification, you move or copy any test data that you added for the rule set. You also move or copy any input that the rule set uses.

Validating a rule specification

Validate a rule specification to verify that the rule specification is ready for use in a Rule Specification transformation.

1. Open the rule specification.

2. Click **Validate** on the asset toolbar.

   The rule specification highlights any rule set that is not valid. Click a rule set to read a message that describes the validation error.

Testing a rule specification in basic mode

Test a rule specification to verify that the data flows through the rule specification in the ways that you expect. If possible, use the business data to test the rule specification.

1. Open the rule specification.
2. Select the primary rule set.
3. In the rule specification properties, click **Test**.

   The properties view displays the input columns that you configured in the rule specification.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter one or more data values in an input column. You can enter data values for any rule set in the rule specification.

   If you enter date data in an input column, use one of the following date formats:

   - yyyy-MM-dd
   - yyyy-MM-dd hh:mm:ss
   - yyyy-MM-dd hh:mm:ss.SSS

   Note: To test a rule statement that reads a list of date values as an input, enter the date data in the format yyyy-MM-dd.
6. Click **Run Test**.

   The Outputs view displays the path of the data values through the rule specification.

   If your rule statement contains the status value *Invalid* and you configured the action to describe exception data, the test results include columns for the exception data options that you specified.
7. Verify that the rule sets read the input values and write the output values that you expect.

   - A data value from a child rule set might generate output values on multiple rule sets. Verify that the test results indicate a data value on each rule set.
   - If a data value does not generate an output on a rule set, the test results show an empty field for the rule set.

Testing a rule set

Test a rule set to verify that the rule statements generate the correct output values. If possible, use the business data to test the rule specification.

To verify the full operation of the rule specification, select the top-level rule shape and test the rule specification.

1. Open the rule specification.
2. Select a rule set.
3. In the rule set properties, click **Test**.

   The properties view displays the input columns that you configured in the rule set.
4. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
5. Enter one or more data values in an input column. You can enter data values for any rule set in the rule specification.

   If you enter date data in an input column, use one of the following date formats:

   - yyyy-MM-dd
   - yyyy-MM-dd hh:mm:ss
   - yyyy-MM-dd hh:mm:ss.SSS

   Note: To test a rule statement that reads a list of date values as an input, enter the date data in the format yyyy-MM-dd.
6. Click **Run Test**.

   The Outputs view displays the path of the data values through the rule specification.

Testing a rule specification in advanced mode

Test a rule specification asset to verify that the data flows through the asset in the ways that you expect.

1. Open the rule specification that you created in advanced mode.
2. Select the **Configuration** tab.
3. Select a Secure Agent to run the test.

   To refresh the list of active Secure Agents, click the **Refresh** icon.
4. Enter one or more data values in the test panel.
5. Click **Test**.

   The test runs the rule logic in the asset. The output columns display the test results.

   If you configure exception indicators in the expression logic, the test results include columns for the exception data options that you specified.
6. Verify that the rule logic reads the input values and writes the output values that you expect.

Part VII: Verifier assets

This part contains the following chapters:

- Introduction to verifier assets, 195
- Verifier configuration, 203
- Verifier asset properties, 211
- Frequently asked questions, 225
- Address verification properties, 229
- Validation and testing, 233
- Input address fields, 234
- Output address fields, 237

CHAPTER 25

Introduction to verifier assets

A verifier asset evaluates the accuracy and deliverability of address records. Use a verifier to determine the accuracy of input addresses, fix errors in addresses, and enhance addresses where possible with additional information.

You can also use a verifier to measure and report on the quality of each address, to retrieve global and country-specific enrichments for addresses, and to verify addresses to the certification standards that a country defines.

To evaluate and update input addresses, the verifier compares the input address data with address reference data. The Secure Agent downloads the reference data to the local system when the mapping runs.

You create and test a verifier asset in Data Quality or Data Integration, and then add the asset to a Verifier transformation in a mapping in Data Integration. In Data Quality, you create the verifier asset from the **New** dialog box. In Data Integration, you create the verifier asset either from the **New Asset** dialog box or from a mapping.

Verifier operations

When you configure a verifier asset, you create a template for the input and output addresses that a Verifier transformation will read and write in a mapping. To create the template, you select a set of input fields that correspond to the address data columns that you will connect to the transformation. Likewise you select a set of output fields that represent the data structure that you want to write to downstream objects in the mapping.

When you run a mapping with a transformation that specifies a verifier, the transformation compares the input address fields to the reference data and performs the actions that the asset defines.

The verifier can perform the following operations:

Verify

A verifier verifies the address data to fix errors and to complete partial address records. To fix an address, the transformation looks for a positive match with an address in the reference data. The transformation replaces any incorrect or absent data elements in the input address with the correct elements in the address reference data.

Format

You can configure a verifier to format the output address data to meet the requirements that the local postal authorities specify. You can also configure a verifier to create an output address in a structure that suits your project or data structure requirements.

Suggest

A verifier can provide you with a list of address suggestions from which you can select the most relevant address. The address list includes the best matches between the input address and the reference data.

Measure

A verifier can measure the quality of an address and help you to understand and analyze the results of the verification process. You select the fields that contain the address status information when you configure the output address structure.

Enrich

A verifier can provide additional information as enrichments to help you better understand and use your address data. You can configure a verifier to provide enrichments that are specific to addresses in individual countries or enrichments that apply to every country. You can also configure a verifier to add geocoordinates as an enrichment to verified addresses.

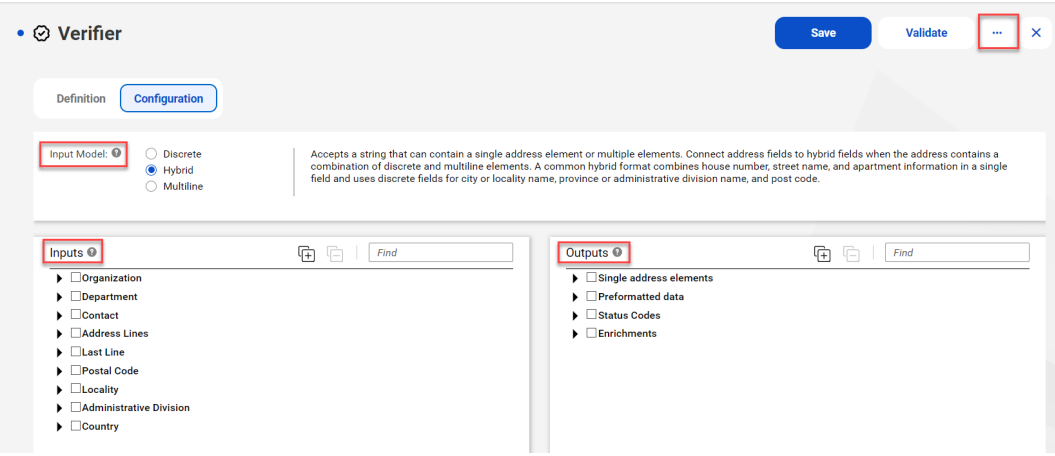
Certify

Postal certification is an additional level of address verification and standardization in Australia, Canada, France, New Zealand, and the United States. Certification requires that a software application validates address accuracy and prepares address records in accordance with the format and quality levels that the national postal carriers expect. The verification engine in Data Quality performs address verification in accordance with the latest certification standards.

Verifier structure

The verifier has configurable options on two tabs. Use the **Definition** tab options to provide a name, description, and location for the asset. Use the **Configuration** tab options to define a template for the addresses that the mapping will read and write at run time.

The following image shows the **Configuration** tab:



The **Configuration** tab has the following options:

1. Input Model. Indicates the type of address template that you will use to define the input address structure. Each input model has a corresponding set of address elements that represent the common ways in which a data source might store address data values.

   Note: The input model selection does not affect the output fields that you can choose from.

2. Inputs. Indicates the available input fields on the model that you select.

3. Outputs. Indicates the output fields that you can use to define the structure of the output address records.
4. Actions. Provides options that you can use to update the address verification properties.

Input model

An input model is a set of address elements that you can use to define the input address fields that the Verifier transformation can read at run time.

Select the model that best represents the data structure of the address records that the Verifier transformation will read.

Select one of the following input models for the verifier:

Discrete

Accepts input fields that contain a single address element. Connect address fields to discrete fields when each field represents a unique address element, such as a house number, apartment number, street name, city or locality name, or post code.

Hybrid

Accepts input fields that can contain a single address element or multiple elements. Connect address fields to hybrid fields when the address contains a combination of discrete and multiline elements. A common hybrid format combines house number, street name, and apartment information in a single field and uses discrete fields for city or locality name, province or administrative division name, and post code.

Multiline

Accepts input fields that contain multiple address elements. Connect address fields to multiline fields when each field represents multiple address elements. For example, Address Line 1 in a multiline address might contain "100 Main Street, Suite 1" and Address Line 2 might contain "New York, NY 10001."

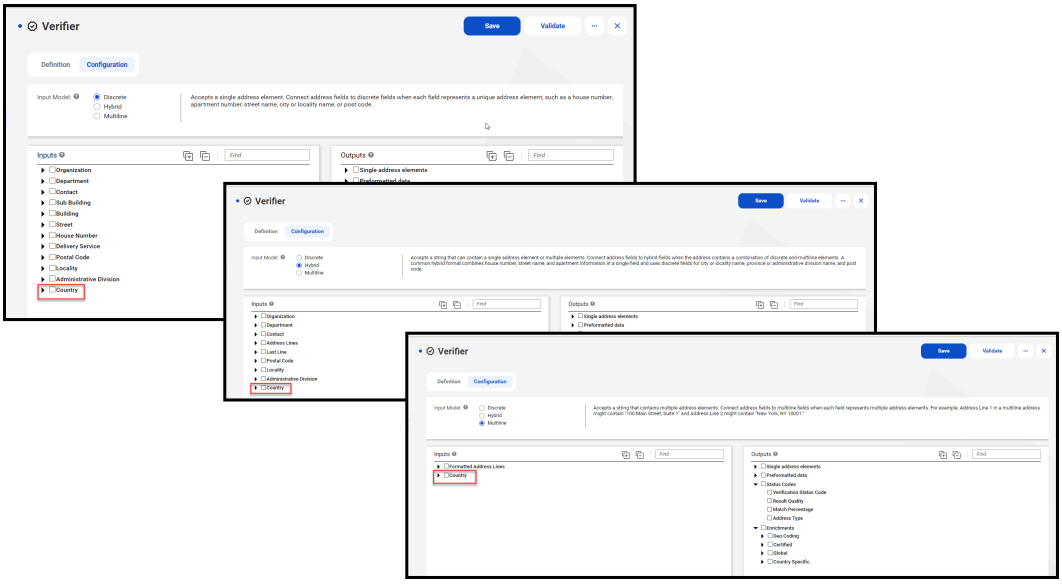
For more information, see “Selecting an input model” on page 203.

Inputs

The Inputs section shows the address fields that you can use to define the structure of your input address data. Select the appropriate fields for the address data that the verification process will analyze at run time.

The address fields that display are based on the input model that you select. You can expand an address field to drill-down to lower levels of the address structure.

The following image shows the input address fields:



The Inputs section includes the following fields:

- Discrete fields.
  Displays input fields that contain data for a single address element. Use when your input address structure assigns individual address elements to discrete fields.
- Hybrid fields.
  Displays input fields that combine discrete fields with fields that can contain multiple address elements. Use when your input address contains a mix of field types.
- Multiline fields.
  Displays input fields that contain multiple address elements. Use when your input address assigns multiple address elements to a single field.

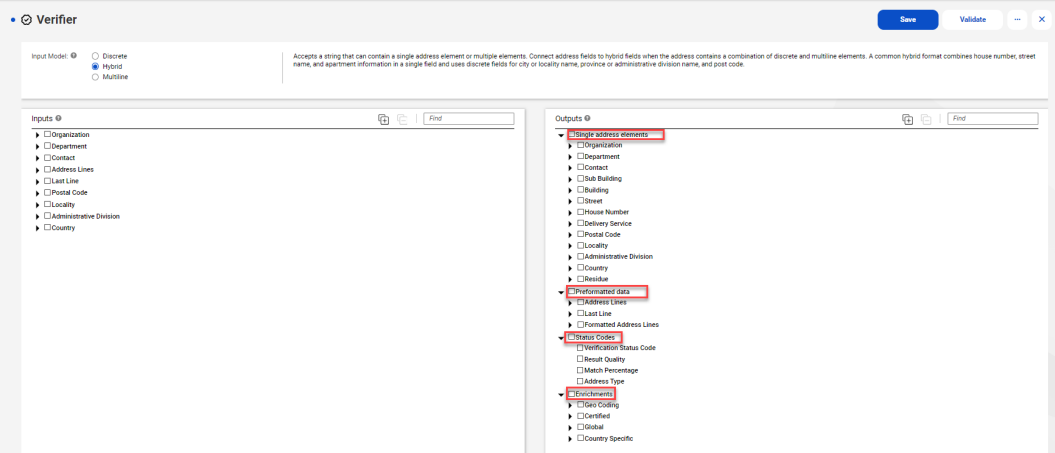
Outputs

The Outputs section shows the address fields that you can use to create an address record. Select the fields that represent the address information that you want the output records to contain.

The output fields in the Single address elements and Preformatted data field groups carry the same types of information as the input fields of the same name. The Single address output fields additionally contain the Residue field group. The Status Codes fields provide information about the output address quality. The Enrichments field add postally-relevant demographic and geographic information to the address records.

You can expand an address field to drill down to lower levels of the address structure.

The following image shows the output groups:



The Outputs section includes the following output groups:

- Single address elements group.
  Contains output fields that correspond to individual address elements. Use to create an address record that assigns the primary elements in the address to individual fields.
- Preformatted data group.
  Contains output fields that correspond to the lines of an address as they appear on an envelope or address label. Use to create an address record that you can print in a manner that is ready for postage.
- Status Codes group.
  Contains status reporting fields that report on the quality of the address. Use to examine the deliverability and accuracy of an address.
- Enrichments group.
  Contains multiple enrichment fields that provide additional information about an addresses and that can increase the usefulness of the address record.

Understanding the address structure

Before you can select the input model and configure the input and output address fields, you must understand the structure of the address data that the Verifier transformation will read and the structure of the addresses that the transformation will write.

You might work with a data steward to learn more about the source data structure and to understand the required structure of the transformation output. You might also run a profile on the source data to discover the input data structure.

Note: The input and output address structures do not need to be the same.

Address verification process flow

To verify an address, you perform steps in Data Quality and in Data Integration.

The verification process includes the following steps:

1. Analyze the structure of the source data, so that you can identify the fields that contain the address elements. You might run a profile to understand the data structure. You might also consult with a data steward who is responsible for the data. You will connect each address field to the appropriate input field in the Verifier asset that you create.
2. Configure a Verifier asset that can process the address data.

   To configure the asset, perform the following steps:

   a. In the Verifier, select the input fields that match the address fields in the source data.

   b. Select the output fields that represent the structure of the address data that you want to write to the mapping target.

      Optionally, add status reporting fields to the output fields. Status reporting fields are optional but important elements that report on the success or otherwise of the verification operation.

      Optionally, add enrichment fields to the output fields.

   c. Review the verification properties, and update the property values if necessary.
3. In Data Integration, define a mapping that can run the verification operation.

   As part of the mapping definition, do the following:

   a. Add the verifier asset to a Verifier transformation.

   b. Connect the verifier asset input and output fields to the upstream and downstream objects in the mapping.
4. Run the mapping and review the outcome of the verification operation. You can modify the Verifier and re-run the mapping to obtain better results.

Certified verification

Postal certification is an additional standard of address verification in Australia, Canada, France, New Zealand, and the United States. You can configure a verifier asset to create output addresses that meet the certification standards.

The following postal services define the certification standards in each country:

Australia Post

Australia Post defines the Address Matching Approval System (AMAS) standard for software applications that validate Australian addresses.

Canada Post

Canada Post defines the Software Evaluation and Recognition Program (SERP) standard for software applications that validate Canadian addresses.

La Poste

La Poste defines the National Address Management Service (SNA) standard for software applications that validate French addresses.

New Zealand Post

New Zealand Post defines the SendRight standard for software applications that validate New Zealand addresses.

United States Postal Service (USPS)

The USPS defines the Coding Accuracy Support System (CASS) standard for software applications that validate United States addresses.

To generate addresses that meet the certification standards, select *Certified verification* on the Process tab of the **Properties** dialog box.

Note: The verification engine can verify addresses to the certification standards that the postal authorities in each country define. The engine can generate certification reports for addresses that you verify in certified mode in Australia, Canada, New Zealand, and the United States. The engine is formally certified by the postal authorities in Australia, New Zealand, and the United States.

Address enrichments

Address enrichments provide additional information that helps you better understand and use the address data. The enrichments include data that increases the precision of the address, provides additional information about the administrative or statistical regions to which the address belongs, and helps the postal service to find the destination mailbox more easily.

For example, a verifier can return the latitude and longitude geocoordinates of an address. For more information, see " Enrichments: Geocoding fields" on page 240.

Many enrichments are specific to addresses in individual countries, such as the census-based information that a verifier can return for United States addresses. The output fields that accompany certified verification are also a form of address enrichment. Some enrichments apply equally to every country. For more information, see " Enrichments: Country Specific fields" on page 251 and " Enrichments: Global fields" on page 250.

Verifier assets and mappings

The verifier that you create is available to the users who create mappings in Data Integration. You add a Verifier transformation to a mapping and then add a verifier asset to the transformation. The Verifier transformation applies the logic in the verifier asset to the data source that the mapping identifies.

A Verifier transformation works in a similar way to a Mapplet transformation. You connect the transformation inputs and outputs to other objects in the mapping in the same manner as a mapplet connects to other mapping objects. When you run the mapping, the Verifier transformation applies the verifier logic to the input data and generates output data for downstream objects.

Verifier assets and dimensions

The data quality issues that you may find in your data can fall into a range of common categories. Data quality assets can identify the categories as dimensions. When you configure an asset, you can use the **Dimension** property to indicate the type of data quality issue that you want the asset to examine.

Find the Dimension property on the **Definition** tab of the asset.

For more information about data quality dimensions, see the *Introduction* help in Data Quality.

Reference data

Address reference data contains authoritative data for the postal addresses in a country. For many countries, the address reference data contains complete data for every postal address. When you run a mapping with a Verifier transformation, the transformation compares the input address data to the reference data files.

The mapping compares the elements in each input address separately and collectively against the reference data to verify that the address identifies a single deliverable address. The mapping results include the verified or corrected versions of the address and any additional information that you requested.

Consider the following rules and guidelines for address reference data:

- The Secure Agent downloads the current versions of the files that you need. If the current version of a file is already present on the Secure Agent host machine, the file is not downloaded again.

  The download operation checks the files that the mapping requires and does not necessarily check the status of every reference data file on the system.
- The download operation downloads a hash file with each reference data file. When you run a mapping, the Secure Agent uses the hash file data to verify that the reference data location contains the current version of the file.

  The address reference data files and hash data files are read-only files. Do not move or delete any file that the Secure Agent downloads for address verification.
- The default location of the reference data files is *[Informatica_root_directory]/avdata*. You can review or update the reference data location in the Administrator service.
- If you change the reference data location, the Secure Agent downloads the reference data to the new location on the next occasion that you run a mapping.
- Ensure that you have enough disk space available in the system for the download files. The disk space that you require depends on the countries for which you download data and the number and type of files that you download. The full set of global reference data files takes up approximately 18 GB of disk space.

  If the current mapping requires a large volume of reference data, the download process may take some time.
- The reference data files require a valid license. The verifier reads the license information from one or more license files that the Secure Agent can access. You enter the path to the license files as a data property on the Secure Agent.
- The address reference data that enables certified address verification for United States addresses is not licensed for use outside the United States.

For more information on the reference data properties, see Chapter 29, “Address verification properties” on page 229.

CHAPTER 26

Verifier configuration

When you configure a verifier asset, you select the appropriate address model for the input address structure and you specify the input and output fields. Select the input fields based on the structure of the source data that the mapping will read. Select the output fields based on the task that you want the mapping to perform.

You can also review and update the properties that determine how the Verifier transformation reads, writes, and processes data at run time.

You can configure a verifier to perform the following tasks on address data:

- Verify and correct address data
- Update the address structure
- Generate address suggestions
- Measure address quality
- Generate geocodes
- Retrieve country-specific address enrichments
- Retrieve global address enrichments
- Retrieve certification data

Note: Every task that you perform with a verifier will attempt to verify and correct the address data.

Selecting an input model

Review the source data structure and select the input model for the address data.

1. Examine the input address structure. You can run a profile to understand the source data. If a data steward is assigned to the data, you might also consult with the data steward.
2. Select one of the following address models:
   - **Discrete**. The discrete input fields contain address information on a single data element, such as a house number, street, or post code.
   - **Hybrid**. The hybrid input fields can contain a single address element or multiple address elements.
   - **Multiline**. The multiline input model contains multiple address elements. Each input column corresponds to a line of an address.

For more information, see "Input model" on page 197.

Selecting the input fields

Select the input fields that correspond to the address fields in the data that the Verifier transformation will read.

For example, if the input data contains the house number, street, and apartment in a single field, you might select an Address Lines field in the Hybrid input model. You can enter the remaining address information, such as the locality, province, and postal code in their respective fields.

If the input data contains street and locality in a single field, select a Formatted Address Lines field in the Multiline input model.

If the input data contains the address information in individual address fields, select the corresponding address fields in the Discrete input model.

Note: An address might use more than one level within a given field. For example, an address in India can contain four locality levels. The element of the largest size is level 1. Levels 2, 3 and other levels are sub-divisions of the preceding element. Disregard any level that the address does not use.

For more information about the uses of the input address fields, see Chapter 31, “Input address fields” on page 234.

Selecting the output fields

The verifier provides you with a range of output address fields. The output fields that you select depend on your objectives for the address verification process.

In some cases, the selection of the output fields might overlap. For example, when you select output fields to verify an address or generate address suggestions, you might select the same output fields.

You can use the output fields to update the structure of the address data. Additionally, you can select Status Codes and Enrichments fields to perform tasks such as reporting on the quality of each address and retrieving enrichment data for valid addresses.

For more information about the uses of the output address fields, see Chapter 31, “Input address fields” on page 234.

Selecting output fields to verify and correct addresses

Verify your address records to determine the accuracy and deliverability of the addresses.

Address verification is the basic objective of all tasks that a verifier can perform. All tasks that you perform with a verify asset will try to verify and correct the input address data.

If your objective is to verify your addresses and to preserve the input address structure, you can choose the same fields for input and output.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve.

For example, the source data might include the following address:

```
1960 W CHELSEA AVE STE 2006R
ALLENTOWN PA 18104
USA
```

The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
ALLENTOWN PA 18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

The following table shows the output fields that you can select to verify the address data:

Output Fields	Address Information
Formatted Address Lines 1	1960 W CHELSEA AVE STE 2006R
Formatted Address Lines 2	ALLENTOWN PA 18104
Country ISO3 1	USA

Selecting output fields to update the address structure

Use the output fields to organize the address data into a set of columns that suits your project or data storage requirements. For example, your current address records might contain multiple address elements in a single field. You can specify individual fields for each address element in the output address.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve.

   For example, the source data might include the following address:

   ```
   1960 W CHELSEA AVE STE 2006R
   ALLENTOWN PA 18104
   USA
   ```

   The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
ALLENTOWN PA 18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

The following table shows the output fields that you can select to update the address structure of the address data:

Output Fields	Address Information
Sub Building 1	STE 2006R
Street 1	W CHELSEA AVE
House Number 1	1960
Postal Code 1	18104
Locality 1	ALLENTOWN
Administrative Division Name 1	PA
Country Name 1	UNITED STATES

Selecting output fields to generate address suggestions

If your source data contains an incomplete address, or an address of uncertain quality, the address verification mapping can return valid suggestions for the addresses from the reference data. To receive the list of address suggestions, set the verification mode in the verifier properties to *Verification with suggestions*.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve.

   For example, the source data might include the following address:

   ```
   1960 W CHELSEA AVE STE 2006R
   18104
   USA
   ```

   The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

The following table shows the output fields that you can select for the address data:

Output Fields	Address Information
Sub Building 1	STE 2006R
Street 1	W CHELSEA AVE

Output Fields	Address Information
House Number 1	1960
Postal Code 1	18104
Locality 1	ALLENTOWN
Administrative Division Name 1	PA
Country Name 1	UNITED STATES

Note: The mapping will return the complete address, including the city and state information.

4. Choose the *Verification with suggestions* option from the **Verification mode** field of the **Process** tab property to generate the list of address suggestions for the address.

Selecting output fields to measure address quality

You measure the quality of the address data to check the deliverability of the addresses and to identify any address that the address verification mapping cannot correct. Select output fields that write each address data element to a separate field. Also, select the address validation status field that provide detailed information on the accuracy and completeness of the output data.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve. Include the Status Codes fields in the output.

   For example, the source data might include the following address:

   ```
   1960 W CHELSEA AVE STE 2006R
   ALLENTOWN PA 18104
   USA
   ```

   The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
ALLENTOWN PA 18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

   The following table shows the output fields that you can select for the address data:

Output Fields	Address Information
Sub Building 1	STE 2006R
Street 1	W CHELSEA AVE

Output Fields	Address Information
House Number 1	1960
Postal Code 1	18104
Locality 1	ALLENTOWN
Administrative Division Name 1	PA
Country Name 1	UNITED STATES
Verification Status Code	V
Match Percentage	100

Selecting output fields to generate geocodes

You can add geocoordinate data to address records to improve the likelihood of delivery, for example in rural areas. The geocodes indicate the closest latitude and longitude coordinates for each address that verification can provide, based on the level of precision in the reference data and the precision level that you select. Select the Geo Coding fields to add geocodes to the address records.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve.

   Include the geocoding fields that you require in the output. Find the fields under **Enrichments** > **Geo Coding** in the Outputs section.

   For example, the source data might include the following address:

   ```
   1960 W CHELSEA AVE STE 2006R
   ALLENTOWN PA 18104
   USA
   ```

   The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
ALLENTOWN PA 18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

   The following table shows the output fields that you can select for the address data:

Output Fields	Address Information
Sub Building 1	STE 2006R
Street 1	W CHELSEA AVE

Output Fields	Address Information
House Number 1	1960
Postal Code 1	18104
Locality 1	ALLENTOWN
Administrative Division Name 1	PA
Country Name 1	UNITED STATES
Arrival Point Coordinates	[Latitude and longitude coordinates]
Arrival Point Accuracy Code	[Geocoding accuracy code value]

Selecting output fields to retrieve certification data

You can retrieve certification data to verify that your addresses meet the data standards that a local postal service may define.

The certification standards require that each address in an address set includes values that can affirm the deliverability and accuracy of the address. To receive the certification values, set the verification mode in the verifier properties to *Certified verification*.

1. Select the input model for the address data.
2. Select the input fields. The source data structure defines the input fields for the address.
3. Select the output fields. You define the output fields based on the result that you want to achieve.

   Include the CASS Status, Barcode, and Carrier Route fields in the output. Find the fields under **Enrichments** > **Certified** > **UnitedStates-CASS** in the Outputs section.

   For example, the source data might include the following address:

   ```
   1960 W CHELSEA AVE STE 2006R
   ALLENTOWN PA 18104
   USA
   ```

   The following table shows the input fields that correspond to the address data:

Address Information	Input Fields	Input Model
1960 W CHELSEA AVE STE 2006R	Formatted Address Lines 1	Multiline
ALLENTOWN PA 18104	Formatted Address Lines 2	Multiline
USA	Country	Multiline

The following table shows the output fields that you can select for the address data:

Output Fields	Address Information
Sub Building 1	STE 2006R
Street 1	W CHELSEA AVE
House Number 1	1960
Postal Code 1	18104
Locality 1	ALLENTOWN
Administrative Division Name 1	PA
Country Name 1	UNITED STATES
Certification Status	[Certification status code value]
Barcode	[An 11-digit number that represents the delivery point for the record.]
Carrier Route	[A four-character code assigned to a mail delivery or collection route within a 5-digit ZIP Code.]

CHAPTER 27

Verifier asset properties

You can configure properties on a verifier asset that determine how a mapping that includes the asset will read, write, and process addresses.

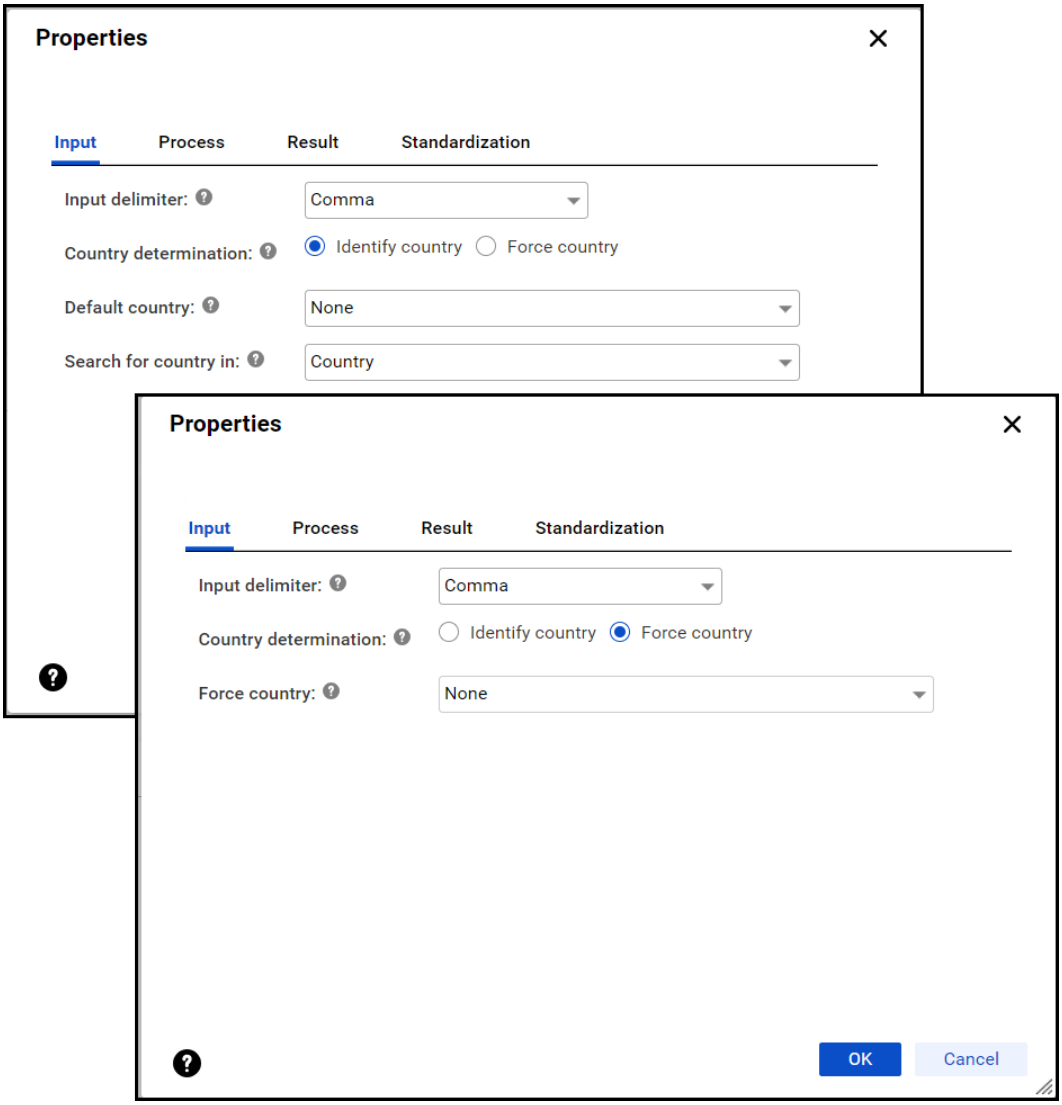
The **Properties** dialog box displays the options on **Input**, **Process**, **Result** and **Standardization** tabs. On the **Input** tab, you can specify the delimiter that the mapping uses to identify the input fields and set options that identify the destination country for the addresses. On the **Process** tab, you can specify the type of verification that you want the mapping to perform and the depth at which level you want to verify the addresses. On the **Result** tab, you can specify how the mapping returns address suggestions and how the mapping can update addresses that return an I value for the verification status code. On the **Standardization** tab, you can specify the language and formatting options that the mapping applies to the output data.

Edit the properties to suit your source data and your project requirements.

Input properties

The **Input** tab properties determine how the mapping parses the input fields and can also assign a destination country to the addresses.

The following image shows the **Input** tab properties:



You can review and update the following properties:

Input delimiter

Specifies the field separator that the input addresses use. For example, if you set the Input delimiter to comma, the verifier treats strings on either side of a comma as separate fields. The mapping treats each line of data as a separate field.

You can select one of the following options:

- Comma. Identifies a comma as the field separator. Comma is the default value.
- Semicolon. Identifies a semicolon as the field separator.
- Tab. Identifies a tab character as the field separator.

Country determination

Determines how verification identifies the destination country for an address.

Select one of the following options:

- Identify country. Searches for country information in the field or fields that you specify.

- Force country. Instructs the mapping to assign all addresses to the country that you specify, regardless of any country information in the address.

Default country

Applies when you select *Identify country* as the Country determination option. Assigns a country to any address that does not contain country information in the field that you specify. Select the country from the drop-down list.

Force country

Applies when you select *Force country* as the Country determination option. Assigns the country that you select to all input addresses. Select the country from the drop-down list.

If you select *None*, the asset ignores the property.

Search for country in

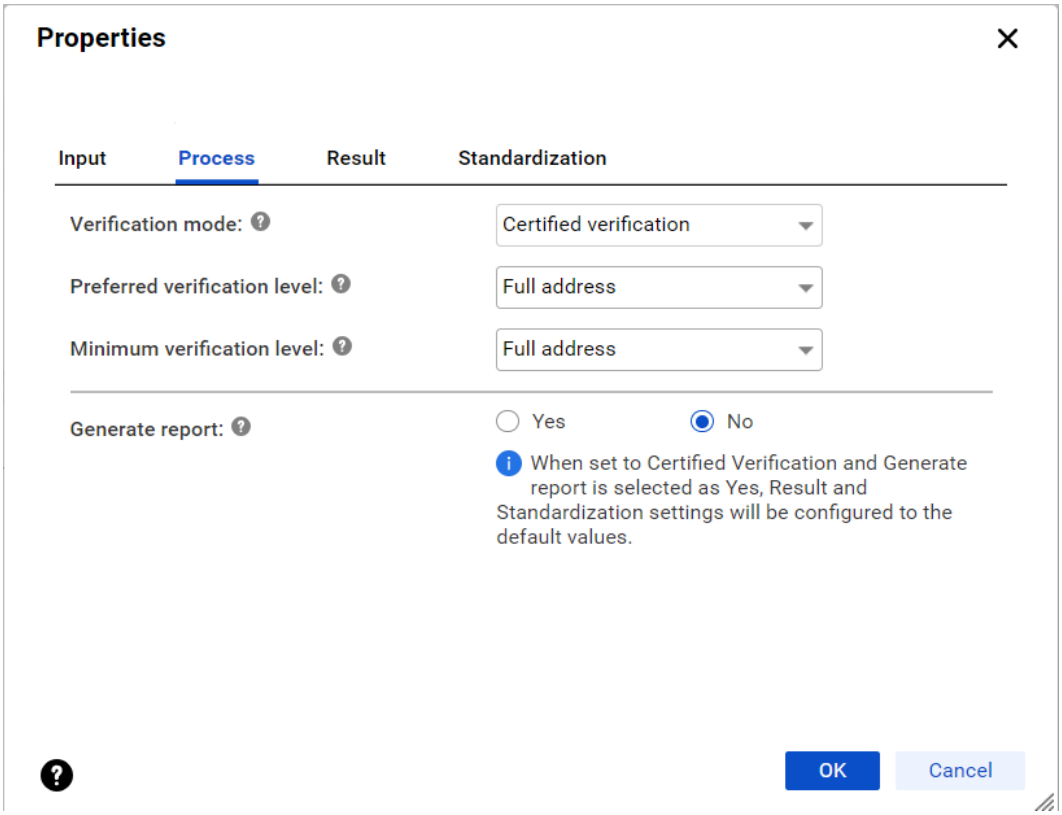
Applies when you select *Identify country* as the Country determination option. Identifies the address field or fields in which the mapping looks for country information.

You can specify that the mapping searches the Country field, any formatted address lines in the address, or a combination of the Country and formatted address line fields. Country is the default value.

Process properties

The **Process** tab properties specify the verification process that the address verification mapping applies to the data and also specify the depth to which the mapping verifies the address data.

The following image shows the **Process** tab properties:



You can set the following properties:

Verification mode

Identifies the address verification process that the mapping runs on the input data.

You can select one of the following options:

- Verification only. Default value. The mapping verifies and corrects the input address data.
- Verification with suggestions. The mapping verifies and corrects the input address data and returns address suggestions for any input address that is ambiguous or incomplete.
- Certified verification. The mapping verifies and corrects the input address data to the certification standards that the postal authority in a country specifies. You can perform certified verification on addresses in Australia, Canada, France, New Zealand, and the United States.

  The verifier complies with the following postal certifications:

  - Address Matching Approval System (AMAS) certification for Australia Post.
  - Coding Accuracy Support System (CASS) certification for the United States Postal Service.
  - SendRight certification for New Zealand Post.
  - Service National de L'Adresse (SNA) certification for La Poste of France.
  - Software Evaluation and Recognition Program (SERP) certification for Canada Post.

The mapping can also generate a certification report for addresses that you verify in Australia, Canada, New Zealand, and the United States.

Preferred verification level

Specifies the depth of address information that the mapping seeks to verify in the input data. The property works in tandem with the **Minimum verification level** property.

Minimum verification level

Specifies the minimum depth of information that the mapping can accept in an address that it considers valid. The property works in tandem with the **Preferred verification level** property.

You can select one of the following options on the Preferred verification level and Minimum verification level properties:

- Full address. Default value on both properties. The mapping attempts to verify all available information.
- Premise level. The mapping attempts to verify all information to building or house number level.
- Street level. The mapping attempts to verify all information to street or delivery service level.
- Locality level. The mapping attempts to verify all information to locality level. The mapping ignores delivery service information. The mapping can also verify postal codes.
- Primary locality level. The mapping attempts to verify all information to locality level 1 and ignores more granular or lower-level locality information. The mapping can also verify postal codes.

If you set both levels to Full and the mapping cannot verify an input data value, address verification invalidates the address.

Generate report

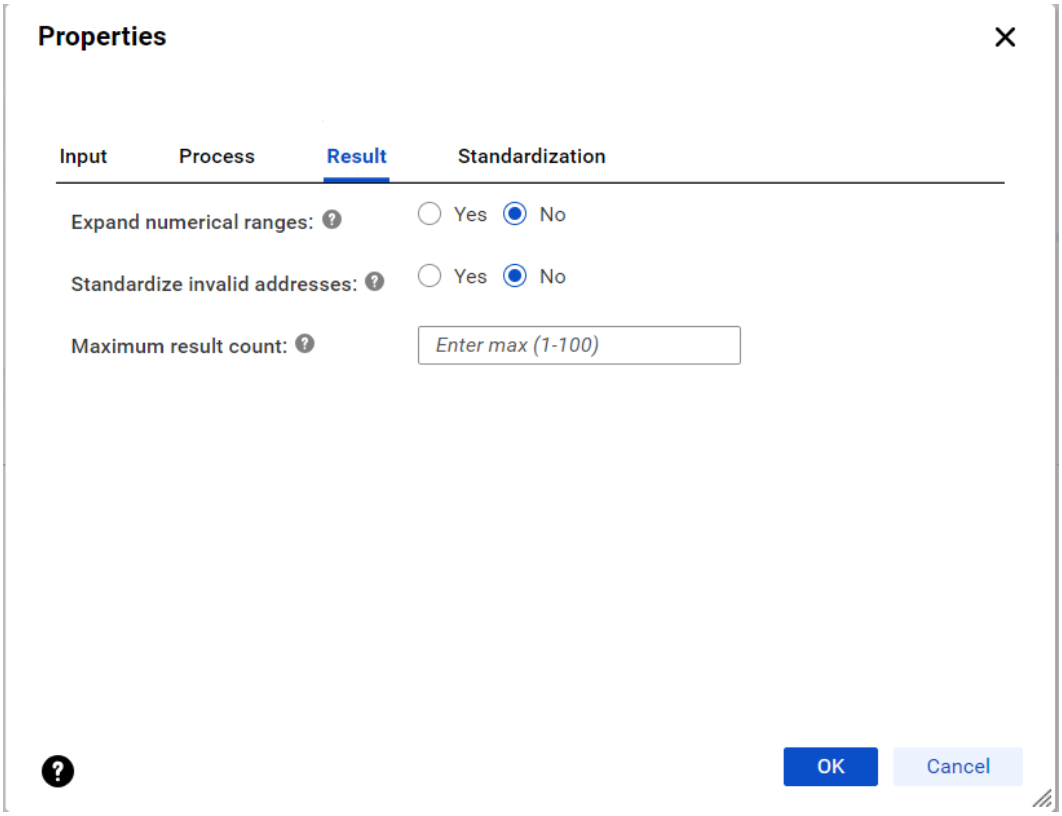
Specifies whether to generate a report that describes the address set that you verify in certified mode. Choose Yes to generate the report.

The **Generate report** property appears when you set the **Verification mode** property to Certified verification. For more information about certification reports, see "Certification report details" on page 220.

Result properties

The **Result** tab properties specify how the mapping returns address suggestions and whether the mapping updates addresses that return an I value for the verification status code.

The following image shows the **Result** tab properties:



You can set the following properties:

Expand numerical ranges

Specifies whether to expand house, building, or sub-building number ranges into individual numbers in address suggestions when the individual numbers are available in the reference data. The default value is No.

Standardize invalid addresses

Determines whether to standardize common address elements in addresses that do not otherwise pass validation, such as addresses that return an I score in the Verification Status Code field. Use the property when you do not expect to find valid versions of the address data and you want to standardize terms that occur throughout the data. The default value is No.

Address verification can standardize the following address elements:

- Street types, for example Road to Rd.
- Pre-directional and post-directional indicators, for example West to W.
- Delivery service terms, for example Post Office Box to P. O. Box.
- Sub-building descriptors, for example Apartment to Apt.
- State, province, or regions, for example California to CA.

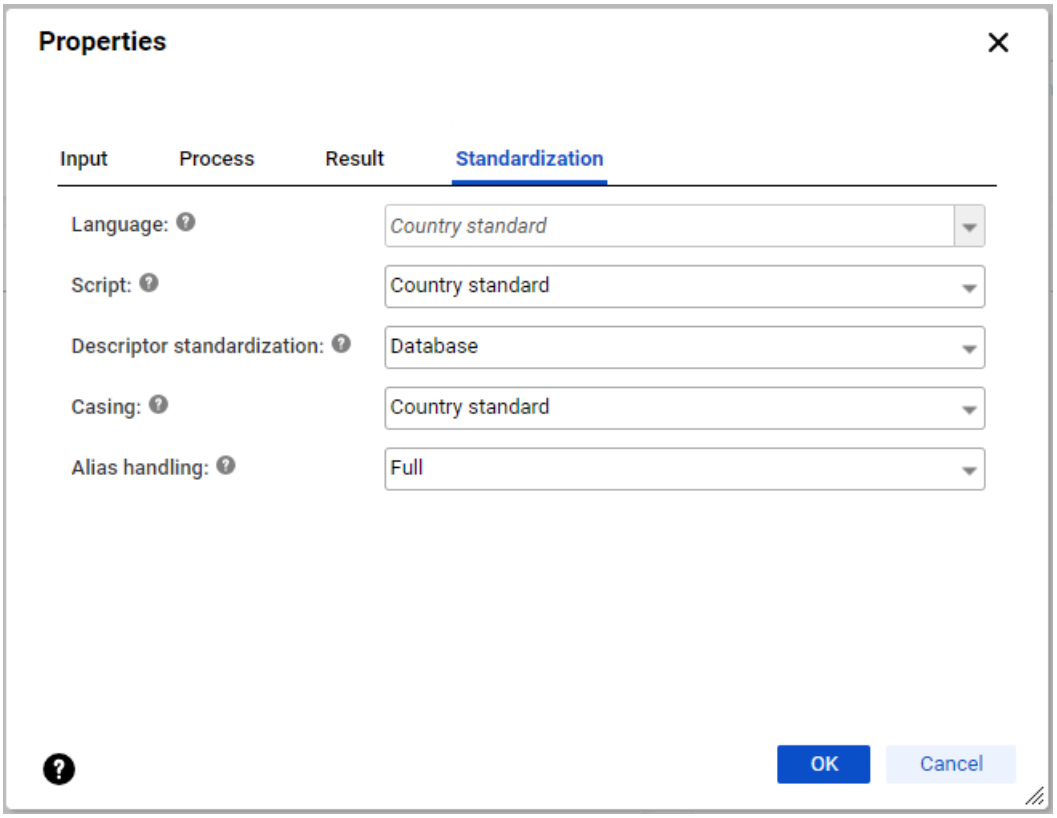
Maximum result count

Specifies the maximum number of address suggestions that address verification can return for an input address when the **Verification mode** property is set to *Verification with suggestions*. You can set the property to a value from 1 through 100. The default value is 20.

Standardization properties

The **Standardization** tab properties specify the language and formatting options that the mapping applies to the output data.

The following image shows the **Standardization** tab properties:



You can configure the following properties:

Language

Indicates how the verifier selects the language in which to return an output address. You can set the property to return address data in the default language for the address country, or you can opt to preserve the input language in the output address data where it is possible to do so. Additionally, the **Language** property options include a list of ISO 639-3 codes that identify a range of languages around the world. You can select one or more language codes from the list.

Before you select a non-default option on the property, ensure that the Secure Agent that runs address verification mappings has access to reference data files for the country that you specify.

You can select one of the following options:

Country standard

Default value. Returns all available address data in the default language that the reference data files specify for the country.

Preserve input

Returns the address data in the language that the input data uses. Preserves the language if the reference data contains the address in the input language.

Select a country from a list of ISO language codes

Returns all available address data in the language that you select.

You can select multiple languages. Select the languages in order of preference. If the verifier cannot return the address data in a language that you specify, it tries the next language.

Script

Indicates the character set to use for the output data when the country supports more than one character set.

Country standard

Default value. Returns the address in the principal character set in which the reference data stores the address.

Country alternate

Returns the address in an alternative character set that the reference data stores for the address, if the character set is present. Otherwise uses the principal script.

Latin

Returns the address in a Latin script.

Preserve input

Returns the address in the character set that the input data uses.

Transliteration type

Indicates whether the transliteration process uses the default transliteration type for the address or an alternative transliteration type. Applies when you select *Latin* or *Preserved input* in *Script* property.

Limit Latin characters

Determines the extent to which the verifier restricts the use of characters from Latin character sets in an output address. Applies when you select *Latin* or *Preserved input* in *Script* property.

You can select one of the following options:

NoLimit

Returns the transliterated output in full. NoLimit is the default value.

Latin1

Limits the Latin characters to the Latin-1 (ISO-8859-1) character set. For example, replaces the following characters with Latin-1 characters:

Input	Output (Latin-1)
ä	ä
á	á
à	à
ả	a
ā	a

ASCIISimplified

Changes non-ASCII Latin characters to characters in the ASCII character set. Uses basic conversion, for example ö to o.

ASCIIExpanding

Changes non-ASCII Latin characters to characters in the ASCII character set. Uses extended conversion, for example ö to oe.

Note: Verify that your reference data files support the characters that the **Script** options specify. For example, a database that supports ASCII characters only will not store all Latin characters.

Descriptor Standardization

Specifies whether to abbreviate street and directional descriptors when abbreviations are available in the reference data.

You can select one of the following options:

- Database. Default value. Returns the standard element descriptors from the reference data.
- Long version. Returns the expanded form of the element descriptors. For example, a mapping returns STREET for the input element ST.
- Preserve input. Copies the element descriptors from the input to the output.
- Short version. Returns the abbreviated form of the element descriptors. For example, a mapping returns AVE for the input element AVENUE.

Casing

Specifies the character case for the output address data.

You can configure one of the following options:

- Country standard. Default value. Returns the output in the standard casing style that the reference data uses for the country rules.
- Lower. Returns the output in lowercase letters.
- Mixed. Returns the output in mixed case according to country-specific casing guidelines.
- Upper. Returns the output in uppercase letters.

Alias handling

Identifies the policy that address verification applies to recognized alternative names for address elements in an input address.

You can select one of the following options:

- Full. Retains the alias for the address element. The default value is Full.
- PreserveOfficial. Returns the address element name that the postal authority prefers. The name may be the official name or the alias, depending on the local usage.
- PostalAdmin. Returns the official name of the address element.
- Abbreviation. Returns the address element in its abbreviated form.

Certification report details

When you submit a certified address set to a postal authority, you include a certification report with the address set. The Secure Agent can generate a certification report for an address data set in Australia, Canada, New Zealand, and the United States. The report contains information that you enter as properties on the verifier asset.

The Secure Agent generates the report when you run a mapping that contains the verifier asset. The Secure Agent also downloads the report to a directory on the Secure Agent machine.

The default download location for certification reports is [Informatica_root_directory]/avreports. You can review and update the download location on the CertifiedReportLocation property in the CDQAV property set. For information about CDQAV properties, see "Verification properties in the CDQAV property set" on page 230.

Use the **Choose report to generate** property on the verifier asset to identify the certification standard for which you'll generate a report. Find the property on the **Process** tab in the verifier asset properties.

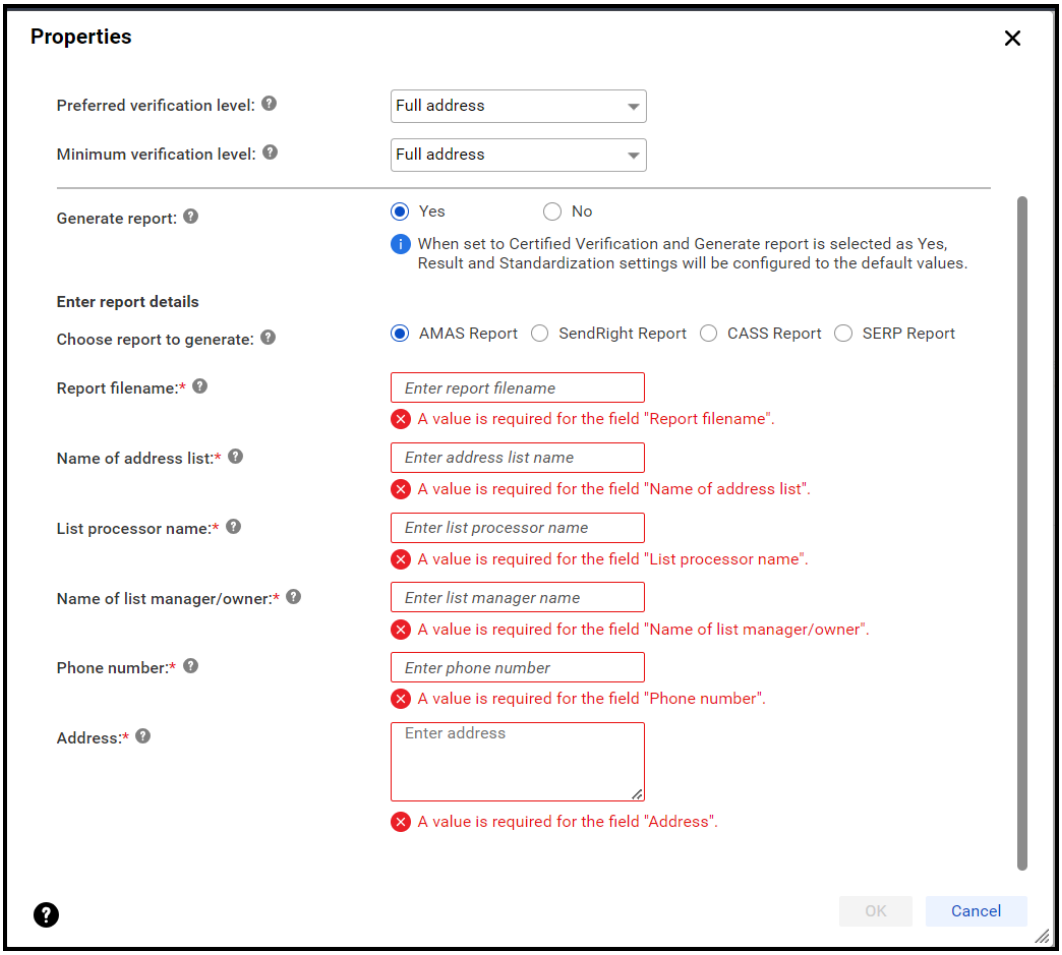
You can select one of the following options:

- AMAS Report. Select for certified addresses in Australia.
- CASS Report. Select for certified addresses in the United States.
- SendRight Report. Select for certified addresses in New Zealand.
- SERP Report. Select for certified addresses in Canada.

When you select an option, the property sheet displays a series of fields that you can complete. Enter the data that each field requires.

AMAS report fields

The following image shows the AMAS report fields:

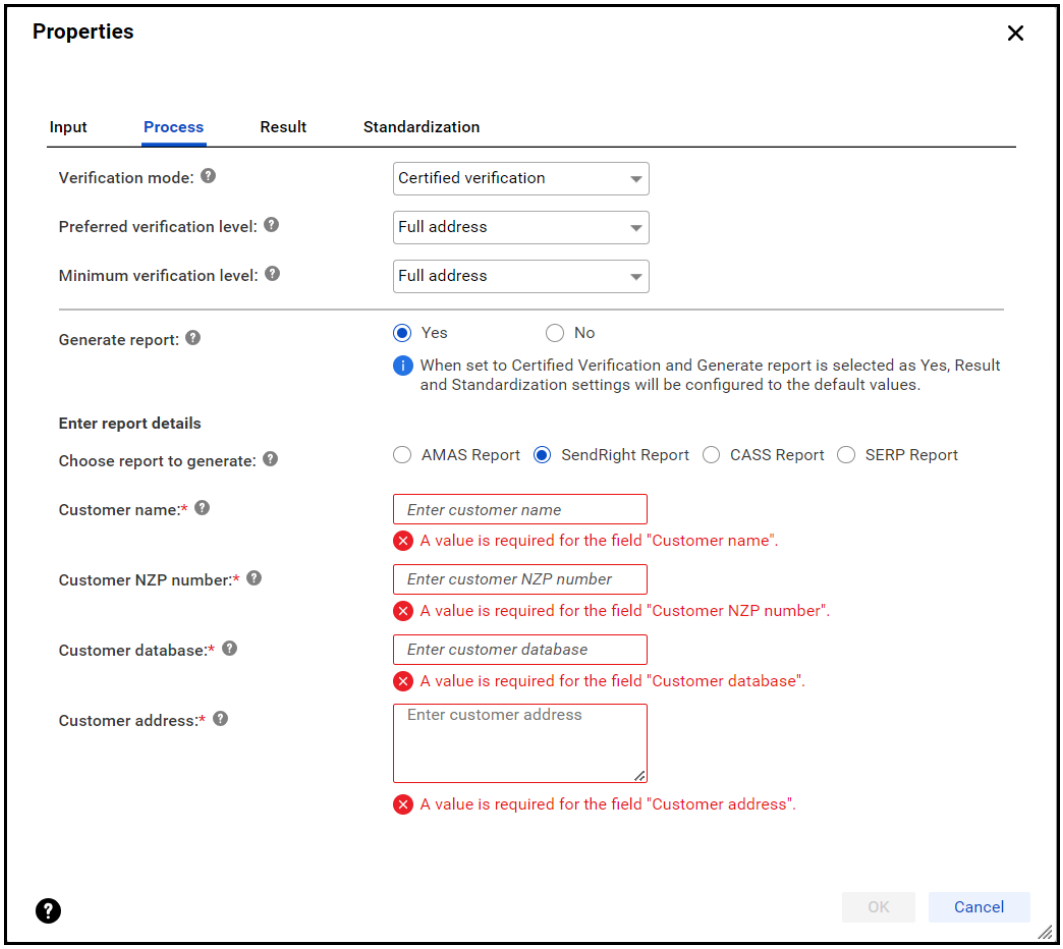


The following table describes the AMAS report fields:

Field	Description
Report filename	Name of the report that the address verification operation creates.
Name of address list	Name of the address record set that you submit to Australia Post.
List processor name	Name of the organization that submits the address record set.
Name of list manager/owner	Name of the manager or owner of the address data in the organization.
Phone number	Contact telephone number of the organization that submits the address record set.
Address	Address of the organization that submits the address record set.

SendRight report fields

The following image shows the SendRight report fields:

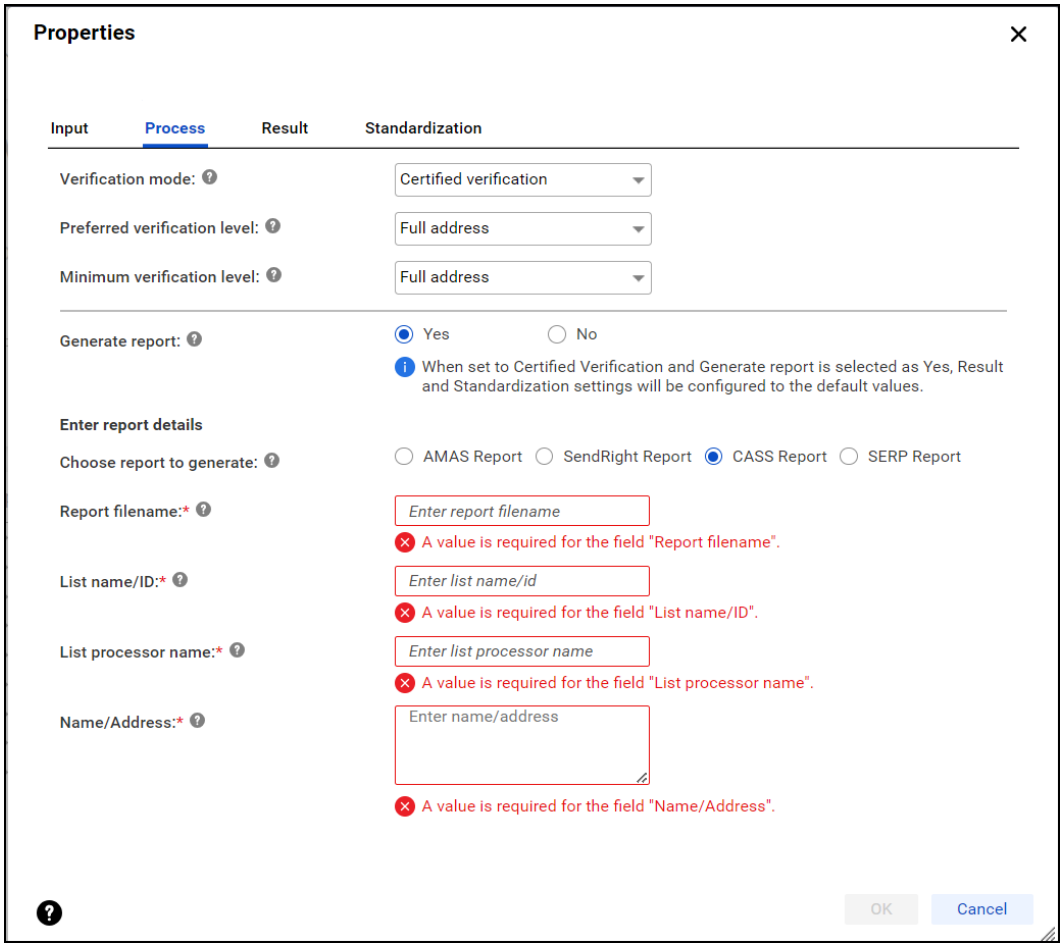


The following table describes the SendRight report fields:

Field	Description
Customer name	The name of the organization that submits the address record set.
Customer NZP number	The New Zealand Post account number of the organization that submits the address record set. If a mailhouse submits the records on behalf of the organization, enter the mailhouse transport identification (TPID) number.
Customer database	The name of the file that contains the address record set.
Customer address	The address of the organization that submits the address record set.

CASS report fields

The following image shows the CASS report fields:

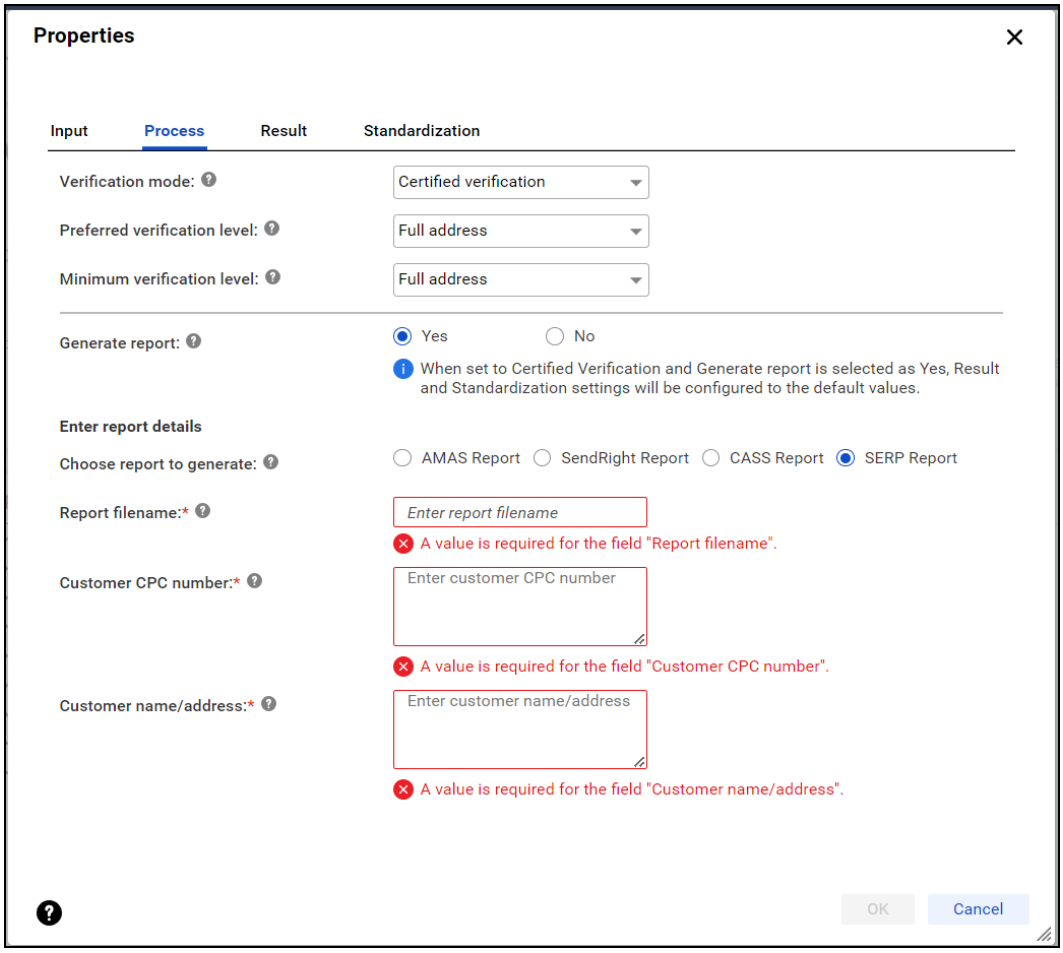


The following table describes the CASS report fields:

Field	Description
Report filename	Name of the report that the address verification operation creates.
List name/ID	Name or identification number of the address list that your organization submits to the mail carrier.
List processor name	Name of the organization that performs the address verification.
Name/Address	Postal name and address of the organization that performs the address verification.

SERP report fields

The following image shows the SERP report fields:



The following table describes the SERP report fields:

Field	Description
Report filename	Name of the report that the address verification operation creates.
Customer CPC number	Customer number that the Canada Post Corporation issues to the organization that performs the address verification.
Customer name/address	Name and address of the organization that performs the address verification.

CHAPTER 28

Frequently asked questions

The frequently asked questions provide quick solutions on how to set up various operations in a verifier. Each question corresponds to a value on a property or option in the verifier.

Why does the verifier asset show multiple instances of a single field?

The verifier asset displays multiple input and output fields for many field types. Use the fields to define an input and output address structure that best suits your address records and your data needs.

In many cases, the fields allow the asset to verify addresses that contain multiple levels of information for a single element. For example, a United Kingdom address might include the district of BETHNAL GREEN and the town of LONDON. To verify the address, you select the Locality 1 and Locality 2 input and output fields on the asset. At run time, the Data Integration user who adds the asset to a Verifier transformation can assign the fields that contain BETHNAL GREEN and LONDON to the Locality 2 and Locality 1 fields respectively.

Additionally, a single Address Lines or Formatted Address Lines field can contain data for multiple address elements. Your address records might use the fields to create address data that matches the standard layout of a printed address. For example, you might select the Formatted Address Lines 1 and Formatted Address Lines 2 fields for the following address data:

```
Field 1: 350 5TH AVE
Field 2: NEW YORK NY 10118
```

Note: You can select the Address Lines 1 field and the Last Line 1 field for the same address.

How do I set the input and output address properties for an address?

The toolbar on the **Configuration** tab includes an **Actions** menu option. Use the menu option to open the **Properties** dialog box. The dialog box contains properties on four tabbed pages that you can configure based on the type of information that you want the address verification mapping to read and write.

For more information about configuring the properties, see Chapter 27, “Verifier asset properties” on page 211.

How do I standardize street and directional descriptors?

In the **Properties** dialog box, select the **Standardization** tab. The **Standardization** tab includes the **Descriptor standardization** property, which you can use to standardize the descriptor data.

You can choose one the following options to standardize the address elements:

- Database, or the reference data standard
- Long version
- Preserve input
- Short version

For more information about setting the **Standardization** tab properties, see "Standardization properties" on page 217.

How do I activate suggestions for addresses that I cannot verify?

In the **Properties** dialog box, select the **Process** tab. The **Process** tab includes the **Verification mode** property, which you can use to activate suggestions for an address. Choose the *Verification with suggestions* option to receive address suggestions.

For more information about setting the **Process** tab properties, see "Process properties" on page 213.

How do I set the level of verification for an address?

In the **Properties** dialog box, select the **Process** tab. The **Process** tab includes the **Preferred verification level** and **Minimum verification level** properties, which you can use to set the level of verification for an address.

The following list identifies the levels that you can set on each property, from the lowest or most precise level to the highest or least precise level:

- Full address
- Premise level
- Street level
- Locality level
- Primary locality level

Select a preferred verification level at the same level of granularity as the minimum verification level or at a lower level. *Full address* is the default value on both properties.

For more information about setting the **Process** tab properties, see "Process properties" on page 213.

How do I standardize common terms in addresses that fail verification?

Use the **Standardize invalid addresses** property on the **Result** tab to standardize common address terms. For example, you might decide to change all instances of Apartment to Apt. You might select the property as a fallback option for addresses of low quality that do not pass verification. Choose *Yes* to standardize the terms.

For more information about setting the **Result** tab properties, see "Result properties " on page 215.

How do I set the output address data to upper case?

In the **Properties** dialog box, select the **Standardization** tab. The **Standardization** tab includes the **Casing** property, which you can use to set the casing to upper case.

For more information about setting the **Standardization** tab properties, see "Standardization properties" on page 217.

How do I set the destination country for the input addresses?

In the **Properties** dialog box, select the **Input** tab. The **Input** tab includes the **Country determination** property. Use the *Identify country* or *Force country* option on the property to determine how verification identifies the destination country.

Use the **Default Country** property to identify a country for any input address that does not contain country identification data. You can select the types of fields in which address verification searches for country information. The **Default Country** property does not apply to any address that identifies a destination country.

Use the **Force Country** property to specify a common destination country for all input addresses. The **Force Country** property overrides any country information in an input address.

To apply either property, choose a country name from the respective country list.

For more information about setting the **Input** tab properties, see "Input properties" on page 211.

How do I specify the character set for the output address data?

In the **Properties** dialog box, select the **Standardization** tab. The **Standardization** tab includes the **Script** property, which you can use to set the character set for the output addresses. Select a character set that accommodates the verified addresses in the data set.

For more information about setting the **Standardization** tab properties, see "Standardization properties" on page 217.

Can I expand the numerical ranges in address suggestions?

The **Expand numerical ranges** property of the **Result** tab helps you expand the house or building number ranges into individual numbers in address suggestions when the individual numbers are available in the reference data. Choose *Yes* to expand the numerical ranges.

For more information about setting the **Result** tab properties, see "Result properties " on page 215.

How do I change the language of the output address data?

In the **Properties** dialog box, select the **Standardization** tab. The **Standardization** tab includes the **Language** property, which you can use to determine how the verifier selects the language in which to return an output address. The property applies in countries where the reference data contains elements in more than one language.

You can select one of the following options:

Country standard

Default value. Returns all available address data in the default language that the reference data files specify for the country.

Preserve input

Returns the address data in the language that the input data uses. Preserves the language if the reference data contains the address in the input language.

Select a country from a list of ISO language codes

Returns all available address data in the language that you select.

You can select multiple languages. Select the languages in order of preference. If the verifier cannot return the address data in a language that you specify, it tries the next language.

For more information about setting the **Standardization** tab properties, see "Standardization properties" on page 217.

Can the Secure Agent download CASS data files?

If your Secure Agent is located in the United States, it can download CASS data files for certified address verification. You can confirm the download status for CASS data on the **DownloadCASSdata** property on the Secure Agent.

For information about the property, see "Verification properties in the CDQAV property set" on page 230.

CHAPTER 29

Address verification properties

Intelligent Data Management Cloud applies a range of properties to the address verification process. The properties provide configuration information to the runtime environment, including the amount of memory to assign to a verification process and the identities of any reference data files to preload into memory.

Configure the properties on the Secure Agent that will run a mapping with a Verifier transformation. The Secure Agent stores the properties in the CDQAV property set on the Data Integration Server.

For more information about the CDQAV properties, see "Verification properties in the CDQAV property set" on page 230.

Configuration properties in earlier releases

In earlier releases of Intelligent Data Management Cloud, the Data Integration Server stored address verification properties in the IDQAD property set. The IDQAD property set remain visible on the Data Integration Server for user reference. The IDQAD properties do not impact the verifier asset or address verification operations.

Verification properties in the CDQAV property set

The properties in the CDQAV property set determine how the Secure Agent reads address reference data files and allocates memory for address verification. Configure the properties in the Data Integration Server service on the Secure Agent on which you'll run address verification mappings.

The following table describes the properties:

Property	Description
LicenseFileDirectoryPath	Location of the license files that activate the address reference data. You might have more than one license file, for example, if you use batch reference data and geocoding reference data. You might have license files for a single country, for multiple countries, or for the entire world. The Secure Agent uses the following location by default: [Informatica_root_directory]/avLicenseFile If you update the locaton, select a directory that the Secure Agent can read. **Note:** For mappings in advanced mode, you must set the license file location.
RefereceDataDirectoryPath	Location of the parent directory for the address reference data files. Default location: [Informatica_root_directory]/avdata The reference data download process creates a `FileSetA` directory in the parent directory for the data files. Do not configure a common reference data location for more than one Secure Agent if the Secure Agents run mappings on a combination of Microsoft Azure and non-Microsoft Azure pods. **Note:** In advanced mode, the Secure Agent determines the download location.
FullPreLoadCountries	List of countries for which all reference data, including geocoding data, is loaded into memory before address verification begins. Enter the three-character ISO country codes in a comma-separated list. For example, enter DEU, FRA, USA. Enter ALL to load all data sets. The property is empty by default. Load the full reference database to increase performance. Some countries, such as the United States, have large databases that require significant amounts of memory. The Secure Agent downloads reference data for every country that the property identifies.
PartialPreLoadCountries	List of countries for which reference metadata and indexing structures are loaded into memory before address verification begins. The property also applies to geocoding data files. Enter the three-character ISO country codes in a comma-separated list. For example, enter DEU, FRA, USA. Enter ALL to partially load all data sets. When you run address verification mappings in a native runtime environment, the property is empty by default. When you run address verification mappings in an elastic runtime environment, the default is ALL. When you set the value to ALL, it partially loads all data sets. Partial preloading increases performance when not enough memory is available to load the complete databases into memory. The Secure Agent downloads reference data for every country that the property identifies.

Property	Description
NoPreLoadCountries	List of countries for which no reference data is loaded into memory before address verification begins. Enter the three-character ISO country codes in a comma-separated list. For example, enter DEU, FRA, USA. When you run address verification mappings in a native runtime environment, the default is ALL. When you run address verification mappings in an elastic runtime environment, the property is empty by default. The Secure Agent downloads reference data for every country that the property identifies. The default value indicates that the Secure Agent will download all address reference data.
MaximumMemoryinMegabytes	The total amount of memory, expressed in megabytes, that the address verification engine may allocate for data preloading and processing. Default is 1048. When you run address verification mappings in an elastic runtime environment and set the value to ALL for PartialPreLoadCountries, the maximum memory required is 4400. When you run address verification mappings in an elastic runtime environment and set the value to ALL for FullPreLoadCountries , the maximum memory required is 18000. When you run address verification mappings in an elastic runtime environment and set the value to ALL for NoPreLoadCountries , the maximum memory required is 2700.
MaximumNumberofResults	The maximum number of addresses that a node in the runtime environment can return in a single thread. The default value is 100.
DownloadCASSData	Indicates whether the Secure Agent downloads CASS reference data files for certified United States address processing. You can download the CASS data if your Secure Agent is located in the United States. Enter *Yes* if the Secure Agent is located in the United States and you want to download the CASS data. Otherwise, enter *No*. Default is *No*.
CertifiedReportLocation	Indicates the location to which the Secure Agent downloads a certified report when you select the Generate report option in a verifier asset. The default location is [Informatica_root_directory]/avreports' on the Secure Agent machine.
ParallelExecutionInstances	The number of threads that the Secure Agent can create for Verifier transformations at run time. Set the value to the maximum number of Verifier transformations that you expect to run in parallel in a single mapping. Default is 5. **Note:** Set a ParallelExecutionInstances value that is greater than or equal to the number of partitions on the Source transformation in the mapping.

Tuning the verification properties

You can update the Secure Agent properties that apply to address verification mappings to ensure that you derive the optimal performance from each mapping at run time. The Secure Agent properties work with the properties on the Source transformation in the mapping to determine mapping performance.

The Secure Agent properties and the mapping source properties define the following run-time conditions:

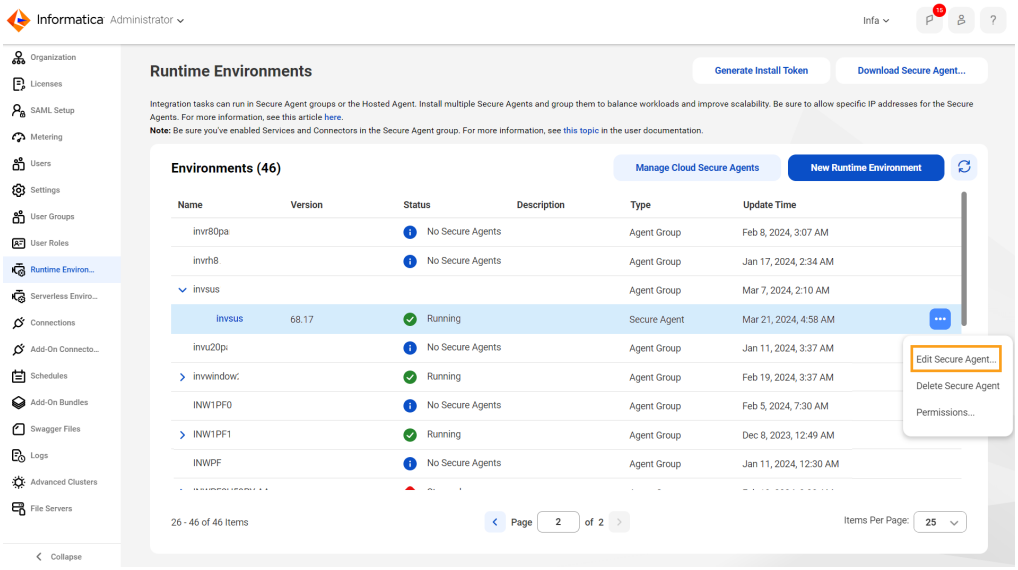
- the number of partitions that the mapping applies to the source data
- the number of threads that can run concurrently
- the number of child instances of the address verification engine that can start concurrently
- the amount of memory to allocate to the mapping

For more information about optimizing mapping performance, see the following Knowledge article: https://knowledge.informatica.com/s/article/HOW-TO-Configure-Secure-Agent-properties-to-optimize-address-verification-mapping-performance?language=en_US.

Configuring the address verification properties

Before you test a verifier asset or run a mapping with a Verifier transformation, configure the properties in the Administrator.

1. From the **My Services** page, select the **Administrator** service.
2. Choose the **Runtime Environments** option.
3. Select the **Secure Agent** that you will use to run address verification mappings.
4. Hover over the action icon for the Secure Agent, and select the **Edit Secure Agent** option.



The **Secure Agent** page appears.

5. Under **System Configuration Details**, select the following options:
   - Select **Data Integration Server** in the **Service** field.
   - Select **CDQAV** in the **Type** field.

   The **System Configuration Details** pane returns a list of properties based on the type that you specified.
6. Review the properties.

   You must provide a license file for each type of reference data that you will download and use in a mapping. You can accept the default values for all other properties. Add the license files to a directory that the Secure Agent can read.

   For example, add the files to the following location:

   ```
   [Informatica_root_directory]/avLicenseFile]
   ```

   Optionally, update the property values to suit your system and the mappings that you will run.

   Note: If you deploy the Secure Agent in a container environment on Linux, review the amount of memory available in the /dev/shm memory space. A Secure Agent that you deploy in a container environment requires access to at least 1 GB memory space for address verification operations.

CHAPTER 30

Validation and testing

Validate a verifier asset before you add it to a Verifier transformation in a mapping.

Validate a verifier

Validate a verifier to verify that the asset is ready for use in a Verifier transformation.

1. Open the verifier asset.
2. Click **Validate** on the asset toolbar.

If the validation process reports any error in the asset, fix the error before using the asset.

CHAPTER 31

Input address fields

A verifier displays a range of fields in the Inputs and Outputs sections of the **Configuration** tab. Each input field appears in one or more input models. Every field appears as an output field.

The following address fields appear in the both Inputs and Outputs sections:

Address Lines

Contains multiple elements of delivery information such as street, house number, and sub-building information, in a single field. Use in conjunction with a Last Line field. The input fields appear in the Hybrid input model.

The number of Address Line fields you need can differ from country to country. Select the number of Address Line fields that you need.

Administrative Division

Contains name of the largest administrative area below the national level, such as a province in Canada, state in India, or county in the United Kingdom. The input fields appear in the Discrete and Hybrid input models.

A verifier includes three levels of Administrative Division fields in the input. Use Administrative Division 1 to identify the primary or largest region. Use Administrative Division 2 to identify the primary subdivision in the province. For example, use Administrative Division 2 for county data in the United States. If the province data is highly complex, use the Administrative Division 3 field.

A verifier includes the following types of Administrative Division output fields:

- Administrative Division Standard
- Administrative Division Name
- Administrative Division Abbreviation
- Administrative Division ISO

Each sub-field contains three levels.

Building

Contains building information. A Building field contains building-level data such as the building descriptor, building name, and building number. If an address record contains more than one building, select additional Building fields. The input fields appear in the Discrete input model.

Contact

Contains contact information such as name, function, and salutation. The input fields appear in the Discrete and Hybrid input models.

A verifier includes the following types of Contact fields:

- Contact 1

- First Name 1
- Middle Name 1
- Surname 1
- Title 1
- Function 1
- Salutation 1

Note: If your address verification mapping will read contact data, select the Contact 1 input field or select one or more of the First Name 1, Middle Name 1, and Surname 1 input fields.

Country

Contains country information. The input fields appear in the Discrete, Hybrid, and Multiline input models.

The input field group contains a single Country field.

The output field group contains the following types of Country field:

- Country Name
- Country ISO 3
- Country ISO 2
- Country ISO Number

Each output field type has two levels. For example, the output fields include Country ISO Number 1 and Country ISO Number 2.

Delivery Service

Contains delivery service information. A delivery service is a mail pickup location managed by the post office, such as a P.O. Box. If the address record contains delivery service elements at more than one level, select additional Delivery Service fields. The input fields appear in the Discrete input model.

Department

Contains information about a department within an organization. The input fields appear in the Discrete and Hybrid input models.

Formatted Address Lines

Contains multiple elements in a single field. You can enter every line of an address on a formatted address line. You can use the Formatted Address Line output to print envelopes or to generate address labels. The input fields appear in the Multiline input model.

Select Formatted Address Lines input fields when the input data consists wholly of address lines instead of fields for different information types. Select Formatted Address Lines output fields to write addresses that you want to format for envelope printing and mail delivery.

House Number

Contains house number information. The house number identifies a building at street level. If the address record contains house numbers at more than one level, select additional House Number fields. The input fields appear in the Discrete input model.

Last Line

Contains last-line address information such as locality, postal code, and province information. Use in conjunction with Address Line fields. The input fields appear in the Hybrid input model.

Select a Last Line input field when you format the input address for postal delivery. Select a Last Line output field to create a formatted address that writes the last line of data to a single field.

Locality

Contains town or city name. Most countries use Locality 1 and optionally Locality 2. You might select additional locality fields if the locality data is highly complex. The input fields appear in the Discrete and Hybrid input models.

Organization

Contains organization information. An Organization field contains organization-level data, such as the organization name and organization descriptor. The input fields appear in the Discrete and Hybrid input models.

Postal Code

Contains postal code information. If the address record contains postal codes at more than one level, select additional Postal Code fields. The input fields appear in the Discrete and Hybrid input models.

Street

Contains street information. A Street field contains street-level data such as the street name, street post-descriptor, street post-directional, street pre-descriptor, and street pre-directional. If an address record contains more than one street, select additional Street fields. The input fields appear in the Discrete input model.

Sub Building

Contains sub-building information, such as an apartment or suite number. A Sub Building field contains sub-building level data such as the sub-building descriptor, sub-building name, and sub-building number. If an address record contains more than one sub-building level, select additional Sub Building fields. The input fields appear in the Discrete input model.

CHAPTER 32

Output address fields

When you process an address, you receive a result that can contains a large number of fields, including address elements, address line elements, and enrichment values. The result can also contain process-related information and status codes that describe the outcome of the verification process for the address.

Select the output fields that suit your address data and your business needs.

Single address elements and preformatted data fields

The output fields that you can select in the *Single address elements* and *Preformatted data* groups carry broadly the same types of information as the input fields of the same name.

For more information about the fields, see Chapter 31, “Input address fields” on page 234.

The output fields that you can select under *Single address elements* include an additional field group:

Residue

Contains residue information. *Residue* fields contain any input values that the verification operations identify as duplicate or redundant. Address verification moves the values from their input fields to the residue fields.

The *Residue* fields carry the following types of information:

- Residue Value. An input value moved to the residue field.
- Residue Type. The reason that address verification assigned the corresponding value to residue. The Residue Type field can return the following values: Necessary, Superfluous, and Unrecognized.

Status code fields

Status code information indicates whether an address is accurate enough for delivery and describes the result of the verification process.

You can select the following status code fields:

Verification Status Code

Provides a summary of the address verification process and indicates the general quality of the address.

Result Quality

Provides a general estimate of the likelihood that a mail carrier can deliver mail to the address

Match Percentage

Indicates the degree of similarity between the input data and reference data. The value ranges from 0 through 100. Values close to 100% indicate high similarity.

Address Type

Indicates the type of mail box that an address identifies.

Verification Status Codes

The Verification Status Code value summarizes the results of the comparison of the input address to the reference data. The code also summarizes any correction that the mapping makes to the address. Find the field in the Status Codes output group.

The following table describes the Verification Status Code output values:

Status Code	Description
V	The mapping verified all postally relevant elements. The input data matches the reference data for the address.
A	The mapping added one or more relevant address elements to the address.
C	The mapping corrected one or more input address elements to match the address elements in the reference data.
I	The mapping cannot correct the address. The address is not valid.
N	The mapping cannot verify the address because the address is not available in the reference data or the country is not supported.

Result Quality

The Result Quality field provides a general estimate of the likelihood that a mail carrier can deliver mail to an address. Use the result quality value with other status values, such as the verification status code, to evaluate the address quality and deliverability.

The following table describes the result quality values:

Value	Summary	Description
6	Completely confident	Indicates that the verification mapping verified all postally relevant elements.
5	Very confident	Indicates that the verification mapping cannot completely verify the address. For example, the verification mapping may be unable to verify secondary delivery information such as sub-building data. Or, the address might be a high-rise default address.
4	Secondary delivery point not verified	Indicates that the verification mapping can check the primary delivery identifier but cannot verify the secondary delivery information in the address record. If an input address contains a unique ZIP Code, corrects other input values and returns the address with a result quality value of 4.

Value	Summary	Description
3	Primary delivery identifier not verified	Indicates that the verification mapping cannot check the primary delivery identifiers such as house number and P.O. Box number. In countries where the house number information is not available, the primary delivery identifier might be a building name.
2	Delivery information not verified	Indicates that the verification mapping cannot find a match for the input street or delivery service information within the reference data. The address is verified to locality level.
1	Incomplete or contradictory address	Indicates that the verification mapping cannot find a match for the address in the reference data. This can occur when locality information is not available in the reference data or the address includes contradictory postal code data.
0	Not an address	Indicates that the input is not a deliverable address.

Note: The verification mapping is likely to return a lower score for a poor-quality address in **Verification only** mode than in **Verification with suggestions** mode. **Verification with suggestions** mode can apply a relatively higher tolerance when calculating the result quality score, as it returns multiple addresses from which you can select the correct address.

Address Type

The Address Type field identifies the physical type of mail box that will receive mail at the address.

Postal authorities around the world use a common set of alphabetic codes to represent address types. However, differences in address type assignment exist between countries. Also, the logic within the processing modes in address verification can result in differences in assignment of address type values across the modes.

The following table describes a set of address type values that apply in many countries:

- B. The address identifies a building.
- F. The address identifies an organization.
- G. The address is a general delivery address. In a general delivery address, the postal code and the recipient data identify the address.
- H. The address identifies a high-rise building. The address contains subbuilding elements such as apartment or suite.
- L. The address post code identifies the organization as a large volume receiver. The reference data adds or validates the organization name.

  Address verification can determine that the address is a large volume receiver in one of the following ways:

  - The address post code identifies the organization as a large volume receiver.
  - The reference data does not contain street or building information.
- N. The address reference data matches the input address data to locality level, but the reference database does not contain street information for the address. The verifier returns the address with N as the address type value.
- P. The address identifies a Post Office Box or a delivery service.
- R. The address is a rural route/highway contract address.

- S. The address identifies a street. S is the default address type. If verification cannot determine the address type from the address data, it returns the default value.
- U. Unidentified. The address is not valid, and verification does not assign an address type.

Enrichments: Geocoding fields

To increase the chance of successful delivery, for example in rural areas, you can enrich the verified addresses with geocodes. Geocodes identify the latitude and longitude of an address.

A verifier asset supports the following levels of geocoding:

Rooftop

Rooftop geocoordinates map to the center of the roof of the primary building on a parcel of land.

ArrivalPoint

ArrivalPoint geocoordinates map to a point in the center of the street segment in front of a house or a building.

StreetCenter

StreetCenter geocoordinates map to an approximate mid-point of the street, based on known geocodes for locations nearby.

LocalityCenter

LocalityCenter geocoordinates map to an approximate mid-point of the locality, based on known geocodes for locations nearby.

PostalCodeCenter

PostalCodeCenter geocoordinates map to a post office that handles mail for the address.

The maximum available precision of the geocodes depends on the depth of the geocoding data that the reference data stores for the country. If verification cannot return geocodes to the level of precision that you select, it returns the geocodes to the closest possible level of precision.

You can select the following fields for each level of geocoding precision:

[Geocoding Level] Accuracy Code

Returns the level of accuracy to which a verifier can return geocodes for an address in a given country.

[Geocoding Level] Coordinates

Returns the closest latitude and longitude coordinates that a verifier can return for the input address.

[Geocoding Level] Latitude

Returns the closest latitude coordinates that a verifier can return for the input address.

[Geocoding Level] Longitude

Returns the closest longitude coordinates that a verifier can return for the input address.

Geocoding Accuracy Codes

The geocoding accuracy code fields describe the result of the geocoding process for an output address. The fields can indicate the level of precision to which address verification returns the geocodes. Find the fields in the Geo Coding output group.

The verifier asset includes geocoding accuracy code data fields for every level of geocoding precision. For example, the asset provides an Arrival Point Accuracy Code field that contains status values for addresses that return Arrival Point geocode data.

The geocoding accuracy code fields can return the following geocoding status values:

ER_DATA_NOT_AVAILABLE

Verification cannot find the geocoding database.

ER_NOT_UNLOCKED

The geocoding database is not unlocked.

ER_DATA_CORRUPT

The geocoding database is corrupted.

NOTHING_FOUND

Verification cannot append geocoordinates to the input address because no geocoordinates are available for the address.

POCO_CENTER

Geocoordinates are accurate to the postal code level.

LOCALITY_CENTER

Geocoordinates are accurate to the locality level.

STREET_CENTER

Geocoordinates are accurate to the street level.

INTERPOLATED

Geocoordinates are accurate to the house number level. (Estimated location of the parcel of land with street-side offset.)

POINT_ARRIVAL_POINT

Geocoordinates are accurate to arrival point level. (Measured entryway to the parcel of land.)

POINT_ROOFTOP

Geocoordinates are accurate to rooftop level.

Enrichments: Certified fields

You can use *Certified verification* mode to verify addresses to the certification standards that postal services define in Australia, Canada, France, New Zealand, and the United States. The verifier asset includes output fields that can add enrichment and status information to addresses from each country.

The asset also includes a *Certification Status* field that reports on the overall outcome of the certified verification process. For more information about the Certification Status field, see "Enrichment and certification status fields" on page 261.

Australia-AMAS

The following table describes the values and data indicators that a verifier can return when you verify an Australia address in certified mode:

Field	Description
AMAS Error Code	A two-character code that represents the validity of the address with respect to the AMAS standard. The code can indicate that the address is fully valid, or the code can describe the reason why the address does not meet the AMAS standard.
Delivery Point ID	An eight-digit identifier that Australia Post assigns to a delivery point. Each DPID is unique and randomly generated. Australia Post creates DPIDs in a range from 30,000,000 to 99,999,999.
Lot Number	The lot reference number that a government department assigned to a property during the sub-division of a parcel of land and prior to road numbering.
Postal Delivery Number	The number information in the delivery service. For example, the post office box number in a P.O. Box address. Australia Post stores a postal delivery number as a five-digit number in the Postal Address File (PAF). Therefore, the PAF stores the postal delivery number 500 as 00500. If the address does not contain a postal delivery number, a verifier does not populate the field.
Postal Delivery Number Prefix	The prefix character in a postal delivery number. For example, A in A500.
Postal Delivery Number Suffix	The suffix character in a postal delivery number. For example, A in 500A.
AUS House Number 1	The primary house number in a street address. Australia Post stores a house number as a five-digit number in the PAF. Therefore, the PAF stores the house number 123 as 00123.
AUS House Number 1 Suffix	An alphabetical suffix to a House Number 1 value. For example, A in 123A.
AUS House Number 2	The secondary house number in a street address. Australia Post stores a house number as a five-digit number in the PAF. Therefore, the PAF stores the house number 456 as 00456. If the address does not contain a secondary house number, a verifier returns 00000 as the field value.
AUS House Number 2 Suffix	An alphabetical suffix to a House Number 2 value. For example, B in 456B.

Canada-SERP

The following table describes the values and data indicators that a verifier can return when you verify a Canada address in certified mode:

Field	Description
SERP Category	The verification category for the address. The category summarizes the result of the verification operation for the address. Contains one of the following values: - V. Address valid. - C. Address corrected. - N. Not valid. - VQ. Considered valid, but questionable because of insufficient data or input. For example, some "general delivery" addresses. - V1A. Valid apartment type record. - V2A. Valid commercial type record. - C1A. Corrected apartment type record. - C2A. Corrected commercial type record. The 1A and 2A types usually refer to database records containing building information.
Excluded	Indicates that the value is excluded from SERP calculations because the sub-building input data is incorrect. Occurs in conjunction with SERP category N. The field contains the value EXCLUDED for an ignored record. Otherwise, the field is empty.

France-SNA

The following table describes the values and data indicators that a verifier can return when you verify a France address in certified mode:

Field	Description
SNA Category	The verification category for the address. The category summarizes the result of the verification operation for the address. Contains one of the following values: - ORI. Address valid, output identical to input. - RES. Address valid or corrected, output not identical to input. - AVE. Address rejected, user input required. - NOK. Address rejected.

New Zealand-Send Right

The following table describes the values and data indicators that a verifier can return when you verify a New Zealand address in certified mode:

Field	Description
NZL Address Type	The address type. Contains one of the following values: - Bag - Box - CMB Rural - CMB Urban - Counter - Rural - Urban
SOA Record Ignored	Indicates whether the Statement of Accuracy ignores the record. The following address records maybe ignored for SOA calculation: - Poste Restante - Private Bags with no number The field contains the value Excluded for an ignored record. Otherwise, the field is empty.
Delivery Point Identifier	An eight-digit number that uniquely identifies each delivery point in New Zealand.
Delivery Service Type	Specifies the type of the delivery service. It may be PO Box, Private Bag, CMB, Response Bag, Counter Delivery, or Poste Restante.
Delivery Service Number	The box or bag number. It contains no leading zeros or spaces, separators, or other punctuation.
Delivery Service Locality	Part of the Suburb line for New Zealand addresses. Contains Box Lobby as the value.
House Number	Number of the structure erected on the property associated with the delivery point.
House Number Alpha	The alphabetical part of the house number of the property associated with the delivery point.
Rural Delivery Number	A six-character field that identifies the rural delivery route number of an address.
Hygiene	A flag that indicates whether address cleansing is possible.
Validity Code	Helps to determine the validity of the input address. Contains one of the following values: - Unique match - VALID-U - Base Address match - VALID-B - Not valid - INVALID

United States-CASS

The following table describes the values and data indicators that a verifier can return when you verify a United States address in certified mode:

Field	Description
CASS Error Code	Indicates whether the fields in the US Specific field group contain data for the address. Verifier does not currently generate output on the CASS Error Code field.
Barcode	An 11-digit number that represents the delivery point for the record. It consists of the nine-digit ZIP+4 code and the two-digit Delivery Point Answer.
Carrier Route	A four-character code assigned to a mail delivery or collection route within a 5-digit ZIP Code. The first character of the code is alphabetical and the last three are numeric: - Bnnn = PO box - Hnnn = Highway contract - Rnnn = Rural route - Cnnn = City delivery - Gnnn = General delivery
Congressional District	A standard value that identifies a geographic area served by a member of the United States House of Representatives or Senate. If the address is an Army/Air Force (APO) address or a fleet post office (FPO) address, the field is blank. If there is only one member of congress within the state, the value is AL (At Large).
Default Flag	Indicates that the record matched to a high-rise, rural route, or street default record in the ZIP + 4 data. The flag has the value Y for a matching record.
Delivery Point Answer	The final two digits of the house/box number, or if a High-rise record is matched, the secondary unit number that represents the delivery point information. The Delivery Point Answer contributes to the 11-digit delivery point barcode (DPBC).
Delivery Point Check Digit	A number that you can add to the sum of the other digits in the delivery point barcode (DPBC) to yield a number that is a multiple of ten. For example: ZIP+4 code = 123456789 Delivery Point Answer = 01 Sum of digits (1+2+3+4+5+6+7+8+9+0+1) = 46 Check digit = 4 (46+4 = 50)
DPV CMRA Indicator*	Indicates the result of the call to the DPV CMRA (Commercial Mail Receiving Agent) hash table: - Y =Address was found in the CMRA table. - N =Address was not found in the CMRA table. - Blank = Address was not presented to the hash table.
DPV Confirmation Enhanced*	Provides additional information on the reasons for the DPV Status assignment: - Y= Address was DPV confirmed for both primary and secondary numbers to determine a valid delivery point. - D= Address was DPV confirmed for the primary number only, and secondary number information was missing. - S= Address was DPV confirmed for the primary number only, and the secondary number information was present but not confirmed. Or, a single trailing alpha character on a primary number was dropped to make a DPV match, and secondary information is required. - N= The primary number information failed to DPV confirm. - R= Address was DPV confirmed but assigned to phantom route R777 or R779. The USPS does not provide delivery. - Blank= Address was not presented to the hash table.

Field	Description
DPV Confirmation Indicator*	Indicates the result of the call to the Delivery Point Validation (DPV) Confirmation hash table: - Y =Address was DPV confirmed for both primary and (if present) secondary numbers. - D =Address was DPV confirmed for the primary number only, and secondary number information was missing. - S = Address was DPV confirmed for the primary number only, and secondary number information was present but unconfirmed. - N =Both primary and (if present) secondary number information failed to DPV confirm. - Blank = Address was not presented to the hash table.
DPV 3553 Confirmation*	Indicates whether to add the address to the total number of addresses recorded in the ZIP + 4/DPV Confirmed column on form 3553: - Increment the total number of addresses. - Do not increment the total number of addresses.
DPV Door Not Accessible*	Identifies addresses that do not provide a door or entry point that the postal carrier can access. A Door Not Available (DNA) address might not provide a door for mail delivery, or the mailbox might reside behind a locked gate. The field can contain the following values: - Y = Address does not have a door available for postal delivery. - N = Address has a door available for postal delivery. - Blank = Address was not presented to the hash table.
DPV Drop Indicator*	Indicates the status of the address in the Delivery Sequence File Second Generation (DSF2) Drop Indicator table. The Drop Indicator table identifies mailbox addresses that serve multiple households or businesses. The field can contain the following values: - Y= Address was found in the table. - N= Address was not found in the table. - Blank = Address was not presented to the hash table.
DPV False Positive Indicator*	Indicates the results of the call to the DPV False Positive hash table: - Y = Address was found in the False Positive table. - N = Address was not found in the False Positive table. - Blank = Address was not presented to the hash table.

Field	Description
DPV Footnote 1* DPV Footnote 2* DPV Footnote 3* DPV Footnote 4* DPV Footnote 5*	The footnote(s) returned for an address from DPV processing: - AA - Input address matched to the ZIP + 4 product. - A1 - Input address not matched to the ZIP + 4 product. - BB - Input address matched to DPV for both primary and secondary numbers necessary to determine a valid delivery point. - CC - Input address primary number matched, secondary number not matched, secondary number not required. - C1 - Input address primary number matched, secondary number not matched, secondary number required. - F1 - Input address matched to a military address. - G1 - Input address matched to a general delivery address. - IA - Informed address identified. - M1 - Input address primary number missing. - M3 - Input address primary number not valid. - N1 - Input address primary number matched to DPV, but required secondary number is missing. - PB - Identified Post Office Box Street Address. - P1 - Input address Post Office Box, Rural Route, or Highway Contract box number missing. - P3 - Input address Post Office Box, Rural Route, or Highway Contract box number not valid. - RR - Input address matched to a Commercial Mail Receiving Agency (CMRA), but a Private Mailbox (PMB) designator is present (PMB 123 or # 123). - R1 - Input address matched to CMRA, but PMB designator is not present (PMB 123 or # 123). - R7 - Addresses that are assigned to a phantom route R777 or R779. - TA - Input address primary number matched to DPV by dropping trailing alphabetic character. - U1 - Input address matched to a unique ZIP Code.
DPV Footnote Complete*	All data values from the populated DPV Footnote fields in a single string.
DPV No Secure Location*	Identifies an address that does not provide a reliable mailbox or reception point for mail: - Y = Location is not secure. - N = Location is secure. - Blank = Address was not presented to the hash table.
DPV PBSA*	Indicates that the address is a Post Office Box Street address (PBSA): - Y =Address was found in the PBSA table. - N =Address was not found in the PBSA table. - Blank = Address was not presented to the hash table. A verifier also returns the footnote PB for a PBSA address.
DPV Throwback Indicator*	Indicates a valid street address for which the USPS forwards mail to a Post Office Box: - Y = The address is a Throwback address. - N = The address is not a Throwback address. - Blank = Address was not presented to the hash table.
DSF2 No Stats Indicator*	Indicates the results of the call to the DPV No-Stat table: - Y = Address was found in the No-Stat table. - N = Address was not found in the No-Stat table. - Blank = Address was not presented to the hash table.

Field	Description
DSF2 No Stats Reason*	A single-digit code that identifies the reason why an address returned a Y in the *DSF2 No Stats Indicator* field. The code can have the following values: - *1 - IDA* Internal Drop Address. The verified address does not physically receive mail. Instead, the USPS delivers mail to a 'drop' address associated with the verified address. - *2 - CDS* The address identifies a new construction that cannot yet accept delivery. Or, the address lies on a Rural Route, Highway Contract Route, or Contract Delivery Service route and the delivery point is unoccupied for more than 90 days. - *3 - Collision* The address does not DPV confirm. A verifier users will not see the collision value, because address validation will change the NoStats indicator to N in this case and clear the NoStats Reason code. - *4 - CMZ* The address is in a college, military, or other zone. A CMZ address is a ZIP+4 address that the USPS has added to the reference data. - *5 - Regular No-Stat* The address is no longer deliverable or lies on an R777 route, Or, the address includes a Post Office Box that has never been rented or is not available to rent.
DSF2 Vacant Indicator*	Indicates the results of the call to the DPV Vacant table: - Y = Address was found in the Vacant table. - N = Address was not found in the Vacant table. - Blank = Address was not presented to the hash table.
eLOT Flag	The Enhanced Line of Travel (eLOT) order in which delivery points are delivered within a given add-on code. For example, if the code is A (Ascending) and the house numbers are 1, 3, and 5, the carrier delivers 1, 3, and 5, in that order (low to high). If the code is D (Descending), the carrier delivers first to 5, then 3, and then 1 (high to low).
eLOT Sequence Number	A number that indicates the order in which add-on codes are arranged within a given carrier route.
Early Warning System (EWS) Return Code*	Indicates whether the address was found in the EWS data: Y = Address was found in the EWS data, thus resulting in a ZIP + 4 No Match. Blank = Address was not found in the EWS data.
High-rise Default	A flag that indicates that the record is assigned to a default high-rise record.
High-rise Exact	Identifies high-rise addresses that contain unit identifiers.
LACS Indicator*	Identifies an address that matched to a LACS address in the ZIP + 4 file. A LACS address is an address that was converted to the city-style address format so that emergency vehicles can more easily find the address location. Return values: - L = Address matched to a LACS address in the ZIP + 4 file. - Blank = Address is not a LACS address in the ZIP + 4 file.

Field	Description
LACSLink Indicator*	An indicator returned when the LACSLink hash tables are queried: - Y = The input record matched a record in the master file. - N = The input record did not match a record in the master file. - Blank = Address was not presented to the hash table.
LACSLink False Positive*	Indicates the results of the call to the LACSLink False Positive hash table: - Y = Address was found in the table. - N = Address was not found in the table. - Blank = Address was not presented to the table.
LACSLink Return Code*	A return value from LACSLink processing: - *A = LACS record match* A new address can be furnished. The input record matched to a record in the master file. - *00 = No match* A new address cannot be furnished. The input record cannot be matched to a record in the master file. - *14 = Found LACS record, but new address did not convert at run time.* The new address cannot be converted to a deliverable address. The input record matched to a record in the master file. - *92 = LACS record, secondary number dropped from input address.* The record is a ZIP + 4 street-level or high-rise match. The input record matched to a master file record, but the input address had a secondary number and the master file record did not.
NonDelivery Days*	A seven-character code that identifies the day or days of the week on which an address cannot receive mail. The code contains a seven-character string that represents the days of the week from Sunday through Saturday. A verifier returns the first letter of a weekday in the corresponding position in the code if the address does not receive mail on that day. A verifier returns a dash symbol in the corresponding position otherwise.
NonDelivery Days Flag*	A single-character field that indicates if the address is not deliverable on one or more days of the week. Address verification populates the NonDelivery Days field for every address that returns Y in the NonDelivery Days Flag field. The NonDelivery Days Flag field can contain the following values: - Y= Address was found in the table. - N= Address was not found in the table. - Blank = Address was not presented to the hash table.
PO Box Only	Indicates if the address is located in a ZIP code that contains post office box addresses only. Y = Address is in a post office box-only delivery zone.
Residential Delivery Indicator *	Indicates if the delivery point is residential: - Y = Indicates residential delivery. - N = Not residential delivery. - Blank = Did not query the Residential Delivery Indicator (RDI) data.

Field	Description
Record Type Code	An alphabetic value that identifies the type of data in the record. Record type codes include the following: - F = Firm - G = General delivery - H = High-rise - P = PO box - R = Rural route/highway contract - S = Street - U = Unidentified. Verification does not assign a record type.
Rural Route Default	A flag that indicates that the record is assigned to a default rural route record. This occurs when the input house number does not match to the primary numbers in the reference data and there is a corresponding rural route record with no primary numbers.
Rural Route Exact	Indicates if the address matches a rural route address in the USPS address reference data set.
SuiteLink Return Code*	A return value of SuiteLink Processing: - *A = SuiteLink Record Match* The input record matched to a record in the master file. An improved business address can be furnished. - *00 = No Match* Business address not improved. The input record cannot be matched to a record in the master file. An improved business address can not be furnished.
ZIPMove Return Code*	A return value from ZIPMove processing: - Y = ZIPMove match was made. - N = ZIPMove match was not made.
ZIP5 Valid Flag	Indicates whether the address record can be added to Form 3553. Five-digit validation requires that the last line values of city, state, and ZIP Code correspond to each other. When CASS certification makes a ZIP+4 match, the five-digit ZIP Code is valid. **Note:** Five-digit validation applies to No Match ZIP+4 records.

Note: Fields marked with * will only be populated for United States customers, in accordance with USPS licensing restrictions.

Enrichments: Global fields

Use the Global field to activate enrichments that can apply in the same way to addresses in every country.

You can select the following global enrichment fields:

TimeZone Code

A one- to three-character numeric value that indicates the offset between the address time zone and Greenwich Mean Time. For example, the time zone code for Eastern Standard Time is -5.

TimeZone Name

A three-character code that identifies the time zone to which the address belongs. For example, EST identifies Eastern Standard Time.

The global enrichments TimeZone Code and TimeZone Name are available only for United States addresses.

Enrichments: Country Specific fields

The verifier asset includes output fields that can add enrichments to addresses across a range of countries. Find the fields in the *Country Specific* field group.

The *Country Specific* group includes a *Country Specific Status* field that reports on the overall outcome of the process to add the enrichments. For more information about the Country Specific Status field, see "Enrichment and certification status fields" on page 261.

Enrichments for Australia addresses

You can configure a verifier to include address enrichments in the validated Australia addresses. The verifier bases the enrichments on geographical areas that the Australian Bureau of Statistics defines.

A verifier provides enrichment data for the following statistical elements:

Geocoded National Address File Identifier

A 14-digit code that identifies an address in the Geocoded National Address File (GNAF). Australian government departments and organizations recognize the code as a persistent, unique identifier for the address.

Local Government Area Name

The name of the Local Government Area (LGA) in which an address is located. An LGA is a region that is governed by a local council.

Local Government Area Code

A five-digit numeric code that identifies the Local Government Area (LGA) in which an address is located.

The Local Government Area Code consists of a single-digit state or territory identifier and a four-digit Local Government Area identifier that is unique within the state or territory.

Level One Statistical Area 11-Digit

An 11-digit code that identifies the level-1 statistical area to which an address belongs. A level-1 statistical area is typically the smallest area for which the Australian Bureau of Statistics releases statistical data.

The 11 digits of the Level One Statistical Area 11-Digit code include the values of the larger areas in the statistical area hierarchy. That is, the Level One Statistical Area 11-Digit includes a state and territory identifier, SA4 identifier, SA3 identifier, and SA2 identifier in addition to the SA1 identifier. The final two digits in the code identify the level-1 statistical area. In this way, the Level One Statistical Area 11-Digit uniquely identifies the area to which the address belongs.

Level One Statistical Area 7-Digit

A seven-digit code that identifies the level-1 statistical area that an address belongs to but that does not provide the full hierarchy of geographical information that Level One Statistical Area 11-Digit provides. The seven-digit code comprises the state and territory identifier, SA2 identifier, and SA1 identifier.

A level-1 statistical area is typically the smallest area for which the Australian Bureau of Statistics releases statistical data.

Level Two Statistical Area 9-Digit

A nine-digit code that identifies the level-2 statistical area to which an address belongs. A level-2 statistical area is a physical area that broadly defines a single social or economic community. A level-2 statistical area comprises multiple level-1 statistical areas.

The nine digits of the Level Two Statistical Area 9-Digit code include the values of the larger areas in the statistical area hierarchy. That is, the Level Two Statistical Area 9-Digit includes a state and territory identifier, SA4 identifier, and SA3 identifier in addition to the SA2 identifier. The final four digits in the code identify the level-2 statistical area.

Level Two Statistical Area 5-Digit

A five-digit code that identifies the level-2 statistical area that an address belongs to but that does not provide the full hierarchy of geographical information that SA2_MAIN_NO provides. The five-digit code comprises the state and territory identifier and the four-digit SA2 identifier.

A level-2 statistical area is a physical area that broadly defines a single social or economic community. A level-2 statistical area comprises multiple level-1 statistical areas.

Level Two Statistical Area Name

The name of the level-2 statistical area to which the address belongs. Each level-2 name is unique, and each name contains no more than forty characters. In an urban area, the name is based on the suburb or suburbs that the area covers. In rural areas, the name is based on the locality that the area covers.

Level Three Statistical Area 5-Digit

A five-digit code that identifies the level-3 statistical area to which an address belongs. A level-3 statistical area comprises multiple level-2 statistical areas.

The five digits of the level-3 statistical areas include the values of larger geographical regions. That is, the Level Three Statistical Area 5-Digit includes a state and territory identifier and SA4 code in addition to the SA3 identifier. The final two digits in the code identify the level-3 statistical area.

A level-3 statistical area generally covers a population of between 30,000 and 130,000 people.

Level Three Statistical Area Name

The name of the level-3 statistical area to which the address belongs. Each level-3 name is unique, and each name contains no more than forty characters. SA3 names reflect the names of the cities, towns, or rural areas that they cover.

Level Four Statistical Area 3 Digit

A three-digit code that identifies the level-4 statistical area to which an address belongs. A level-4 statistical area comprises multiple level-3 statistical areas.

The three digits of the level-4 statistical areas include the values of the largest sub-state regions in the main structure of the Australian Statistical Geography Standard (ASGS). That is, the Level Four Statistical Area 3 Digit includes a state and territory identifier and SA4 identifier. The final two digits in the code identify the level-4 statistical area.

A level-4 statistical area contain at least 100,000 people. In regional areas, a level-4 statistical area might include between 100,000 and 300,000 people. In urban areas, a level-4 statistical area might include between 300,000 and 500,000 people.

Level Four Statistical Area Name

The name of the level-4 statistical area to which the address belongs. Each level-4 name is unique, and each name contains no more than forty characters. SA4 names reflect the names of the cities, towns, or rural areas that they cover.

Greater Capital City Statistical Area 5-Char

A five-character alphanumeric code that identifies the greater capital city (GCCSA) statistical area to which an address belongs. A greater capital city statistical area comprises multiple level-4 statistical areas.

There are 16 spatial GCCSA regions, including eight regions that represent the Australian state and territory capital cities and eight regions that cover the rest of each state and the Northern Territory. In addition, there are 18 non-spatial greater capital city statistical areas.

The five character code includes a single-digit state and territory identifier and a four-character GCCSA identifier.

Greater Capital City Statistical Area Name

The name of the greater capital city statistical area to which an address belongs.

Greater capital city statistical areas are named for the cities they represent or, if they do not identify a city, for the rest of the state or territory that they represent.

State or Territory Name

Represents the name of a state or territory.

State or Territory Code

An unique one-digit code that represents a state or territory.

Census Collection District Code 2006

A seven-digit code that represents a census collection district that the Australia Bureau of Statistics defined for the 2006 census. A census collection district is an area that a census data collector might cover in a ten-day period. In urban areas, the district might comprise 220 homes. In rural areas, the district might comprise fewer homes and the geographical area might increase.

Mesh Block 11-Digit 2011

An 11-digit code that identifies the mesh block to which an address belongs. Mesh Block 11-Digit 2011 represents the areas that the Australian Bureau of Statistics defined for the 2011 census. A mesh block represents the smallest geographical area that the Australian Bureau of Statistics uses to generate statistical information.

Mesh blocks are building blocks for statistical information rather than areas for which the Australian Bureau of Statistics releases information. The Australian Bureau of Statistics builds statistical areas and regions from mesh blocks. Mesh blocks broadly align with land use, such as residential use, commercial use, or parkland. A mesh block might contain thirty to sixty dwellings, although some mesh blocks are designed to contain no dwellings.

Mesh Block 11-Digit 2016

An 11-digit code that identifies the mesh block to which an address belongs. Mesh Block 11-Digit 2016 represents the areas that the Australian Bureau of Statistics defined for the 2016 census.

Note: The Australian Bureau of Statistics might maintain statistical information for some mesh blocks that it defined for the 2016 census.

See also *Mesh Block 11-Digit 2011*.

Enrichments for Austria addresses

You can configure a verifier to include unique identification codes in the output for valid Austria addresses.

A verifier can return the following code values:

Postal Address Code AT

The Postal Address Code AT is a unique identifier for a current Austria address.

Postal Address Code Indentifier AT

The Postal Address Code Indentifier AT is the Postal Address Code AT of the address at which a building receives mail if the building has more than one address.

Postal Address Code AT enrichment

The Postal Address Code is a unique identifier for the current version of an address in Austria. For example, the following address returns a Postal Address Code value of 105176447:

```
Plättenstraße 7
2380 Perchtoldsdorf
Niederösterreich
AUT
```

Postal Address Code Indentifier AT enrichment

An Austria address has a Postal Address Code Indentifier AT value when the address identifies a mailbox that receives mail at another address. For example, a building at an intersection of two streets might have an address on both streets and might specify one of the addresses as the mailbox address.

Note: The address that gives access to the mailbox is called the Ident address.

A street address that does not receive mail has a Postal Address Code value and a Postal Address Code Indentifier AT value. The Postal Address Code Indentifier AT value is the Postal Address Code of the Ident address that receives the mail. The postal carrier delivers mail to the address that the Postal Address Code Indentifier AT identifies.

The following table lists street addresses that identify a single mail destination:

Address	Postal Address Code AT	Postal Address Code Indentifier AT
Hauptplatz 4 8010 Graz AUT	100001915	100004254
Neue-Welt-Gasse 2 8010 Graz AUT	100004254	Not applicable

The address "Hauptplatz 4" does not receive mail because the mailbox is at another address at the same location. The building receives mail at “Neue-Welt-Gasse 2” and therefore “Neue-Welt-Gasse 2” is the Ident address. The Postal Address Code Indentifier AT is the Postal Address Code AT of the address that receives the mail. The Postal Address Code value for the Ident address is 100004254.

Enrichment for Belgium addresses

You can configure a verifier to add the National Institute of Statistics (StatBel) or NIS Code as an enrichment to validated Belgium addresses.

NIS codes are five-digit codes that uniquely identify geographic areas in Belgium. If you enable enrichment for Belgium, a verifier returns a nine-digit code that contains the five-digit NIS code and a four-digit Neighborhood ID.

For example, a verifier returns `21004A001` as an enrichment to the following address:

```
Rue au Beurre 1
1000 Bruxelles
BEL
```

In this example, `21004` is the NIS code and `A001` is the Neighborhood ID.

Enrichments for Brazil addresses

You can configure a verifier to include the Institute of Geography and Statistics (IBGE) Code as enrichment to validated Brazil addresses.

The IBGE code is a seven-digit numeric code that identifies cities and states in Brazil. The IBGE code is useful for e-commerce operations as you can use this code for taxation and audit purposes.

Example: Institute of Geography and Statistics Code

When you validate the following address with the address enrichment enabled, a verifier returns an IBGE code of `Institute of Geography and Statistics Code: 2606101` as an enrichment to the validated output.

```
Rua da Matriz 9
Centro
Glória do Goitá-pe
55620-000
Brazil
```

Enrichment for Czech Republic addresses

You can configure a verifier to add RUIAN ID codes as an enrichment to a valid Czech Republic address. The Czech Office for Surveying, Mapping and Cadastre (ČÚZK) maintains the RUIAN code data.

The RUIAN ID enrichment comprises the following codes:

- RUIAN Delivery Point Identifier. Uniquely identifies the address delivery point.
- RUIAN Building Identifier. Identifies the address to building level.
- RUIAN Building Entrance Identifier. Identifies the building entrance.

The database for the Czech Republic includes RUIAN Delivery Point Identifier and RUIAN Building Identifier values for ninety-nine percent of Czech Republic addresses. The database includes RUIAN Building Entrance Identifier values for a small percentage of addresses.

Example: RUIAN ID Codes

The supplementary database stores all types of RUIAN ID data for the following address:

```
Musorgského 320/2
Kohoutovice
623 00 Brno 23
CZE
```

A verifier returns the following RUIAN ID codes for the address from the database:

```
RUIAN Delivery Point Identifier: 19382481
RUIAN Building Identifier: 19204035
RUIAN Building Entrance Identifier: 76838
```

Enrichments for France addresses

You can configure a verifier to include the INSEE-5 Code and the INSEE-9 Code in the validated output for France addresses.

The INSEE code is a numeric indexing code that the French National Institute for Statistics and Economic Studies (INSEE) use to identify entities such as French communes and departments. INSEE codes are particularly helpful in uniquely identifying French communes that share the same name, spelling, and pronunciation. Of a five-digit INSEE code for a commune, the first two digits represent the department and the last three denote the commune. INSEE codes are also used as National Identification Numbers for French citizens.

The INSEE-9 code is also known as the IRIS code. IRIS stands for aggregated units for statistical information in French, and represents a demographic group that contains a maximum of 2000 people. France is composed of around 16,100 IRIS units including 650 units in overseas departments.

For example, you receive INSEE-5 Code `47001` and INSEE-9 Code `470010115` when you validate the following France address:

```
6 RUE DU PUITS DU SAUMON
47000 AGEN
FRA
```

Enrichments for Germany addresses

You can configure a verifier to include multiple address enrichments in validated Germany addresses.

You can configure a verifier to include the following enrichments in validated Germany addresses.

Official Municipality Key DE

The Official Municipality Key DE is an eight-digit identification number for a German municipality. A municipality is administrative area in the German federal system.

Locality Identifier DE

The Locality Identifier DE is a variable length code that uniquely identifies a locality in Germany.

Street Identifier DE

The Street Identifier DE is a variable length code that uniquely identifies a street address in Germany.

Street Code

The Street Code is a three-digit code that identifies a street in Germany. Positions 6, 7, and 8 of the Frachtleitcode or Freight code form the street code. A street code value of 994 indicates that the address points to a packstation.

For example, when you validate

```
Röntgenstr. 9
67133 Maxdorf
Germany
```

A verifier returns the following additional information in the validated output:

```
DEU_AGS: 07338018
Locality Identifier DE: 68015519
Street Identifier DE: 100560690
Street Code: 057
```

Enrichments for Italy addresses

You can configure a verifier to add ISTAT code data as an enrichment to a valid Italy address. The ISTAT code contains a series of values that identify the province and municipality to which the address belongs. The Italian National Institute of Statistics (ISTAT) maintains the ISTAT codes.

For example, the ISTAT code for the city of Venice is 027042.

Enrichments for Japan addresses

You can configure a verifier to add enrichments to valid Japan addresses.

A verifier offers the following enrichments for Japan:

- Choumei Aza Code JP

- New Choumei Aza Code JP
- Current Choumei Aza Code JP
- Gaiku Code

Choumei Aza Codes

A Choumei Aza code is an 11-digit string that represents an address to delivery-point level in Japan. When Japan Post updates the address information for a delivery point, Japan Post also issues a new Choumei Aza code for the delivery point. Therefore, a delivery point might have a current address and one or more legacy addresses and a corresponding set of current and legacy Choumei Aza codes.

You can submit a Japan address in batch or interactive mode and return one or more Choumei Aza codes that represent different versions of the address.

A verifier returns the following types of Choumei Aza code:

Choumei Aza Code JP

The code that corresponds directly to the validated version of the address that you submit.

New Choumei Aza Code JP

The code that corresponds to the next iteration of the input address that you submit. For example, if you enter the first or oldest version of the address, the New Choumei Aza Code JP value that the verifier returns represents the second version of the address.

Current Choumei Aza Code JP

The code that corresponds to the current version of the address that you submit.

Example

The Japan reference data contains the current version of an address and two older versions. You have the first or oldest version of the address. You select the Japan enrichments, and you submit the address in batch mode. A verifier verifies the address and returns the Choumei Aza code enrichments.

Note: The New Choumei Aza Code JP returns the Choumei Aza code for the update that directly followed the address that you submitted. You can rerun the steps with the New Choumei Aza Code JP value to find each version of the address in the reference data.

Gaiku Code

A Gaiku code is a four-digit code that identifies a city block in Japan.

Append the Gaiku code to a current Choumei Aza code to create a 15-digit string that you can submit in address code lookup mode to find an address.

Enrichments for Poland addresses

You can configure a verifier to include the Gmina Code PL, TERYT Locality Identifier PL, and TERYT Street Identifier PL as enrichments for validated Poland addresses.

Official Register of the Territorial Division of the Country (TERYT) is the agency responsible for identifiers and names of territories, localities, roads, and buildings in Poland. Gmina is the Polish equivalent of communes or municipalities. TERYT assigns and manages Gmina code and TerytIDs.

For example, when you validate

```
ul. Laskowa 1
50-510 Wrocław
POL
```

A verifier returns the following enrichment values:

```
Gmina Code PL: 2183
TERYT Locality Identifier PL: 0986544
TERYT Street Identifier PL: 10666
```

Enrichments for Russia addresses

You can configure a verifier to include the Federal Information Addressing System (FIAS) ID in the validated output for Russia addresses.

The FIAS ID is an alphanumeric string.

Enrichments for Serbia addresses

You can configure a verifier to include the Postal Address Code RS as an enrichment to the validated output for Serbia addresses.

The Postal Address Code RS is a six-digit code that maps to the street level. Including the Postal Address Code RS in an address ensures correct and prompt delivery to recipients in Serbia. You do not need the Postal Address Code RS for items that you address to a P.O. Box, *poste restante*, or to a military address.

Enrichments for South Africa addresses

You can configure a verifier to include the National Address Database Identifier ZA in the validated output for South Africa addresses. The National Address Database Identifier ZA is a unique numeric ID that is assigned to street addresses in South Africa.

For example, a verifier returns the National Address Database Identifier ZA value of `2170232` in the output when you validate the following address:

```
4 Balmoral Road
Vincent
East London
5247
South Africa
```

Enrichments for Spain addresses

You can configure a verifier to add INE code data as an enrichment to a valid Spain address. The INE code contains a series of values that identify the province, municipality, and street to which the address belongs. The National Statistics Institute of Spain (INE) maintains the INE codes.

A verifier returns the following INE codes:

- INE Province Code
- INE Municipality Code
- INE Street Code

Enrichments for Switzerland addresses

You can configure a verifier to include the additional postal code characters in the validated output for Switzerland addresses.

A verifier returns the additional postal code characters in an enrichment field called Postal Code Extension.

For example, when you validate the following Switzerland address with the address enrichment enabled, a verifier returns a Postal Code Extension value of 05.

```
Hohlen 1
3800 Sundlauenen
Switzerland
```

Enrichments for United Kingdom addresses

You can configure a verifier to include address enrichments in validated United Kingdom addresses.

Delivery Point Suffix

A two-character suffix that the Royal Mail assigns to a mailbox in a United Kingdom post code area. The first character in a delivery point suffix is a number and the second character is a letter. A combination of a post code and the delivery point suffix identifies a mailbox.

Delivery Point Type

A single-character code that indicates whether the address points to a residence (R), a small organization (O), or a large organization (L).

Unique Delivery Point Reference Number GB

An eight-character code that uniquely identifies each postal address in the Royal Mail PAF database. The Unique Delivery Point Reference Number (UDPRN) remains uniquely tied to the physical delivery point regardless of changes in the address.

Address Key

An eight-digit numeric code that maps to an address in the Postcode Address File (PAF) from the Royal Mail. An Address Key in conjunction with Organization Key and the Post Code Type uniquely identifies an address.

Organization Key

A unique 8-digit numeric code that Royal Mail assigns to small organizations.

UPRN

A numeric code that uniquely identifies a land or property unit in the United Kingdom. The Unique Property Reference Number (UPRN) is a code that the United Kingdom government assigns and can contain a maximum of 12 digits.

Examples

The following examples show enrichment values for different delivery point type addresses.

Delivery Point Type	Input Address	Enrichment Values
Residence	FLAT 17 GROVE HOUSE WAVERLEY GROVE LONDON N3 3PU UNITED KINGDOM	Address Key: 18161676 Delivery Point Suffix: 1H Delivery Point Type: R Organization Key: 00000000 Unique Delivery Point Reference Number GB: 15498195 UPRN: 200123099
Large Organization	PO BOX 43078 LONDON NW1 1SF UNITED KINGDOM	Address Key: 02356470 Delivery Point Suffix: 1A Delivery Point Type: L Organization Key: 00000000 Unique Delivery Point Reference Number GB: 17635833 UPRN: 10015054387
Small Organization	17A THE GROVE LONDON N3 1QN UNITED KINGDOM	Address Key: 28470295 Delivery Point Suffix: 1H Delivery Point Type: O Organization Key: 01464593 Unique Delivery Point Reference Number GB: 15491057 UPRN: 200210632

Enrichments for United States addresses

You can configure a verifier to add multiple address enrichments when you verify United States addresses.

Country Federal Information Processing Standard Code

A three-digit number that identifies a county in the United States.

Note: The Federal Information Processing Standards (FIPS) include a set of numbers that identify states, counties, and other territorial possessions in the United States. A two-digit state code identifies each state. A three-digit county code identifies a county within a state. Together, the five digits of the state and county codes uniquely identify a county.

State Federal Information Processing Standard Code

A two-digit number that identifies a state in the United States.

Metropolitan Statistical Area Identification

The Metropolitan Statistical Area (MSA) identification number is a five-digit number that identifies an urban area that has a population of 50,000 or more.

Core Based Statistical Area Identification

A Core-Based Statistical Area (CBSA) identification number is a five-digit number that identifies an urban area with a population greater than 10,000. A CBSA can be a Metropolitan Statistical Area or Micropolitan Statistical Area. A Metropolitan Statistical Area has over 50,000 inhabitants. A Micropolitan Statistical Area has between 10,000 and 50,000 inhabitants.

Census Tract Number

A six-digit number that identifies a Census Tract. A Census Tract is a statistical subdivision of a county.

Census Block Number

A four-digit number that identifies a Census Block. A Census Block is the smallest entity for which the Census Bureau collects census information.

Census Block Group

A 12-digit number that identifies a Census Block Group. A Census Block Group is a group of Census Blocks that share the same first digit. The first digit in the Census Block number is the last digit in the 12-digit Census Block Group number.

Finance Number

A finance number is a six-digit number that the United States Postal Service (USPS) assigns to post offices and other postal facilities to support the collection of cost and statistical data. The first two digits of the finance number identify the state. The final four digits identify the post office or postal facility.

Minor Civil Division Identification

A Minor Civil Division (MCD) identification number is a five-digit number. An MCD is a primary legal subdivision of a county.

Place Federal Information Processing Standard Code

A five-digit number that identifies a locality in the United States.

Enrichment and certification status fields

The verifier asset output fields include a *Country Specific Status* field and a *Certification Status* field that describe the delivery of data in any country-specific enrichment or country-specific certification field that you select. Find the Country Specific Status field in the *Country Specific* output field group. Find the Certification Status field in the *Certified* output field group.

You can find the fields in the following locations:

- Find the Country Specific Status field in the *Country Specific* output field group.
- Find the Certification Status field in the *Certified* output field group.

Address verification can return the following status codes in each field:

- AllValuesAvailable. Verification returns all requested data in the output address.
- SomeValuesAvailable. Verification returns some but not all of the requested data in the output address.
- NoValuesAvailable. Verification did not return any of the requested data.

Each status code field is empty when you do not request the enrichment or certification item concerned. The status code fields are also empty when address verification does not find a match for the input address.

INDEX

B

business rules
  facts 137

C

cleanse assets
  and dimensions 27
  steps 23
  testing a cleanse asset 37
Cloud Application Integration community
  URL 11
Cloud Developer community
  URL 11
conditions
  Define Action dialog box 150
copy and paste operations
  rule sets 141
  rule statements 163

D

Data Integration community
  URL 11
deduplicate assets
  mandatory and required fields 59
  testing a deduplicate asset 56

F

facts in business data 137

I

Informatica Global Customer Support
  contact information 12
Informatica Intelligent Data Management Cloud
  web site 11

L

labeler assets
  output fields 90
  testing a labeler asset 90

M

maintenance outages 12

P

parse assets
  and dimensions 96
  parsing modes 94
  testing a parse asset 116

R

rule sets
  configuration 137
  configuration rules and guidelines 137
  copy and paste operations 141
  properties 140
  testing a rule set 193
rule specifications
  configuration prerequisites 126
  dimensions 126
  output properties 132
  rule sets 122, 137
  rule statements 123, 143
  status values 123
  testing a rule specification 192
  verify business rule requirements 126
rule statements
  actions 123, 148
  AND relationships 178
  conditions 123, 144
  configuration 143
  copy and paste operations 163
  current date analysis 144
  default rule statement 123
  discovering business information 178
  improving data usability 177
  List of Values function 139
  menu options 159
  null value analysis 144
  operators 123, 147
  searching a list of values 181
  standardizing data 176
  steps to configure 160
  verify data against business regulations 179
  verifying data accuracy 174

S

status
  IDMC 12
steps
  in cleanse assets 23
system status 12

T

trust site
description 12

U

upgrade notifications 12

V

verifier assets
and dimensions 202

W

web site 11